TECHNICAL DOCUMENTATION

AEGIS

Advanced Evidence, Generation Intelligence & Synthetic-Media Forensics System

A complete guide to the multimodal forensic workstation that helps authorised analysts examine whether text, images, audio or video may have been generated or materially altered by AI — its principles, architecture, every module, the statistics behind its methods, deployment, operations, security and governance.

Reports only what the evidence supports

Repository
kryptonite012/AEGIS-AI-Content-Intelligence-Forensics-System
Software version 0.1.0 · MIT licence

Edition
November 2026

Contents

About This Document 9
How it was produced 9
Conventions 9
A note on honesty 10

PART I · FOUNDATIONS

1 Introduction 12
1.1 What AEGIS is 12
1.2 The problem it addresses 13
1.3 Who uses it 13
1.4 What AEGIS deliberately does not do 14
1.5 What works today 14
1.6 Technology at a glance 15
1.7 How this book is organised 15

2 Principles and Evidence Classes 16
2.1 The seven evidence classes 16
2.2 Rules the system enforces 17
2.3 The method status vocabulary 17
2.4 Raw scores are not probabilities 18
2.5 Outcome language 18
2.6 Honesty about results 19

3 System Architecture 20
3.1 The four layers 20
3.2 Runtime components 21
3.3 Evidence and analysis data flow 22
3.4 The investigation state machine 24
3.5 Key design properties 25
3.6 Architecture decision records 25
3.7 Scaling 26

4 Repository Layout and Toolchain 27
4.1 Top-level layout 27
4.2 The Python workspace 27
4.3 The TypeScript workspace 29
4.4 The Makefile 29
4.5 Getting started: three ways to run AEGIS 30
4.6 Continuous integration 31
4.7 What is not committed 31

PART II · THE APPLICATION PLATFORM

5 The API Service: Foundations 34
5.1 Package structure 34
5.2 Configuration 34
5.3 The application factory 36
5.4 Request middleware 37
5.5 Structured logging 37
5.6 Uniform error responses 37

5.7 Rate limiting 38
5.8 Database access 39
5.9 Migrations 39
5.10 Bootstrapping roles and the model registry 39

6 The Data Model 41
6.1 Overview by area 41
6.2 Core relationships 42
6.3 Identity tables 42
6.4 Case tables 43
6.5 Evidence tables 44
6.6 Analysis tables 44
6.7 Registry tables 45
6.8 Research tables 45
6.9 Governance tables 45
6.10 Database object store 46

7 Identity, Access and Tenancy 47
7.1 Passwords 47
7.2 Access and refresh tokens 47
7.3 The sign-in flow 48
7.4 Encrypting MFA secrets 50
7.5 Roles and permissions 50
7.6 Enforcing permissions: get_principal and require 51
7.7 Tenant isolation 52
7.8 Restricted cases 52
7.9 Summary of controls 53

8 Investigations and the Case Workflow 54
8.1 Opening a case 54
8.2 Listing and searching cases 54
8.3 The state machine 55
8.4 Editing, assigning and legal hold 56
8.5 Annotations 57
8.6 Human review 57
8.7 Appeals and corrections 58
8.8 Deletion and the two-person rule 59
8.9 Case history 59
8.10 Separation of duties: the complete list 60

9 The Evidence Vault 61
9.1 The acquisition pipeline 62
9.2 Write-once storage backends 66
9.3 Integrity verification 67
9.4 Redaction 67
9.5 Downloads, thumbnails, text and artefacts 67
9.6 Cross-case evidence views 68
9.7 Retention purge 68

10 The Hash-Chained Audit Trail 69
10.1 The hash formula 69
10.2 Appending an event 69
10.3 What is audited 70
10.4 Verifying the chain 71
10.5 What it detects and what it does not 71

10.6 Chain of custody 72
10.7 Viewing the audit trail 72

11 The Analysis Pipeline 73
11.1 Analysis profiles 73
11.2 Creating a job 74
11.3 The Celery worker 74
11.4 Executing a job 75
11.5 Cancelling a job 78
11.6 Recovering from lost workers 78
11.7 Live progress over server-sent events 78
11.8 Reading results 79

12 Reproducible Reports 80
12.1 Two representations, one source 80
12.2 What a report contains 80
12.3 The PDF layout 81
12.4 Generating a report 82
12.5 Finalising 82
12.6 Corrections 83
12.7 Exporting 83
12.8 Verifying 83

13 Administration, Health and Operations Commands 85
13.1 Dashboard and search 85
13.2 Notifications 85
13.3 Audit endpoints 85
13.4 Truthful health checks 86
13.5 Detectors and the model registry 87
13.6 User and organisation administration 88
13.7 Settings, retention and job recovery 88
13.8 Governance endpoints 89
13.9 The command-line tool 89

PART III · ANALYSIS METHODS

14 The Analysis Method Framework 92
14.1 Shared types 92
14.2 The uniform run wrapper 93
14.3 The method catalogue 93
14.4 Profiles and method selection 95
14.5 The per-frame video method 95
14.6 Running external tools safely 95

15 Text Methods 96
15.1 Extraction 96
15.2 Tokenisation 97
15.3 Language and script identification 98
15.4 Segmentation 98
15.5 Descriptive statistics 98
15.6 Model-based detectors 99
15.7 Why the text detectors are unavailable by default 101

16 Image, Audio and Video Methods 102
16.1 Images 102

16.2 Audio 104
16.3 Video 106
16.4 External detection providers 107

17 Provenance and Watermark Verification 108
17.1 C2PA Content Credentials 108
17.2 Declared (unsigned) generation metadata 110
17.3 The AEGIS reference text watermark 110
17.4 SynthID-Text boundary 112

18 Evidence Fusion: Policy aegis-fusion-v1 113
18.1 Why not combine scores? 113
18.2 Inputs 113
18.3 Categories 114
18.4 Uncertainty reasons and next steps 115
18.5 The primary outcome 115
18.6 Output 116
18.7 Worked examples 116

PART IV · RESEARCH AND EVALUATION

19 The Statistical Reliability Engine 118
19.1 Conventions 118
19.2 Confidence intervals for proportions 118
19.3 How many negatives does a strict FPR need? 119
19.4 Confusion-matrix metrics 119
19.5 Predictive values and base rates 120
19.6 Score-based metrics 120
19.7 Sensitivity at a fixed false-positive rate 121
19.8 Threshold sweeps and deployment projection 121
19.9 Calibration 121
19.10 Corroboration: combining two signals 122
19.11 Fairness: where do false flags land? 122
19.12 The full evaluation bundle 122
19.13 Tests 122

20 The Nine Reliability Experiments 123
20.1 Reproducing the results 123
20.2 Experiment 1 — Base-rate effects (analytical) 123
20.3 Experiment 2 — Subgroup error concentration (analytical) 124
20.4 Experiment 3 — Corroborating evidence (simulation) 125
20.5 Experiment 4 — AUROC versus strict operating points (simulation) 126
20.6 Experiment 5 — Watermark detection versus length and entropy (simulation) 127
20.7 Experiment 6 — Watermark robustness under random token replacement (simulation) 127
20.8 Experiment 7 — Threshold selection (analytical) 128
20.9 Experiment 8 — Watermark strength versus distortion (simulation) 129
20.10 Experiment 9 — Exact versus approximate false-positive calibration (analytical) 129
20.11 The built-in benchmark of the reference watermark 130

21 The Benchmark Laboratory and Research Bench 132
21.1 Evaluation methodology 132
21.2 Registering a dataset 132
21.3 Running a benchmark 133
21.4 Importing scores 134
21.5 The built-in reference-watermark benchmark 134

21.6 Calibration profiles 134
21.7 Research Bench endpoints 135
21.8 Independent evaluation protocol 135
21.9 Multilingual evaluation plan 136

PART V · THE WEB WORKSTATION

22 The Next.js Workstation 138
22.1 Technology 138
22.2 Source layout 138
22.3 One origin, no secrets in the browser 139
22.4 The API client 139
22.5 Authentication context 140
22.6 Navigation and permissions 140
22.7 Uploads with local hashing 141
22.8 Live analysis view 141
22.9 Design system 142
22.10 Shared types 142

23 Screen by Screen 143
23.1 Sign-in 143
23.2 Overview 143
23.3 New investigation 144
23.4 Case files 144
23.5 The case workspace 144
23.6 Text, Image, Audio and Video forensics 145
23.7 Provenance & watermarks 145
23.8 Evidence vault 145
23.9 Research bench 145
23.10 Benchmark laboratory 145
23.11 Reports 146
23.12 Audit & compliance 146
23.13 Administration 146
23.14 System health 146

PART VI · QUALITY, DEPLOYMENT AND OPERATIONS

24 Testing Strategy 148
24.1 Test suites 148
24.2 The API test harness 148
24.3 What each API test proves 149
24.4 Analysis-method tests 150
24.5 Research tests 151
24.6 Web tests 151
24.7 Static checks in CI 152

25 Deployment 153
25.1 Container images 153
25.2 Docker Compose 154
25.3 Production configuration 154
25.4 Campus or college server 155
25.5 Render (free plan) 156
25.6 Choosing a deployment 158

26 Operations 159

26.1 Configuring a real detector 159
26.2 Backup and recovery 160
26.3 Incident response 161
26.4 Performance 161
26.5 Troubleshooting 162

PART VII · SECURITY AND GOVERNANCE

27 Threat Model and Security Controls 165
27.1 Assets and trust boundaries 165
27.2 Threats and controls 166
27.3 Security controls matrix 168
27.4 Software bill of materials 169

28 Governance and Responsible Use 170
28.1 Status and intended use 170
28.2 The pilot readiness package 170
28.3 Human review and appeal procedure 171
28.4 Evidence-handling procedure (template) 171
28.5 Data-retention policy (template) 172
28.6 Privacy impact assessment (template) 172
28.7 Compliance control mapping 172
28.8 Limitations and residual risks 172
28.9 Roadmap 173
28.10 Responsible use 174

A Configuration Reference 175
A.1 Application settings (AEGIS_* , read by Settings) 175
A.2 Analysis-method settings (read by aegis_ml) 176
A.3 Operational and deployment variables 176

B API Endpoint Reference 178
Authentication and sessions 178
Investigations, review, appeals and retention 178
Evidence vault and uploads 179
Analysis jobs 180
Reports 180
Research bench and benchmark laboratory 181
Platform: overview, audit, health, registry, administration, governance 181
Request and response schemas 183

C Database Schema Reference 186
analysis_artifacts 186
analysis_jobs 186
analysis_signals 187
annotations 187
appeals 188
audit_events 188
benchmark_runs 189
benchmark_samples 189
benchmark_suites 189
calibration_profiles 190
datasets 190
deletion_requests 191
detector_runs 191

detector_versions 192
evidence_items 193
evidence_versions 193
experiment_runs 194
file_hashes 194
investigations 194
memberships 195
model_providers 196
notifications 196
organizations 196
permissions 196
provenance_records 197
reports 197
retention_policies 197
review_decisions 198
role_permissions 198
roles 198
segment_findings 199
stored_object_chunks 199
stored_objects 199
system_health_events 199
system_settings 200
upload_sessions 200
user_sessions 200
users 200
workspaces 201

D Role and Permission Matrix 202

D.1 Roles 202
D.2 Matrix 202

E Compliance Control Mapping 205

F Source File Inventory 211

(root) 211
.github 211
apps/api 211
apps/web 214
apps/worker 215
docs/api 216
docs/architecture 216
docs/assets 216
docs/governance 216
docs/operations 217
docs/research 217
docs/security 217
infrastructure/deployment 217
infrastructure/docker 218
infrastructure/render 218
ml 218
packages 220
research 220
scripts 220

G Glossary 221

About This Document

This document is a complete technical description of **AEGIS — Advanced Evidence, Generation Intelligence & Synthetic-Media Forensics System**, as contained in the repository `kryptonite012/AEGIS-AI-Content-Intelligence-Forensics-System`. It was written from a full reading of the source code, tests, configuration and documentation of the repository, and is intended for:

- **developers** who will maintain or extend the code;
- **operators and IT teams** who will deploy and run it;
- **investigators, analysts and reviewers** who want to understand exactly what each result means and does not mean;
- **evaluators, auditors and governance bodies** assessing the system for a pilot;
- **students and researchers** studying the reliability of AI-content detection.

How it was produced

Every statement about behaviour in this book is based on the code as it stands in the repository at the time of writing (version 0.1.0 of the API, analysis and research packages). Code excerpts are quoted from the source and occasionally shortened; omissions are marked with "…". The reference appendices — configuration variables, API operations, database schema, the role–permission matrix, the compliance mapping and the file inventory — were **generated by script directly from the source**, so they match the code exactly.

The nine reliability experiments were **re-run** while this document was prepared, and the charts in the Experiments chapter are the plots produced by that run; every number matched the tables published in the repository. The research and analysis-method test suites (52 tests) were also run and passed. The API test suite needs a PostgreSQL server and was not re-run in the preparation environment; it runs in the project's CI pipeline.

Conventions

Convention	Meaning
`apps/api/app/services/evidence.py`	A path relative to the repository root
`AEGIS_SECRET_KEY`	An environment variable
`POST /investigations/{id}/evidence`	An API operation; all paths are under `/api/v1`
`text.binoculars`	An analysis method identifier
`case.read`	A permission
unavailable, *failed*, …	Method run statuses
Shaded **Note** boxes	Important clarifications

The screenshots come from the repository's documentation (`docs/assets/`) and show the demonstration deployment with synthetic **DEMO FIXTURE** data. No figure in this document describes a real deployment or real casework.

A note on honesty

AEGIS is unusual in how carefully it separates what it can do from what it cannot. This book follows the same rule. Where a capability is an integration point rather than a working detector, where a number is simulated rather than measured, or where a control is documented but has not been exercised, the text says so.

PART I

Foundations

Purpose, principles, evidence classes, architecture and the repository.

CHAPTER 1

Introduction

1.1 What AEGIS is

AEGIS stands for *Advanced Evidence, Generation Intelligence & Synthetic-Media Forensics System*. It is a multimodal forensic workstation. Authorised analysts use it to examine whether a piece of text, an image, an audio recording or a video may have been generated or materially altered by artificial intelligence. Its guiding sentence, repeated in the README, the API description and the user interface, is that it *reports only what the evidence supports*.

That sentence matters more than any single feature. Most "AI detectors" return a single percentage and invite the reader to treat it as a verdict. AEGIS was designed the other way around. It treats detection as an investigation: a case is opened with a documented purpose, evidence is acquired and sealed with cryptographic hashes, a set of independent methods is run, an independent human reviewer decides what the results mean, and a reproducible report is exported. Every step is written to a tamper-evident audit trail. A score is one input to that process and is never allowed to become the conclusion by itself.

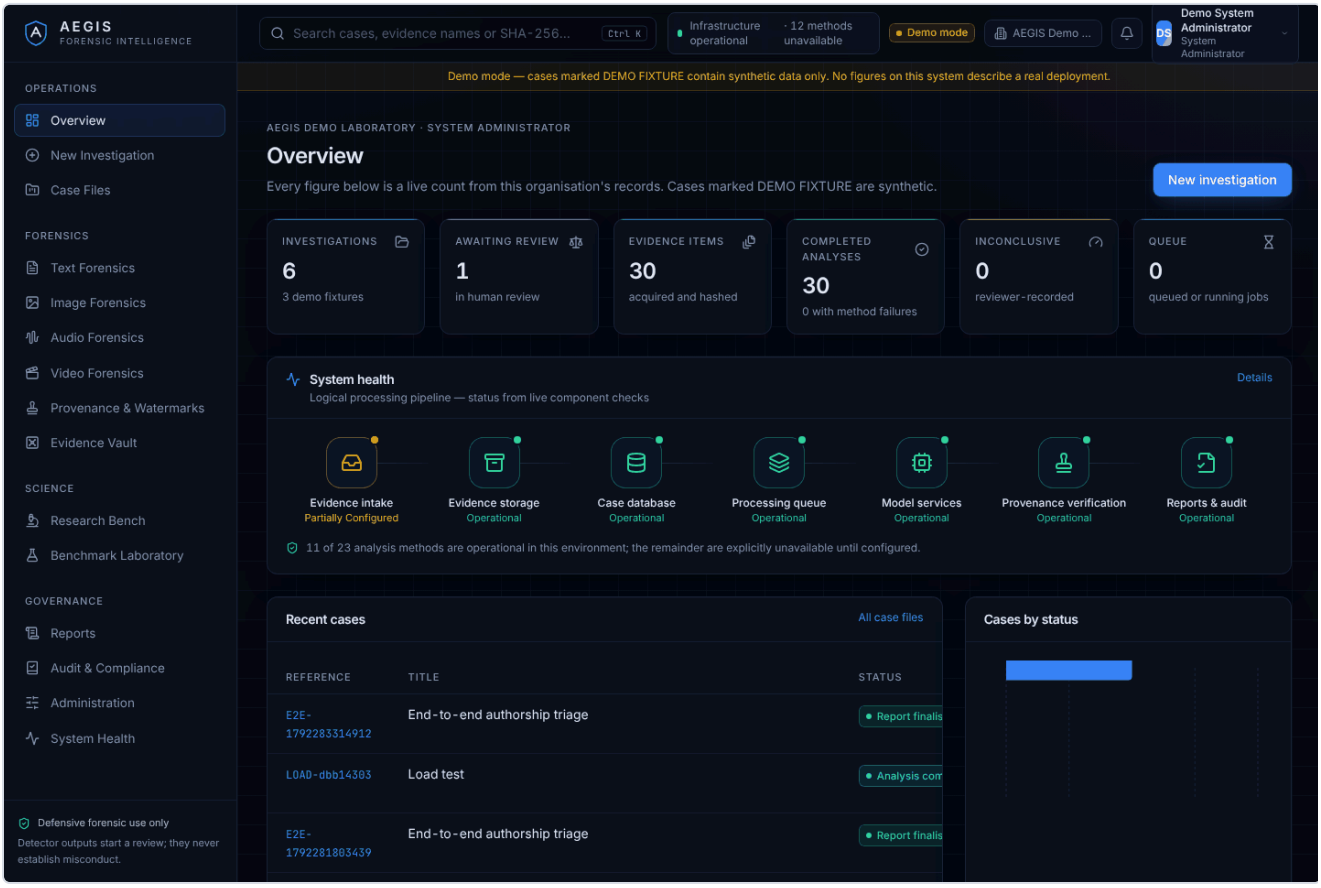


Figure 1. The AEGIS overview dashboard. The header shows the deployment mode, the signed-in role and the live status of the system; the cards summarise open cases, queued analyses and items needing review.

1.2 The problem it addresses

Generative models can now produce fluent essays, photorealistic images, convincing synthetic voices and edited video. Institutions such as universities, newsrooms, courts, regulators and investigative units increasingly face questions like “was this assignment written by a model?”, “is this photograph authentic?” or “is this a cloned voice?”. The tools that claim to answer those questions have well-documented problems:

- **Statistical detectors are not proof.** A classifier that says text “resembles” model output is measuring similarity to its training distribution, not authorship. Detectors generalise poorly to unseen generators, to edited or paraphrased text, to short passages and to non-native writers.
- **Base rates dominate.** Even a detector with 90% sensitivity and a 5% false-positive rate produces more false flags than true ones when only 5% of submissions are AI-generated. The research chapter of this book reproduces that calculation exactly.
- **Different kinds of evidence get blended.** A cryptographic provenance signature, a watermark test, a statistical score and a reviewer’s opinion are fundamentally different things. Averaging them into one number destroys the information that makes each one meaningful.
- **Process failures cause harm.** A score shown without its limitations, its calibration status or the population it applies to invites misuse: a student penalised, a journalist discredited, an exhibit wrongly admitted or excluded.

AEGIS is built around these failure modes. Instead of trying to be a better oracle, it gives investigators a disciplined workflow in which each kind of evidence keeps its own meaning, uncertainty is shown rather than hidden, and a qualified human makes the final assessment.

1.3 Who uses it

The system defines seven roles (the Identity, Access and Tenancy chapter explains the permission model in detail):

Role	Typical person	What they do in AEGIS
Investigator	Case officer, academic-integrity officer	Opens cases, acquires evidence, runs analyses, drafts reports
Forensic analyst	Technical examiner	Runs analyses, registers datasets, runs benchmarks
Independent reviewer	Senior examiner, panel member	Records the human assessment; finalises reports
Auditor	Internal audit, oversight body	Read-only access to cases, reports and the audit chain
Read-only researcher	Methods researcher	Uses the research bench and benchmark results; never sees case data
Organisation administrator	Department lead	Manages users, settings, retention and the model registry
System administrator	Platform operator	Full access including system-wide settings

The roles are designed to enforce *separation of duties*: the person who opens a case cannot review it, the author of a report cannot finalise it, and the person who requests deletion of evidence cannot approve that deletion.

1.4 What AEGIS deliberately does not do

The project is explicit about scope. AEGIS is defensive investigative software. It contains **no** surveillance, account-access or device-monitoring capability; it performs **no** biometric identification, speaker recognition or face recognition; and it never infers a person's identity, nationality, ethnicity or intent. It claims no government approval or certification. Its documentation states, in bold, that *a detector score alone must never lead to punishment, rejection, prosecution or any other adverse decision about a person.*

1.5 What works today

The repository is candid about what is operational and what is only an integration point. The table below is taken from the project's own capability statement and is reflected throughout the code by explicit method statuses (`completed`, `unavailable`, `failed`, and so on).

Capability	Status
Case management: state machine, separation of duties, appeals, corrections	Operational and tested
Evidence vault: content sniffing, SHA-256, write-once storage (local disk, S3 or PostgreSQL), versions, redaction, integrity alerts, two-person deletion	Operational and tested
Text extraction (TXT, PDF, DOCX), segmentation, language and script identification (including Hindi and other Indic scripts)	Operational and tested
C2PA Content Credentials verification	Tested with signed, tampered and unsigned files
AEGIS reference text watermark verifier	Verifies text produced by the AEGIS reference generator under the deployment's key
Image, audio and video forensic observations (Pillow, NumPy/SciPy, ffmpeg)	Operational and tested
Reports: deterministic PDF plus canonical JSON with SHA-256, re-verifiable	Operational and tested
Security: Argon2id, JWT with revocable sessions, rotating refresh cookie, TOTP MFA, RBAC (7 roles, 32 permissions), tenant isolation, rate limits, hash-chained audit	Operational and tested
Supervised, token-statistic, Fast-DetectGPT and Binoculars text detectors	Adapters implemented; no model weights bundled, so they report `unavailable` until configured
Synthetic-speech and image classifiers (ONNX), external provider services	Integration points that run once a model or authorised provider is configured
SynthID-Text and other provider watermarks	Placeholder boundary; needs a provider agreement, keys and weights
Calibrated probabilities and validated thresholds	None yet: no detector has been validated on locally measured real data

> **Reading the status column.** "Unavailable" is not a bug. It is one of the system's central design decisions (ADR-03, see the Architecture chapter): if a model is not configured, the method says so and gives the reason, instead of silently substituting a heuristic and producing a number that looks authoritative.

1.6 Technology at a glance

Layer	Technology
Web workstation	Next.js 15 (App Router), React 18, Tailwind CSS, TanStack Query and Table, Recharts
API	FastAPI, SQLAlchemy 2, Alembic, PostgreSQL 16, server-sent events for live job progress
Background jobs	Celery with Redis, late acknowledgement and idempotent runs; an *inline* mode for single-container hosts
Analysis library	`aegis_ml`: content sniffing, extraction, every forensic and detection method, C2PA verification, provider adapters, fusion
Research library	`aegis_research`: reliability statistics, reference watermark, nine reliability experiments
Storage	Write-once, content-addressed object storage on local disk, S3 or PostgreSQL
Packaging	uv workspace for Python, pnpm workspace for TypeScript, Docker images, Render Blueprint

1.7 How this book is organised

This document explains the whole repository: its purpose, its design, every major module, the mathematics behind its methods, how it is deployed and operated, and how it is governed.

- **Part I — Foundations** covers the principles, the evidence classes, the architecture and the repository layout.
- **Part II — The Application Platform** walks through the FastAPI service: configuration, the data model, identity and security, investigations, the evidence vault, the audit chain, the analysis pipeline, reports and administration.
- **Part III — Analysis Methods** describes the `aegis_ml` library method by method: text, image, audio, video, provenance, watermarking, external providers and the evidence-fusion policy.
- **Part IV — Research and Evaluation** explains the reliability statistics, the nine experiments (with charts regenerated for this book), and the benchmark laboratory.
- **Part V — The Web Workstation** covers the Next.js application screen by screen.
- **Part VI — Quality, Deployment and Operations** covers the test suite, container images, deployment targets, CI, operations runbooks and performance.
- **Part VII — Security and Governance** covers the threat model, controls, governance procedures, limitations and the roadmap.
- **Appendices** provide reference material generated directly from the source: every configuration variable, every API operation, every database table and column, the role–permission matrix, the compliance control mapping, a source-file inventory and a glossary.

Throughout the book, file paths are given relative to the repository root, for example `apps/api/app/services/evidence.py`. Code excerpts are quoted from the source; where an excerpt is shortened, the omission is marked.

CHAPTER 2

Principles and Evidence Classes

AEGIS is easier to understand once its few governing principles are clear. Almost every design decision in the code can be traced back to one of them. This chapter states them and introduces the vocabulary used in the rest of the book.

2.1 The seven evidence classes

Every result AEGIS produces belongs to exactly one *evidence class*. The classes are never mixed, multiplied or averaged together. Each one answers a different question, has a different failure mode and needs a different kind of expertise to interpret.

Evidence class	What it means	Example in AEGIS
Statistical inference	How closely content resembles known AI output. *Not proof of authorship.*	Supervised classifier, token statistics, Fast-DetectGPT, Binoculars (need model weights)
Watermark verification	A keyed statistical test for one specific watermark scheme	The AEGIS reference green-list watermark (Kirchenbauer et al., 2023) with exact binomial p-values
Signed provenance	A cryptographic check of a signed record. A valid signature does not make the record's claims true.	C2PA Content Credentials, checked with the official `c2pa-rs` library
Forensic observation	Measurements for an analyst to interpret; not a classifier	EXIF/ICC/JPEG tables, frequency spectrum, noise residual, error-level analysis, spectrograms, video timelines
Declared metadata	Unsigned claims inside the file, which anyone can edit	PNG text chunks, EXIF `Software`, XMP `DigitalSourceType`
Process evidence	Drafts, revision histories and notes	Evidence uploaded with the `process_evidence` role
Human assessment	A qualified reviewer's conclusion	A review decision, hashed and recorded in the audit trail

In code, the classes are the `EvidenceClass` enumeration in `ml/aegis_ml/core.py`:

```
class EvidenceClass(StrEnum):
    """Different forms of evidence. They are never interchangeable and never averaged."""

    STATISTICAL_INFERENCE = "statistical_inference"
    VERIFIED_WATERMARK = "verified_watermark"
    CRYPTOGRAPHIC_PROVENANCE = "cryptographic_provenance"
    MEDIA_FORENSIC_OBSERVATION = "media_forensic_observation"
    DECLARED_METADATA = "declared_metadata"  # unsigned claims embedded in the file
    PROCESS_EVIDENCE = "process_evidence"  # drafts, revision history, notes
    HUMAN_ASSESSMENT = "human_assessment"
```

Each analysis method declares its class. The database stores it on every `DetectorRun`, the fusion policy groups results by it, the user interface displays results grouped by it, and the exported report keeps the groups apart.

Why the classes must stay separate

Consider three results for the same image: a C2PA signature that verifies, a frequency spectrum with periodic peaks, and an EXIF `Software` field that says "Stable Diffusion". These are not three votes on the same question.

- The **signature** says that a particular signer attested to particular claims about the file and that the bytes have not changed since. It says nothing about whether those claims are true, and its trustworthiness depends on whether the signer is in the configured trust list.
- The **spectrum** is a measurement. Periodic peaks are *consistent with* upsampling, which some generators perform, but also with ordinary resizing. An analyst must interpret it.
- The **metadata** is an unsigned claim that anyone with a text editor can add or remove.

Averaging these into a single "AI likelihood" would hide all of this. AEGIS therefore reports them side by side and lets the human reviewer weigh them, with each result carrying its own limitations.

2.2 Rules the system enforces

The README lists rules that the code enforces rather than merely recommends:

1. **A detector flag starts a review. It never establishes misconduct.** The case workflow requires an independent human review decision before a report can be finalised.
2. **A low score does not prove a human wrote the content.** Outcome categories are phrased to describe evidence ("no reliable conclusion", "some signals consistent with AI"), never to declare authenticity.
3. **A missing watermark or provenance record proves nothing either way.** The watermark verifier and the C2PA verifier report "not detected" or "no manifest" as neutral facts.
4. **A method that cannot run reports `unavailable`; a method that crashes reports `failed`.** Neither ever produces a score. This is enforced centrally by the `AnalysisMethod.run()` wrapper.
5. **Fusion reports disagreement as a contradiction, never as an average.** The `aegis-fusion-v1` policy combines only categorical facts.
6. **Base rates change what a flag is worth.** The research bench shows the positive predictive value (PPV) of a flag at a stated prevalence, and true-positive rates at strict false-positive rates are reported only when the sample is large enough to estimate them.

2.3 The method status vocabulary

Every run of every method ends in exactly one of these statuses (`RunStatus` in `ml/aegis_ml/core.py`):

Status	Meaning	Can it carry a score?
`completed`	The method ran and produced its output	Yes, if the method computes one
`unavailable`	The method cannot run in this deployment (no weights, missing tool, no key). *Not a negative result.*	**No**

Status	Meaning	Can it carry a score?
`insufficient_input`	The input is too short, too small or otherwise below the method's minimum	**No**
`unsupported_input`	The method does not handle this modality or MIME type	**No**
`failed`	The method started but raised an error. *Not a negative result.*	**No**
`blocked_by_policy`	The method would send evidence outside the deployment and external processing is not permitted	**No**

The database adds two transient statuses, `pending` and `running`, for runs that have not finished.

The distinction between `unavailable` and `failed` is deliberate. `unavailable` is an honest, expected state ("this deployment has no text classifier configured"). `failed` is an incident that an operator should look at. Neither is allowed to fall back to a heuristic that would produce a misleading number.

2.4 Raw scores are not probabilities

A detector's raw output, whether a logit, a perplexity ratio or a vendor's "probability", is not a calibrated probability for the population being examined. AEGIS records raw scores with a `score_semantics` string that says what the number is (for example "Binoculars score: log-perplexity divided by cross-perplexity; lower values are more typical of machine text"). Two fields are kept empty unless strict conditions are met:

- `calibrated` is true only when a calibration profile, fitted on locally measured validation data and marked as validated, is attached to that exact method version.
- `flagged` is set only when such a profile supplies a threshold. Without one, a score is displayed but never interpreted.

Calibration is also dropped if the evidence is in a language the model was not evaluated on, and the run carries an out-of-distribution warning.

2.5 Outcome language

The fusion policy (`Outcome` in `ml/aegis_ml/fusion.py`) and the review workflow use a fixed set of outcome categories. They describe what the evidence shows, not what the truth is:

Outcome	Meaning
`no_reliable_conclusion`	No method produced interpretable evidence
`inconclusive`	Evidence exists but does not support a conclusion
`some_signals_consistent_with_ai`	One validated statistical signal is above its threshold
`multiple_compatible_signals_consistent_with_ai`	More than one validated signal agrees
`evidence_of_recognized_watermark`	A supported watermark verified under a known key
`provenance_record_verified`	A signed provenance record verified
`provenance_verification_failed`	A provenance record was present but failed verification
`contradictory_evidence`	Signals point in different directions

Outcome	Meaning
`human_review_required`	Added by fusion whenever a category that needs a human (any AI-consistent signal, a watermark, a failed provenance check or a contradiction) is present
`further_analysis_requested`	Review-only outcome: the reviewer sends the case back for more analysis

The human review and appeal procedure (see the Governance chapter) instructs reviewers to use the same style: “consistent with”, “inconclusive”, “verification succeeded for the supported provenance record”. Statements such as “100% AI-generated” or “definitively human-written” are explicitly prohibited.

2.6 Honesty about results

The final principle runs through the documentation as much as the code: **say exactly what was measured and nothing more.** Benchmark results are stored with a `result_kind` (`locally_measured`, `simulated`, `published`, `reproduced` or `vendor_reported`) enforced by a database CHECK constraint. Every experiment result records whether it is *analytical* or *simulation*. Performance figures state the hardware they were measured on. The governance package lists what is *not* implemented as carefully as what is. This book follows the same convention.

CHAPTER 3

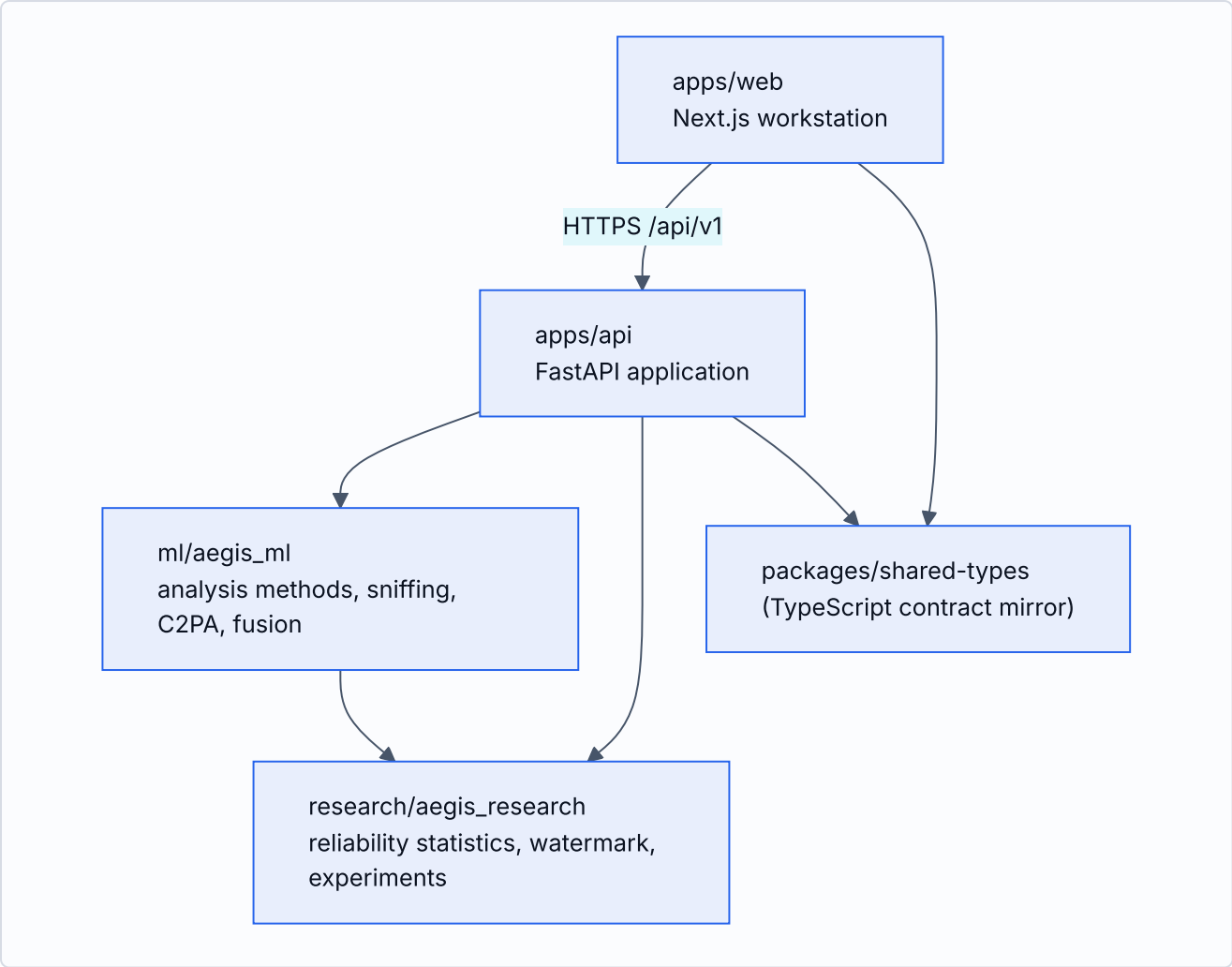
System Architecture

AEGIS is a modular monorepo. It is organised in four layers that are deliberately kept apart, so that each one can be tested, versioned and reasoned about independently. This chapter describes the layers, the runtime components, the main data flows and the architecture decision records (ADRs) that shaped them.

3.1 The four layers

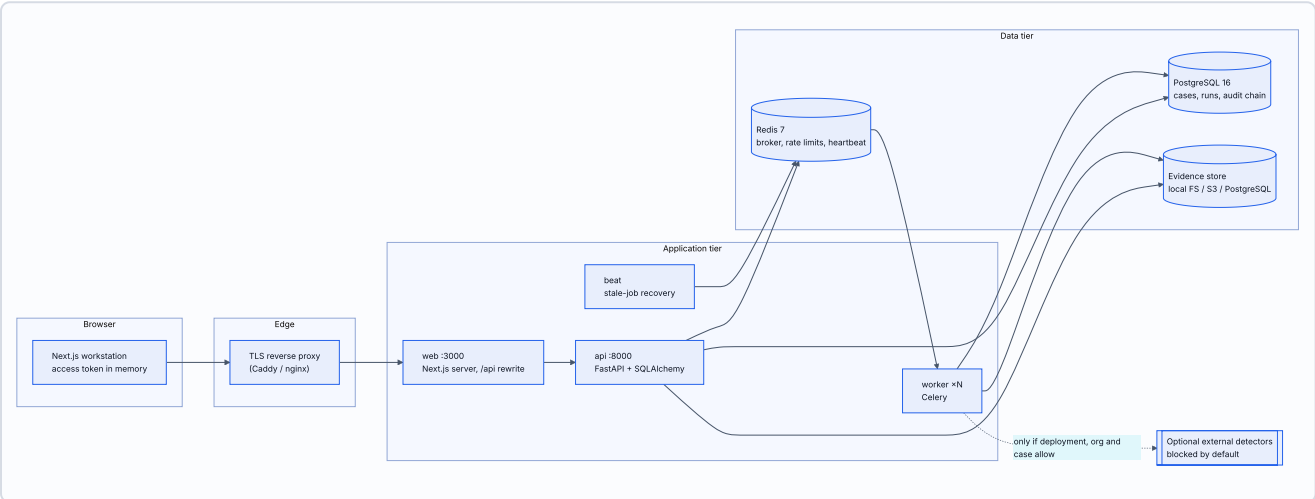
Layer	Package	Responsibility	What it must not do
Research	`research/aegis_research`	Pure statistics: Bayes PPV/NPV, exact intervals, ROC/PR curves, calibration, strict-FPR operating points, the reference watermark and the nine experiments	No I/O, no database, no web framework
Analysis methods	`ml/aegis_ml`	Content sniffing, text extraction, every forensic, detection and verification method, provider adapters, the evidence-fusion policy	No database; methods receive an input object and return a result object
Application	`apps/api/app`	Identity, RBAC, tenancy, cases, evidence vault, job orchestration, reports, audit, benchmarks, administration	No analysis logic of its own; it calls `aegis_ml` and `aegis_research`
Presentation	`apps/web`	The investigative workstation	Talks only to the versioned REST API; no direct database or storage access

Dependencies point downwards only: the web app depends on the API's HTTP contract; the API depends on `aegis_ml` and `aegis_research`; `aegis_ml` depends on `aegis_research` (for the watermark and statistics); `aegis_research` depends only on NumPy and SciPy. This makes the lower layers reusable from the command line, from benchmarks and from tests without starting a server.



3.2 Runtime components

In a full deployment AEGIS runs as a set of cooperating processes. The Docker Compose file (see the Deployment chapter) starts them all.



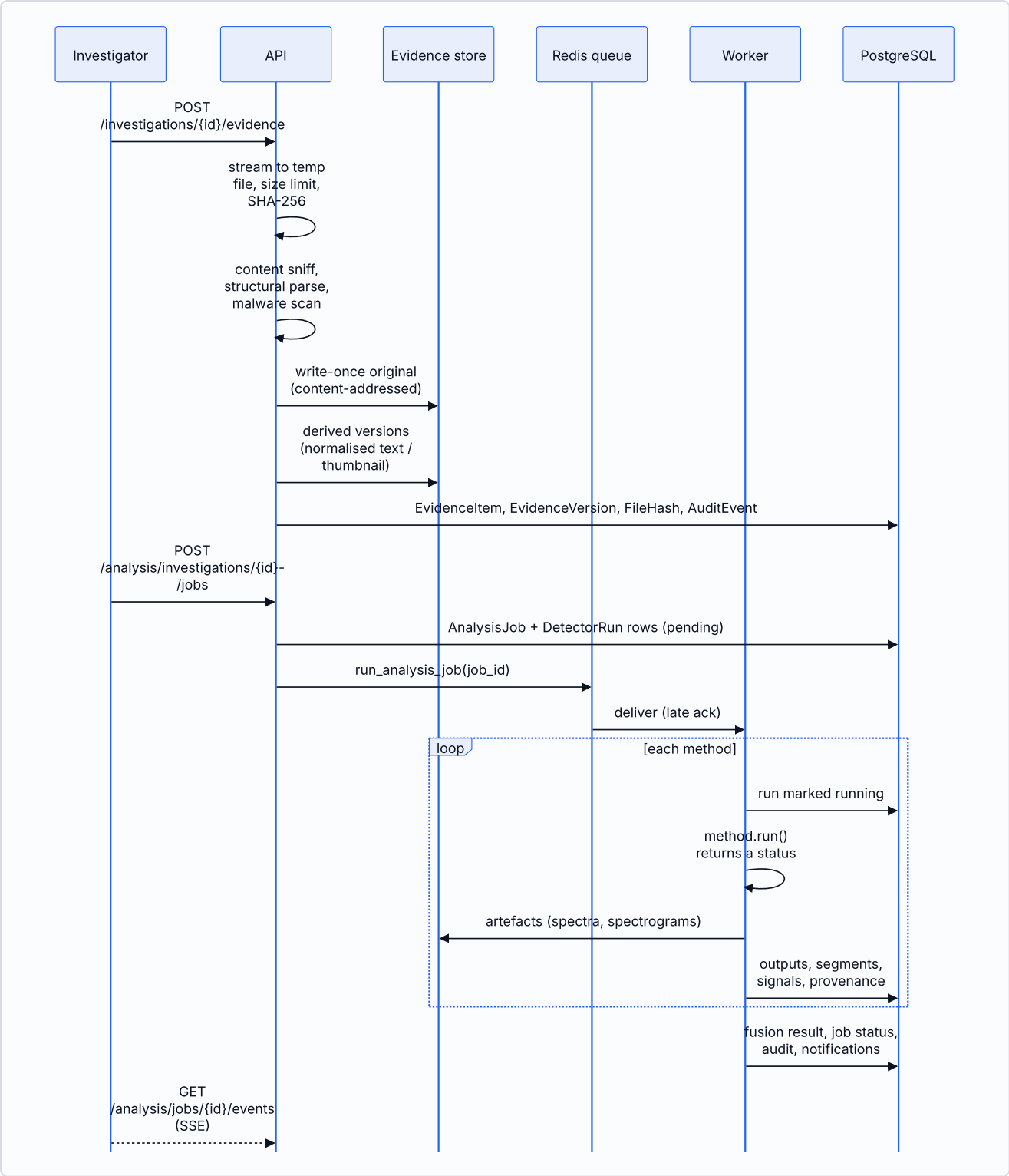
Component	Process	Role
web	`next start` (Node 22)	Serves the workstation and rewrites `/api/*` to the API, so the browser sees a single origin
api	`uvicorn app.main:app`	Stateless REST API; authenticates every request, enforces permissions and tenancy, writes the audit chain
worker	`celery -A app.workers.celery_app worker -Q aegis,aegis-research`	Runs analysis jobs and benchmarks; writes results to PostgreSQL and artefacts to the evidence store
beat	`celery ... beat`	Every 10 minutes, finds jobs abandoned by a lost worker and requeues them (at most three attempts)
migrate	`alembic upgrade head && python -m app.cli sync`	One-shot container that applies migrations and synchronises roles and the model registry before the others start
PostgreSQL 16	database	All structured state: users, cases, evidence metadata, runs, reports, the audit chain
Redis 7	broker and cache	Celery broker, rate-limit counters, worker heartbeat; holds no authoritative state
Evidence store	filesystem, S3 or PostgreSQL tables	Write-once, content-addressed binary objects: originals, derivatives, artefacts, report PDFs, datasets

Single-container mode

For hosts that cannot run a separate worker or Redis (for example Render's free plan), AEGIS supports a single-container mode: `AEGIS_TASK_MODE=inline` runs analysis jobs in a background thread inside the API process, `AEGIS_REDIS_URL` is left empty so rate limiting falls back to an in-process counter, and `AEGIS_STORAGE_BACKEND=database` keeps evidence in PostgreSQL because the container's filesystem is reset on every restart. The API refuses inline mode when `AEGIS_ENVIRONMENT=production`.

3.3 Evidence and analysis data flow

The following sequence shows what happens from the moment an investigator uploads a file until the analysis results are visible.



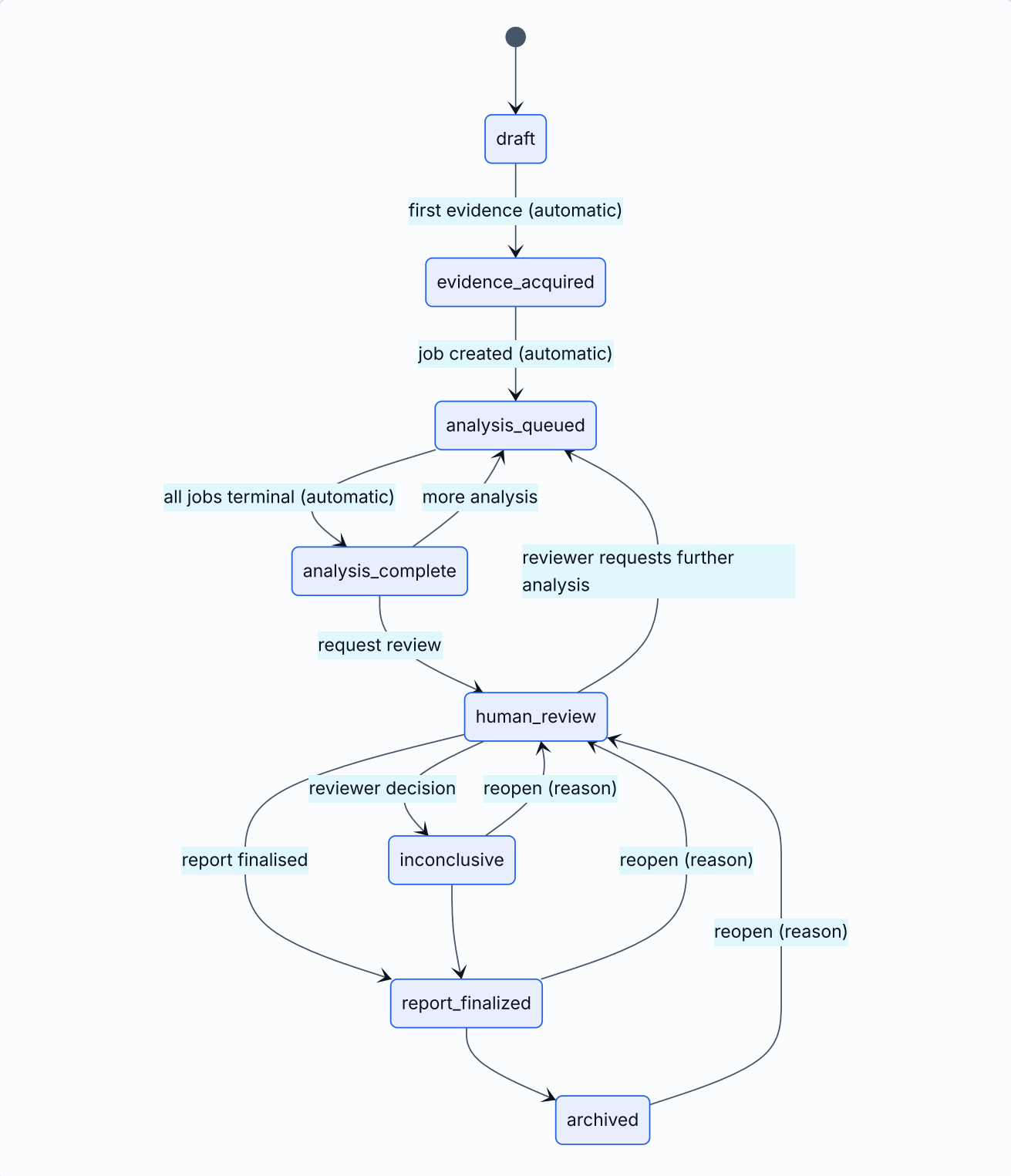
Three properties of this flow are worth stating explicitly:

1. **Only identifiers cross the queue.** The Redis message contains the job id and nothing else. All state lives in PostgreSQL. A refreshed browser, a lost worker or a redelivered message therefore cannot corrupt the job.
2. **Progress is truthful.** The progress bar is `methods_done / methods_total`, read from the database. The server-sent event stream emits a new snapshot only when the persisted state changes; nothing is driven by a timer.

3. **Every artefact is evidence-grade.** Visualisations such as spectra and spectrograms are stored in the same write-once, content-addressed store as the evidence, with their own SHA-256, and are served with an `X-AEGIS-SHA256` header.

3.4 The investigation state machine

Cases move through a fixed set of states. Some transitions are automatic (driven by evidence and jobs); others require a person with the right permission, and reopening a closed case requires a written reason.



Finalised and archived cases refuse edits and new evidence. Corrections create a new report version and preserve the previous one. The state machine is implemented in `apps/api/app/services/investigations.py` and is described in detail in the Investigations chapter.

3.5 Key design properties

- **No fabricated output.** `AnalysisMethod.run()` wraps every method. Unsupported input, a policy block, missing configuration or an exception each produce an explicit status. A score is only ever set by a method that actually computed it.
- **Idempotent, resumable jobs.** `DetectorRun` rows are unique per `(job_id, method_id)`. A redelivered task skips completed runs. The `recover_stale` task requeues jobs abandoned by a lost worker, at most three attempts.
- **Evidence classes never mix.** The fusion policy never averages or multiplies scores. It reports agreeing, conflicting, missing and failed signals and a contradiction state.
- **Separation of duties.** The case creator cannot review their own case. A report's author cannot finalise it. A deletion requester cannot approve their own request. An appellant cannot resolve their own appeal.
- **Defence in depth on tenancy.** Every query is scoped by organisation membership. Identifiers belonging to another tenant return 404, identical to "missing", so they cannot be probed.
- **Reproducible reports.** The PDF is rendered from the canonical JSON payload in ReportLab's invariant mode, so verification can re-render it and compare hashes byte for byte.

3.6 Architecture decision records

The repository keeps short ADRs in `docs/architecture/decisions.md`. They are reproduced here with commentary, because they explain *why* the code looks the way it does.

ADR-01 — Python services, TypeScript workstation. Detection research and forensic tooling (NumPy, SciPy, Pillow, ffmpeg, the `c2pa-rs` bindings, PyTorch and ONNX) live in Python. The analyst interface is Next.js with typed API contracts in `packages/shared-types`. *Consequence:* one language per concern, with the contract mirrored in TypeScript.

ADR-02 — Methods are libraries, not services. Analysis methods live in `aegis_ml` and run inside workers. This keeps one versioned code path for the API, the worker, the CLI and benchmarks. External services connect through `HttpDetectorProvider` with a strict JSON contract. *Consequence:* a benchmark measures exactly the code that runs in production.

ADR-03 — Explicit capability status instead of fallbacks. No method substitutes a heuristic when a model is missing. Unconfigured methods are `unavailable`, and the interface shows the reason. *Consequence:* silent fake scores are impossible by construction.

ADR-04 — Fusion by evidence class, not by score arithmetic. No detector pair has held-out joint validation and detector dependence is unknown, so `aegis-fusion-v1` combines only categorical facts (flagged under a validated threshold, verified, failed). It reports contradictions instead of resolving them. A learned combiner may be added only with documented held-out validation. Experiment 3 in the research chapter shows numerically why multiplying false-positive rates of correlated detectors is wrong.

ADR-05 — Raw scores are not probabilities. `flagged` and `calibrated` are set only when a calibration profile validated on locally measured data is attached through the model registry, or ships beside the model as `aegis_calibration.json` with `validated: true`.

ADR-06 — Content-addressed, write-once evidence storage. Keys derive from SHA-256. Writes use a temporary file plus a hard link to prevent overwrite races, and objects are made read-only. Deletion is possible only through the audited two-person retention purge, which keeps tombstones.

ADR-07 — Hash-chained audit trail in the primary database. This is simple and transactional with the operation it records: an event commits or rolls back with it. It is tamper-evident but not tamper-proof against a privileged database administrator; anchoring the chain head externally is the documented mitigation.

ADR-08 — Celery and Redis with database-persisted job state. The broker carries only job ids. All state lives in PostgreSQL, so refreshes, worker loss and redelivery are safe. Late acknowledgement and per-run uniqueness make execution idempotent.

ADR-09 — Access token in memory, refresh token in an httpOnly SameSite=Strict cookie scoped to `/api/v1/auth`. The refresh endpoint also requires a custom header, which cross-site forms cannot send. The browser talks to the API through a same-origin rewrite.

ADR-10 — Reference watermark as a real, narrow verifier. No provider watermark verifier was available. AEGIS implements the published green-list scheme with its own keyed generator, so the verification pathway (keys, exact p-values, length checks, robustness benchmarks) is exercised end to end. It is labelled everywhere as able to verify only AEGIS-generated reference text.

ADR-11 — Static ffmpeg in container images. Images copy pinned static ffmpeg and ffprobe builds (`mwader/static-ffmpeg:7.1`). Builds then need no distribution package mirror, and the media toolchain version is pinned.

ADR-12 — Next.js 15 with React 18. Moving to Next 15.5 cleared all `pnpm audit` advisories. React 18.3 is kept to limit churn; Next 15 supports it.

3.7 Scaling

The API is stateless; run multiple uvicorn workers or replicas behind the proxy. Workers scale horizontally by adding containers, and `-c` sets per-container concurrency according to available memory. Model-backed methods should run in dedicated worker pools on a separate queue, sized to model memory and placed on GPU nodes where available. PostgreSQL holds only structured data; binaries live in object storage, except with the `database` storage backend, which is intended for small deployments only. Measured numbers are given in the Operations chapter.

CHAPTER 4

Repository Layout and Toolchain

This chapter is a map of the repository. It explains where each piece lives, how the Python and TypeScript workspaces are wired together, and which commands a developer uses day to day.

4.1 Top-level layout

```
AEGIS-AI-Content-Intelligence-Forensics-System/
├── apps/
│   ├── api/            FastAPI service: models, migrations, routes, services, Celery tasks, CLI,
tests
│   ├── web/            Next.js 15 workstation, Vitest unit tests, Playwright end-to-end tests
│   └── worker/         README only: the worker runs the API image with a Celery command
├── ml/                 aegis_ml: analysis methods, content sniffing, C2PA, providers, fusion
├── research/           aegis_research: reliability statistics, reference watermark, experiments
├── packages/
│   └── shared-types/   TypeScript mirror of the API contract
├── infrastructure/
│   ├── docker/         api, web and single-container (Render) Dockerfiles
│   ├── deployment/     Caddy reverse proxy and server Compose override
│   └── render/         Render start script, static/API proxy, external-database Blueprint
├── docs/               architecture, security, API, research, operations, governance, screenshots
├── scripts/            dev runner, server setup, load test
├── .github/workflows/  CI pipeline
├── docker-compose.yml  full local/server stack
├── render.yaml         Render Blueprint (one-click deploy)
├── Makefile            task runner
├── pyproject.toml      uv workspace root (virtual)
├── package.json        pnpm workspace root
├── uv.lock, pnpm-lock.yaml   locked dependency graphs
└── .env.example        documented configuration template
```

The repository contains 213 tracked files. Excluding the two lock files, there are roughly 24,000 lines of source, tests, configuration and documentation. The largest modules are the initial database migration (about 1,000 lines), the platform administration router (about 880 lines), the investigations router (about 790 lines), the reliability statistics module (about 700 lines) and the ORM models (about 660 lines). Appendix F lists every file with its size and purpose.

4.2 The Python workspace

The Python code is managed by uv as a *workspace* with three member packages. The root `pyproject.toml` is a virtual project (`package = false`) that ties them together:

```
[tool.uv.workspace]
members = ["apps/api", "ml", "research"]

[tool.uv.sources]
aegis-ml = { workspace = true }
aegis-research = { workspace = true }
aegis-api = { workspace = true }
```

Package	Import name	Key dependencies
`aegis-research`	`aegis_research`	`numpy ⩾ 2.1`, `scipy ⩾ 1.14`; optional `matplotlib` for plots
`aegis-ml`	`aegis_ml`	`aegis-research`, NumPy, SciPy, Pillow ≥ 12.1.1, pypdf ≥ 6.21, python-docx, langdetect, `c2pa-python` 0.38, httpx, cryptography; optional extra `transformers` (torch + transformers)
`aegis-api`	`app`	`aegis-ml`, `aegis-research`, FastAPI 0.135–0.143, Starlette, uvicorn, Pydantic 2, pydantic-settings, SQLAlchemy 2.0, Alembic, psycopg 3, redis, Celery 5, argon2-cffi, PyJWT, pyotp, python-multipart, ReportLab, boto3, email-validator, cryptography

The development dependency group adds pytest, pytest-cov, Ruff, mypy, moto (for S3 tests), pip-audit and httpx. All versions are pinned in `uv.lock`, and every Makefile target runs through `uv run --frozen`, so no command silently upgrades a dependency.

Lint and format configuration

Ruff is configured at the root with a 120-character line length, a Python 3.11 target and a rule set that includes pyflakes, pycodestyle, import sorting, bugbear, pyupgrade, **bandit security rules (`S`)** and **blind-except detection (`BLE`)**. Blind `except Exception` clauses are therefore visible in review: in the code base each one carries a `# noqa: BLE001` comment explaining why catching everything is correct at that point (for example, “failures become explicit FAILED results”).

Test configuration

```
[tool.pytest.ini_options]
testpaths = ["research/tests", "ml/tests", "apps/api/tests"]
addopts = "-q"
```

A single `pytest` invocation runs all three suites. The API suite needs PostgreSQL; the research and ML suites need nothing beyond the Python dependencies (and ffmpeg for media tests).

4.3 The TypeScript workspace

The front end uses pnpm 10 workspaces. `pnpm-workspace.yaml` includes `apps/web` and `packages/*`. The root `package.json` pins `pnpm@10.28.0`, restricts post-install build scripts to `esbuild` only, and overrides `postcss` to a patched version.

Package	Purpose	Notable dependencies
`@aegis/web`	The workstation	Next 15.5, React 18.3, TanStack Query 5 and Table 8, react-hook-form with zod, Recharts 2.15, lucide-react icons, Inter and JetBrains Mono variable fonts; dev: Tailwind 3.4, TypeScript 5.7, Vitest 2, Playwright 1.56, ESLint 8
`@aegis/shared-types`	Typed API contract	TypeScript only; consumed as source (`main: src/index.ts`)

Exact versions (not ranges) are used for every web dependency, which together with `pnpm install --frozen-lockfile` makes front-end builds reproducible.

4.4 The Makefile

All routine tasks go through `make`. Every target that touches the database loads `.env` first.

Target	What it does
`make setup`	Creates `.env` from `.env.example` if missing (rewriting the storage path to an absolute one), then `uv sync --frozen` and `pnpm install --frozen-lockfile`
`make services`	Starts PostgreSQL and Redis in Docker
`make migrate`	`alembic upgrade head`, then `python -m app.cli sync` to seed roles, permissions, providers and the model registry
`make seed-demo`	Creates the demo organisation, one user per role and three DEMO FIXTURE cases (refused in production)
`make dev`	Runs `scripts/dev.sh`: API on port 8000 with reload, a Celery worker, and the web app on port 3000; Ctrl-C stops all three
`make api`, `make worker`, `make web`	Run one component alone
`make test`	`test-py` (all Python suites) plus `test-web` (type check and Vitest)
`make test-e2e`	Playwright browser tests against a running, seeded stack
`make lint` / `make format`	Ruff check and format check, ESLint / auto-format
`make benchmark`	The built-in simulated benchmark plus the nine experiments, written to `benchmarks/runs/`
`make experiments`	Only the nine experiments, as JSON and PNG, into `research/results/`
`make loadtest`	Repeatable API load test (`scripts/loadtest.py`)
`make verify`	Recompute every stored evidence hash and verify the audit chain

Target	What it does
`make build`, `make up`, `make down`	Build images and start or stop the Compose stack
`make security-check`	Export locked requirements, run pip-audit (with a CycloneDX SBOM), `pnpm audit --prod`, and list Node licences
`make clean`	Remove build output and caches; evidence and databases are kept

The development runner

`scripts/dev.sh` starts the three development processes and cleans them up together:

```
(cd apps/api && uv run --frozen uvicorn app.main:app --reload --host 127.0.0.1 --port 8000) & pids+=($!)
(cd apps/api && uv run --frozen celery -A app.workers.celery_app worker -Q aegis,aegis-research -c 2 --loglevel=INFO) & pids+=($!)
pnpm --filter @aegis/web dev & pids+=($!)
echo "AEGIS: web http://localhost:3000  ·  API docs http://localhost:8000/api/docs"
wait -n
```

`wait -n` returns as soon as any one process exits, and the `trap` then kills the others, so a crash in one component is never hidden behind the other two.

4.5 Getting started: three ways to run AEGIS

Option	Best for	Requirements
A. Render	A free public `https://….onrender.com` link for demonstrations	GitHub and Render accounts
B. Docker Compose	The full stack on one machine or a server	Docker 24+ with Compose v2
C. Local development	Changing the code	Python 3.11+, uv, Node 22, pnpm 10, PostgreSQL 16, Redis 7, ffmpeg

Option B in four commands:

```
cp .env.example .env                                  # set AEGIS_SECRET_KEY to a long random value
docker compose up --build -d                          # postgres, redis, migrate, api, worker, beat, web
docker compose exec api python -m app.cli seed-demo   # demo users and DEMO FIXTURE cases (not in production)
# open http://localhost:3000
```

Option C in five:

```
make setup       # locked Python and Node dependencies, creates .env
make services    # PostgreSQL and Redis in Docker (or use local ones)
make migrate     # Alembic migrations + roles and model registry
```

```
make seed-demo   # demo organisation, one user per role, three DEMO FIXTURE cases
make dev         # API on :8000 (OpenAPI at /api/docs), Celery worker, web on :3000
```

Demo accounts

The demo seeder creates one account per role, all sharing the password in `AEGIS_DEMO_PASSWORD` (default `Aegis-Demo-Only-2026` for local use; refused on Render, where the operator must choose one of at least 12 characters):

Account	Role
`investigator@aegis.demo`	Investigator — the recommended starting point
`analyst@aegis.demo`	Forensic analyst
`reviewer@aegis.demo`	Independent reviewer
`auditor@aegis.demo`	Auditor
`researcher@aegis.demo`	Read-only researcher
`orgadmin@aegis.demo`	Organisation administrator
`admin@aegis.demo`	System administrator

After ten consecutive failed sign-in attempts an account is locked for fifteen minutes. Demo seeding is refused when `AEGIS_ENVIRONMENT=production`; real administrators are created with `python -m app.cli create-admin`.

4.6 Continuous integration

`.github/workflows/ci.yml` runs on pushes to `main` and `educational` and on every pull request, with read-only repository permissions. It has three jobs:

Job	Steps
python	Ubuntu 24.04 with a PostgreSQL 16 service container; install uv 0.5.11 and ffmpeg; `uv sync --frozen`; `ruff check`; `ruff format --check`; `pytest` (the harness builds the schema from empty with Alembic); export requirements and run `pip-audit`
web	pnpm 10.28 and Node 22 with caching; frozen install; ESLint (zero warnings allowed); `tsc --noEmit`; Vitest; production `next build`; `pnpm audit --prod`
images	Build the API and web Docker images to prove the Dockerfiles work

A dependency advisory therefore fails the build in the same way as a failing test.

4.7 What is not committed

`.gitignore` and `.dockerignore` make sure that runtime data never enters the repository or an image: `var/`, `storage/`, `evidence/`, SQLite files, `research/results/`, `benchmarks/runs/`, every `.env` file except `.env.example`, private keys (`*.pem`, `*.key`), Python and Node caches, and Playwright output. The ignore

rules for `var/`, `storage/` and `evidence/` are anchored to the repository root with a leading slash, because those words are also the names of source folders (for example `apps/api/app/storage/`), and an earlier unanchored rule had accidentally excluded source code.

PART II

The Application Platform

The FastAPI service: configuration, data model, identity, cases, evidence, audit, analysis pipeline, reports and administration.

CHAPTER 5

The API Service: Foundations

The AEGIS API is a FastAPI application in `apps/api/app`. This chapter covers the plumbing that every request passes through: configuration, application start-up, middleware, logging, error handling, rate limiting and database access. Later chapters build on it.

5.1 Package structure

```
apps/api/app/
├── main.py                 application factory, router registration, lifespan
├── cli.py                  management commands (python -m app.cli ...)
├── db.py                   engine, sessions, naming convention, utcnow()
├── core/
│   ├── config.py           Settings from AEGIS_* environment variables
│   ├── middleware.py       correlation ids, security headers, request log
│   ├── logging.py          JSON log formatter with redaction
│   ├── errors.py           AppError and uniform error responses
│   └── ratelimit.py        fixed-window limiter (Redis or in-process)
├── security/
│   ├── passwords.py        Argon2id hashing and password policy
│   ├── tokens.py           JWT access tokens, refresh tokens, field encryption
│   ├── permissions.py      the 32 permissions and 7 roles
│   └── deps.py             get_principal(), require(), require_system_admin()
├── models/__init__.py      every SQLAlchemy model (39 tables)
├── schemas/common.py       shared Pydantic schemas (Page, UserRef, ...)
├── api/v1/                 routers: auth, investigations, evidence, analysis, reports, research,
platform
├── services/               business logic: investigations, evidence, analysis, reports, benchmarks,
...
├── storage/                Storage interface and local, S3 and database backends
├── audit/service.py        hash-chained audit trail
└── workers/                Celery application and tasks
```

Routers are thin: they validate input with Pydantic, check permissions through dependencies, and delegate to a service module. Services contain the business rules and write the audit trail. This keeps rules such as “the case creator cannot review” in one place regardless of which endpoint triggers them.

5.2 Configuration

All configuration comes from environment variables with the `AEGIS_` prefix, read by a Pydantic `BaseSettings` class in `app/core/config.py`. A `.env` file is also read if present. Unknown variables are ignored.

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="AEGIS_", env_file=".env", extra="ignore")

    mode: Mode = Mode.DEMO
    environment: str = "development"  # development | test | hosted-demo | production
    secret_key: str = Field(default="dev-only-insecure-secret-change-me-0123456789", min_length=32)
    database_url: str = "postgresql+psycopg://aegis:aegis-dev-only@localhost:5432/aegis"
    redis_url: str = "redis://localhost:6379/0"
    task_mode: str = "celery"  # celery | inline
    ...
```

Two settings describe *where* the system runs and are easy to confuse:

- `mode` (`demo`, `research`, `connected-production`) is a label shown in the user interface so that users always know what kind of deployment they are looking at.
- `environment` (`development`, `test`, `hosted-demo`, `production`) changes behaviour: production enables the safety checks below, `test` disables Redis for rate limiting, and `hosted-demo` is the Render configuration.

The full list of variables, with defaults, appears in Appendix A. The most important groups are:

Group	Variables
Core	`MODE`, `ENVIRONMENT`, `SECRET_KEY`, `DATABASE_URL`, `REDIS_URL`, `TASK_MODE`, `RESUME_INLINE_JOBS_ON_STARTUP`
Storage	`STORAGE_BACKEND` (`local`, `s3`, `database`), `STORAGE_ROOT`, `S3_BUCKET`, `S3_ENDPOINT_URL`, `S3_REGION`, `S3_OBJECT_LOCK`
Sessions and CORS	`CORS_ORIGINS`, `ACCESS_TOKEN_MINUTES` (15), `REFRESH_TOKEN_DAYS` (7), `COOKIE_SECURE`
Uploads and scanning	`MAX_UPLOAD_MB` (512), `MAX_TEXT_UPLOAD_MB` (25), `UPLOAD_CHUNK_MB` (8), `REQUIRE_MALWARE_SCAN`, `CLAMAV_HOST`, `CLAMAV_PORT`
Rate limits	`LOGIN_RATE_LIMIT_PER_MINUTE` (10), `UPLOAD_RATE_LIMIT_PER_HOUR` (120), `API_RATE_LIMIT_PER_MINUTE` (600)
Policy	`ALLOW_EXTERNAL_PROCESSING` (false) — the deployment-wide kill switch for external providers

Method-specific variables (model paths, the watermark key, C2PA trust anchors, provider URLs and tokens) are read directly by the methods in `aegis_ml`, so that the analysis library has no dependency on the API's settings class.

Accepting hosted database URLs

Managed PostgreSQL services (Render, Neon, Heroku) hand out URLs that start with `postgres://` or `postgresql://`. SQLAlchemy would pick the wrong driver for those, so a validator rewrites them to the psycopg 3 dialect:

```
@field_validator("database_url")
@classmethod
def _psycopg_driver(cls, v: str) -> str:
    for prefix in ("postgres://", "postgresql://"):
        if v.startswith(prefix):
            return "postgresql+psycopg://" + v[len(prefix) :]
    return v
```

Production safety checks

`Settings.check_production_safety()` returns a list of problems when `environment == "production"`. The application factory refuses to start if the list is not empty:

Check	Why
`SECRET_KEY` must not start with `dev-only`	The development key is public; it signs every access token and derives the MFA encryption key
`TASK_MODE` must not be `inline`	Inline jobs run in the API process, which is not durable or scalable enough for production
`COOKIE_SECURE` must be true	The refresh cookie must never travel over plain HTTP

```
problems = s.check_production_safety()
if problems:
    raise RuntimeError("Unsafe production configuration: " + "; ".join(problems))
```

5.3 The application factory

`app/main.py` builds the FastAPI application in `create_app()`:

1. Configure JSON logging.
2. Run the production safety checks and refuse to start on failure.
3. Create the `FastAPI` instance with the OpenAPI UI at `/api/docs`, ReDoc at `/api/redoc` and the specification at `/api/openapi.json`. The API description itself restates the evidence-class principle and the error format.
4. Add `RequestContextMiddleware` and `CORSMiddleware`. CORS allows only the origins in `AEGIS_CORS_ORIGINS`, allows credentials, and lists exactly the headers AEGIS uses (`Authorization`, `Content-Type`, `X-Requested-With`, `X-AEGIS-Organization`, `Idempotency-Key`, `X-Request-ID`). It exposes `X-Request-ID` and `X-AEGIS-SHA256` to the browser.
5. Install the error handlers.
6. Mount the seven routers under `/api/v1`: `auth`, `investigations`, `evidence`, `analysis`, `reports`, `research` and `platform`.

A *lifespan* handler runs at start-up. In inline task mode with `RESUME_INLINE_JOBS_ON_STARTUP=true`, it calls `resume_inline_jobs()`, which picks up any analysis job that was interrupted by the previous process stopping. This is what allows the free Render deployment to survive being put to sleep in the middle of a video analysis.

5.4 Request middleware

`RequestContextMiddleware` (in `app/core/middleware.py`) wraps every request:

- **Correlation id.** If the client sends an `X-Request-ID` that matches `^[A-Za-z0-9-]{8,64}$`, it is reused; otherwise a new UUID is generated. The id is stored in a context variable so that log lines and audit events written during the request carry it, and it is returned in the `X-Request-ID` response header. An analyst who reports a problem can quote the id, and an operator can find every log line and audit event for that request.
- **Security headers** on every response:

Header	Value	Purpose
`X-Content-Type-Options`	`nosniff`	Stops browsers guessing content types
`X-Frame-Options`	`DENY`	Prevents click-jacking
`Referrer-Policy`	`no-referrer`	Case URLs never leak in referrers
`Permissions-Policy`	`camera=(), microphone=(), geolocation=()`	No device access
`Cache-Control`	`no-store` (unless set)	Evidence and case data are not cached
`Content-Security-Policy`	`default-src 'none'; frame-ancestors 'none'`	The API serves data, never active content
`Strict-Transport-Security`	`max-age=31536000; includeSubDomains`	Only when the request arrived over HTTPS

The documentation pages are the only exception to the strict CSP: `/api/docs` and `/api/redoc` load the Swagger UI from `cdn.jsdelivr.net`, so they get a policy that permits exactly that.

- **Request log.** One structured line per request with method, path, status and duration in milliseconds.

5.5 Structured logging

`app/core/logging.py` replaces the root handler with a JSON formatter. Each line contains a timestamp, level, logger name, message and correlation id, plus any extra fields whose names start with `ctx_`. Fields whose names contain `password`, `token`, `secret`, `authorization`, `api_key` or `cookie` are replaced by `[redacted]`. Uvicorn's own access log is reduced to warnings because the middleware already logs every request in the structured format.

```
{"ts": "2026-10-18T09:12:44.102+00:00", "level": "INFO", "logger": "aegis.http",
 "msg": "request", "correlation_id": "3f0c…", "method": "POST",
 "path": "/api/v1/investigations", "status": 201, "ms": 38}
```

5.6 Uniform error responses

Every error, whatever its source, is returned in the same envelope:

```
{"error": {"code": "invalid_transition",
           "message": "Cannot move from draft to human_review.",
```

```
            "correlation_id": "3f0c…",
            "details": {"allowed": ["evidence_acquired"]}}}
```

`app/core/errors.py` defines an `AppError(status, code, message, details)` exception and helpers: `not_found()`, `forbidden()`, `conflict()` and `bad_request()`. Four exception handlers are installed:

Handler	Produces
`AppError`	The given status, code, message and details
Starlette `HTTPException`	Maps 401 → `unauthenticated`, 403 → `forbidden`, 404 → `not_found`, 405 → `method_not_allowed`, 413 → `payload_too_large`, 429 → `rate_limited`
`RequestValidationError`	422 `validation_error` with a list of `{loc, msg, type}`
Any other exception	500 `internal_error` with the generic message "An internal error occurred."; the stack trace is logged with the correlation id but never returned

The helper `not_found()` carries a comment that explains an important security property: *"Same response for 'missing' and 'not yours' so identifiers cannot be probed across tenants."*

Error codes used across the API include `unauthenticated`, `forbidden`, `not_found`, `validation_error`, `conflict`, `invalid_transition`, `reason_required`, `unsupported_media`, `malformed_media`, `media_too_long`, `payload_too_large`, `rate_limited`, `malware_detected`, `scanner_required`, `account_locked`, `mfa_required`, `invalid_credentials`, `weak_password`, `csrf`, `purged` and `extraction_failed`.

5.7 Rate limiting

`app/core/ratelimit.py` implements a fixed-window counter. The key combines the limit name, the subject and the window number:

```
key = f"aegis:rl:{name}:{subject}:{int(time.time() // window_seconds)}"
```

When Redis is configured and reachable, the counter is an atomic `INCR` with an `EXPIRE` slightly longer than the window, executed in a pipeline. When Redis is not configured, is unreachable, or the environment is `test`, an in-process dictionary protected by a lock is used instead. Exceeding the limit raises `AppError(429, "rate_limited", ...)` with the limit and window in `details`.

Limit	Subject	Default
`login`	client IP + email	10 per minute
`upload`	user id	120 per hour
`api`	user id (checked on every authenticated request)	600 per minute

The in-process fallback is per process, so with several API workers the effective limit is multiplied by the number of workers. That is acceptable for the single-container demo it was designed for; production deployments use Redis.

5.8 Database access

`app/db.py` creates one SQLAlchemy engine per database URL (cached with `lru_cache`) with `pool_pre_ping=True`, a pool of 10 connections and up to 20 overflow connections. Sessions are created with `expire_on_commit=False` so that objects remain usable after a commit when they are serialised in the response.

- `get_db()` is the FastAPI dependency that yields a session per request and closes it afterwards. Routes commit explicitly.
- `session_scope()` is a context manager for workers and CLI commands that commits on success and rolls back on any exception.
- `utcnow()` returns a timezone-aware UTC timestamp, and the declarative base maps every `datetime` column to `DateTime(timezone=True)`, so no naive timestamps are ever stored.

A naming convention is set on the metadata so that every index, unique constraint, check constraint, foreign key and primary key has a deterministic name (`ix_…`, `uq_…`, `ck_…`, `fk_…`, `pk_…`). This keeps Alembic autogeneration stable and makes migration diffs readable.

5.9 Migrations

The schema is created only by Alembic (`apps/api/migrations`). There are two revisions:

Revision	Date	Content
`0001` initial schema	2026-10-17	All 37 application tables, indexes and constraints
`0002` database object store	2026-10-18	`stored_objects` and `stored_object_chunks` for the PostgreSQL storage backend

`migrations/env.py` imports `app.models` so that every table is registered, takes the URL from the application settings (escaping `%` for the configuration parser), and enables `compare_type=True` so that column type changes are detected. The commands are:

```
cd apps/api
alembic upgrade head        # apply
alembic downgrade base      # roll back everything (development only)
alembic check               # fails if models and migrations diverge
alembic revision --autogenerate -m "describe change"
```

The API test harness drops the test schema and runs `alembic upgrade head` from empty at the start of every test session, so the migrations are exercised from a clean database on every run. In production, migrations are treated as forward-only; `downgrade` is used only on restored copies for testing.

5.10 Bootstrapping roles and the model registry

After migrating, `python -m app.cli sync` runs two idempotent functions from `app/services/bootstrap.py`:

- **`sync_roles()`** inserts any missing permission, creates or updates each role's display name and description, and then reconciles the `role_permissions` table with the code: missing grants are added

and grants that the code no longer contains are deleted. The code in `security/permissions.py` is therefore the single source of truth.

- **`sync_registry()`** creates three model providers (`aegis-local`, `operator-local-model` and `external-http`) and one `detector_versions` row for every method in the `aegis_ml` registry. It refreshes descriptive fields from the method's `MethodInfo` (name, provider, supported media, limitations, evidence class, implementation status, references) but never overwrites the operator's validation decisions. Statistical text methods get a default minimum input of 50 tokens.

The migrate container in Docker Compose runs both commands before the API, worker and beat containers start.

CHAPTER 6

The Data Model

The relational schema is the backbone of AEGIS. Everything that matters for accountability — who did what, to which evidence, with which method version, and what the result was — is a row in PostgreSQL. This chapter explains the schema by area. Appendix C lists every table and column generated directly from the models.

The source of truth is `apps/api/app/models/__init__.py`. It defines **39 tables**. All primary keys are UUIDs, with three exceptions: reference tables keyed by a readable identifier (`roles`, `permissions`, `model_providers`, `system_settings`), the database object store keyed by storage key, and `audit_events`, which uses a monotonically increasing `seq` so that the hash chain has a total order.

Two helpers keep columns consistent:

```
JSONType = JSON().with_variant(JSONB(), "postgresql")   # JSONB on PostgreSQL, JSON elsewhere

def _uuid() -> Mapped[uuid.UUID]:
    return mapped_column(Uuid, primary_key=True, default=uuid.uuid4)

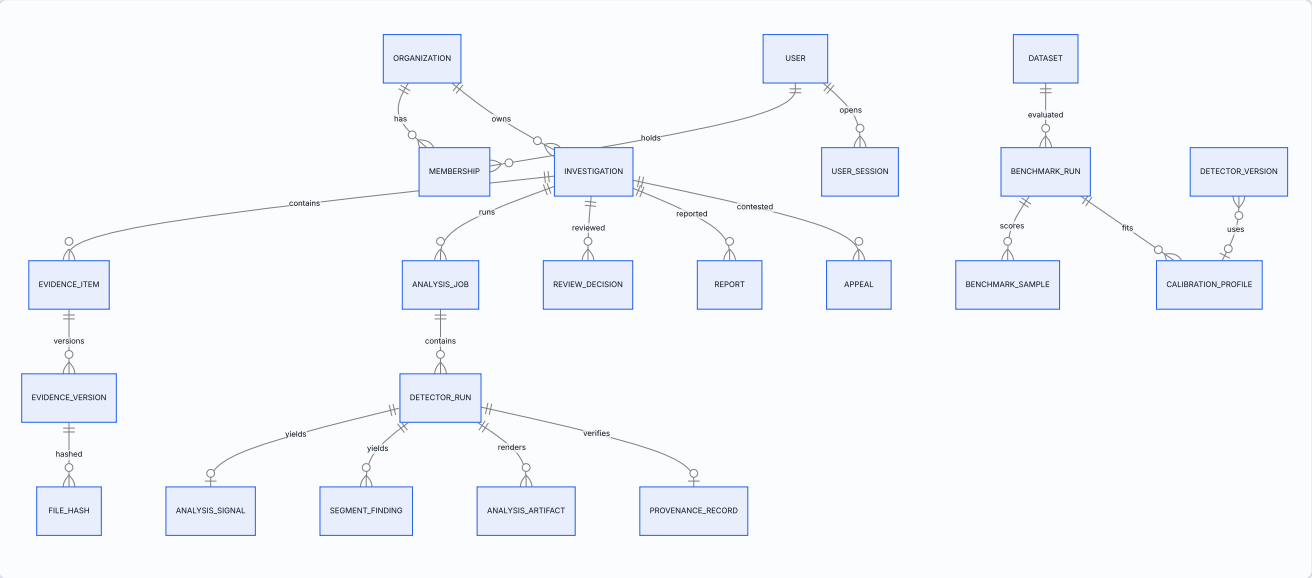
def _created() -> Mapped[datetime]:
    return mapped_column(default=utcnow, nullable=False)
```

6.1 Overview by area

Area	Tables	Notes
Identity	`organizations`, `workspaces`, `users`, `roles`, `permissions`, `role_permissions`, `memberships`, `user_sessions`	Roles and permissions are seeded from code. Sessions store only a SHA-256 of the refresh token. MFA secrets are encrypted.
Cases	`investigations`, `annotations`, `review_decisions`, `appeals`, `deletion_requests`	`(organization_id, case_reference)` is unique. Review decisions carry a content hash and the hash of their audit event.
Evidence	`evidence_items`, `evidence_versions`, `file_hashes`, `upload_sessions`	`(evidence_item_id, version_no)` is unique. `file_hashes` records the acquisition hash and every later integrity re-check.
Analysis	`analysis_jobs`, `detector_runs`, `analysis_signals`, `segment_findings`, `analysis_artifacts`, `provenance_records`	`(job_id, method_id)` is unique, which makes job execution idempotent.
Registry	`model_providers`, `detector_versions`, `calibration_profiles`	`(method_id, version)` is unique. Validation requires a benchmark run (enforced in the API).

Area	Tables	Notes
Research	`datasets`, `benchmark_suites`, `benchmark_runs`, `benchmark_samples`, `experiment_runs`	`benchmark_runs.result_kind` is restricted by a CHECK constraint.
Governance	`reports`, `audit_events`, `retention_policies`, `notifications`, `system_health_events`, `system_settings`	`(investigation_id, version)` is unique for reports; `audit_events.event_hash` is unique.
Object store	`stored_objects`, `stored_object_chunks`	Used only with `AEGIS_STORAGE_BACKEND=database`.

6.2 Core relationships



6.3 Identity tables

`organizations` are tenants. Each has a unique `slug` and an `allow_external_processing` flag, the organisation-level switch for sending evidence to external providers. **`workspaces`** subdivide an organisation (unique per organisation by name) and can be attached to investigations.

`users` hold the email address (unique), display name, Argon2id `password_hash`, `is_active`, `is_system_admin`, the encrypted TOTP secret (`mfa_secret_encrypted`) and `mfa_enabled`, the lockout counters `failed_login_count` and `locked_until`, `last_login_at` and an `is_demo_account` flag that the interface uses to show a demo banner.

`memberships` link a user to an organisation with exactly one role (unique per user and organisation). A user can belong to several organisations with different roles; the `X-AEGIS-Organization` header selects which membership applies to a request.

`user_sessions` are the server-side half of authentication. Each stores `refresh_token_hash` (a SHA-256 hex digest, unique), `expires_at`, `revoked_at`, and the user agent and IP address for the user's own session list. Access tokens carry the session id, so revoking a session invalidates its access tokens immediately.

`roles`, `permissions` and `role_permissions` mirror `security/permissions.py` and are reconciled by `app.cli sync`.

6.4 Case tables

`investigations` are cases. Important columns:

Column	Meaning
`case_reference`	Human reference, unique per organisation, restricted to `[A-Za-z0-9._/-]`
`title`, `purpose`, `description`	The purpose is mandatory (at least 10 characters): a case cannot be opened without saying why
`media_category`	`text`, `document`, `image`, `audio`, `video` or `mixed`
`priority`	`low`, `normal`, `high`, `urgent`
`status`	One of the eight workflow states
`legal_basis`	Optional legal or organisational basis
`retention_class`	`standard`, `extended`, `legal_minimum` or `research`
`access_classification`	`standard` or `restricted`
`legal_hold`	Blocks deletion while true
`allow_external_processing`	Case-level switch for external providers (settable only by administrators)
`disposition`	The latest review decision
`pending_questions`	JSON list of open questions
`assigned_reviewer_id`, `created_by_id`	Used for separation-of-duties checks
`is_demo_fixture`	Marks synthetic demo cases so they are labelled everywhere

A composite index `ix_investigations_org_status` on `(organization_id, status)` supports the case list's most common filter.

`annotations` are notes attached to a case and optionally to an evidence item or a detector run. Their `kind` is `note`, `finding`, `question` or `contradiction`.

`review_decisions` record the human assessment: the reviewer, the decision (one of the outcome categories), a rationale, whether the reviewer agrees with the automated outcome, the automated outcome at that moment, an optional `supersedes_id`, a `content_sha256` of the canonical decision content and the `audit_event_hash` of the audit event that recorded it. That last column links the decision cryptographically into the audit chain.

`appeals` record contests and correction requests: `grounds`, an optional report, an optional free-text `on_behalf_of` (deliberately minimal, to avoid storing personal data), a status (`open`, `under_review`, `upheld`, `corrected`, `rejected`), the resolution and who resolved it.

`deletion_requests` implement the two-person rule: requester, reason, status (`pending`, `approved`, `rejected`, `executed`, `blocked`), decider and timestamps.

6.5 Evidence tables

The evidence model separates the *item* (the logical piece of evidence) from its *versions* (each concrete set of bytes).

`evidence_items` carry the display name, the role (`primary` or `process_evidence`), the detected modality and MIME type, a status (`active`, `integrity_alert` or `purged`), the uploader, the acquisition time and, for text, the JSON extraction report (method, warnings, language, statistics).

`evidence_versions` are immutable. Each has a `version_no` (unique per item), a `kind` (`original`, `normalized_text`, `thumbnail`, `redacted` or `correction`), a `parent_version_id`, the `sha256`, size, MIME type, the opaque `storage_key`, the original filename, a list of `transformations` applied to produce it, a `metadata_status` summary (EXIF, ICC, XMP presence, duration, streams and so on), the malware `scan_status`, an optional reason, the creator and `purged_at`. When a version is purged under the retention policy, its bytes are removed but the row — including the hash — remains as a tombstone.

`file_hashes` keep a history of hashes per version. The first row has `purpose = "acquisition"`. Every integrity check adds a row with `purpose = "integrity_check"`, the recomputed value (or `missing`) and `matches_recorded`. The table is therefore a complete record of every time the evidence was checked and by whom.

`upload_sessions` track resumable chunked uploads: total size, bytes received and status.

6.6 Analysis tables

`analysis_jobs` represent one request to analyse one evidence version with one profile. They store the profile id, status, `methods_total` and `methods_done` (the source of the progress bar), options, whether external processing was permitted at creation time, an optional unique `idempotency_key`, the Celery task id, the number of attempts, a `cancel_requested` flag, the fusion result (JSON), an error message and timestamps. An index on `status` supports the queue view and stale-job recovery.

`detector_runs` are the heart of the analysis record. One row per method per job, unique on `(job_id, method_id)`. Each row records, for reproducibility:

Column	Recorded value
`method_id`, `method_version`, `method_name`, `evidence_class`	Exactly which code ran
`input_sha256`	Exactly which bytes it ran on
`config`	The configuration it ran with (model identifiers, weight fingerprints, parameters)
`threshold`	The operating threshold, if a validated one applied
`status`, `summary`, `error`	The outcome in words
`output`, `output_schema`	The full structured output, with schema `aegis.method_result/v1`
`score`, `score_semantics`, `calibrated`, `flagged`, `uncertainty`	The numerical result, what it means, and whether it may be interpreted
`warnings`, `limitations`	Shown alongside the result in the interface and the report
`started_at`, `finished_at`, `duration_ms`	Timing

`analysis_signals` are a normalised summary of each completed run for evidence fusion: evidence class, verification status, calibration status (`validated`, `uncalibrated` or `not_applicable`), the applicable population, input characteristics, uncertainty, known dependencies and a supporting reference.

`segment_findings` hold localised results — a span of characters in text, a time range in audio or video, or a frame index — with a label, an optional score and details.

`analysis_artifacts` are generated visualisations. Each has a name, media type, storage key, SHA-256, a description and, importantly, a `cannot_establish` text that states what the visualisation does *not* show. The interface displays this beside every image.

`provenance_records` store the outcome of a C2PA verification per run: standard, state, summary and the trust anchor used.

6.7 Registry tables

`detector_versions` form the model registry: one row per `(method_id, version)` with the provider, model identifier, licence, supported media, languages evaluated, minimum input, known limitations, data documentation, evidence class, implementation status, **validation status**, the benchmark run that validated it, a note, the attached calibration profile, the operating threshold and an `enabled` switch. Disabling a method here makes later runs report `unavailable`.

`calibration_profiles` hold a fitted calibration: method and version, the calibration method (`platt` or `threshold_only`), parameters, threshold, target FPR, the population it applies to, languages, the benchmark run it was fitted on, metrics and a `validated` flag.

6.8 Research tables

`datasets` require `provenance`, `license` and `consent_basis`; they record a SHA-256, storage key, sample count, split sizes, fields and a duplicate report. `benchmark_suites` group runs by task. `benchmark_runs` store the method and version, seed, configuration, a full manifest, the computed metrics, an optional source citation and the `result_kind`, constrained by:

```
CheckConstraint(
    "result_kind in ('published','reproduced','locally_measured','simulated','vendor_reported')",
    name="result_kind",
)
```

`benchmark_samples` store each scored sample (unique on run, sample key and transform) with split, label, group, transform, score and status. `experiment_runs` store each execution of one of the nine research experiments with its configuration, results and manifest.

6.9 Governance tables

`reports` are versioned per investigation (unique on investigation and version), with status `draft`, `final` or `superseded`, an optional `supersedes_id` and `correction_reason`, the schema version `aegis.report/v1`, the canonical JSON `payload` and its `json_sha256`, and the PDF's storage key and `pdf_sha256`.

`audit_events` are the hash chain (see the Audit Trail chapter). Indexes support queries by organisation and time and by entity.

`retention_policies` map an organisation's retention classes to a number of days. `notifications` are per-user messages with a severity and a link; an index on `(user_id, read_at)` serves the unread count. `system_health_events` log component status changes. `system_settings` is a key–value store for platform settings.

6.10 Database object store

When evidence must live in PostgreSQL (hosts without persistent disks), objects are stored in two tables. `stored_objects` holds the key (primary key), content type, size and creation time; `stored_object_chunks` holds the bytes in 1 MiB chunks keyed by `(key, seq)`, with cascade delete. An object and all its chunks are inserted in one transaction and never updated; a second write to the same key fails on the primary key. The Evidence Vault chapter explains how this preserves the write-once guarantee.

CHAPTER 7

Identity, Access and Tenancy

AEGIS handles sensitive evidence and records decisions that may affect people. Its identity and access layer is therefore strict by default. This chapter explains authentication (passwords, tokens, sessions, MFA), authorisation (roles and permissions), tenant isolation and the access classification of cases.

7.1 Passwords

`app/security/passwords.py` uses **Argon2id** through `argon2-cffi`'s `PasswordHasher` with the library defaults, which follow the RFC 9106 low-memory profile.

```
_ph = PasswordHasher()
MIN_PASSWORD_LENGTH = 12
_DUMMY = _ph.hash("timing-equalisation-dummy-password")

def verify_password(password: str, hashed: str | None) → bool:
    try:
        return _ph.verify(hashed or _DUMMY, password) and hashed is not None
    except (VerificationError, InvalidHashError):
        return False
```

The dummy hash deserves attention. When someone tries to sign in with an email address that does not exist, there is no stored hash to verify against. Returning immediately would make the "unknown user" path measurably faster than the "wrong password" path, and an attacker could enumerate valid accounts by timing responses. AEGIS instead verifies the password against a dummy Argon2 hash, so both paths take the same time, and then returns `False` because `hashed is None`.

The password policy (`password_problems`) requires at least 12 characters, a mix of upper- and lower-case letters, and at least one digit. It is enforced when a password is changed or an administrator is created.

7.2 Access and refresh tokens

AEGIS uses two kinds of token with different lifetimes and storage:

	Access token	Refresh token
Format	JWT, HS256, signed with `AEGIS_SECRET_KEY`	48 random bytes, URL-safe base64
Lifetime	15 minutes (`ACCESS_TOKEN_MINUTES`)	7 days (`REFRESH_TOKEN_DAYS`)
Where the browser keeps it	In JavaScript memory only	In an httpOnly, SameSite=Strict cookie scoped to `/api/v1/auth`
Where the server keeps it	Nowhere (stateless), but bound to a session	Only its SHA-256 in `user_sessions.refresh_token_hash`
Sent as	`Authorization: Bearer …`	Cookie `aegis_refresh`, only to the auth endpoints

The access token's claims are:

```
{"sub": str(user_id), "sid": str(session_id), "iat": now, "exp": exp,
 "iss": "aegis", "typ": "access", "jti": secrets.token_hex(8)}
```

Decoding requires `exp`, `sub`, `sid` and `iss`, accepts only the HS256 algorithm (preventing algorithm-confusion attacks) and checks the issuer.

The `sid` claim is what makes stateless tokens revocable. On every request, `get_principal()` looks up the session and rejects the token if the session has been revoked or has expired, even though the JWT signature and expiry are still valid. Logging out, changing a password, an administrator deactivating an account and revoking a session from the session list all take effect immediately.

Why this split

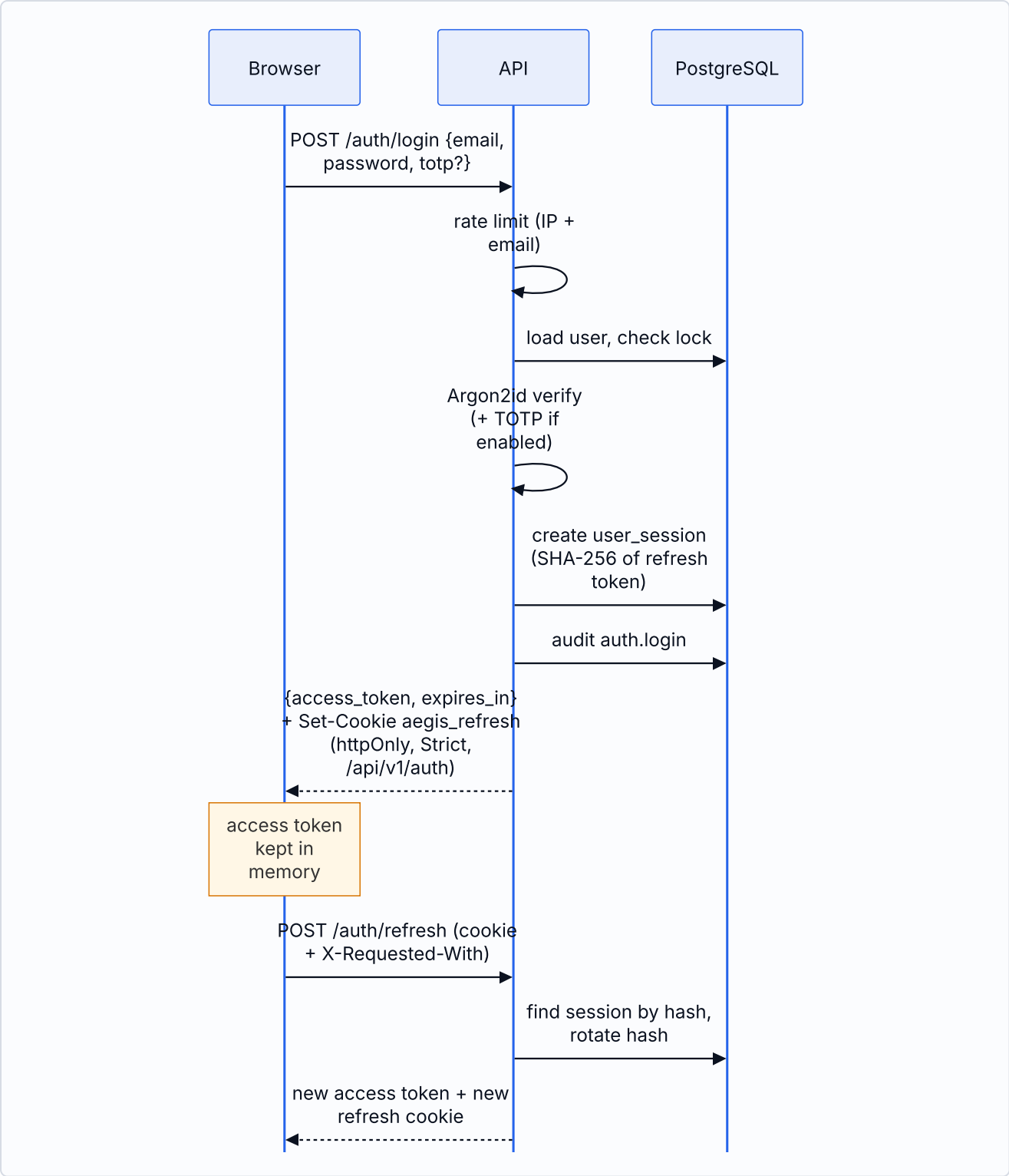
Keeping the access token in memory means it is not readable by other tabs, does not persist after the tab closes and is not sent automatically by the browser, so it is immune to cross-site request forgery. Keeping the refresh token in an httpOnly cookie means scripts — including injected ones — cannot read it. Scoping the cookie path to `/api/v1/auth` means it is sent only to the login, refresh and logout endpoints, not with every API call. SameSite=Strict stops other sites from causing the browser to send it. As a final layer, the refresh endpoint also requires the header `X-Requested-With: aegis-web`, which a cross-site HTML form cannot set:

```
def _require_csrf_header(x_requested_with: str | None) → None:
    # Cookie-authenticated endpoints require a custom header that cross-site forms cannot set.
    if x_requested_with ≠ "aegis-web":
        raise AppError(403, "csrf", "Missing X-Requested-With: aegis-web header.")
```

7.3 The sign-in flow

`POST /api/v1/auth/login` (in `app/api/v1/auth.py`) proceeds as follows:

1. Apply the login rate limit: 10 attempts per minute per client IP and email.
2. Look up the user by lower-cased email.
3. If the account is locked (`locked_until` in the future), return `423 account_locked`.
4. Verify the password (with timing equalisation for unknown users).
5. If the password is correct and MFA is enabled, require a TOTP code: a missing code returns `401 mfa_required`, which tells the interface to show the code field. The code is checked with a tolerance of one 30-second step either side.
6. On any failure, increment `failed_login_count`. At 10 failures the account is locked for 15 minutes and the counter resets. The failure is written to the audit chain with the attempted email and IP. The response is always the same generic `401 invalid_credentials` ("Invalid email, password or code."), so it does not reveal which part was wrong.
7. On success, reset the counters, record `last_login_at`, create a `UserSession` with a new refresh token hash, write `auth.login` to the audit chain, issue an access token and set the refresh cookie.



Refresh rotation

`POST /api/v1/auth/refresh` requires the CSRF header and the cookie, looks up the session by the hash of the presented token, checks that it is neither revoked nor expired and that the user is active, then **rotates** the refresh token: a new random token is generated, its hash replaces the old one, and the new token is set in the cookie. A stolen refresh token therefore works at most once; after the legitimate client refreshes, the stolen copy no longer matches any session.

Other authentication endpoints

Endpoint	Behaviour
`POST /auth/logout`	Revokes the current session, writes `auth.logout`, deletes the cookie
`GET /auth/me`	Returns the user, current organisation, role and role name, sorted permission list, all memberships, and the deployment environment (mode, environment, task mode, whether external processing is enabled). The interface uses this to decide what to show.
`POST /auth/change-password`	Verifies the current password, enforces the policy, re-hashes, **revokes every other session** of the user, audits `auth.password_changed`
`POST /auth/mfa/setup`	Generates a TOTP secret, stores it encrypted with MFA still disabled, returns the secret and an `otpauth://` URI for an authenticator app
`POST /auth/mfa/enable`	Enables MFA only after a valid current code proves the authenticator was set up correctly; audits `auth.mfa_enabled`
`GET /auth/sessions`	Lists the user's 50 most recent sessions with user agent, IP and which one is current
`DELETE /auth/sessions/{id}`	Revokes one of the user's own sessions; another user's session id returns 404

7.4 Encrypting MFA secrets

TOTP secrets must be recoverable (the server needs them to check codes), so they cannot be hashed. They are encrypted with Fernet (AES-128-CBC with HMAC-SHA256). The Fernet key is derived from `AEGIS_SECRET_KEY` with HKDF-SHA256, using a fixed salt and a purpose-specific `info` string so that the derived key cannot be confused with any other use of the secret:

```
def _fernet() → Fernet:
    key = HKDF(algorithm=hashes.SHA256(), length=32, salt=b"aegis-field-encryption",
               info=b"mfa-secrets").derive(get_settings().secret_key.encode())
    return Fernet(base64.urlsafe_b64encode(key))
```

A consequence documented in the incident-response plan is that rotating `AEGIS_SECRET_KEY` invalidates every access token *and* every stored MFA secret, so users must re-enrol MFA after a key rotation.

7.5 Roles and permissions

Authorisation is permission-based. `app/security/permissions.py` defines 32 permissions and 7 roles; the code is the single source of truth and is copied into the database by `app.cli sync`.

Group	Permissions
Cases	`case.create`, `case.read`, `case.update`, `case.assign`, `case.archive`, `case.reopen`, `case.read_restricted`
Evidence	`evidence.upload`, `evidence.read`, `evidence.download`, `evidence.verify`, `evidence.redact`

Group	Permissions
Analysis	`analysis.run`, `analysis.read`
Review	`review.annotate`, `review.decide`
Reports	`report.generate`, `report.finalize`, `report.read`
Appeals	`appeal.submit`, `appeal.resolve`
Retention	`retention.request_deletion`, `retention.approve_deletion`
Audit	`audit.read`
Research	`research.use`, `benchmark.read`, `benchmark.run`, `dataset.register`
Administration	`admin.users`, `admin.models`, `admin.settings`, `system.health`

The roles are defined as sets of these permissions. Two definitions are especially telling:

```
"system_admin": ("System Administrator", "Full access, including system settings.", set(PERMISSIONS)),
"org_admin": ("Organization Administrator", "Administers an organisation's users, settings and
models.",
              set(PERMISSIONS) - {"review.decide", "report.finalize"}),
```

An organisation administrator has every permission *except* recording a review decision and finalising a report. Administrative power over an organisation therefore does not include the power to decide cases, which keeps the human assessment independent of whoever manages the system. Appendix D gives the full role–permission matrix.

Summary of what each role can do:

- **Investigator** — create, read, update and assign cases; upload, read, download, verify and redact evidence; run and read analyses; annotate; generate and read reports; submit appeals; request deletion; use the research bench; read benchmarks; view system health.
- **Forensic analyst** — like an investigator but cannot create or assign cases or request deletion; additionally can run benchmarks and register datasets.
- **Independent reviewer** — read cases including restricted ones; reopen cases; read, download and verify evidence; read analyses; annotate; **decide reviews; finalise reports; resolve appeals; approve deletions**; use the research bench; read benchmarks. Cannot upload evidence or run analyses.
- **Auditor** — read cases (including restricted), evidence, analyses and reports; verify evidence; read the audit trail; read benchmarks; view system health. Nothing that changes state except integrity verification.
- **Read-only researcher** — only `research.use` and `benchmark.read`. A researcher never sees case data.

7.6 Enforcing permissions: `get_principal` and `require`

Every protected endpoint declares its required permissions with a FastAPI dependency:

```
@router.post("", response_model=InvestigationOut, status_code=201)
def create_investigation(body: InvestigationIn,
```

```
                        p: Principal = Depends(require("case.create")),
                        db: Session = Depends(get_db)) → Investigation:
```

`require(*perms)` depends on `get_principal()`, which performs the full authentication on every request:

1. Parse `Authorization: Bearer …`; reject if missing (`401 unauthenticated`).
2. Decode the JWT; distinguish an expired token ("Access token expired.") from an invalid one.
3. Load the session from `sid`; reject if missing, revoked or expired.
4. Load the user from `sub`; reject if inactive or if the session belongs to someone else.
5. Load the user's memberships; reject with 403 if there are none.
6. Choose the organisation: the first membership by default, or the one named in `X-AEGIS-Organization`. A header naming an organisation the user does not belong to returns `403`.
7. Compute the permission set from the membership's role.
8. Apply the per-user API rate limit (600 requests per minute).
9. Return a `Principal` holding the user, session id, organisation, role, permissions and client IP.

`require()` then checks that every listed permission is present and returns `403 forbidden` naming the missing ones. A separate `require_system_admin()` dependency checks the `is_system_admin` flag for platform-wide operations.

Because the permission check is a dependency declared on each route, it is visible in the code next to the route and cannot be forgotten by a downstream service. The test `test_role_permissions_enforced_server_side` exercises it for every role.

7.7 Tenant isolation

An AEGIS deployment can host several organisations in one database. Isolation is enforced in three ways:

1. **Every case query starts from `visible_cases(principal)`**, which filters on `Investigation.organization_id == principal.org_id`. Evidence, jobs, reports and annotations are reached through their case, so they inherit the filter.
2. **Foreign identifiers return 404.** If a user requests a case, evidence item, job or report that belongs to another organisation, the response is exactly the same `404 not_found` as for an identifier that does not exist. An attacker cannot tell whether a UUID exists in another tenant.
3. **The organisation header is validated against memberships**, so a user cannot simply claim another organisation's id.

The test `test_cross_tenant_isolation` creates two organisations and checks that each one's identifiers are invisible to the other.

7.8 Restricted cases

Cases have an `access_classification` of `standard` or `restricted`. A restricted case is visible only to:

- users holding `case.read_restricted` (reviewers, auditors and administrators), and
- the case's creator and its assigned reviewer.

```
def visible_cases(p: Principal) → Select:
    q = select(Investigation).where(Investigation.organization_id == p.org_id)
```

```
    if not p.can("case.read_restricted"):
        q = q.where(or_(
            Investigation.access_classification != "restricted",
            Investigation.created_by_id == p.user.id,
            Investigation.assigned_reviewer_id == p.user.id,
        ))
    return q
```

Because the restriction is applied in the same query builder used for tenant scoping, a restricted case that a user may not see behaves exactly like a case in another tenant: it is absent from lists and returns 404 when addressed directly. This is verified by `test_restricted_classification`.

7.9 Summary of controls

Threat	Control
Password guessing	Argon2id, 10/min rate limit per IP and email, 15-minute lockout after 10 failures, optional TOTP
Account enumeration	Timing-equalised verification, identical error message for every failure
Token theft from storage	Access token in memory only; refresh token httpOnly and path-scoped
CSRF	Bearer tokens for the API; SameSite=Strict cookie plus custom-header check on refresh
Token replay after logout	Access tokens bound to server-side sessions checked on every request
Stolen refresh token	Rotation on every use; hash-only storage
Privilege escalation	Server-side permission dependency on every route; org admins cannot decide reviews
Cross-tenant access	Membership-scoped queries; 404 for foreign ids; validated organisation header

CHAPTER 8

Investigations and the Case Workflow

An *investigation* (also called a *case*) is the unit of work in AEGIS. Evidence, analyses, annotations, review decisions, reports, appeals and deletion requests all belong to a case. This chapter describes the case life-cycle, the separation-of-duties rules, human review, appeals and the retention workflow. The code lives in `apps/api/app/services/investigations.py` (rules) and `apps/api/app/api/v1/investigations.py` (endpoints).

8.1 Opening a case

`POST /api/v1/investigations` requires `case.create`. The request body is validated by the `InvestigationIn` schema:

Field	Rule
`title`	3–300 characters
`case_reference`	2–80 characters, pattern `^[A-Za-z0-9._/-]+$`, unique within the organisation (409 on conflict)
`purpose`	**10–4,000 characters, mandatory**
`media_category`	`text`, `document`, `image`, `audio`, `video` or `mixed`
`description`	Up to 20,000 characters
`priority`	`low`, `normal` (default), `high`, `urgent`
`assigned_reviewer_id`	Optional; must be a member of the organisation whose role has `review.decide`, and must not be the creator
`legal_basis`	Optional, up to 4,000 characters
`retention_class`	`standard` (default), `extended`, `legal_minimum`, `research`
`access_classification`	`standard` (default) or `restricted`
`allow_external_processing`	Default false

Requiring a written purpose is a governance choice: AEGIS will not let anyone start examining evidence without stating why. The creation is audited (`investigation.created`, with reference, classification and the external-processing flag), and if a reviewer is assigned they receive a notification.

8.2 Listing and searching cases

`GET /api/v1/investigations` returns a paginated list (`{items, total, page, page_size}`, at most 200 per page) of the cases the caller may see. Filters: `status` (comma-separated list), `priority`, `q` (case-insensitive search in title and reference), `assigned_to_me`. Sorting: `updated_at`, `created_at`, `priority` or `title`, ascending or descending. The list always starts from `visible_cases()`, so tenancy and restricted classification apply automatically.

8.3 The state machine

The workflow has eight states. The allowed transitions are a plain dictionary, which makes the rules easy to audit:

```
TRANSITIONS: dict[str, set[str]] = {
    "draft": {"evidence_acquired"},
    "evidence_acquired": {"analysis_queued"},
    "analysis_queued": {"analysis_complete"},
    "analysis_complete": {"human_review", "analysis_queued"},
    "human_review": {"inconclusive", "report_finalized", "analysis_queued"},
    "inconclusive": {"report_finalized", "human_review"},
    "report_finalized": {"archived", "human_review"},
    "archived": {"human_review"},
}
REOPEN_FROM = {"report_finalized", "inconclusive", "archived"}
EVIDENCE_MUTABLE_STATES = {"draft", "evidence_acquired", "analysis_queued", "analysis_complete",
"human_review"}
LOCKED_STATES = {"report_finalized", "archived"}
```

State	Meaning	How a case enters it
`draft`	Opened, no evidence yet	Creation
`evidence_acquired`	At least one evidence item	Automatically on first upload
`analysis_queued`	One or more analysis jobs are pending or running	Automatically when a job is created; or by a reviewer requesting further analysis
`analysis_complete`	All jobs have reached a terminal status	Automatically when the last job finishes
`human_review`	Waiting for the independent reviewer	Investigator requests review (a reviewer must be assigned); or reopening
`inconclusive`	Reviewer recorded an inconclusive assessment	Review decision
`report_finalized`	A final report exists	Finalising a report
`archived`	Closed	`case.archive` permission

`transition()` enforces the rules:

- A target that is not in the allowed set returns `409 invalid_transition` with the list of allowed targets in `details`.
- **Reopening** (moving from `report_finalized`, `inconclusive` or `archived` back to `human_review`) requires the `case.reopen` permission and a reason of at least 10 characters (`422 reason_required` otherwise).
- Moving to `archived` sets `archived_at`.
- Every transition writes an `investigation.status_changed` audit event with the previous and new state, the reason, and whether it was automatic. Automatic transitions are attributed to the actor label `system:workflow`.

`advance_if()` performs automatic progression: for example, `ingest()` calls `advance_if(db, inv, {"draft"}, "evidence_acquired")` after storing the first item, so a case that already has evidence is not moved.

The manual transition endpoint `POST /investigations/{id}/transition` accepts only the four targets a person may request directly — `human_review`, `analysis_queued`, `inconclusive`, `archived` — and maps each to a permission:

Target	Required permission
`human_review`	`case.update` (or `case.reopen` when reopening)
`analysis_queued`	`analysis.run`
`inconclusive`	`review.decide`
`archived`	`case.archive`

Requesting human review from `analysis_complete` additionally requires an assigned reviewer (`409` otherwise) and notifies that reviewer.

Locked states

Once a case is `report_finalized` or `archived`, `ensure_unlocked()` refuses edits, reviewer changes and new uploads with `409 conflict` and a message telling the user to reopen it with a recorded reason. Evidence can be added only in `EVIDENCE_MUTABLE_STATES`. This guarantees that a finalised report always describes the evidence and analyses as they were when it was finalised.

Concurrency

Endpoints that modify a case load it with `get_case(..., for_update=True)`, which on PostgreSQL adds `SELECT … FOR UPDATE`. Two people acting on the same case at once are therefore serialised, and a state check followed by a transition cannot be interleaved with another request.

8.4 Editing, assigning and legal hold

- `PATCH /investigations/{id}` (requires `case.update`) changes title, description, priority, legal basis, classification, retention class, pending questions or the external-processing flag. Only users with `admin.settings` may *enable* external processing for a case. The audit event records the values before and after.
- `POST /investigations/{id}/assign` (requires `case.assign`) sets or clears the reviewer. The reviewer must hold `review.decide`, must not be the person assigning, and must not be the case creator. The change is audited and the new reviewer notified.
- `POST /investigations/{id}/legal-hold` (requires `admin.settings`) sets or clears the legal hold with a mandatory reason. While a hold is in place, deletion requests are created in the `blocked` state and approvals are refused.
- `GET /investigations/{id}/reviewers` lists the members eligible to review this case: active users whose role has `review.decide`, excluding the creator.

8.5 Annotations

`POST /investigations/{id}/annotations` (requires `review.annotate`) adds a note of kind `note`, `finding`, `question` or `contradiction`, optionally linked to an evidence item (which must belong to the same case) or to a detector run. Annotations are the place where investigators and reviewers record reasoning that does not fit into a structured field, for example "the EXIF timestamp contradicts the claimed capture date".

8.6 Human review

The review decision is the central act of the workflow. `POST /investigations/{id}/reviews` requires `review.decide` and enforces:

1. The case must be in `human_review` (`409` otherwise).
2. **The case creator cannot record the review** (`403`, "Separation of duties: the case creator cannot record the independent review.").
3. The decision must be one of the outcome categories plus `further_analysis_requested`.
4. The rationale must be at least 20 characters.

The endpoint looks up the latest finished job with a fusion result and records its `primary_outcome` as the automated outcome at the moment of decision. It then builds a canonical content object, hashes it and links the decision into the audit chain:

```
content = {
    "investigation_id": str(inv.id), "reviewer_id": str(p.user.id),
    "decision": body.decision, "rationale": body.rationale,
    "agrees_with_automated": body.agrees_with_automated,
    "automated_outcome": automated, "recorded_at": utcnow().isoformat(),
}
digest = hashlib.sha256(canonical(content).encode()).hexdigest()
...
ev = audit.record(db, "review.decision_recorded", "review_decision", rd.id, ...,
                  details={"decision": body.decision, "content_sha256": digest,
                           "automated_outcome": automated,
                           "disagrees_with_automated": body.agrees_with_automated is False})
rd.audit_event_hash = ev.event_hash
```

The decision therefore carries both its own content hash and the hash of the audit event that recorded it. Altering the decision later would break either the content hash or the chain.

Disagreement with the automated outcome is recorded explicitly (`disagrees_with_automated`). This is useful for oversight: a high rate of reviewer disagreement with a particular method is a signal that the method is not performing as expected on this population.

Some decisions also move the case:

- `further_analysis_requested` → `analysis_queued`
- `inconclusive` → `inconclusive`

The decision is stored as the case's `disposition`, and the case creator is notified.

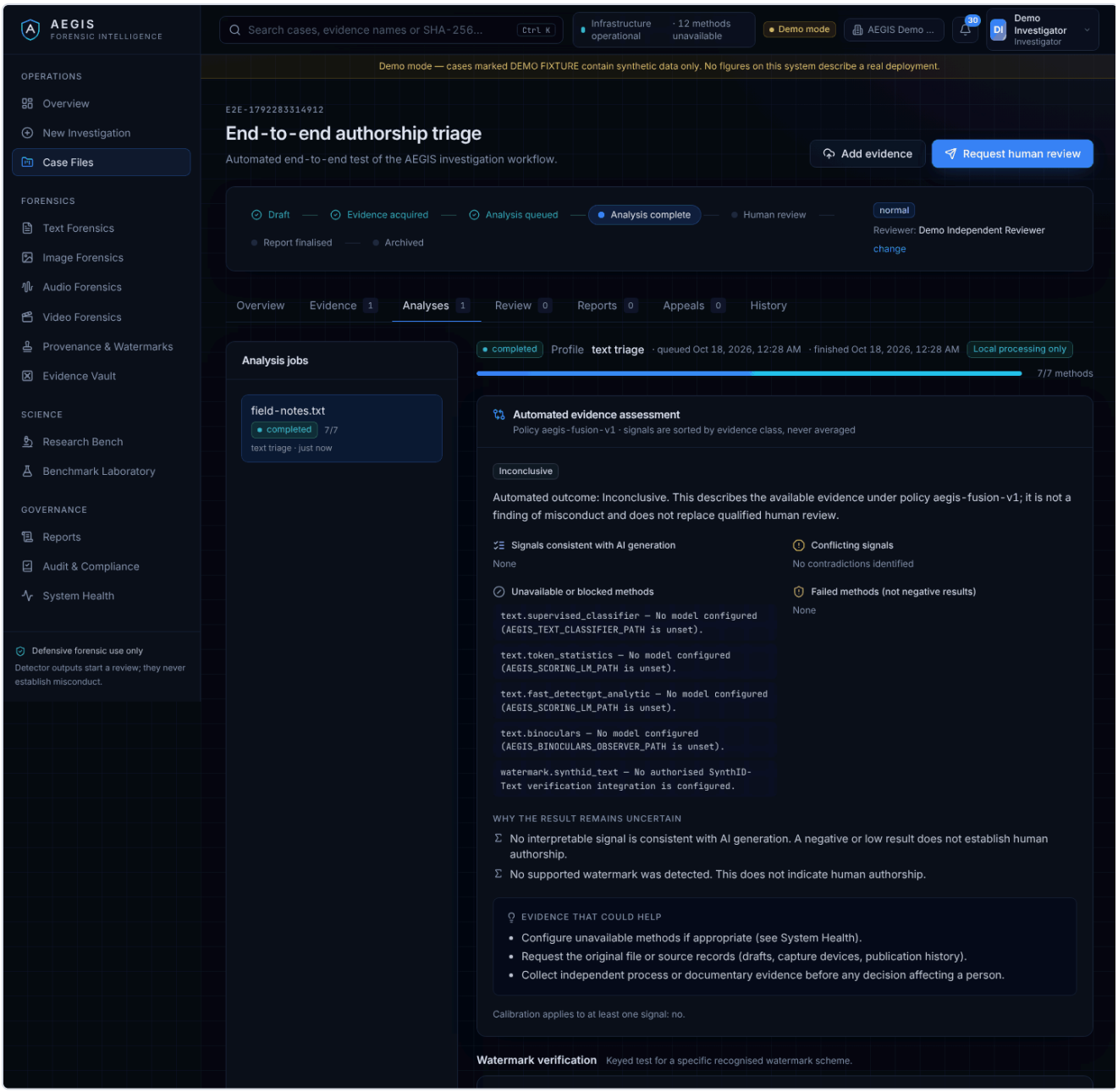


Figure 2. The case workspace shows the automated assessment, the methods that were unavailable and why, and results grouped by evidence class. The reviewer records the human assessment from this screen.

8.7 Appeals and corrections

Anyone with `appeal.submit` can file an appeal against a case, optionally naming a specific report and optionally stating on whose behalf it is filed (`on_behalf_of`, free text up to 200 characters — the procedure recommends a role or reference rather than personal details). Grounds must be at least 20 characters. The assigned reviewer receives a warning-level notification.

Resolution (`POST /investigations/{id}/appeals/{appeal_id}/resolve`, requires `appeal.resolve`) moves the appeal to `under_review`, `upheld`, `corrected` or `rejected` with a resolution text of at least 10 characters. **The person who submitted an appeal cannot resolve it.** An appeal that is already resolved cannot be changed. A correction is then made by issuing a new report version that supersedes the previous one (see the Reports chapter); the original is preserved.

8.8 Deletion and the two-person rule

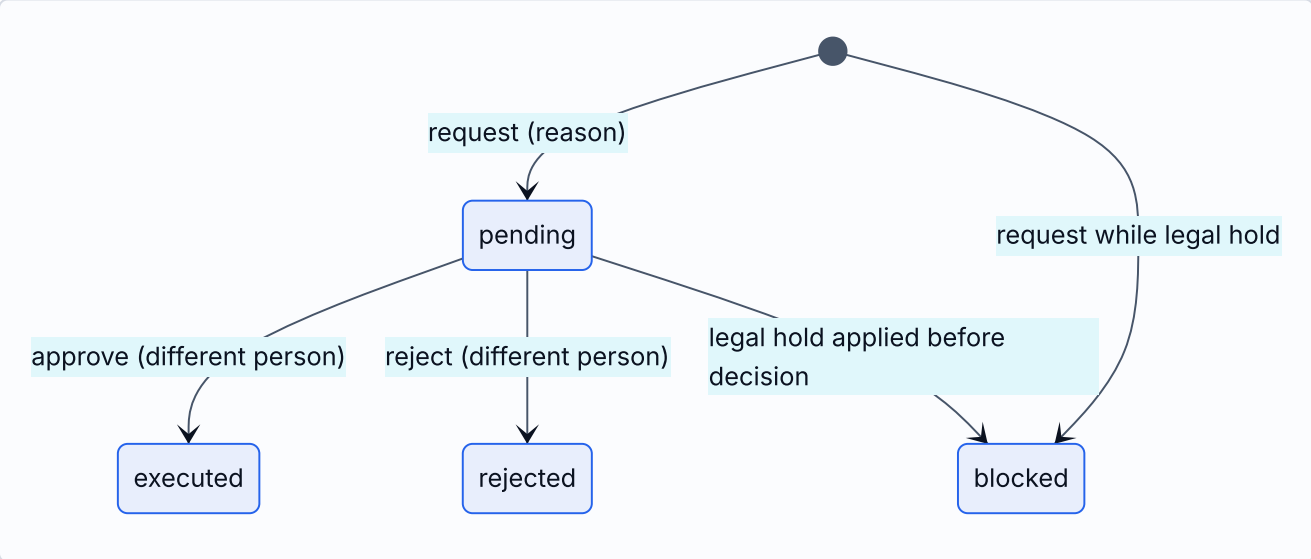
Evidence is never deleted casually. The retention workflow has two steps performed by two different people:

1. **Request.** `POST /investigations/{id}/deletion-requests` (requires `retention.request_deletion`) records a reason of at least 10 characters. If the case is under legal hold, the request is created as `blocked`; otherwise it is `pending` "awaiting approval by a second authorised person".
2. **Decide.** `POST /investigations/{id}/deletion-requests/{request_id}/decide` (requires `retention.approve_deletion`) approves or rejects with a reason.

The decision step enforces:

- The request must be `pending`.
- **The requester cannot approve their own request** ("Two-person rule").
- If the case has come under legal hold in the meantime, the request becomes `blocked` and the decision is refused.

On approval, `purge_binaries()` runs for every evidence item in the case. It deletes the stored bytes of each version — unless another unpurged version elsewhere still references the same content-addressed object — sets `purged_at` on the version, sets the item status to `purged`, and writes an `evidence.purged` audit event. The response states plainly: *"Stored bytes were removed; hash records, audit events and tombstones are retained."* Anyone later examining the case can still see what evidence existed, its hashes, who acquired it, who requested and approved the deletion, and why.



8.9 Case history

`GET /investigations/{id}/history` returns the audit events for the case, newest first (up to 1,000), each with sequence number, time, actor, action, entity, reason, details and the event hash. The case page shows this as a timeline, so anyone with access to the case can see its complete custody record without needing audit-trail permissions for the whole organisation.

8.10 Separation of duties: the complete list

Rule	Where enforced
A user cannot assign themselves as reviewer of their own case	`_validate_reviewer()`
The case creator cannot be its reviewer	`assign_reviewer()` and `eligible_reviewers()`
The case creator cannot record the review decision	`record_review()`
Organisation administrators cannot record review decisions or finalise reports	Role definition (`org_admin` lacks `review.decide` and `report.finalize`)
A report's author cannot finalise it	Reports service
A deletion requester cannot approve their own request	`decide_deletion()`
An appellant cannot resolve their own appeal	`resolve_appeal()`
Only administrators can enable external processing for a case	`update_investigation()`

CHAPTER 9

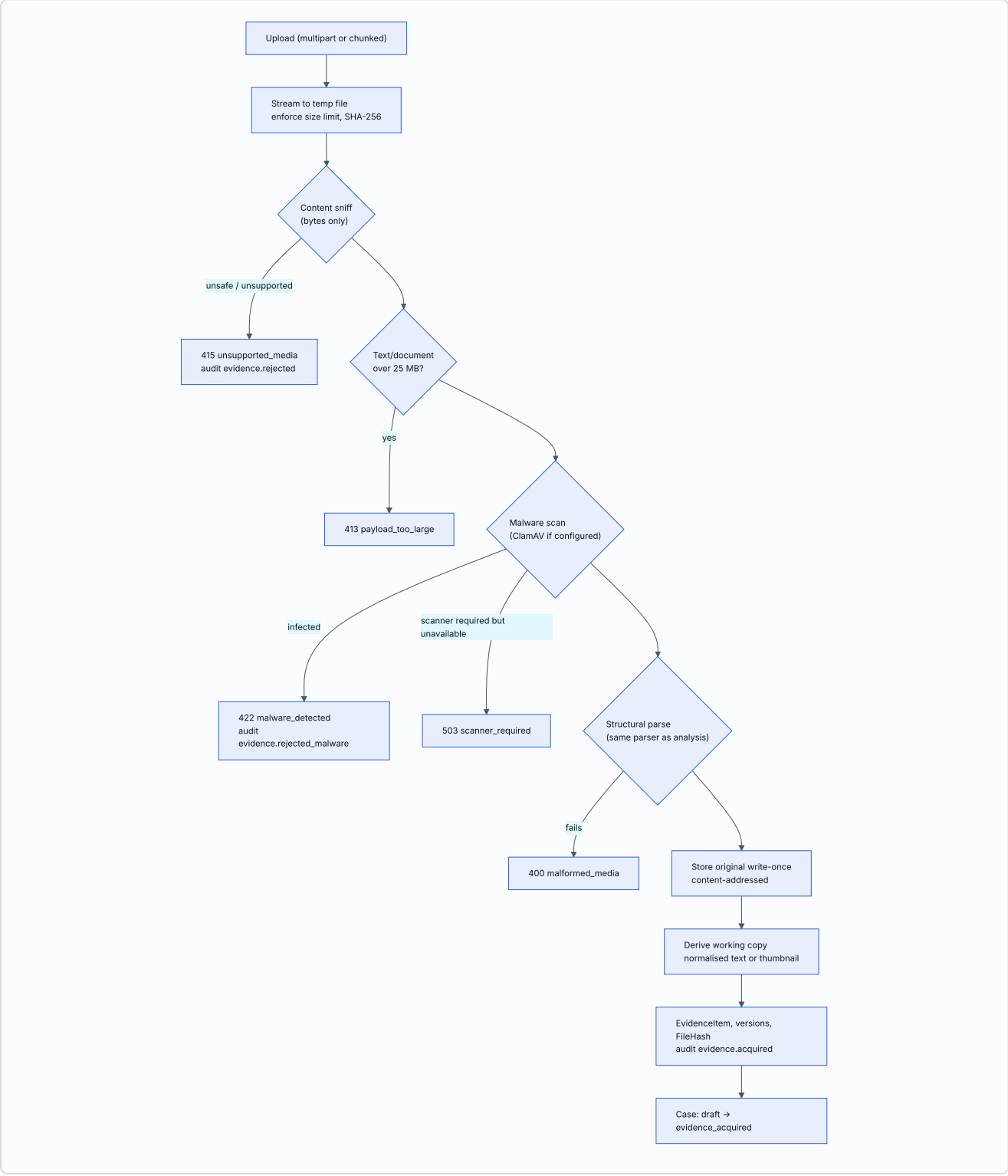
The Evidence Vault

Forensic findings are only as trustworthy as the custody of the evidence they describe. The evidence vault is the part of AEGIS that receives files, decides whether they are safe and supported, seals them with cryptographic hashes, stores them so that they cannot be overwritten, produces working copies, and lets anyone with permission verify later that nothing has changed. The code lives in `apps/api/app/services/evidence.py`, `apps/api/app/api/v1/evidence.py`, `apps/api/app/storage/`, `apps/api/app/services/scanning.py` and `ml/aegis_ml/evidence/sniff.py`.

The module docstring states the scope of what a hash proves:

> *Recording a hash proves only that AEGIS can later detect changes to the stored bytes. It says nothing about the file's origin or authenticity before acquisition.*

9.1 The acquisition pipeline



Receiving the bytes

There are two ways to upload.

Multipart upload — `POST /investigations/{id}/evidence` with a `file` field, an optional `role` (`primary` or `process_evidence`) and an optional description. It requires `evidence.upload`, applies the upload rate limit (120 per hour per user), locks the case row, and streams the body to a temporary file with

`copy_upload_to_temp()`, which reads 1 MiB at a time, updates a SHA-256 as it goes, and aborts with `413 payload_too_large` as soon as the byte count exceeds `MAX_UPLOAD_MB` (512 MB by default). The file is never held entirely in memory.

Chunked, resumable upload — for large media over unreliable connections:

1. `POST /investigations/{id}/uploads` with `filename`, `total_size` and `role` creates an `UploadSession` (refusing sizes above the limit or locked cases) and returns the `upload_id` and the chunk size (8 MB by default).
2. `PUT /uploads/{upload_id}?offset=N` sends one chunk. The offset must equal the number of bytes already received (`409` with the current count otherwise), so a client that lost its place can ask and resume. A chunk larger than the chunk size, or one that would exceed the declared total, is rejected.
3. `POST /uploads/{upload_id}/complete` checks that every byte arrived and runs the same `ingest()` pipeline on the assembled file, then deletes the partial file.

Upload sessions belong to the user who created them; another user's `upload_id` returns 404.

Content sniffing: trust only the bytes

The declared file extension and the client's MIME type are ignored. `sniff()` in `ml/aegis_ml/evidence/sniff.py` reads the first 8 KiB and identifies the format from its signature. Only formats with an implemented and tested parser are accepted:

Detected type	Signature checked	Modality
PDF	`%PDF-`	document
DOCX	ZIP with `[Content_Types].xml` and `word/document.xml`	document
Plain text	Valid UTF-8 (or UTF-8/UTF-16 BOM), no NUL bytes	text
PNG	`\x89PNG\r\n\x1a\n`	image
JPEG	`\xff\xd8\xff`	image
WebP	`RIFF….WEBP`	image
WAV	`RIFF….WAVE`	audio
MP3	`ID3` or an MPEG audio frame sync	audio
M4A	ISO BMFF `ftyp` with brand `M4A` or `M4B`	audio
MP4	ISO BMFF `ftyp` with brands such as `isom`, `mp41`, `mp42`, `avc1`, `dash`, `M4V`	video

Everything else is rejected with an `UnsafeFileError`:

- **Executables** — files starting with `MZ` (Windows PE), `\x7fELF` (Linux), `#!` (scripts) or `\xca\xfe\xba\xbe` (Mach-O/Java) — "executable content is rejected".
- **Unknown ISO BMFF brands** (for example QuickTime variants AEGIS has not tested).
- **ZIP archives other than DOCX**.
- **Empty files** and unrecognised formats.

A small detail in the text detector shows the care taken: if the 8 KiB window ends in the middle of a multi-byte UTF-8 character, decoding fails near the end; the sniffer treats an error within the last three bytes as a truncation artefact rather than a binary file.

DOCX archive safety

DOCX files are ZIP archives, which brings the risks of zip bombs and path traversal. `_check_docx()` inspects every entry before accepting the file:

Check	Limit
Number of entries	≤ 2,000
Total uncompressed size	≤ 200 MB
Per-entry compression ratio	≤ 200:1
Entry names	No absolute paths, no `..` components, no backslashes, no drive letters (`:`)
Encryption	Encrypted entries rejected
Macros	`word/vbaProject.bin` present → "macro-enabled documents are rejected"

These checks are covered by `test_zip_traversal_and_bomb_rejected` and `test_malicious_uploads_rejected`.

Malware scanning

`scan_file()` in `app/services/scanning.py` implements ClamAV's `clamd` INSTREAM protocol directly over a TCP socket: it sends `zINSTREAM\0`, then the file in 64 KiB chunks each prefixed with a 4-byte big-endian length, then a zero-length terminator, and parses the reply. It returns one of `clean`, `infected` (with the signature name), `error` or `not_scanned` (no scanner configured).

- An **infected** file is rejected with `422 malware_detected`, and the rejection — with the file's SHA-256 and the signature — is audited even though the evidence is not stored.
- If `AEGIS_REQUIRE_MALWARE_SCAN=true` and the result is anything other than `clean`, the upload fails with `503 scanner_required`. Production deployments are advised to enable this.
- The scan status is stored on the original version, so a later reviewer can see whether the evidence was scanned.

The governance documents note honestly that the ClamAV integration has not been exercised against a live scanner in the build environment.

Structural validation

After sniffing, `_validate_structure()` opens the file with *the same parser the analysis methods will use*. If the parser fails, the file is rejected with `400 malformed_media` and the exception type. This prevents a file that passes the signature check but is truncated or crafted from reaching the workers. The function also records a short metadata summary on the original version:

Modality	Checks	Recorded
Image	Open with Pillow (with decompression-bomb protection); read EXIF	Width, height, EXIF present/absent/malformed, ICC profile, XMP

Modality	Checks	Recorded
PDF	Open with pypdf; **encrypted PDFs rejected**	Page count, document info, XMP
DOCX	Open with python-docx	Paragraph count, core properties present
Audio/video	`ffprobe`; the required stream type must exist; duration must be positive and within limits (audio ≤ 3 h, video ≤ 2 h)	Duration, stream types, container tags
Text	Encoding validated during extraction	—

Storing the original

The original is written with `_store()` under a content-addressed key:

```
def evidence_key(org_id: str, sha256: str, kind: str = "original") → str:
    return f"orgs/{org_id}/{kind}/{sha256[:2]}/{sha256}"
```

Keys include the organisation id, so tenants never share objects, and the first two hex digits of the hash form a fan-out directory. If an object with that key already exists — the same bytes uploaded twice — the `ObjectExists` error is ignored, because the stored bytes are by definition identical.

The filename is sanitised before it is stored or displayed: directory components are removed (including Windows backslash paths), the name is NFC-normalised, control characters are stripped, leading dots are removed (no hidden files), and the result is truncated to 200 characters. A file named `../../etc/passwd` becomes `passwd`. This is tested by `test_filename_is_sanitised`.

Versions and derivatives

Each item gets an `original` version and, depending on its modality, a derived working copy:

- **Text and documents** — the extracted text (see the Text Methods chapter) is stored as a `normalized_text` version (UTF-8, `text/plain`) whose `transformations` list records the extraction method and every normalisation step. The extraction report (warnings, language, statistics) is stored on the item.
- **Images** — a PNG thumbnail at most 512 px on its longest side, with transformations recorded as “EXIF orientation applied”, “converted to RGB”, “downscaled to <=512 px”, “PNG”.

Each version gets its own SHA-256, its own write-once object and an `acquisition` row in `file_hashes`. Versions are numbered per item and record their parent, so the derivation tree is explicit. Originals are never modified; analysts work on derivatives.

Finally, `evidence.acquired` is audited with the hash, size, MIME type, sanitised filename, role, scan status and list of derived versions, and the case advances from `draft` to `evidence_acquired` if it was still a draft.

Process evidence

Uploading with `role = "process_evidence"` marks drafts, revision histories and notes. Such items are stored and hashed exactly like primary evidence but form a separate evidence class. They are evaluated by the reviewer, never scored automatically.

9.2 Write-once storage backends

All storage goes through the abstract `Storage` interface in `app/storage/base.py`, with `put_file`, `put_bytes`, `open` (an iterator of chunks), `exists`, `purge`, `health`, a `local_path()` context manager and a streaming `sha256()`. The backend is chosen by `AEGIS_STORAGE_BACKEND`. Every backend validates keys:

```
def check_key(key: str) -> None:
    """Keys are relative, slash-separated names; they can never address anything outside the store."""
    if not key or key.startswith("/") or ".." in key.split("/") or "\\" in key or "\0" in key:
        raise StorageError("invalid storage key")
```

Local filesystem

`LocalStorage` writes under `AEGIS_STORAGE_ROOT`. Besides `check_key`, it resolves the final path and confirms that the root is one of its parents, so even symbolic-link tricks cannot escape the root. A write proceeds as follows:

```
def _write(self, key: str, writer) -> None:
    dest = self._path(key)
    if dest.exists():
        raise ObjectExists(key)
    dest.parent.mkdir(parents=True, exist_ok=True)
    tmp = dest.with_name(dest.name + ".partial")
    writer(tmp)
    os.chmod(tmp, stat.S_IRUSR | stat.S_IRGRP)       # read-only
    try:
        os.link(tmp, dest)  # fails if dest exists: no overwrite race
    except FileExistsError as exc:
        raise ObjectExists(key) from exc
    finally:
        tmp.unlink(missing_ok=True)
```

The bytes are first written to a temporary file and made read-only. They are then published with `os.link()`, which — unlike `rename()` — fails atomically if the destination already exists. Two concurrent writers of the same key cannot overwrite each other, and a reader never sees a half-written object. The health check writes and removes a probe file and reports free disk space.

S3-compatible storage

`S3Storage` uses boto3 against AWS S3 or MinIO. It refuses to write if the key exists (`head_object`), requests server-side encryption (`AES256`) on every object, and, when `AEGIS_S3_OBJECT_LOCK` is enabled, relies on the bucket's default Object Lock retention in governance mode. The deployment guide recommends bucket versioning with Object Lock and cross-region replication. S3 behaviour is tested against the `moto` mock.

PostgreSQL storage

`DatabaseStorage` exists for hosts without a persistent disk, such as free container platforms whose filesystem is wiped on every restart. An object's header row and its 1 MiB chunks are inserted in **one transaction**, so a reader sees all of the object or none of it. The header insert uses `ON CONFLICT DO NOTHING …`

`RETURNING key`; if nothing is returned, the key already existed and the write fails with `ObjectExists`. Rows are never updated. Reads stream chunks in order with `yield_per`. Because the evidence then lives in the database, database backups cover it too. The design is intended for small deployments; it is what the Render Blueprint uses.

9.3 Integrity verification

`POST /evidence/{id}/verify` (requires `evidence.verify`, available to auditors) calls `verify_integrity()`, which re-reads every unpurged version from storage, recomputes its SHA-256 and size, and compares them with the recorded values. Each check is stored as a new `file_hashes` row with `purpose = "integrity_check"` and `matches_recorded`. A missing object counts as a mismatch.

If any version does not match:

- the item's status becomes `integrity_alert`;
- an `evidence.integrity_mismatch` audit event records every result;
- the uploader and every organisation and system administrator receive a **critical** notification: *"The stored evidence no longer matches its recorded SHA-256 hash. It must not be treated as unchanged."*

If everything matches, `evidence.integrity_verified` is audited. Either way the response includes the same caveat as the module docstring: a matching hash shows the bytes are unchanged since acquisition by AEGIS, not that the file is authentic. The `make verify` target and `python -m app.cli verify-evidence` run the same check over every item. The incident-response plan's first playbook covers what to do after a mismatch (see the Operations chapter).

9.4 Redaction

Personal data that is not needed for an analysis can be redacted. `POST /evidence/{id}/redactions` (requires `evidence.redact`) takes up to 500 character spans and a reason. `redact_text()` starts from the current text version, replaces every non-whitespace character inside each span with a full block `▮` — keeping whitespace so that the layout, word boundaries and offsets stay intact — and stores the result as a new `redacted` version whose parent is the text it was made from. The original and the unredacted text remain untouched. The redaction is audited with the span count and the new and parent version ids. Reading the text endpoint with `version=redacted` returns the redacted copy.

9.5 Downloads, thumbnails, text and artefacts

Endpoint	Permission	Behaviour
`GET /evidence/{id}/versions/{vid}/download`	`evidence.download`	Audited (`evidence.downloaded`, with hash and kind); served as `application/octet-stream` with `Content-Disposition: attachment` and `X-AEGIS-SHA256`; purged versions return `410 purged`
`GET /evidence/{id}/thumbnail`	`evidence.read`	The derived PNG thumbnail
`GET /evidence/{id}/text?version=normalized|redacted`	`evidence.read`	The extracted or redacted text

Endpoint	Permission	Behaviour
`GET /artifacts/{id}`	`analysis.read`	A generated visualisation, after checking that its job belongs to the caller's organisation and that the caller can see the case; served with `X-AEGIS-SHA256`

Serving originals as opaque attachments means a browser never renders untrusted content inline, and the storage key itself is never exposed.

9.6 Cross-case evidence views

`GET /evidence` lists evidence across all cases the caller can see, optionally filtered by `modality` (comma-separated) and by `q`, which matches either the display name or a SHA-256 prefix. Each entry includes the latest analysis job and its fusion outcome. The Text, Image, Audio and Video forensics screens are built on this single endpoint (for example `modality=text,document`), so evidence access control is enforced in one place rather than in four modality-specific route families.

9.7 Retention purge

The only way bytes leave the store is `purge_binaries()`, reachable only through an approved two-person deletion request (see the Investigations chapter). It deletes an object only when no other unpurged version references the same key — important because storage is content-addressed — and keeps every database row as a tombstone with its hash.

CHAPTER 10

The Hash-Chained Audit Trail

Every state-changing operation in AEGIS appends an event to an append-only audit trail. Each event includes the hash of the previous one, forming a chain: if any event is edited, deleted or reordered, verification fails at that point. The implementation is in `apps/api/app/audit/service.py`, and its documentation in `docs/security/audit-chain.md`.

10.1 The hash formula

```
event_hash = SHA-256( prev_hash || canonical_json({id, occurred_at, organization_id, actor_user_id,
                                                   actor_label, action, entity_type, entity_id,
                                                   investigation_id, reason, details}) )
```

- `canonical_json` sorts keys, uses no whitespace and encodes UTF-8 without ASCII escaping, so the same event always serialises to the same bytes.
- The first event links to the *genesis* value, 64 zeros.
- `prev_hash` is stored on each row, and `event_hash` is unique across the table.

```python
GENESIS = "0" * 64
_LOCK_KEY = 0x4AE615  # advisory lock id serialising chain appends on PostgreSQL

def canonical(obj: Any) -> str:
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), default=str, ensure_ascii=False)

def compute_hash(prev_hash: str, ev: AuditEvent) -> str:
    return hashlib.sha256((prev_hash + canonical(_hash_fields(ev))).encode()).hexdigest()
```

The correlation id and IP address are stored on the row for investigation but are not part of the hashed fields; the hashed fields describe *what happened*, *to what*, *by whom*, *when* and *why*.

10.2 Appending an event

`record()` is called by services inside the same database transaction as the operation it describes:

```python
def record(db, action, entity_type, entity_id=None, *, actor=None, actor_label=None,
           organization_id=None, investigation_id=None, reason=None, details=None, ip=None):
    """Append an event in the caller's transaction (it commits or rolls back with the operation)."""
    if db.get_bind().dialect.name == "postgresql":
        db.execute(text("SELECT pg_advisory_xact_lock(:k)"), {"k": _LOCK_KEY})
    last = db.execute(select(AuditEvent.event_hash).order_by(AuditEvent.seq.desc()).limit(1)).scalar()
    prev = last or GENESIS
    ev = AuditEvent(..., prev_hash=prev)
    ev.event_hash = compute_hash(prev, ev)
    db.add(ev)
```

```
    db.flush()
    return ev
```

Two properties follow from this design:

1. **Atomicity with the operation.** Because the event is written in the caller's transaction, an operation that rolls back leaves no audit event, and an audit event can never exist for an operation that did not happen. Conversely, an operation cannot commit without its event.
2. **No forks.** Two concurrent requests could otherwise both read the same "last hash" and append two events with the same `prev_hash`, forking the chain. A PostgreSQL *transaction-level advisory lock* (`pg_advisory_xact_lock`) serialises appends; it is released automatically when the transaction ends.

The `details` dictionary is normalised through `canonical()` before storage, so what is stored is exactly what was hashed.

Actors are recorded both as a user id (when a person acted) and as a label: the user's email, or a system label such as `system:workflow` for automatic state transitions or `system:worker` for job completion.

10.3 What is audited

Some of the actions recorded across the code base:

Area	Actions
Authentication	`auth.login`, `auth.login_failed`, `auth.logout`, `auth.password_changed`, `auth.mfa_enabled`, `auth.session_revoked`
Cases	`investigation.created`, `investigation.updated` (with before/after), `investigation.reviewer_assigned`, `investigation.status_changed`, `investigation.legal_hold`
Review	`annotation.added`, `review.decision_recorded`, `appeal.submitted`, `appeal.updated`
Evidence	`evidence.acquired`, `evidence.rejected`, `evidence.rejected_malware`, `evidence.downloaded`, `evidence.integrity_verified`, `evidence.integrity_mismatch`, `evidence.redacted`, `evidence.purged`
Retention	`retention.deletion_requested`, `retention.deletion_decided`
Analysis	`analysis.queued`, `analysis.completed`, `analysis.completed_with_failures`, `analysis.failed`, `analysis.cancel_requested`
Reports	generation, finalisation, corrections, verification
Administration	user, membership, model-registry, settings and retention-policy changes, each with a reason

Read operations are not generally audited, with one deliberate exception: downloading original evidence is, because it is the act that takes evidence outside the system's custody.

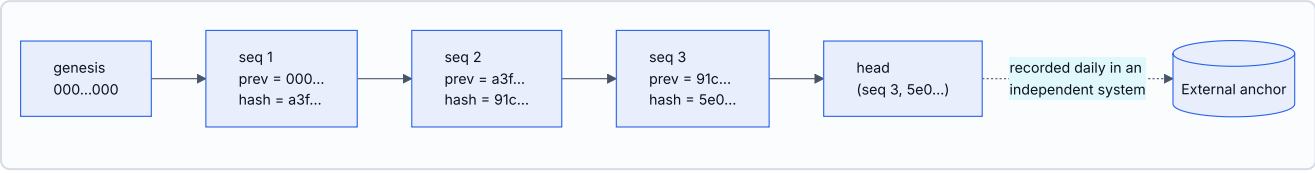
10.4 Verifying the chain

```
def verify_chain(db: Session, limit: int | None = None) -> dict[str, Any]:
    prev = GENESIS
    checked = 0
    q = select(AuditEvent).order_by(AuditEvent.seq)
    for ev in db.execute(q.execution_options(yield_per=500)).scalars():
        if ev.prev_hash != prev:
            return {"valid": False, "checked": checked, "broken_at_seq": ev.seq, "reason": "prev_hash
mismatch"}
        if compute_hash(prev, ev) != ev.event_hash:
            return {"valid": False, "checked": checked, "broken_at_seq": ev.seq, "reason": "event
content altered"}
        prev = ev.event_hash
        checked += 1
    return {"valid": True, "checked": checked, "head": prev}
```

Verification streams events in sequence order 500 at a time and distinguishes two failure modes:

- `prev_hash mismatch` — an event was deleted or inserted, or the order changed.
- `event content altered` — a field of this event was changed after it was written.

It is available as `POST /api/v1/audit/verify` (requires `audit.read`), as `python -m app.cli verify-audit`, and as part of `make verify`. `GET /api/v1/audit/head` returns the latest `(seq, event_hash)`. The test `test_audit_tampering_detected` edits an event directly in the database and checks that verification reports the exact sequence number.



10.5 What it detects and what it does not

The chain detects:

- editing any field of any event;
- deleting or reordering events (the next event's `prev_hash` no longer matches).

It does **not** prevent a person with write access to the database from recomputing the whole chain after an edit. Hash chaining alone cannot detect that. The documented mitigation is **external anchoring**:

1. Record the head hash on a schedule — for example daily — in a system the database administrators do not control: an append-only ticket, WORM object storage, a signed email to a governance mailbox, or an RFC 3161 timestamping service.
2. During audits, check that every recorded head still appears in the chain at the recorded sequence number.

Reports also embed the latest audit event hash for their case at generation time, giving each exported report an independent anchor: a report printed last month carries a hash that must still exist in the chain today. Automatic RFC 3161 timestamping of the head is on the roadmap.

10.6 Chain of custody

The audit trail, the per-version SHA-256 records and the write-once store together form a *technical* custody record. The documentation is explicit that they do **not** amount to a legally certified chain of custody. Certification requires a validated operational procedure (who may acquire evidence, how originals are obtained, how devices are handled) and legal review in the relevant jurisdiction. The evidence-handling procedure template in the governance package is a starting point for that work.

10.7 Viewing the audit trail

The **Audit & Compliance** screen lets auditors browse events filtered by action, entity or case, run chain verification and see the head. Each case page also shows that case's own history (see the Investigations chapter). The same screen displays the compliance control mapping described in the Governance chapter.

CHAPTER 11

The Analysis Pipeline

This chapter explains how an analysis request becomes a set of persisted method results and an automated assessment: profiles, job creation, dispatch to workers, idempotent execution, calibration lookup, result persistence, fusion, cancellation, recovery and live progress. The code is in `apps/api/app/services/analysis.py`, `apps/api/app/api/v1/analysis.py` and `apps/api/app/workers/`.

11.1 Analysis profiles

A *profile* is a named set of methods for a kind of investigation. Profiles are defined in `ml/aegis_ml/registry.py`:

Profile id	Name	Modalities	Methods
`text_triage`	Text authenticity triage	text, document	descriptive statistics, supervised classifier, token statistics, Fast-DetectGPT, Binoculars, AEGIS reference watermark, SynthID-Text, C2PA
`image_investigation`	Image authenticity investigation	image	image metadata, declared generation claims, C2PA, frequency spectrum, noise residual, error-level analysis, local image model, external image detector
`synthetic_speech`	Synthetic speech investigation	audio	audio signal analysis, declared claims, C2PA, local synthetic-speech model, external audio detector, transcription placeholder
`deepfake_video`	Deepfake video investigation	video	container inspection, temporal consistency, audio signal analysis, declared claims, C2PA, per-frame model, external video detector
`provenance_verification`	Provenance verification	all	C2PA, declared claims, reference watermark, SynthID-Text
`comprehensive`	Comprehensive multimodal examination	all	every method applicable to the evidence type

Each profile also carries a resource estimate (for example “CPU: ~10–60 s per minute of video at 2 fps sampling”) and its own limitations (“Frame-level observations do not establish that a whole video is fabricated.”). `methods_for(profile, modality, registry)` selects the profile’s methods that apply to the evidence’s modality; text methods also apply to documents.

`GET /analysis/profiles` returns every profile with, for each method, its evidence class, implementation status, current availability and the reason if it is unavailable. The new-analysis dialog uses this to show the user *before* they start which methods will actually run.

11.2 Creating a job

`POST /analysis/investigations/{case_id}/jobs` (requires `analysis.run`, returns `202 Accepted`) takes an evidence item id, a profile id and options (only `sample_fps` is accepted; other keys are dropped). It locks the case and calls `create_job()`:

1. **Idempotency.** If an `Idempotency-Key` header is supplied and a job with that key exists, the existing job is returned — provided it belongs to the same organisation and evidence item; otherwise `409`. A client that retries a request after a network error therefore never creates duplicate jobs.
2. **Validation.** The profile must exist; the evidence must belong to the case; the evidence status must be `active` (an item with an integrity alert cannot be analysed: "analysis is blocked until integrity is resolved"); the case must not be locked; the profile must have at least one method for the modality.
3. **Policy snapshot.** `external_processing_permitted` is computed once and stored on the job. It is true only if **all three** levels allow it:

```
def external_permitted(db: Session, inv: Investigation) → bool:
    org = db.get(Organization, inv.organization_id)
    return bool(get_settings().allow_external_processing
                and org and org.allow_external_processing
                and inv.allow_external_processing)
```

4. **Rows.** One `AnalysisJob` (status `queued`, `methods_total` = number of methods, bound to the *original* version) and one `DetectorRun` per method (status `pending`, with method id, version, name, evidence class, input SHA-256 and the method's limitations).
5. **Workflow.** The case moves to `analysis_queued` if it was in `evidence_acquired`, `analysis_complete` or `human_review`.
6. **Audit.** `analysis.queued` with the profile, evidence id, input hash, method list and the external-processing decision.

After the transaction commits, `dispatch()` hands the job to an executor according to `AEGIS_TASK_MODE`:

Mode	Behaviour
`celery`	`run_analysis_job.delay(job_id)`; the Celery task id is stored on the job
`inline`	A daemon thread in the API process runs `execute_job(job_id)`
`manual`	Nothing; the test suite drives execution explicitly

Dispatching *after* commit matters: a worker can never receive a job id whose rows are not yet visible.

11.3 The Celery worker

`app/workers/celery_app.py` configures Celery for reliability rather than throughput:

```
celery_app.conf.update(
    task_acks_late=True,                # acknowledge only after the task finishes
    task_reject_on_worker_lost=True,    # redeliver if the worker process dies
    worker_prefetch_multiplier=1,       # one long job at a time per process
    task_time_limit=3600,               # hard limit: 1 hour
    task_soft_time_limit=3300,
    broker_connection_retry_on_startup=True,
    task_default_queue="aegis",
    task_routes={"app.workers.tasks.run_benchmark": {"queue": "aegis-research"}},
    beat_schedule={"recover-stale-jobs": {"task": "app.workers.tasks.recover_stale", "schedule":
600.0}},
)
```

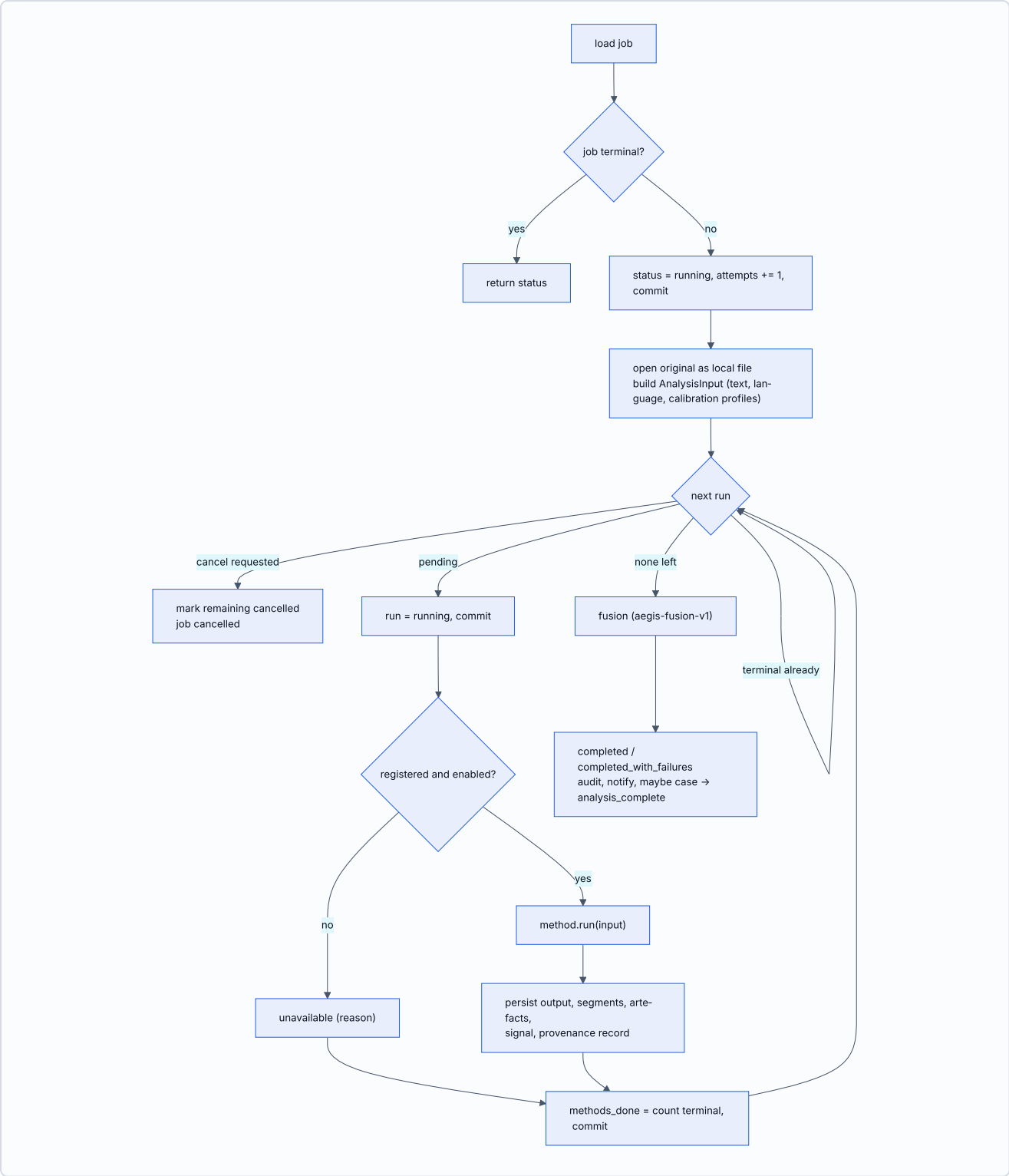
There is no result backend: results are written to PostgreSQL, not returned through Celery. Three tasks exist:

Task	Queue	Purpose
`run_analysis_job(job_id)`	`aegis`	Calls `execute_job`; retries up to five times with exponential back-off on database `OperationalError`
`run_benchmark(run_id)`	`aegis-research`	Executes a benchmark run (see the Benchmark Laboratory chapter)
`recover_stale()`	`aegis`	Scheduled every 600 s by beat; requeues abandoned jobs

Separate queues let operators run research workloads on different machines from casework. When a worker starts, a daemon thread writes a heartbeat (`aegis:worker:heartbeat`, the current time with a 60-second expiry) to Redis every 20 seconds. System Health reports the worker as down if the key is missing.

11.4 Executing a job

`execute_job()` is written to be called any number of times for the same job. Its docstring: *"Run (or resume) a job. Safe to call repeatedly: completed runs are never repeated."*



Step by step:

1. Load the job. If it is already terminal (`completed`, `completed_with_failures`, `failed`, `cancelled`), return immediately.
2. Mark it `running`, increment `attempts`, set `started_at` if unset, and commit.
3. Obtain a local path to the original (for remote stores this downloads to a temporary file) and build the `AnalysisInput`: modality, MIME type, path, SHA-256, the current text version (normalised or redacted), the detected language, the extraction report, the job options and **the validated calibration profiles from the model registry**.

4. For each run in position order:
   - skip it if it is already terminal (this is what makes redelivery safe);
   - re-read the job and stop if cancellation was requested, marking the remaining runs `cancelled`;
   - mark it `running` and commit, so the interface shows which method is active;
   - if the method is no longer registered, or an administrator disabled it in the registry, record `unavailable` with that reason;
   - otherwise call `method.run(input)` — which never raises — and persist the result;
   - if the registry entry is validated, record its operating threshold on the run;
   - update `methods_done` from the count of terminal runs and commit.
5. When all runs are done, compute the fusion result from the stored runs, and set the job to `completed`, or to `completed_with_failures` if any run `failed`.
6. Audit `analysis.completed` (or `completed_with_failures`) with every run's status and the automated outcome; notify the requester and the assigned reviewer; and if no other job for the case is still queued or running, move the case to `analysis_complete`.

If anything outside the per-method wrapper raises — for example the storage backend fails — the transaction is rolled back, the job is marked `failed` with the error, an `analysis.failed` event is audited as `system:worker`, the requester gets an error notification, and the case is still allowed to progress. A job failure is recorded, never silently dropped.

Feeding calibration into methods

`_calibration_profiles()` builds a dictionary of calibration data for every method whose registry entry is `validated` *and* has an attached calibration profile that is itself `validated`. It contains the calibration method and parameters, threshold, population, languages evaluated, the benchmark run and profile ids, and the score direction (`higher_is_ai` by default). The dictionary is passed to methods through `AnalysisInput.options["calibration_profiles"]`. A method may set `calibrated` and `flagged` only when it finds a profile for itself there — and, for text, only if the evidence language is among the evaluated languages. This is how ADR-05 ("raw scores are not probabilities") is enforced at run time.

Persisting a result

`_persist_result()` copies the `MethodResult` onto the `DetectorRun`:

- all fields are passed through `jsonable()`, which converts NumPy scalars to Python values, turns sets and tuples into lists, and **replaces NaN and infinity with `null`** so that no invalid number ever reaches JSON or the database;
- a non-finite score is stored as `NULL`;
- each artefact (for example a spectrum PNG) is hashed and written to `orgs/{org}/artifacts/{sha[:2]}/{sha}` in the write-once store, and an `AnalysisArtifact` row records its name, media type, key, hash, description and `cannot_establish` text;
- up to 2,000 segments are stored as `SegmentFinding` rows;
- for completed runs, an `AnalysisSignal` row summarises the run for fusion (verification status, calibration status — `validated`, `uncalibrated` for statistical methods or `not_applicable` — population, input characteristics such as token count and language, uncertainty and *known dependencies*);
- for completed C2PA runs, a `ProvenanceRecord` stores the verification state, summary and trust anchors.

The *known dependencies* field deserves a note. `_dependencies()` lists other methods in the same job that share a configuration key with this one — for example Fast-DetectGPT and token statistics both use `AEGIS_SCORING_LM_PATH`, so they score text with the same language model. Their outputs are therefore not independent, and the signal record says so ("shares configuration/model with text.token_statistics"). This is the same concern that leads the fusion policy to refuse to treat agreement between detectors as independent corroboration.

11.5 Cancelling a job

`POST /analysis/jobs/{id}/cancel` (requires `analysis.run`) sets `cancel_requested`. A job that is still `queued` is cancelled immediately and its runs are marked `cancelled`. A running job is cancelled cooperatively: the worker checks the flag before each method, so the method currently running finishes and is recorded, and the rest are marked "Cancelled before execution." Cancelling a terminal job returns `409`. The request is audited.

11.6 Recovering from lost workers

If a worker machine dies mid-job, Celery redelivers the message because acknowledgement is late. If the message is lost too — for example Redis was restarted — the job stays `running` for ever. `recover_stale_jobs()` handles this:

```
stale = select(AnalysisJob).where(AnalysisJob.status == "running", AnalysisJob.started_at < cutoff)
for job in stale:
    if job.attempts >= 3:
        job.status, job.error = "failed", "Exceeded retry attempts after worker loss."
        continue
    for r in job.runs:
        if r.status == "running":
            r.status = "pending"
    job.status = "queued"
...
for job in stale:
    if job.status == "queued":
        dispatch(job.id)
```

Jobs running for more than 30 minutes are reset to `queued`, their interrupted run goes back to `pending`, and they are dispatched again. Runs already completed are kept, so no work is repeated. After three attempts a job is failed rather than retried for ever. Beat runs this every ten minutes; operators can also run `python -m app.cli recover-jobs` or trigger it from the administration screen.

In inline mode there is no broker to redeliver anything, so `resume_inline_jobs()` runs at API start-up (when enabled): it redispatches every `queued` job and recovers every `running` job regardless of age. Its docstring warns that this is only correct when that API process is the sole executor — which is exactly the single-container case.

11.7 Live progress over server-sent events

`GET /analysis/jobs/{id}/events` streams the job's state as server-sent events:

```
async def stream():
    last = None
    idle = 0
    while not await request.is_disconnected():
        with get_sessionmaker()() as s:
            job = s.get(AnalysisJob, job_id)
            snap = _snapshot(job) if job else None
        if snap != last:
            last = snap
            idle = 0
            yield f"event: job\ndata: {json.dumps(snap)}\n\n"
            if snap is None or snap["status"] in svc.TERMINAL_JOB:
                return
        else:
            idle += 1
            if idle % 15 == 0:
                yield ": keep-alive\n\n"
        await asyncio.sleep(1.0)
```

Each snapshot contains the job id, status, `methods_total`, `methods_done`, the status of every run and the primary outcome. The server polls the database once a second with a fresh short-lived session and emits an event **only when the snapshot changes**, sends a comment line every 15 seconds of inactivity to keep proxies from closing the connection, and closes the stream when the job reaches a terminal state or the client disconnects. The response sets `X-Accel-Buffering: no` so that nginx does not buffer the stream. The request's own database session is closed before streaming begins, so a long-lived stream does not hold a pooled connection.

Because the stream reflects only persisted state, the progress bar can never run ahead of reality. If the browser reloads, it reconnects and receives the current state immediately.

11.8 Reading results

Endpoint	Returns
`GET /analysis/investigations/{case_id}/jobs`	All jobs for the case, newest first
`GET /analysis/jobs/{id}`	The job with every run (status, summary, output, score, semantics, calibration, flag, warnings, limitations, config, timing, artefacts) and the fusion result
`GET /analysis/runs/{id}/segments`	The localised findings of one run
`GET /artifacts/{id}`	A visualisation PNG with its SHA-256 header

Every read goes through `require_job()`, which checks the organisation and then calls `get_case()` so that restricted-classification rules apply to jobs exactly as they apply to cases.

CHAPTER 12

Reproducible Reports

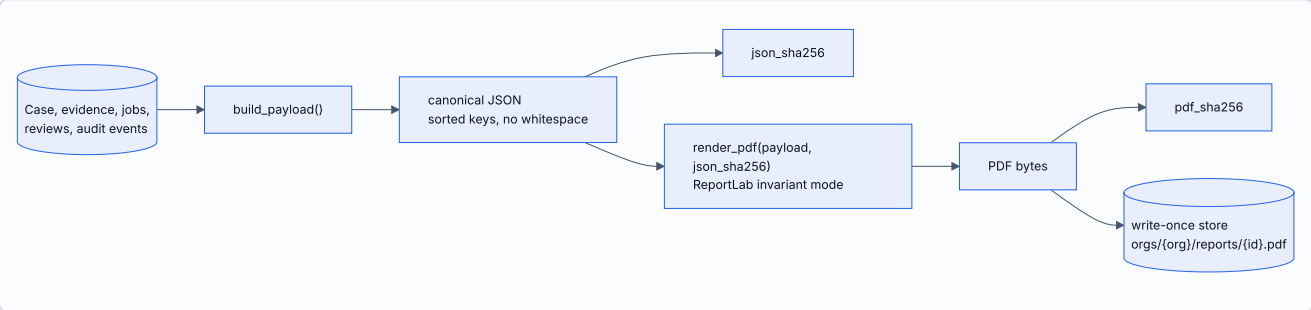
A report is the artefact that leaves AEGIS: it is printed, attached to a file, shown to a panel or submitted to a court. AEGIS reports are designed to be **verifiable** (anyone can check they have not been altered), **reproducible** (the PDF can be regenerated bit-for-bit from the JSON) and **honest** (automated findings and human conclusions are kept in separate sections, with standard limitations). The code is in `apps/api/app/services/reports.py` and `apps/api/app/api/v1/reports.py`.

12.1 Two representations, one source

Every report exists in two forms:

1. a **canonical JSON payload** with schema `aegis.report/v1`, stored in the `reports.payload` column and hashed as `json_sha256`;
2. a **PDF** rendered *from the payload alone*, stored write-once in the evidence store and hashed as `pdf_sha256`.

The PDF is generated with ReportLab's `invariant=1` option, which removes the creation timestamp and random document identifier that PDF writers normally embed. The same payload therefore always produces the same PDF bytes. This allows a verifier to re-render the PDF from the stored JSON and compare hashes, proving that the PDF says exactly what the JSON says.



12.2 What a report contains

`build_payload()` assembles the payload from the database:

Section	Content
`schema`	`aegis.report/v1`
`report`	Report id, version, generation time, author, AEGIS version, deployment mode, the report it supersedes and the correction reason
`case`	Reference, title, purpose, media category, description, priority, status, legal basis, retention and access classification, reviewer, creator, creation time, demo-fixture flag, pending questions
`scope_and_methodology`	A fixed description of acquisition, persisted jobs and the fusion policy, and the policy id `aegis-fusion-v1`

Section	Content
`evidence_inventory`	Every item with role, modality, MIME type, status, acquisition time and uploader; the extraction summary (method, token count, tokenizer, language, warnings, OCR flag, pages); and **every version** with id, number, kind, SHA-256, size, MIME type, parent, transformations, metadata status, scan status and whether it was purged
`automated_findings`	Every job with evidence and version ids, profile, status, requester, timing, external-processing decision and the full automated assessment; and every method run with id, name, version, evidence class, status, input hash, configuration, threshold, summary, score and its semantics, calibration, flag, uncertainty, warnings, limitations, output schema, timing, artefacts (with hashes and `cannot_establish`), segment count and, for C2PA, the provenance output
`contradictory_signals`	The union of conflicting methods across all jobs
`human_assessment`	Every review decision (reviewer, decision, rationale, agreement with the automated outcome, the automated outcome at the time, content hash, audit event hash, time) and every annotation — under the note “Human conclusions below are separate from automated findings.”
`appeals`	Every appeal with status, grounds and resolution
`limitations`	The six standard limitations (below)
`audit_references`	The number of audit events for the case, the latest event hash (`chain_head_for_case`) and the last 200 events with sequence, action, time, actor and hash

The standard limitations

Every report carries the same six statements, defined once in code:

1. No detector can reliably establish authorship or origin from content alone. Statistical outputs describe resemblance to known generator behaviour under the stated conditions.
2. A low or negative detector output does not prove human authorship; absence of a watermark or provenance record does not indicate either AI generation or human creation.
3. Detector performance varies with length, language, domain, generator, editing, compression and other transformations; uncalibrated scores are not probabilities.
4. A valid provenance signature shows the record is intact and signed; it does not establish that its claims are true.
5. Hash records show the stored bytes are unchanged since acquisition by AEGIS; they do not establish origin.
6. This report supports investigation and review. It must not by itself be the basis of an adverse decision about any person; independent corroboration and qualified human review are required.

12.3 The PDF layout

`render_pdf()` produces an A4 document with 18 mm margins, a blue rule at the top of each page and a footer showing the report id, version, case reference and page number. Its structure is:

Page / section	Content
Cover	Title, a table with case reference, report id, version, generation time, author, deployment mode, access classification and the **payload SHA-256**; a "DEMO FIXTURE" warning for synthetic cases; the correction notice when the report supersedes another; a note that automated outcomes describe the evidence and are not findings of misconduct
1. Executive summary	The automated outcome of each completed analysis
2. Scope and methodology	How evidence was acquired and analysed
3. Evidence inventory and hashes	Each item and version with SHA-256
4. Processing history and method versions	Jobs and the exact method versions used
5. Individual results	Each method's status, summary, score semantics and limitations
6. Statistical reliability	Calibration status and the meaning of scores
7. Provenance findings	C2PA verification results
8. Contradictory signals	Methods in conflict, or "None identified by the fusion policy."
9. Limitations	The standard limitations
10. Human reviewer assessment	Review decisions with rationale and hashes; annotations
Appendix A. Audit references	Event count, latest event hash, and the last 60 events

All text from the database is XML-escaped before being placed in ReportLab paragraphs, so a case title containing markup cannot alter the document.

12.4 Generating a report

`POST /investigations/{case_id}/reports` (requires `report.generate`) calls `generate()`:

1. Determine the next version number for the case.
2. Refuse if the case is `report_finalized` or `archived` — a finalised case's report can only be changed through a correction.
3. Build the payload, compute `json_sha256`, render the PDF, compute `pdf_sha256`.
4. Store the PDF at `orgs/{org}/reports/{report_id}.pdf` (write-once).
5. Insert a `Report` row with status `draft`.
6. Audit `report.generated` with the version and both hashes.

Drafts can be generated as many times as needed while the case is open; each is a new version.

12.5 Finalising

`POST /reports/{id}/finalize` (requires `report.finalize`, held by reviewers and system administrators) calls `finalize()`, which enforces:

- the report must be a `draft`;
- **at least one human review decision must exist** for the case ("A human review decision is required before a report can be finalised.");

- **the report's author cannot finalise it** ("Separation of duties: the report must be finalised by someone other than its author."). The only exception is a system administrator, which keeps a single-administrator installation usable.

On success the report becomes `final` with the finaliser and time recorded; if it supersedes an earlier report, that one becomes `superseded`; the case moves from `human_review` or `inconclusive` to `report_finalized`; and `report.finalized` is audited with both hashes.

12.6 Corrections

A final report is never edited. `POST /reports/{id}/corrections` (requires `report.generate`, reason of at least 10 characters) is accepted only for `final` reports. It generates a new version whose payload records `supersedes_report_id` and `correction_reason`, and whose cover page carries the correction notice. When the correction is finalised, the original becomes `superseded` but remains stored and verifiable. This is the mechanism the appeal procedure relies on: an upheld appeal leads to a correction, and both versions remain on record.

12.7 Exporting

Endpoint	Output
`GET /reports/{id}/json`	A JSON download containing the payload and a manifest with the schema, `json_sha256`, `pdf_sha256` and the canonicalisation rule ("JSON, sorted keys, no whitespace")
`GET /reports/{id}/pdf`	The stored PDF with `X-AEGIS-SHA256`
`GET /reports`, `GET /investigations/{id}/reports`	Report lists

Both exports are audited as `report.exported` with the format.

12.8 Verifying

`POST /reports/{id}/verify` (requires `report.read`) performs three independent checks and audits the result:

```
def verify(rep: Report) → dict[str, Any]:
    json_ok = payload_sha256(rep.payload) == rep.json_sha256
    pdf_ok = None
    if rep.pdf_storage_key:
        try:
            actual, _ = get_storage().sha256(rep.pdf_storage_key)
            pdf_ok = actual == rep.pdf_sha256
        except Exception:
            pdf_ok = False
    rerender_ok = hashlib.sha256(render_pdf(rep.payload, rep.json_sha256)).hexdigest() ==
rep.pdf_sha256
    return {..., "json_integrity": json_ok, "pdf_integrity": pdf_ok,
            "pdf_reproducible_from_json": rerender_ok,
            "intact": bool(json_ok and pdf_ok is not False and rerender_ok)}
```

Check	Detects
`json_integrity`	Someone edited the payload in the database
`pdf_integrity`	The stored PDF object was altered or is missing
`pdf_reproducible_from_json`	The stored PDF does not correspond to the stored payload

The test `test_report_tampering_detected` modifies a stored payload and checks that verification fails. Outside AEGIS, anyone holding a PDF can compute its SHA-256 and compare it with the hash recorded in the report row and the audit chain; anyone holding the JSON export can canonicalise it and compare with `json_sha256`.

CHAPTER 13

Administration, Health and Operations Commands

The `platform` router (`apps/api/app/api/v1/platform.py`, the largest router at about 880 lines) collects everything that is not tied to a single case: the dashboard, global search, notifications, the audit trail, health checks, the detector and model registry, user and organisation administration, settings, retention and governance. This chapter describes it together with the health service and the command-line tool.

13.1 Dashboard and search

`GET /overview` (requires `case.read`) returns the figures for the overview dashboard. Its docstring states the rule: *"Every number is a live database count for the caller's organisation."* It counts visible cases by status, demo-fixture cases, evidence items, jobs by status, automated outcomes by category and detector runs by status — all computed from `visible_cases()`, so a user never sees numbers that include cases they cannot open.

`GET /search?q=…` (at least two characters) powers the global search box (Ctrl-K). It returns up to ten matching cases (by title or reference) and, if the caller has `evidence.read`, up to ten evidence items matched by display name or by **SHA-256 prefix**. Searching by hash lets an investigator check instantly whether a file they have been sent is already in the system.

13.2 Notifications

Services create notifications with `notify(db, user_id, org_id, kind, title, body, link, severity)`. Kinds include `assignment`, `review_requested`, `review_recorded`, `appeal`, `job_completed`, `job_failed` and `integrity_alert`; severities are `info`, `warning`, `error` and `critical`. `GET /notifications` returns the user's 50 most recent notifications (optionally unread only) with the unread count, and `POST /notifications/read` marks all of them, or a given list of ids, as read. The bell icon in the workstation header shows the unread count and links each notification to the relevant case.

13.3 Audit endpoints

Endpoint	Permission	Purpose
`GET /audit/events`	`audit.read`	Audit events for the organisation, filterable by action and case, paged backwards by sequence number (`before_seq`)
`POST /audit/verify`	`audit.read`	Recompute the whole chain (see the Audit Trail chapter)
`GET /audit/head`	`audit.read`	The latest sequence number and event hash, for external anchoring

13.4 Truthful health checks

The health service (`apps/api/app/services/health.py`) has a one-line docstring that summarises its approach: *"Truthful component health: every status comes from an actual check at request time."* Nothing is cached or assumed.

Component	Check	Statuses
PostgreSQL	`SELECT 1`, with round-trip time	operational / unavailable
Redis	`PING` (1 s timeout); "not configured" when `REDIS_URL` is empty	operational / unavailable / unknown
Analysis workers	In Celery mode, the age of the heartbeat key; operational if under 90 s. In inline mode, reported as in-process and "not for production"	operational / unavailable
Analysis queue	Length of the `aegis` list in Redis	operational / unknown
Evidence storage	The backend's own `health()`: a write probe and free space (local), `head_bucket` (S3), object count and size (database)	operational / unavailable
ffmpeg / ffprobe	Present on `PATH`; otherwise "audio and video methods unavailable"	operational / unavailable
Malware scanner	Configured or not ("uploads are recorded as not scanned")	operational / unknown
C2PA library	Import `c2pa` and report the `c2pa-rs` SDK version	operational / unavailable

Three endpoints expose this:

- `GET /health/live` — no authentication, always `{"status": "ok"}` while the process runs. Used by container orchestrators for liveness.
- `GET /health/ready` — no authentication; `ready` only if both the database and storage are operational, otherwise `degraded`. Used for readiness probes and by the Render start script.
- `GET /health/components` — requires `system.health`; returns every component above, every method with its availability and reason, and job counts by status.

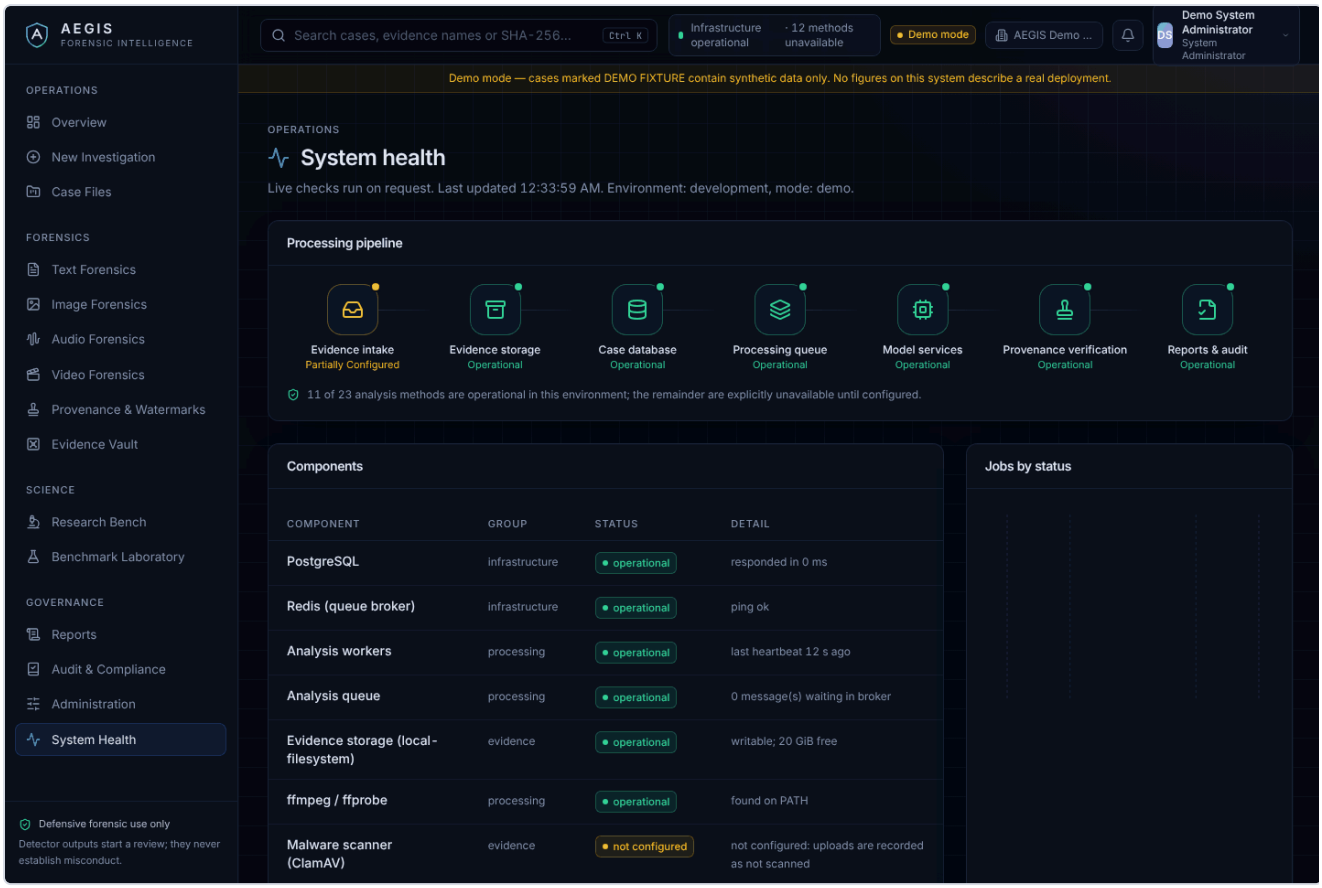


Figure 3. System Health shows a live check of every component and the true status of every analysis method, including the reason a method is unavailable.

13.5 Detectors and the model registry

`GET /detectors` lists every registered method with its `MethodInfo` (id, name, version, modalities, evidence class, implementation status, description, output semantics, limitations, references, requirements, whether it sends evidence externally) and its current availability. `GET /provenance/records` lists C2PA verification records, and `GET /watermarks/verifiers` lists the watermark verifiers and their status.

`GET /models` returns the model registry: every `detector_versions` row with the latest completed benchmark run for that method (id, result kind, time, AUROC) and whether the method is operational right now.

`PATCH /models/{id}` (requires `admin.models`) changes a registry entry: enable or disable it, set the licence, model identifier, languages evaluated, validation status (`unvalidated`, `validated`, `deprecated`), the validating benchmark run, the calibration profile, the operating threshold and a validation note. A reason of at least 10 characters is mandatory. The endpoint enforces the validation rules that make ADR-05 meaningful:

```
if target_status == "validated":
    run = db.get(BenchmarkRun, run_id) if run_id else None
    if run is None or run.status != "completed" or run.method_id != r.method_id:
        raise bad_request("Marking a detector validated requires a completed benchmark run of this method.")
    if run.result_kind != "locally_measured":
        raise bad_request("Validation requires locally measured results on registered real data, not "
                          f"'{run.result_kind}' results.")
```

```
    if prof_id is not None:
        prof = db.get(CalibrationProfile, prof_id)
        if prof is None or not prof.validated or prof.method_id != r.method_id:
            raise bad_request("Calibration profile must be validated and belong to this method.")
```

A method cannot be marked validated on the strength of a simulated, published or vendor-reported result. It needs a completed, **locally measured** benchmark run of that exact method, and any attached calibration profile must itself be validated and belong to the method. Every change is audited with the values before and after.

13.6 User and organisation administration

Endpoint	Permission	Behaviour
`GET /users`	`admin.users`	Members of the current organisation with role, status, MFA and last login
`POST /users`	`admin.users`	Creates a user (password policy enforced) or adds an existing user to this organisation; audited `admin.user_added`
`PATCH /users/{id}`	`admin.users`	Changes role or active status. **Administrators cannot change their own role or status.** Users with memberships in several organisations can only be deactivated by a system administrator. Deactivation revokes all the user's sessions. Audited `admin.user_updated`
`POST /users/{id}/revoke-sessions`	`admin.users`	Revokes every session of a member
`GET /roles`	authenticated	Roles with their permissions
`GET /organizations`	authenticated	The caller's organisations
`POST /organizations`	system administrator	Creates an organisation
`PUT /organizations/current/settings`	`admin.settings`	Sets the organisation's external-processing switch with a reason; the response reminds the caller of the deployment kill switch and that all three levels must permit external processing
`GET /reviewers`	`case.read`	Members who may review

The role that can be assigned through `PATCH` excludes `system_admin`; system administrators are created only with the CLI.

13.7 Settings, retention and job recovery

- `GET /admin/settings` and `PUT /admin/settings/{key}` manage a small allow-list of platform settings — `feature.research_bench` (boolean), `feature.chunked_uploads` (boolean) and `ui.banner` (string) — with a reason. Writing settings requires a system administrator.

- `GET /admin/retention` lists the organisation's retention policies and the cases **due for review** (created longer ago than their class's retention period), with their legal-hold status. The response notes that nothing is deleted automatically: *"Deletion always needs a request and a second person's approval, and is blocked under legal hold."*
- `PUT /admin/retention/{classification}` changes a class's retention period and description with a reason; audited.
- `POST /admin/jobs/recover` (system administrator) runs stale-job recovery immediately and audits the number of jobs handled.

Default retention classes are `standard` (365 days), `extended` (7 years), `legal_minimum` (90 days) and `research` (3 years). They are templates for the organisation to review.

13.8 Governance endpoints

- `GET /governance/compliance` serves `docs/governance/compliance-mapping.json` (see the Governance chapter) so that the Audit & Compliance screen always shows the version shipped with the code.
- `GET /governance/registry-summary` counts registered methods by implementation status (operational, requires configuration, experimental, placeholder).

13.9 The command-line tool

`python -m app.cli` (in `apps/api/app/cli.py`) provides operational commands that need direct database access:

Command	Purpose
`sync`	Synchronise roles, permissions, providers and the model registry with the code (idempotent; run after every migration)
`create-admin --email --name --org-name --org-slug`	Create the organisation if needed and a system administrator. The password is read from `AEGIS_ADMIN_PASSWORD` or prompted for, and must satisfy the password policy. Audited as `admin.bootstrap_admin` by `cli`
`seed-demo`	Create the demo organisation, the seven demo users and three DEMO FIXTURE cases, ingest synthetic evidence and run the analyses. **Refused in production** ("seed-demo is disabled in production.")
`benchmark [--out DIR] [--plots]`	Run the built-in simulated reference-watermark benchmark and the nine experiments; write JSON (and optionally PNG) results
`verify-audit`	Verify the hash chain; print the result as JSON; exit status 1 if invalid
`verify-evidence`	Recompute the hash of every non-purged evidence item; print `OK` or `MISMATCH` per item; exit status 1 on any mismatch
`recover-jobs`	Requeue or fail stale jobs

The non-zero exit codes make `verify-audit` and `verify-evidence` suitable for cron jobs and monitoring: a scheduled run that fails can raise an alert.

The demo seeder in detail

`seed-demo` is worth describing because the three demo cases appear in screenshots throughout this book. It creates the organisation "AEGIS Demo Laboratory" (`demo-lab`) and these cases, each marked as a demo fixture, assigned to the demo reviewer, with the legal basis "Demonstration only; no personal data.":

Case	Evidence (all synthetic, generated at seeding time)	Profile
`DEMO-TXT-001` Field-notes authorship triage	`field-notes.txt` (a human-style sample text), `reference-watermarked.txt` (220 tokens from the AEGIS reference generator, if a watermark key of at least 16 characters is configured), `field-notes.docx`	`text_triage`
`DEMO-IMG-001` Image provenance check	`signed-generated.jpg` (a JPEG with a C2PA manifest signed by a locally generated test certificate), `test-pattern.jpg` (EXIF Software "DemoCam 1.0"), `declared-generator.png` (a PNG whose text chunk declares a generator)	`image_investigation`
`DEMO-AV-001` Audio and video structure review	`tone.wav` (generated tones), `test-pattern.mp4` (a 4-second 320×240 test pattern)	`synthetic_speech`, `deepfake_video`

The fixtures come from `ml/aegis_ml/fixtures.py`. Each case's evidence is ingested through the normal pipeline and analysed with real jobs, so the demo shows genuine method output on synthetic input. The text case is then moved to `human_review` so that the reviewer account has something to review. Re-running the seeder skips cases that already exist.

PART III

Analysis Methods

The aegis_ml library: the method framework, text, image, audio and video methods, provenance, watermarks and evidence fusion.

CHAPTER 14

The Analysis Method Framework

The `aegis_ml` package contains every analysis method AEGIS can run. It has no database and no web framework: a method receives an `AnalysisInput` and returns a `MethodResult`. This chapter describes the shared types, the uniform run wrapper, the method registry and the full catalogue of methods. The following chapters go through the methods by modality.

14.1 Shared types

All shared types are in `ml/aegis_ml/core.py`.

`Modality` — `text`, `image`, `audio`, `video`, `document`.

`EvidenceClass` — the seven evidence classes introduced in the Principles chapter.

`ImplementationStatus` — how real a method is:

Status	Meaning
`operational`	Implemented and runnable with bundled dependencies
`requires_configuration`	A real adapter that needs model weights or credentials supplied by the operator
`experimental`	Implemented but not validated
`placeholder`	An integration boundary only; never produces results

`RunStatus` — `completed`, `unavailable`, `insufficient_input`, `unsupported_input`, `failed`, `blocked_by_policy`.

`MethodInfo` — a frozen dataclass describing a method: `id`, `name`, `version`, `modalities`, `evidence_class`, `implementation`, `description`, `output_semantics`, `limitations`, `references`, `requires` (configuration keys or optional packages), `external_processing` (true if evidence leaves the deployment) and `validated_granularity` (for example `("document",)`).

`AnalysisInput` — what a method may read: modality, MIME type, the server-side path to the original, its SHA-256, the extracted text, the detected language, the extraction report, options (including validated calibration profiles from the registry) and whether external processing is allowed.

`MethodResult` — what a method returns: method id and version, evidence class, status, summary, structured output, optional score with `score_semantics`, `calibrated`, `flagged` (documented as "only set when a validated operating threshold exists"), uncertainty, warnings, limitations, segments, artefacts, the configuration used, duration and error.

`Segment` — a localised finding: start and end (character offsets for text, seconds for audio and video, or frame indices), unit, label, optional score and details.

`Artifact` — a derived visualisation: name, media type, bytes, a description and a mandatory `cannot_establish` statement. The docstring is explicit: an artefact is "stored as a processing derivative, never as a replacement for the original evidence".

14.2 The uniform run wrapper

Every method subclasses `AnalysisMethod` and implements `_run()`. Callers never call `_run()` directly; they call `run()`, which applies the same rules to every method:

```
def run(self, inp: AnalysisInput) -> MethodResult:
    """Run with uniform handling of unavailability, policy and failures."""
    started = time.monotonic()
    if not self.supports(inp):
        res = self.result(RunStatus.UNSUPPORTED_INPUT, f"{self.info.name} does not support
{inp.mime_type}.")
    elif self.info.external_processing and not inp.allow_external_processing:
        res = self.result(RunStatus.BLOCKED_BY_POLICY,
                          "External processing is disabled for this investigation; evidence was not
sent.")
    else:
        ok, reason = self.availability()
        if not ok:
            res = self.result(RunStatus.UNAVAILABLE, reason)
        else:
            try:
                res = self._run(inp)
            except Exception as exc:  # failures become explicit FAILED results
                res = self.result(RunStatus.FAILED, f"{self.info.name} failed: {type(exc).__name__}",
                                  error="".join(traceback.format_exception_only(type(exc),
exc)).strip()[:2000])
    res.duration_ms = int((time.monotonic() - started) * 1000)
    return res
```

The order of checks matters:

1. **Support** — a method asked to process a modality it does not handle returns `unsupported_input`.
2. **Policy** — a method that sends evidence off the machine returns `blocked_by_policy` *before* anything else happens, so a misconfigured provider cannot leak evidence.
3. **Availability** — `availability()` must be cheap and side-effect free: it checks for environment variables, model directories, optional packages or external tools, and returns a human-readable reason. That reason is what users see on the System Health page and in the case workspace (“No model configured (AEGIS_TEXT_CLASSIFIER_PATH is unset).”).
4. **Execution** — any exception becomes `failed` with the exception type in the summary and the message (truncated to 2,000 characters) in `error`.

The helper `self.result(status, summary, **kw)` fills in the method id, version and evidence class, and defaults `limitations` to the method’s declared limitations, so every result carries its limitations even if the method author forgets.

14.3 The method catalogue

`build_registry()` in `ml/aegis_ml/registry.py` instantiates every method. The registry currently holds 23 methods:

Method id	Name	Modality	Evidence class	Status
`text.descriptive_statistics`	Descriptive text statistics	text	forensic observation	operational
`text.supervised_classifier`	Supervised AI-text classifier (local transformer)	text	statistical inference	requires configuration
`text.token_statistics`	Language-model token statistics	text	statistical inference	requires configuration
`text.fast_detectgpt_analytic`	Fast-DetectGPT (analytic, single scoring model)	text	statistical inference	requires configuration
`text.binoculars`	Binoculars (observer/performer cross-perplexity)	text	statistical inference	requires configuration
`watermark.aegis_reference_text`	AEGIS reference text watermark verifier	text	verified watermark	operational
`watermark.synthid_text`	SynthID-Text watermark verification	text	verified watermark	placeholder
`image.metadata`	Image structure and metadata	image	forensic observation	operational
`image.frequency_spectrum`	Frequency-domain spectrum	image	forensic observation	operational
`image.noise_residual`	Noise residual and local consistency	image	forensic observation	operational
`image.error_level_analysis`	Error level analysis (JPEG)	image (JPEG only)	forensic observation	operational
`image.local_model`	Local image generation classifier (ONNX)	image	statistical inference	requires configuration
`provider.image_detector`	External image detection service	image	statistical inference	requires configuration (external)
`audio.signal_analysis`	Audio container, waveform and spectrogram	audio, video	forensic observation	operational
`audio.synthetic_speech_model`	Local synthetic-speech classifier (ONNX)	audio, video	statistical inference	requires configuration
`provider.audio_detector`	External synthetic-speech detection service	audio	statistical inference	requires configuration (external)
`audio.transcription`	Speech transcription (separate service)	audio, video	forensic observation	placeholder
`video.container`	Video container and stream inspection	video	forensic observation	operational
`video.temporal_consistency`	Sampled-frame temporal consistency	video	forensic observation	operational

Method id	Name	Modality	Evidence class	Status
`video.frame_model`	Per-frame image model (sampled frames)	video	statistical inference	requires configuration
`provider.video_detector`	External deepfake detection service	video	statistical inference	requires configuration (external)
`provenance.c2pa`	Content Credentials (C2PA) verification	image, audio, video, document	cryptographic provenance	operational
`metadata.declared_generation_claims`	Declared (unsigned) generation metadata	image, audio, video	declared metadata	operational

Out of the box, every method marked *operational* runs; every method marked *requires configuration* reports `unavailable` with a reason; and the placeholders report `unavailable` permanently. On a fresh install the System Health header therefore reads, for example, "12 methods unavailable" — which is the truthful state of a deployment without model weights or provider agreements.

14.4 Profiles and method selection

Profiles (described in the Analysis Pipeline chapter) group methods by investigation type. `methods_for()` filters a profile's methods by the evidence's modality, and treats documents as text so that a PDF or DOCX runs the text methods on its extracted text.

14.5 The per-frame video method

One method is defined in the registry module itself because it composes two others. `VideoFrameModelMethod` takes the configured local image model and applies it to video frames:

- frames are extracted with ffmpeg at 1 fps, at most 60 frames, from the first video stream;
- each frame is scored by the image model;
- each score is recorded as a one-second `Segment` labelled `frame_model_output`;
- the output records the number of frames scored and the maximum frame output.

Its summary deliberately says the outputs are "per-frame observations, not a video-level conclusion", and its limitations note that image models trained on still images degrade on compressed video frames and that no face recognition or identity inference is performed. It is available only when the image model is.

14.6 Running external tools safely

Several methods call ffmpeg or ffprobe. They do so through `run_tool()` in `ml/aegis_ml/providers.py`, which runs the executable with an argument list (never through a shell, so filenames cannot inject commands), with a timeout (120 s by default, longer for frame extraction), captured output, an output-size cap of 50 MB, and a `RuntimeError` carrying the last 400 bytes of standard error if the tool exits with a non-zero status. Callers also pass `-nostdin` to ffmpeg so it can never block waiting for input. Combined with the content sniffing and structural validation done at upload, this limits what a malicious media file can do to the worker.

CHAPTER 15

Text Methods

Text is the modality where demand for "AI detection" is highest and where the evidence is weakest. AEGIS's text pipeline therefore puts as much effort into describing the input honestly — how it was extracted, how long it is, what language it is in — as into scoring it. This chapter covers extraction, tokenisation, language and script identification, segmentation, descriptive statistics and the four model-based detectors. The watermark verifier is covered in its own chapter.

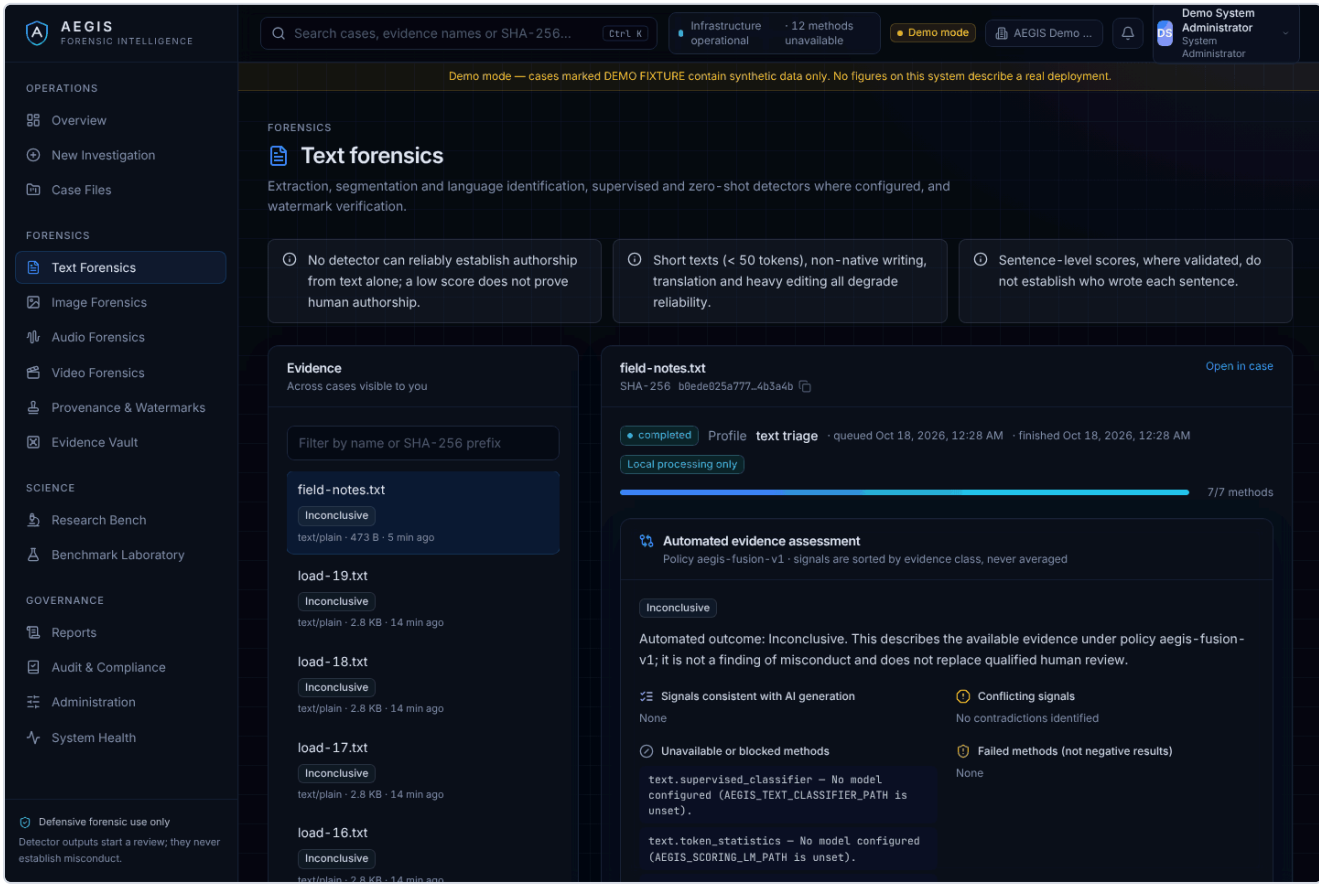


Figure 4. The Text forensics view lists text evidence across visible cases. The banner cards state the limits of text detection; the assessment panel shows the automated outcome, the conflicting signals and every unavailable method with its reason.

15.1 Extraction

`extract_text(path, mime)` in `ml/aegis_ml/text/extract.py` runs at upload time and produces an `ExtractionResult`. Its module docstring sets the security posture: *"Extracted text is untrusted input: it is only ever treated as data to analyse. Nothing in a document can change AEGIS behaviour."*

Format	Extractor	Warnings it can add
Plain text	Decode with BOM detection: UTF-8-SIG, UTF-16, UTF-8; falls back to UTF-8 with replacement characters	"File was not valid UTF-8; undecodable bytes were replaced."

Format	Extractor	Warnings it can add
PDF	`pypdf` embedded-text extraction, page by page; encrypted PDFs refused	"*N* of *M* pages contain no embedded text (possibly scanned). OCR is not applied automatically…"; "PDF extraction may reorder columns, drop ligatures or merge hyphenated words."
DOCX	`python-docx` paragraphs, then table rows joined with `|`	"Table cell text was appended after body paragraphs."

Every extraction then applies the same steps and checks:

1. **Truncation** at 2,000,000 characters, with a warning.
2. **Zero-width characters** (U+200B, U+200C, U+200D, U+2060) are counted and reported: they can be formatting artefacts or deliberate obfuscation intended to defeat detectors.
3. **Instruction-like phrases** are detected with a regular expression covering phrases such as "ignore previous instructions", "system prompt", "you are now an AI assistant" and "disregard the above". If found, a warning records that the text contains phrases addressed to AI systems and that AEGIS treats document content as data only. This is the threat model's control T10 for prompt injection: no language-model agent processes documents, and the phrase is recorded as an observation, *not* as evidence of authorship.
4. **Normalisation** (`normalise()`), recording each step that actually changed something:
   - Unicode NFC normalisation;
   - line endings converted to LF;
   - zero-width characters removed (including the BOM U+FEFF);
   - runs of spaces and tabs collapsed, and three or more newlines reduced to two.
5. **Language identification**, **token counting** and **sentence and paragraph counts**.
6. **Sufficiency**: if there are fewer than 50 tokens (`MIN_TOKENS_FOR_ANALYSIS`), a warning says statistical detectors are unreliable on short text and will report insufficient input, and `sufficient_for_detection` is false.

The result keeps both `original_text` (exactly as extracted) and `text` (normalised), the method name, source MIME type, an `ocr_derived` flag (always false — OCR is not implemented) and `extraction_confidence` (`direct` for embedded text). The normalised text becomes the `normalized_text` evidence version; the rest of the report is stored on the evidence item and shown to users.

15.2 Tokenisation

AEGIS needs a token count that does not depend on any particular language model, for length checks and descriptive statistics. `ml/aegis_ml/text/tokenize.py` defines a documented, dependency-free tokenizer called `aegis-regex-v1`: a token is either a run of word characters — letters, digits, combining marks (U+0300–U+036F) and the Indic blocks (U+0900–U+0DFF), plus zero-width joiners — or a single punctuation symbol.

Including the Indic combining-mark range matters. Python's `\w` does not match vowel signs and viramas (Unicode categories Mc and Mn), so a naive tokenizer would split Hindi words in the middle. The module states clearly that this is *not* the tokenizer of any language model; model-based detectors use their own tokenizers and report their own counts.

15.3 Language and script identification

`identify_language()` in `ml/aegis_ml/text/language.py` uses two layers:

1. **Unicode script proportions** — deterministic and exact. Each alphabetic character is assigned to Latin, Devanagari, Bengali, Gurmukhi, Gujarati, Oriya, Tamil, Telugu, Kannada, Malayalam, Arabic, Cyrillic, CJK or Other, and the proportions are reported.
2. **`langdetect`** — an n-gram naive Bayes classifier (Nakatani Shuyo) run with a fixed seed (0) on the first 20,000 characters, returning up to three candidate languages with probabilities. Among Indian languages it supports Hindi, Bengali, Gujarati, Kannada, Malayalam, Marathi, Nepali, Punjabi, Tamil, Telugu and Urdu.

If the text is shorter than 20 characters, no language is reported. If more than one script accounts for over 15% of the letters, a note warns that "Mixed scripts detected; language-specific detector validity may not apply." — important for code-mixed text such as Hinglish.

The module docstring makes the key point: *identifying a language does not mean any detector has been validated for it.* Detector support is tracked separately per model through `languages_evaluated`, and calibration is withdrawn for languages outside it.

15.4 Segmentation

`ml/aegis_ml/text/segment.py` splits text into paragraphs (runs of non-empty lines separated by blank lines) and sentences, keeping character offsets so that findings can be highlighted in the original. Sentence boundaries are punctuation runs — including the Devanagari danda `।` and double danda `॥` used in Hindi, Marathi, Nepali and Sanskrit — optionally followed by closing quotes or brackets, then whitespace. A boundary after a common abbreviation (Mr, Mrs, Dr, Prof, St, vs, etc., e.g., i.e., Fig, No, Vol) is skipped.

15.5 Descriptive statistics

`text.descriptive_statistics` is operational on every text. It is a *forensic observation*, not a detector, and its docstring is emphatic: *"They are NOT an AI-detection signal and AEGIS never converts them into a score."* The measurements help reviewers judge whether the text is suitable for analysis and whether it is out of distribution for a detector.

Statistic	Definition
characters, tokens, words, sentences, paragraphs	Counts (`aegis-regex-v1`)
`type_token_ratio`	Distinct words ÷ words
`mattr_50`	Moving-average type-token ratio over 50-word windows — less dependent on length than raw TTR
`hapax_ratio`	Words occurring exactly once ÷ words
`mean_sentence_words` , `sentence_words_stdev`	Mean and population standard deviation of words per sentence
`sentence_length_cv`	Coefficient of variation of sentence length (sometimes called "burstiness")
`mean_word_length`	Mean characters per word
`zlib_compression_ratio`	Compressed size at level 9 ÷ raw UTF-8 size

Statistic	Definition
`punctuation_per_100_tokens`	Punctuation tokens per 100 tokens
`unigram_entropy_bits`	Shannon entropy of the word distribution, $H = -\sum_w p_w \log_2 p_w$

Each paragraph (up to 500) is returned as a segment with its word count, so the interface can show the document structure. The limitations state that these features vary with genre, proficiency, language and editing, and that low sentence-length variation or lexical diversity occurs naturally in many human texts.

15.6 Model-based detectors

Four detectors live in `ml/aegis_ml/text/model_detectors.py`. They are real implementations, but AEGIS ships no weights and never downloads models, so out of the box all four report `unavailable`. When configured, they load models with Hugging Face `transformers` using `local_files_only=True`, run on CPU or CUDA (`AEGIS_INFERENCE_DEVICE`), and cache up to four loaded models.

Shared behaviour

All four share the following rules (implemented in `_TextModelMethod` and helper functions):

- **Availability** requires the model directory variable to be set, the directory to exist, and `torch` and `transformers` to be installed (`uv sync --extra transformers`).
- **Minimum length.** Texts with fewer than 50 tokens return `insufficient_input`.
- **Model identity.** Every result records the model's `config.json` name, type and architecture and a **partial weight fingerprint**: the SHA-256 of the first MiB of the first weight file, truncated to 32 hex digits. The documentation calls it partial so nobody mistakes it for a full checksum; it is fast and sufficient to tell whether the weights changed between runs.
- **Calibration lookup.** A calibration profile from the model registry takes precedence; otherwise an `aegis_calibration.json` file in the model directory is used, but only if it contains `"validated": true`.
- **Applying calibration** (`_apply_calibration`): for Platt scaling, the calibrated probability is

$$p = \frac{1}{1 + e^{-(as + b)}}$$

  where s is the raw score and a, b are the fitted parameters. If a threshold is present, `flagged` is $s \geq t$ when higher scores indicate AI, or $s \leq t$ when lower scores do.
- **Language check** (`_language_ok`): if the profile lists evaluated languages and the evidence language is not among them, the run carries an "Out-of-distribution" warning and **both the probability and the flag are discarded**.
- **Summaries** state explicitly whether the output was interpreted: "no validated threshold, so the output is not interpreted", "exceeds the validated threshold (consistent with AI generation)", or "below the validated threshold. This does not prove human authorship."

Supervised classifier (`text.supervised_classifier`)

Configuration: `AEGIS_TEXT_CLASSIFIER_PATH` (a local sequence-classification model) and `AEGIS_TEXT_CLASSIFIER_AI_LABEL` (the label name that means "AI").

The method refuses to guess which output means AI. If the configured label does not match one of the model's labels, the run **fails** with the list of model labels in the output: "refusing to guess which output is 'AI'". This prevents a silent inversion where a model's "Human" probability is reported as an AI score.

The text is tokenised with the model's tokenizer and split into chunks of 510 tokens (512 minus special tokens), at most 16 chunks. Each chunk is scored as the softmax probability of the AI label, and the document score is the mean of the chunk scores. The output includes every chunk score, and the uncertainty field reports the range of chunk scores. If more than 16 chunks exist, a warning says only the first 16 were scored.

Paragraph-level scores are produced only if the calibration profile lists `paragraph` in `validated_granularity`. Otherwise, for multi-paragraph texts, a warning says "Segment-level scores withheld: the model is not validated at paragraph granularity." This prevents the common misuse of highlighting individual sentences as "AI-written" on the basis of a document-level model.

Token statistics (`text.token_statistics`)

Configuration: `AEGIS_SCORING_LM_PATH` (a local causal language model).

This method computes features in the spirit of GLTR (Gehrmann et al., 2019) over the first window of up to 512 tokens. For each position i with target token x_i and model distribution $p(\cdot \mid x_{<i})$:

Feature	Formula
Mean log-likelihood	$\bar{\ell} = \frac{1}{n}\sum_i \log p(x_i \mid x_{<i})$
Perplexity	$\exp(-\bar{\ell})$
Log-likelihood standard deviation	standard deviation of $\log p(x_i \mid x_{<i})$
Mean log-rank	mean of $\log r_i$, where r_i is the rank of x_i in the model's distribution (1 = most likely)
Fraction top-1, top-10	fraction of positions with $r_i = 1$ and $r_i \le 10$

It sets **no score and no flag**. Its output semantics: "Feature values under the configured scoring model. None of them is a detection score." Formulaic human writing — legal, technical or non-native — can be highly predictable, which is why AEGIS refuses to turn perplexity into a verdict.

Fast-DetectGPT (`text.fast_detectgpt_analytic`)

Configuration: `AEGIS_SCORING_LM_PATH`.

Fast-DetectGPT (Bao et al., 2024) measures *conditional probability curvature*: whether the observed tokens are more likely under the model than tokens the model itself would typically sample. AEGIS uses the analytic form. With $\log p_i(\cdot)$ the log-distribution at position i:

$$\mu_i = \sum_v p_i(v)\,\log p_i(v), \qquad \sigma_i^2 = \sum_v p_i(v)\,(\log p_i(v))^2 - \mu_i^2$$

$$\text{criterion} = \frac{\sum_i \log p_i(x_i) - \sum_i \mu_i}{\sqrt{\sum_i \sigma_i^2}}$$

Larger values are more consistent with model-generated text. The code is a direct transcription:

```
p = lp.exp()
ll = lp.gather(1, targets[:, None])[:, 0].sum()
mu = (p * lp).sum(dim=1)
var = (p * lp.pow(2)).sum(dim=1) - mu.pow(2)
crit = float((ll - mu.sum()) / var.sum().sqrt())
```

The method uses **one model for both sampling and scoring**. Its limitations say plainly that the paper's strongest results use specific model pairs, that this configuration is not a reproduction of those results, and that performance depends on how well the scoring model matches the unknown source generator.

Binoculars (`text.binoculars`)

Configuration: `AEGIS_BINOCULARS_OBSERVER_PATH` and `AEGIS_BINOCULARS_PERFORMER_PATH` (two causal language models sharing a tokenizer).

Binoculars (Hans et al., 2024) compares how surprising the text is to one model with how surprising one model's predictions are to the other. Following the reference implementation, with performer log-probabilities $\log q_i$ and observer probabilities p_i:

$$\log \text{PPL} = -\frac{1}{n}\sum_i \log q_i(x_i), \qquad \log \text{X-PPL} = -\frac{1}{n}\sum_i \sum_v p_i(v)\, \log q_i(v)$$

$$B = \frac{\log \text{PPL}}{\log \text{X-PPL}}$$

Lower values are more consistent with model-generated text, so the calibration direction defaults to `lower_is_ai`. If the two tokenizers' vocabularies differ, the run fails ("Binoculars requires a shared vocabulary"). The published threshold applies only to the published Falcon-7B pair, and AEGIS does not use it.

15.7 Why the text detectors are unavailable by default

It would be easy to ship a default classifier and show a percentage on every document. The project chose not to, for reasons spelled out in the model cards and limitations documents:

- no third-party model was downloaded, evaluated or validated in the build environment;
- detectors generalise poorly to generators, domains and languages absent from their training;
- without local validation, a score has no known false-positive rate for the population being examined.

An operator who wants model-backed detection follows the procedure in the Operations chapter: obtain weights legitimately, record their checksums, configure the paths, benchmark the method on representative local data, calibrate it on a validation split, and only then mark it validated in the model registry.

CHAPTER 16

Image, Audio and Video Methods

The media methods are mostly *forensic observations*: deterministic signal-processing measurements and visualisations that an analyst interprets. None of them is an AI-generation classifier, and each one states what it cannot establish. Learned classifiers for images, speech and video frames are available as adapters that run once an operator supplies ONNX weights. This chapter describes every media method and the maths behind it.

16.1 Images

All image methods are in `ml/aegis_ml/image/analysis.py`. Two safety measures apply to every image:

- `Image.MAX_IMAGE_PIXELS = 80_000_000` — Pillow's decompression-bomb guard. Pillow warns above this size and raises an error above twice this size, so a tiny file that decompresses to billions of pixels cannot exhaust memory.
- `open_image()` first calls Pillow's `verify()` (a structural check), then reopens and fully loads the image. A truncated or malformed file fails here rather than half-way through an analysis.

For the spectral and residual methods, the image is converted to greyscale and downsampled with Lanczos filtering so that its longest side is at most 1,024 pixels (`ANALYSIS_MAX_SIDE`). This bounds the cost of analysis and is recorded in each result's configuration.

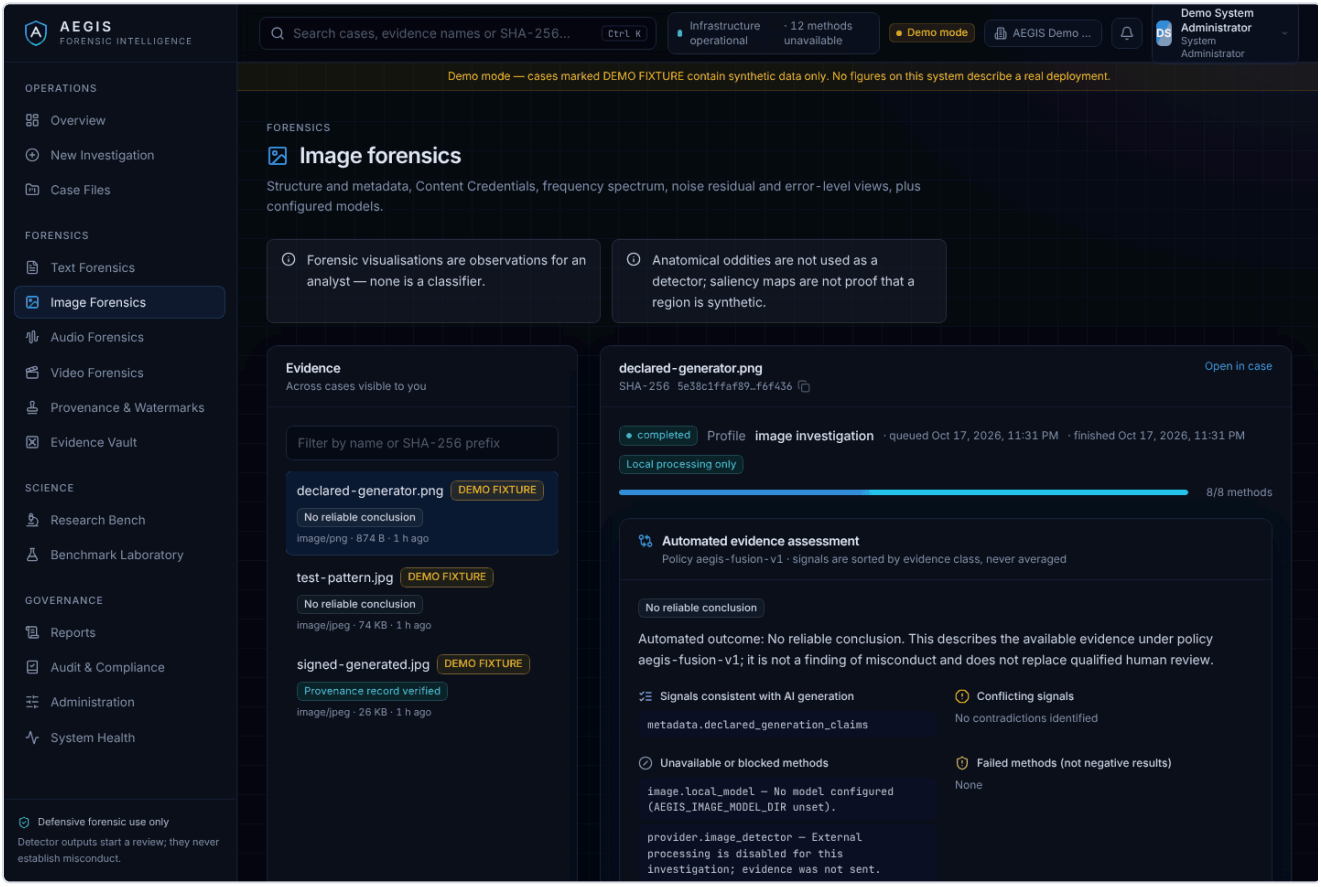


Figure 5. Image forensics: provenance, metadata and visualisations, each stating what it cannot establish.

Image structure and metadata (`image.metadata`)

Reports the format, colour mode, width, height, megapixels, number of frames, orientation, ICC colour-profile description, alpha channel, EXIF status and fields, progressive encoding, DPI, XMP presence, and — for JPEG files — an **estimated quality**.

The quality estimate inverts the standard IJG scaling of the JPEG luminance quantisation table. With Q the file's luminance table and S the standard Annex K table, the scale factor is the mean ratio $s = 100 \cdot \bar{Q}/\bar{S}$, and the quality is

$$q = \begin{cases} (200 - s)/2 & s \le 100 \\ 5000/s & s > 100 \end{cases}$$

clipped to 1–100. The method label explains that this is an estimate and that non-IJG encoders deviate from it.

Privacy. Sensitive EXIF fields — `GPSInfo` , `BodySerialNumber` , `CameraOwnerName` , `LensSerialNumber` , `ImageUniqueID` — are shown as "[present — redacted]", and a warning notes their presence. Maker notes and unknown tags are skipped; byte values are truncated to 64 bytes and strings to 300 characters. The limitations state that missing metadata is common (social media strips it) and indicates nothing, and that present metadata can be forged.

Frequency-domain spectrum (`image.frequency_spectrum`)

Generative upsampling layers can leave periodic artefacts that show up as peaks in the Fourier spectrum (Durall et al., 2020; Frank et al., 2020). The method:

1. subtracts the mean and multiplies by a 2-D Hann window $w(y, x) = h(y)\,h(x)$ to reduce edge leakage;
2. computes the centred magnitude spectrum $|F| = |\mathrm{fftshift}(\mathrm{FFT2}(g \cdot w))|$ and its log $\log(1 + |F|)$;
3. computes the **radial power profile** — mean power $|F|^2$ in rings of integer radius from the centre — and fits a line in log–log space to obtain the **radial power slope** (natural images typically show a power-law fall-off);
4. computes the **high-frequency energy fraction**: the share of radial power at normalised frequencies above 0.5;
5. computes a **periodic-peak strength**: the log spectrum minus a 9×9 median-filtered version of itself, maximised over points away from the centre (beyond 15% of the maximum radius) and away from the axes.

The artefact `frequency_spectrum.png` shows the centred log-magnitude spectrum ("centre = low frequencies, edges = high frequencies"). Its `cannot_establish` text: *"Cannot establish that an image is synthetic or authentic; periodic patterns have benign causes."* Resampling, compression grids, halftone printing and textures all produce peaks, and their absence does not indicate an authentic camera image.

Noise residual and local consistency (`image.noise_residual`)

A high-pass residual is computed as the image minus its 3×3 median filter. The residual is divided into 32×32-pixel blocks, and the standard deviation of each block is computed. The output reports:

- the overall residual standard deviation;
- the block grid size;
- the coefficient of variation of the block standard deviations;
- **outlier blocks**: blocks whose standard deviation differs from the median by more than four times the median absolute deviation (MAD), and their fraction.

Two artefacts are produced: the amplified absolute residual (bright = strong fine-grain detail) and a heat map of block standard deviations. Images smaller than 64×64 pixels return `insufficient_input`. The limitations stress that residual energy varies with texture, focus, denoising and compression, that inconsistency is not proof of splicing, and that this is **not** a PRNU camera fingerprint — no device attribution is attempted.

Error level analysis (`image.error_level_analysis`)

ELA (Krawetz, 2007) re-compresses a JPEG at a known quality and looks at how much each pixel changes: regions with a different compression history may re-compress differently. The method supports only `image/jpeg`. It downsamples to 1,024 px, re-saves at quality 90, and computes the per-pixel maximum channel difference. It reports the mean and 99th-percentile error level and produces a heat map. The limitations are blunt: *"ELA is widely misinterpreted: edges, text and saturated colours always stand out."* The result is qualitative.

Visualisation helpers

`ml/aegis_ml/visual.py` renders artefacts without matplotlib (keeping the API image small). `heat_png()` normalises values between the 1st and 99.5th percentiles and applies a perceptually ordered colour ramp from midnight blue through blue and cyan to near-white. `waveform_png()` draws a peak envelope for audio.

Local image classifier (`image.local_model`)

`LocalOnnxClassifier` (in `ml/aegis_ml/providers.py`) runs an operator-supplied ONNX model. The directory named by `AEGIS_IMAGE_MODEL_DIR` must contain `model.onnx` and `aegis_model.json`:

```
{"name": "...", "version": "...", "license": "...", "positive_index": 1,
 "input": {"kind": "image", "size": 224, "mean": [0.485, 0.456, 0.406], "std": [0.229, 0.224, 0.225]},
 "calibration": {"validated": true, "threshold": 0.9}}
```

The image is resized to the given size, normalised with the given mean and standard deviation, transposed to channel-first order and run on the CPU execution provider. The output is soft-maxed and the positive-class probability is the score. `flagged` is set only if the card contains a calibration block marked validated with a threshold. Suggested model families in the documentation are CNNDetection (Wang et al., 2020) and UnivFD (Ojha et al., 2023), exported to ONNX by the operator.

16.2 Audio

All audio processing is in `ml/aegis_ml/audio/analysis.py`. It depends on ffmpeg and ffprobe; without them, every audio and video method reports `unavailable` ("ffmpeg/ffprobe are not installed on this worker.").

- `ffprobe(path)` runs `ffprobe -v error -print_format json -show_format -show_streams` with a 60-second timeout.
- `decode_pcm(path, sample_rate)` runs ffmpeg to decode the first audio stream to mono 32-bit float PCM at the requested sample rate, **limited to the first 900 seconds** (`MAX_AUDIO_SECONDS`), with a 300-second timeout. Uploads of up to three hours are accepted, but only the first fifteen minutes are analysed, and the result says so.

Audio container, waveform and spectrogram (`audio.signal_analysis`)

This method runs on audio files and on the audio track of videos. It decodes at the native sample rate (capped at 48 kHz) and computes:

Measurement	Definition
Peak and RMS level	$20 \log_{10}$ of the peak absolute sample and of $\sqrt{\bar{x^2}}$, in dBFS
DC offset	Mean sample value
Clipping fraction	Share of samples with $\lvert x \rvert \geq 0.999$
Silence fraction	Share of 20 ms frames with RMS below −50 dBFS
Effective bandwidth (99%)	The frequency below which 99% of spectral energy lies
Mean spectral flatness	Mean over time of the ratio of geometric to arithmetic mean of the power spectrum

The spectrogram uses a 1,024-point Hann-windowed short-time Fourier transform with 768-sample overlap (`scipy.signal.spectrogram`). SciPy's signal module is imported only at this point: the code comments that it "dominates API start-up time otherwise", which mattered for the 512 MB free hosting tier.

Warnings are added for a native sample rate below 16 kHz, more than 0.1% clipped samples, recordings shorter than 2 seconds, truncation at 900 seconds, and — importantly — when 99% of the energy lies below 60% of the Nyquist frequency: *"consistent with band-limiting, resampling or transcoding. Many benign causes exist."* Recordings shorter than 0.5 s return `insufficient_input`.

Runs of non-silent frames lasting at least 0.3 s are returned as `active_signal` segments (up to 500) for the timeline. Two artefacts are produced: a peak-normalised waveform and a log-power spectrogram with low frequencies at the bottom. The output also includes codec, sample rates, channels and layout, bit rate, container and container tags.

The limitations state that transcoding, telephony channels and noise reduction alter every measured quantity, and that **no speaker identity, nationality, ethnicity or intent is inferred**.

Local synthetic-speech classifier (`audio.synthetic_speech_model`)

The same ONNX adapter as for images, configured by `AEGIS_SPEECH_MODEL_DIR`, with an audio input specification:

```
{"name": "my-speech-cm", "version": "2026.1", "license": "...", "positive_index": 1,
 "input": {"kind": "audio", "sample_rate": 16000, "samples": 64600}}
```

Audio is decoded at the given rate, truncated or zero-padded to the given number of samples, and scored. The documentation suggests AASIST-class models (Jung et al., 2022) from the ASVspoof challenge series. It runs on audio files and on video sound tracks.

Transcription placeholder (`audio.transcription`)

A boundary for an optional transcription engine. Transcripts would be labelled separately from authenticity findings, because transcription errors are not authenticity findings. No engine is bundled, so it is always `unavailable`.

16.3 Video

Video methods are in `ml/aegis_ml/video/analysis.py`. The module docstring states that no facial recognition or identity inference is performed.

Video container and stream inspection (`video.container`)

Using ffprobe, the method lists every stream with type, codec, profile, dimensions, pixel format, average and nominal frame rates, frame count, sample rate, channels, start time, duration and tags, plus the container format, duration, bit rate and tags. It then records **structural findings**:

- audio and video streams starting more than 200 ms apart (container level);
- audio and video stream durations differing by more than 1 s;
- average and nominal frame rates differing by more than 0.5 fps ("variable frame rate or edited timeline").

A file with no video stream returns `unsupported_input`; a video without audio gets a warning that audio methods will be skipped. The limitations note that platforms rewrite most container metadata when they re-encode, and that alignment is container-level only — **lip-sync analysis is not implemented**.

Sampled-frame temporal consistency (`video.temporal_consistency`)

Frames are sampled with ffmpeg at a configurable rate (`sample_fps`, default 2, clamped to 0.1–10), scaled to 320 px wide, up to 240 frames (`MAX_SAMPLED_FRAMES`). For each consecutive pair of greyscale frames the mean absolute difference d_i is computed, and for each frame the standard deviation of its 3×3 median residual r_i. Timeline observations are then derived with robust statistics:

Label	Rule
`duplicate_or_frozen_frame`	$d_i < 0.05$
`abrupt_discontinuity`	$d_i > \text{median}(d) + 8\,\text{MAD}(d)$ and $d_i > 8$
`noise_level_shift`	$\lvert r_i - \text{median}(r)\rvert / \text{median}(r) > 0.6$

Keyframe times are listed with ffprobe (`-skip_frame nokey`, first 600 seconds, up to 2,000 keyframes). The output includes the full inter-frame change and residual series. Two artefacts are produced: a **contact sheet** of twelve evenly spaced frames ("Frames are for orientation; visual impressions are not findings.") and an inter-frame change strip ("Cuts and motion are normal; discontinuities are prompts for review."). Fewer than two sampled frames gives `insufficient_input`.

Its output semantics are unambiguous: *"A timeline of observations for an analyst. Not a deepfake detector."* Scene cuts, camera motion and compression cause discontinuities in authentic footage.

The performance documentation records that, for the demo seed's four-second 320×240 test video, `video.container` took 49 ms and `video.temporal_consistency` took 386 ms on a 4-vCPU development container.

Per-frame image model (`video.frame_model`)

Described in the framework chapter: the configured image model is applied to frames at 1 fps (at most 60), producing per-frame segments, never a video-level verdict.

16.4 External detection providers

`HttpDetectorProvider` connects a self-hosted model server or a vendor API behind an adapter the operator controls. Three instances are registered: `provider.image_detector`, `provider.audio_detector` and `provider.video_detector`. Each is configured server-side:

```
AEGIS_PROVIDER_<ID>_URL=https://detector.internal/v1/score   # HTTPS required
AEGIS_PROVIDER_<ID>_TOKEN=...                                # sent as Bearer; never logged
AEGIS_PROVIDER_<ID>_TIMEOUT=60
```

Because evidence leaves the deployment, the method's `MethodInfo` has `external_processing=True`, so the run wrapper blocks it unless the deployment, the organisation **and** the case all permit external processing. A non-HTTPS URL makes the method unavailable (unless an explicit insecure-provider development flag is set).

The request is a multipart POST with the original bytes in a `file` field and the header `X-AEGIS-Evidence-SHA256`. The response must be HTTP 200 with JSON that passes strict validation (`validate_provider_response()`):

Field	Rule
`score`	Required finite number; booleans are rejected (in Python `True` is an `int`, so this check is explicit)
`model_version`	Required, 1–128 characters
`score_semantics`	Optional string, truncated to 300 characters
`request_id`	Optional, `[A-Za-z0-9._:-]{1,128}`
`calibrated_probability`	Optional, in [0, 1]; stored as **vendor-reported** and never treated as AEGIS calibration

Any other status code, malformed JSON or rule violation makes the run `failed` with the reason; no score is stored. Even a valid provider score is recorded with the summary "not interpreted without a validated threshold" until the provider has been benchmarked locally and a validated calibration profile attached in the model registry. The threat model (T8) notes the residual risk: a provider can still return plausible but wrong scores, which is why local benchmarking is required before relying on one.

CHAPTER 17

Provenance and Watermark Verification

Two kinds of evidence in AEGIS rest on cryptography or keyed statistics rather than on resemblance: **signed provenance** (C2PA Content Credentials) and **watermark verification**. They answer narrower questions than detectors, but they answer them much more reliably. This chapter covers the C2PA verifier, the declared-metadata method, the AEGIS reference watermark and the SynthID-Text boundary.

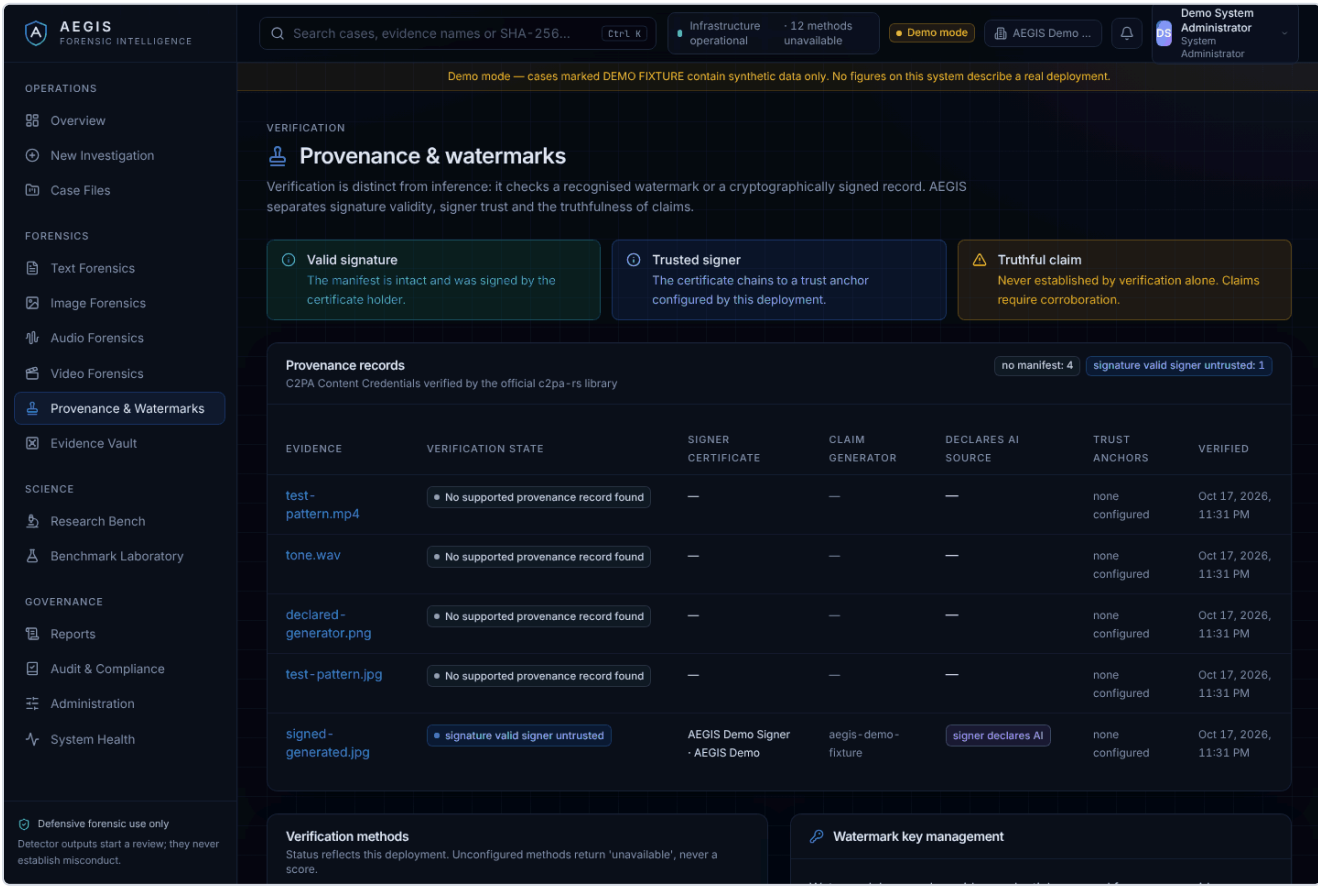


Figure 6. Provenance & watermarks: signature validity, signer trust and truth of claims are kept apart.

17.1 C2PA Content Credentials

The Coalition for Content Provenance and Authenticity (C2PA) defines a way to embed a signed *manifest* in a media file. A manifest contains *assertions* (for example, which actions were performed and with what software, and the IPTC *digital source type*) and *ingredients* (the files it was derived from), and is signed with an X.509 certificate. Hard bindings hash the content so that changes after signing are detectable.

AEGIS verifies manifests with the official `c2pa-python` bindings (version 0.38, wrapping the Rust `c2pa-rs` library 0.91) in `ml/aegis_ml/provenance/c2pa_verifier.py`. The module docstring defines the three things every result keeps distinct:

1. **signature and integrity validity** — the manifest is intact and its signature verifies;
2. **signer trust** — the signing certificate chains to a configured trust anchor;
3. **truthfulness of the claims** — *not* established by either of the above.

Verification states

State	Meaning	Summary shown
`no_manifest`	No C2PA manifest found	"No supported provenance record found."
`signature_valid_signer_trusted`	Signature and hashes valid; signer chains to a configured anchor	"Verification succeeded for the supported provenance record; the signer chains to a configured trust anchor."
`signature_valid_signer_untrusted`	Signature and hashes valid; signer not in the trust list	"The manifest's signature and hashes validate, but the signer is not trusted by this deployment."
`verification_failed`	A manifest exists but is broken or fails validation	"Provenance verification failed (… failure codes…)."
`unsupported_format`	The library cannot read this file structure	— (`unsupported_input`)

Supported formats are JPEG, PNG, WebP, WAV, MP3, M4A, MP4 and PDF.

How the verifier works

1. **Trust context.** If `AEGIS_C2PA_TRUST_ANCHORS` names a PEM bundle, a `c2pa.Context` is created with those anchors and trust verification enabled; otherwise there is no trust context, and valid signatures are reported as "signer untrusted". The anchor file name is recorded in the run configuration.
2. **No network.** Remote (cloud) manifests are **not fetched**; the configuration records `"remote_manifests": "not fetched"`. Verification never makes a network request, which keeps the worker isolated and avoids leaking which evidence is being examined.
3. **Reading.** `c2pa.Reader.try_create(mime, stream, context)` returns `None` when no manifest exists. A `NotSupported` error means the format cannot be read. Any other C2PA error means a manifest exists but cannot be parsed — reported as `verification_failed`, not as a crash.
4. **Classifying.** The library's validation state (`Trusted`, `Valid` or invalid) is combined with the failure codes of the active manifest. The code `signingCredential.untrusted` is not an integrity failure, so it is excluded when deciding between "valid" and "failed"; any other failure code means verification failed.
5. **Summarising** (`summarise_manifest_store()`): the number of manifests, the active manifest label and title, the claim generator(s), signature algorithm, issuer, common name and signing time, every action (action, digital source type, software agent, time), every ingredient (title, relationship, format, whether it has its own manifest), the success, informational and failure codes, and whether the manifest is embedded.

Declared source types

The IPTC *digital source type* vocabulary distinguishes, among other values, captured media from algorithmic media. AEGIS groups them:

Group	Values
AI source types	`trainedAlgorithmicMedia`, `compositeWithTrainedAlgorithmicMedia`, `algorithmicMedia`, `compositeSynthetic`
Capture source types	`digitalCapture`, `negativeFilm`, `positiveFilm`, `print`, `minorHumanEdits`

The summary sets `declared_ai_source` and `declared_capture_source`. When a valid manifest declares an AI source type, a warning says: *"The manifest declares AI-generated content (IPTC digital source type). This is the signer's claim."* The fusion policy uses these flags to detect contradictions — for example a valid manifest claiming a camera capture while statistical detectors flag the image.

Limitations

- "No supported provenance record found" is not evidence of AI generation or of human creation.
- Signer trust depends entirely on the configured trust anchors.
- Manifests are frequently stripped by re-encoding and by platforms.
- Remote manifests are not fetched.

The verifier is tested with valid-untrusted, valid-trusted, tampered (data-hash mismatch) and absent manifests on JPEG files generated by the test fixtures with a locally created test certificate authority.

17.2 Declared (unsigned) generation metadata

`metadata.declared_generation_claims` (`DeclaredMetadataMethod`) looks for unsigned claims about how a file was made. These are listed as a separate evidence class because anyone can add, copy or remove them.

- **Images:** PNG text chunks named `parameters`, `prompt`, `workflow`, `software`, `comment`, `description`, `source` or XMP (Stable Diffusion front-ends, for example, write the generation parameters into a `parameters` chunk); XMP `DigitalSourceType`; EXIF `Software`, `ImageDescription`, `Artist`, `UserComment` and `XPComment`.
- **Audio and video:** container and stream tags whose names contain `encoder`, `comment`, `software`, `description`, `title` or `handler`.

Values are compared case-insensitively with a list of known generator markers: Stable Diffusion, Midjourney, DALL-E, Firefly, Imagen, NovelAI, ComfyUI, AUTOMATIC1111, InvokeAI, Flux, ElevenLabs, Sora, Runway, Pika and Leonardo. IPTC AI source types are also detected. The summary is one of:

- "Unsigned metadata declares generation by: …"
- "*N* unsigned tool/description fields found; none names a known generator."
- "No declared generation metadata found (common; indicates nothing)."

The fusion policy lists declared AI markers among agreeing indicators but adds the uncertainty reason that such declarations "are unverified claims that can be added, copied or removed; they are listed but not treated as verification".

17.3 The AEGIS reference text watermark

Why it exists

A text watermark is a hidden statistical pattern that a generator embeds while producing text, which a key holder can later test for. Commercial schemes (for example SynthID-Text) require provider keys and detector weights that are not public. Rather than ship an empty interface, AEGIS implements the published green-list scheme of Kirchenbauer et al. (2023) with its own keyed generator (ADR-10). This exercises the entire verification pathway — keys, exact p-values, minimum-length checks, robustness benchmarks — end to end. It is labelled everywhere as able to verify **only** text produced by the AEGIS reference generator under the deployment's key.

The scheme

The implementation is split between `research/aegis_research/watermark.py` (the scheme) and `ml/aegis_ml/text/reference_watermark.py` (the verifier method and a demo generator).

Green lists. For each context — the previous `context_width` tokens, here one token — a keyed pseudo-random permutation of the vocabulary is generated, and the first $\gamma|V|$ tokens form the *green list*:

```
digest = hmac.new(self._key, repr(ctx).encode(), hashlib.sha256).digest()
rng = np.random.default_rng(int.from_bytes(digest[:8], "big"))
perm = rng.permutation(self.config.vocab_size)
mask = np.zeros(self.config.vocab_size, dtype=bool)
mask[perm[: self._green_size]] = True
```

Using HMAC-SHA256 with the secret key means that without the key nobody can tell which tokens are green, and that the same key and context always give the same list.

Generation. At each step the generator adds a bias δ to the logits of green tokens before sampling, nudging the text towards green tokens: `logits + delta * green_mask(context)`.

Detection. Given T scored tokens of which g are green, under the null hypothesis "not watermarked with this key" each token is green with probability γ, so

$$z = \frac{g - \gamma T}{\sqrt{T\,\gamma\,(1-\gamma)}}$$

AEGIS reports the z-score and **two** p-values: the normal approximation $1 - \Phi(z)$ and the **exact binomial** tail $P(X \geq g)$ for $X \sim \mathrm{Binomial}(T, \gamma)$. The watermark is reported as detected when $z \geq 4$.

Deduplication. Natural text repeats phrases, and repeated (context, token) pairs are not independent, which inflates z. With `dedupe=True` (used by the verifier) each distinct pair is scored once, restoring the independence assumption behind the p-value.

Minimum length. Even if every token is green, $z = \sqrt{T(1-\gamma)/\gamma}$, so reaching the threshold requires

$$T_{\min} = \left\lceil z_{\mathrm{thr}}^2 \,\frac{\gamma}{1-\gamma} \right\rceil$$

For $\gamma = 0.25$ and $z_{\mathrm{thr}} = 4$ this is $\lceil 16/3 \rceil = 6$ scored tokens in the best case; in practice the verifier marks results as insufficient below this length and returns `insufficient_input`.

Exact false-positive rate. `exact_false_positive_rate(n, γ, z)` computes the true probability that unwatermarked text crosses the threshold:

$$k_{\min} = \lceil \gamma n + z\sqrt{n\gamma(1-\gamma)} \rceil, \qquad \mathrm{FPR} = P(X \geq k_{\min}),\ X \sim \mathrm{Binomial}(n, \gamma)$$

Experiments 7 and 9 in the Experiments chapter use this to show that the normal approximation understates the false-positive rate of short texts by up to an order of magnitude — which is why AEGIS reports the exact p-value.

Configuration in the verifier

Parameter	Value
Vocabulary	2,048 public pseudo-words (consonant–vowel syllable pairs, permuted with a fixed public seed)

Parameter	Value
γ	0.25
δ	2.0
Context width	1 token
Threshold	$z \geq 4$
Deduplication	On
Key	`AEGIS_REFERENCE_WATERMARK_KEY`, at least 16 bytes

The vocabulary is deliberately public: security rests on the key alone. Words outside the vocabulary are mapped into the id space with a public SHA-256 hash so that any text can be tokenised. The key never leaves the server; outputs show only a **key fingerprint**, the first 16 hex digits of `SHA-256("aegis-wm-fp:" + key)`, which identifies the key without revealing it.

Results and wording

The verifier's output includes the scored and green token counts, γ, the z-score, both p-values, the threshold, whether it was detected, whether the length was sufficient, the key fingerprint and the vocabulary name. The summaries are carefully worded:

- "Reference watermark detected (z=..., exact p=...)."
- "Reference watermark not detected (z=...). This does not indicate human authorship."

The method's limitations: it verifies only the AEGIS reference watermark and cannot detect watermarks of commercial models; absence of this watermark says nothing about whether text is AI-generated or human-written; detection weakens with editing, paraphrasing and short length.

The demo generator

`generate_reference_text(key, n_tokens, seed)` produces pseudo-word text from the toy `SyntheticLM` (a first-order Markov model with a Zipf profile), with or without the watermark, grouped into twelve-word "sentences". Its output is meaningless by design and is used only for the demo seed and the simulated benchmark. In the demo, `reference-watermarked.txt` is generated this way so that the verifier has something genuine to detect.

17.4 SynthID-Text boundary

`watermark.synthid_text` is a placeholder for an authorised SynthID-Text integration (Dathathri et al., 2024). Verification requires the provider's watermarking configuration (keys) and a trained Bayesian detector that are not public. The method is permanently `unavailable` with the reason "No authorised SynthID-Text verification integration is configured." and lists its requirements: a provider agreement, watermarking keys and detector weights. It exists so that the plugin interface, the registry entry and the user interface are ready when such an agreement exists, without ever pretending to verify anything in the meantime.

CHAPTER 18

Evidence Fusion: Policy aegis-fusion-v1

After every method in a job has finished, AEGIS produces an *automated assessment*. This is the step where most detection products combine scores into a single number. AEGIS deliberately does not. Its fusion policy, `aegis-fusion-v1` in `ml/aegis_ml/fusion.py`, sorts the results by evidence class, identifies what agrees, what conflicts, what is missing and what failed, and assigns descriptive outcome categories. This chapter explains the policy rule by rule.

18.1 Why not combine scores?

The module docstring gives the reason:

> *Combination of statistical signals into a single probability is deliberately NOT performed: no held-out joint validation exists for any detector pair in this deployment, and detector dependence is unknown.*

Two detectors trained on similar data, or scoring with the same language model, make correlated errors. Multiplying their false-positive rates as if they were independent overstates how much a second agreeing signal adds. Experiment 3 in the research chapters quantifies this: with a within-class correlation of 0.9, the true joint false-positive rate of an AND rule is **12.9 times** the independence estimate. ADR-04 therefore restricts fusion to categorical facts and states that a learned combiner may be added only with documented held-out joint validation.

18.2 Inputs

`fuse(results)` takes the serialised runs of a job — each with method id, evidence class, status, `flagged`, `calibrated`, output and summary — and walks through them once.

Run status	Treatment
`unavailable`, `blocked_by_policy`	Added to **missing** with the reason
`failed`	Added to **failed** with the reason
`insufficient_input`, `unsupported_input`	Ignored for outcomes (listed in the per-class table)
`completed`	Interpreted according to evidence class (below)

For completed runs:

Evidence class	What is extracted
Statistical inference	`flagged = true` → flagged list; `flagged = false` → unflagged list; `flagged = null` (no validated threshold) → **not interpreted**
Verified watermark	Checked; detected if `flagged = true`
Cryptographic provenance	State `signature_valid_signer_trusted` or `…_untrusted` → verified (noting trust, and whether the manifest declares a capture or AI source); `verification_failed` → failed

Evidence class	What is extracted
Declared metadata	Generator markers or an IPTC AI source type → declared AI indicator
Forensic observation	Listed by class only; never interpreted automatically

Note what is *not* used: raw scores. A statistical method contributes to the outcome only through `flagged`, which can be set only with a validated calibration profile. Without one, the method is counted as "not interpreted", however high its raw score.

18.3 Categories

The policy then assigns categories:

1. A verified provenance record → `provenance_record_verified` (with an uncertainty reason if the signer is untrusted).
2. A failed provenance verification → `provenance_verification_failed`, with the next step "Obtain an original copy from the source; verification failure may indicate modification after signing."
3. A detected watermark → `evidence_of_recognized_watermark`.
4. Two or more flagged statistical detectors → `multiple_compatible_signals_consistent_with_ai`, *with* the reason: "Statistical detectors may share training data or failure modes; their dependence is unknown, so agreement is not treated as independent corroboration."
5. Exactly one flagged statistical detector → `some_signals_consistent_with_ai`.

Agreeing indicators are the flagged statistical detectors, detected watermarks, declared-AI metadata and a verified provenance record whose signer declares AI generation.

Contradictions

Two situations are contradictions:

- A **verified provenance record declares a capture pathway** (camera, film, scan, minor human edits) while flagged detectors or a detected watermark indicate AI generation. The reason recorded: "The provenance record is not rejected automatically; the discrepancy requires review."
- **Validated statistical detectors disagree**: at least one is flagged and at least one is validated but not flagged.

Either adds `contradictory_evidence` and lists the conflicting methods. AEGIS does not resolve contradictions by majority vote or by trusting one class over another; it surfaces them for the reviewer.

When nothing points to AI

If no category was assigned:

- if any interpretable result exists (a validated detector, a watermark check or a provenance verification) → `inconclusive`, with the reason "No interpretable signal is consistent with AI generation. A negative or low result does not establish human authorship.";
- otherwise → `no_reliable_conclusion`, "No method with a validated interpretation produced a result for this evidence."

On a deployment without validated detectors, the typical outcome for text is therefore `inconclusive` (when the watermark verifier ran and found nothing) or `no_reliable_conclusion`.

Human review required

If the categories include any AI-consistent signal, a watermark, a failed provenance verification or a contradiction, `human_review_required` is added. Independently of that, the result's `requires_human_review` field is **always true**, with the code comment "every investigation in AEGIS ends with a human assessment".

18.4 Uncertainty reasons and next steps

The policy explains itself in two lists that the interface and the report display.

Uncertainty reasons include:

- how many statistical outputs have no validated operating threshold and are shown without interpretation;
- how many methods failed — "failures are not negative results";
- that unsigned metadata declarations are unverified claims;
- that a valid provenance record itself declares AI generation (the signer's claim);
- that no supported watermark was detected, which "does not indicate human authorship".

Suggested next steps include:

- run the detector through the Benchmark Laboratory on representative labelled data to establish a validated operating point;
- configure unavailable methods if appropriate (see System Health);
- re-run failed methods after investigating the error;
- request the original file or source records (drafts, capture devices, publication history) when no provenance was verified;
- and always: "Collect independent process or documentary evidence before any decision affecting a person."

18.5 The primary outcome

One category is chosen as the *primary* outcome, by priority:

1. `contradictory_evidence`
2. `provenance_verification_failed`
3. `evidence_of_recognized_watermark`
4. `multiple_compatible_signals_consistent_with_ai`
5. `some_signals_consistent_with_ai`
6. `provenance_record_verified`
7. `inconclusive`
8. `no_reliable_conclusion`

Contradictions come first because they are the most important thing for a reviewer to see. The primary outcome is accompanied by a fixed statement:

> *Automated outcome: label. This describes the available evidence under policy aegis-fusion-v1; it is not a finding of misconduct and does not replace qualified human review.*

18.6 Output

The `FusionResult` stored on the job contains:

Field	Content
`policy`	`aegis-fusion-v1`
`primary_outcome`, `primary_label`	The chosen category and its human-readable label
`categories`	Every category assigned
`requires_human_review`	Always true
`agreeing`, `conflicting`	Method ids
`missing`, `failed`	Method ids with reasons
`not_interpreted`	Statistical methods without validated thresholds
`calibration_applies`	Whether any completed statistical method was calibrated
`uncertainty_reasons`, `suggested_next_steps`	The explanations above
`by_class`	Every method grouped by evidence class with status, summary, flag and calibration
`statement`	The fixed sentence above

18.7 Worked examples

Text on a fresh install. The four model-based detectors are unavailable; the reference watermark verifier completes and finds no watermark; descriptive statistics complete. Missing: four methods. Watermark checked, not detected. No categories → `inconclusive` (because the watermark check is interpretable). Reasons: no watermark detected does not indicate human authorship. Next steps: configure unavailable methods; request source records; collect independent evidence.

The demo's signed image. The C2PA verifier finds a valid manifest whose signer is not in the trust list, and which declares a trained-algorithmic-media source; declared metadata finds nothing. Categories: `provenance_record_verified`. Agreeing: the C2PA method (because the signer declares AI). Reasons: the signer is untrusted; the provenance record itself declares AI generation (signer's claim). Primary outcome: *Provenance record verified*.

A hypothetical contradiction. A validated image classifier is flagged, and a trusted C2PA manifest declares `digitalCapture`. Categories: `provenance_record_verified`, `some_signals_consistent_with_ai`, `contradictory_evidence`, `human_review_required`. Primary outcome: *Contradictory evidence — review the discrepancy*. The provenance record is not rejected; the reviewer must decide.

PART IV

Research and Evaluation

The statistical reliability engine, the nine experiments and the benchmark laboratory.

CHAPTER 19

The Statistical Reliability Engine

The question AEGIS is built around — *when can an AI-content flag be trusted?* — is a statistical question. The `aegis_research` package answers it with a small, dependency-light library: `research/aegis_research/reliability.py` (about 700 lines, NumPy and SciPy only, no I/O). It powers the Research Bench calculators, the Benchmark Laboratory, calibration profiles and the nine experiments. This chapter explains each quantity it computes and why it matters.

19.1 Conventions

- Label 1 = AI-generated (the positive class); label 0 = human-written (the negative class).
- Scores are oriented so that higher means more consistent with AI generation.
- Scores are **not** probabilities unless a calibration profile validated on held-out data says so.

Every input rate carries a **source**, displayed next to every number derived from it:

`RateSource`	Meaning
`measured`	Estimated from labelled evaluation data recorded in AEGIS
`assumed`	Entered by a researcher as an assumption
`published`	Copied from a cited publication (not reproduced)
`vendor_reported`	Claimed by a provider, not independently verified
`simulated`	Produced by a synthetic simulation

A positive predictive value computed from a vendor's claimed false-positive rate is therefore visibly different from one computed from a locally measured rate.

19.2 Confidence intervals for proportions

Every rate (TPR, FPR and so on) is a binomial proportion estimated from finite data, so AEGIS reports it with an interval. Two methods are provided.

Wilson score interval. For k successes in n trials, $\hat{p} = k/n$ and $z = \Phi^{-1}(1 - \alpha/2)$:

$$\frac{\hat{p} + \frac{z^2}{2n} \pm z\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z^2}{4n^2}}}{1 + \frac{z^2}{n}}$$

Clopper–Pearson (exact) interval. Based on the beta distribution:

$$\left[\, B\left(\tfrac{\alpha}{2};\, k,\, n-k+1\right),\;\; B\left(1-\tfrac{\alpha}{2};\, k+1,\, n-k\right) \right]$$

with the lower bound 0 when $k = 0$ and the upper bound 1 when $k = n$. It is conservative and is the method AEGIS prefers for rare events such as false positives at strict operating points. When $n = 0$ both methods return the uninformative interval $[0, 1]$.

19.3 How many negatives does a strict FPR need?

This is perhaps the single most important function in the library. If a detector is evaluated on n human-written samples and produces **zero** false positives, what can be said about its false-positive rate? The exact one-sided upper bound at confidence $1-\alpha$ is the p solving $(1-p)^n = \alpha$. For that bound to be below a target FPR f:

$$n \geq \left\lceil \frac{\ln \alpha}{\ln (1-f)} \right\rceil \approx \frac{3}{f} \quad (\alpha = 0.05)$$

```python
def min_negatives_for_fpr(target_fpr: float, confidence: float = 0.95) -> int:
    if target_fpr == 0:
        raise ValueError("an FPR of exactly zero can never be established from finite samples")
    return math.ceil(math.log(1 - confidence) / math.log(1 - target_fpr))
```

Target FPR	Negatives needed (95%)
5%	59
1%	299
0.1%	2,995
0.01%	29,956

The function refuses a target of exactly zero: no finite sample can establish it. These numbers drive the *estimability* checks throughout AEGIS. A benchmark that reports “TPR at 0.1% FPR” from 90 negatives is reporting something the data cannot support, and AEGIS labels it “not estimable”.

19.4 Confusion-matrix metrics

`ConfusionMatrix` holds TP, FP, TN and FN and can be built from labels and predictions. `classification_metrics()` reports, each with counts and an exact interval:

Metric	Formula
TPR (sensitivity, recall)	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FN})$
TNR (specificity)	$\mathrm{TN}/(\mathrm{TN}+\mathrm{FP})$
FPR	$\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$
FNR	$\mathrm{FN}/(\mathrm{FN}+\mathrm{TP})$
Sample precision	$\mathrm{TP}/(\mathrm{TP}+\mathrm{FP})$ — valid only at the *sample’s* prevalence
NPV (sample)	$\mathrm{TN}/(\mathrm{TN}+\mathrm{FN})$
Accuracy	$(\mathrm{TP}+\mathrm{TN})/n$
F1	$2\,\mathrm{TP}/(2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN})$

Precision is called *sample precision* on purpose: benchmark datasets are often balanced 50/50, while real populations are not, so a benchmark’s precision says little about deployment.

19.5 Predictive values and base rates

The positive predictive value — the probability that a flagged item is actually AI-generated — depends on the prevalence π of AI content in the population actually screened. By Bayes' rule:

$$\mathrm{PPV} = \frac{\mathrm{TPR}\cdot\pi}{\mathrm{TPR}\cdot\pi + \mathrm{FPR}\cdot(1-\pi)}, \qquad \mathrm{NPV} = \frac{(1-\mathrm{FPR})(1-\pi)}{(1-\mathrm{FPR})(1-\pi) + (1-\mathrm{TPR})\pi}$$

`predictive_values(tpr, fpr, prevalence, sample_size, sources)` returns both values and, for a screening population of a given size, the expected numbers of true positives, false positives, false negatives, true negatives, total flags, and **false flags per true flag**. It also returns the assumptions every result depends on:

- TPR and FPR are assumed to hold in the deployment population (no distribution shift);
- prevalence is the proportion of AI-generated items in the population actually screened;
- PPV is a population-level expectation, not the probability for any specific item unless the detector's operating characteristics are valid for that item's subgroup.

For a detector with 90% TPR and 5% FPR screening 10,000 items at 5% prevalence: 450 true flags, 475 false flags, PPV 48.65%. More than half of all flags are false. `ppv_sensitivity()` tabulates PPV and NPV across a range of prevalences, which the Research Bench plots as a curve.

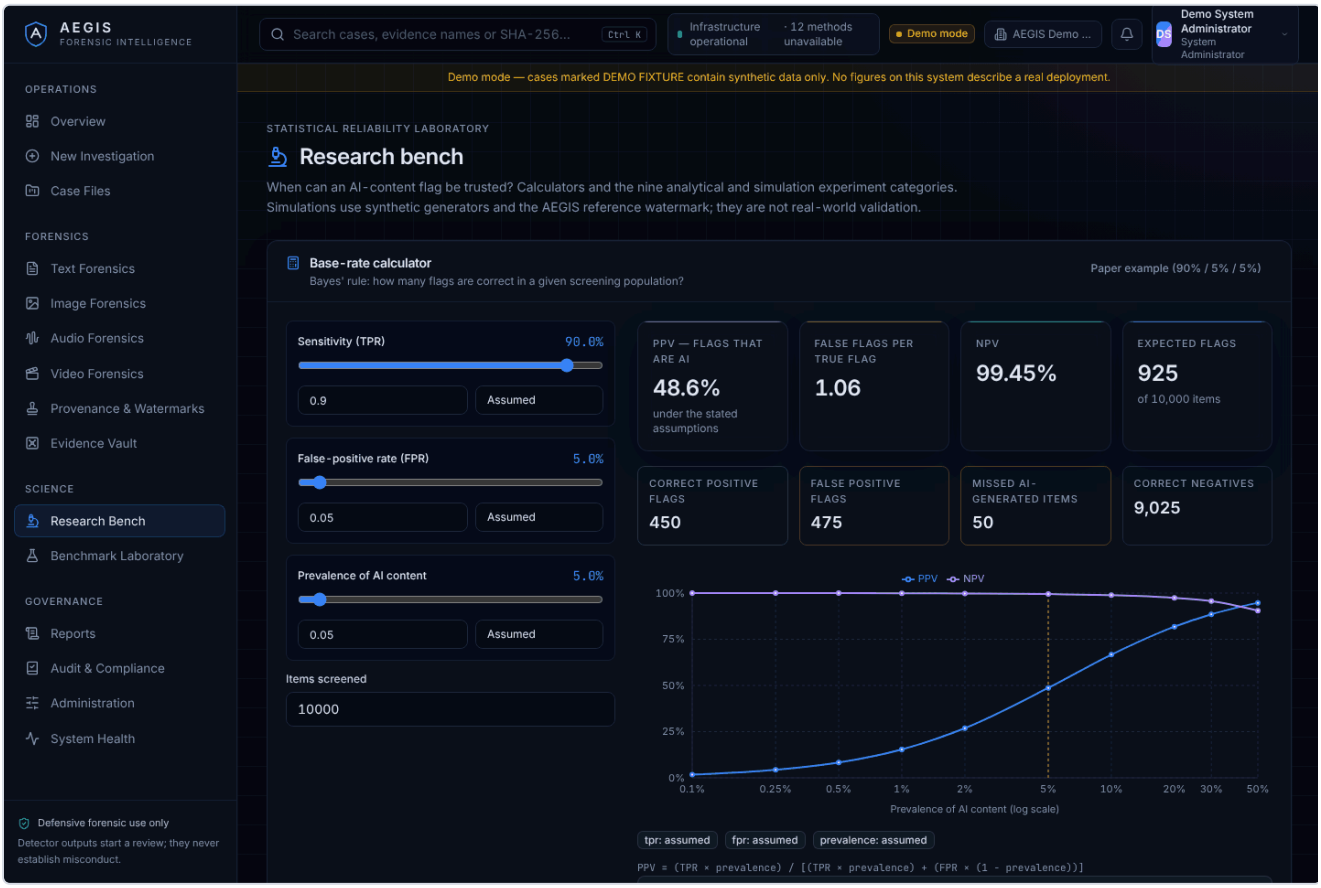


Figure 7. The Research Bench: the Bayes PPV calculator with a labelled source for each input, and the catalogue of the nine reliability experiments.

19.6 Score-based metrics

AUROC is computed exactly through the Mann–Whitney *U* statistic, with ties counting one half:

$$\text{AUROC} = \frac{\sum_{i \in \text{pos}} \text{rank}(s_i) - \frac{n_+(n_+ + 1)}{2}}{n_+ n_-}$$

`auroc_bootstrap_ci()` gives a **stratified bootstrap** interval: positives and negatives are resampled separately (preserving the class balance) 1,000 times by default with a fixed seed, and the 2.5th and 97.5th percentiles are reported.

ROC and PR curves are computed at every distinct score threshold.

AUROC is never reported alone. The evaluation methodology insists on sensitivity at strict false-positive rates, because two detectors with nearly identical AUROC can behave completely differently at screening operating points (experiment 4 demonstrates this).

19.7 Sensitivity at a fixed false-positive rate

`tpr_at_fpr(labels, scores, target_fpr)` finds the most lenient threshold whose empirical FPR does not exceed the target, by walking down the distinct scores from the highest until the FPR would exceed the target. It returns an `OperatingPoint` with the threshold, the achieved FPR, the TPR with its interval, the exact Clopper–Pearson interval of the FPR, the number of negatives, the minimum required, and an `estimable` flag:

- If the sample has enough negatives: "Operating point estimable from this sample."
- If not: "Only *n* negatives; at least *m* are needed to bound FPR at *f* with zero observed false positives. Treat this operating point as not validated."
- If no threshold achieves the target: a note saying so, with no threshold.

19.8 Threshold sweeps and deployment projection

`threshold_sweep()` evaluates a range of thresholds and, for each, projects the sample's TPR and FPR onto a stated deployment prevalence to give the deployment PPV and expected false flags. This powers the **threshold explorer** in the Research Bench, where moving a slider shows how many innocent people would be flagged per true detection in a population of a given size.

19.9 Calibration

Brier score: the mean squared difference between predicted probability and outcome, $\frac{1}{n}\sum (p_i - y_i)^2$.

Reliability bins: probabilities are grouped into equal-width bins; each bin reports its mean predicted probability, its observed frequency of positives and its count.

Expected calibration error (ECE): the count-weighted mean absolute gap between predicted and observed frequency across bins, $\sum_b \frac{n_b}{n} |\bar{p}_b - \bar{y}_b|$.

These are computed only when scores are declared to be calibrated probabilities in [0, 1].

Platt calibration

`PlattCalibrator` maps a raw score s to a probability $p = 1/(1 + e^{-(as+b)})$. It is fitted on a **validation split** by Newton–Raphson minimisation of the log-loss, using Platt's smoothed targets to avoid over-confident fits:

$$t_i = \begin{cases} \dfrac{n_+ + 1}{n_+ + 2} & y_i = 1 \\ \dfrac{1}{n_- + 2} & y_i = 0 \end{cases}$$

The gradient and Hessian of the log-loss with respect to (a, b) are computed in closed form, a tiny ridge term keeps the Hessian invertible, and iteration stops when the step is below 10^{-10} or after 200 iterations. Fitting requires both classes. The docstring warns that the calibrator is "only meaningful for the population it was fitted on", which is why every calibration profile in AEGIS stores the benchmark run, population and languages it came from.

19.10 Corroboration: combining two signals

`combined_rates_assuming_independence()` returns the rates of an AND or OR rule for two detectors *if* their errors are conditionally independent given the class:

$$\text{AND: } \text{TPR} = t_1 t_2, \ \text{FPR} = f_1 f_2; \qquad \text{OR: } \text{TPR} = 1 - (1 - t_1)(1 - t_2), \ \text{FPR} = 1 - (1 - f_1)(1 - f_2)$$

The result includes the assumption as a string, and the docstring notes that it is "rarely guaranteed for detectors trained on similar data". `empirical_joint_rates()` computes the *actual* joint rates from two sets of flags on labelled data, so the two can be compared. Experiment 3 shows how far apart they can be.

19.11 Fairness: where do false flags land?

`subgroup_false_positive_share()` takes, for each group, the number of human-written items and the group's FPR, and computes each group's share of the population and its share of all false flags. Groups smaller than a suppression threshold have their counts withheld to protect privacy. With group A making up 90% of the population at FPR 2% and group B 10% at FPR 10%, group B receives 35.7% of all false flags — more than three times its population share (experiment 2).

`subgroup_metrics()` computes per-group FPR and FNR with exact intervals from labelled predictions, suppressing groups with fewer than 10 members. The Fairness laboratory in the user interface uses these functions **only** on consented evaluation-data fields; AEGIS never asks for or infers demographic attributes of the people whose evidence it examines.

19.12 The full evaluation bundle

`evaluate_scores(labels, scores, threshold, groups, probabilities_are_calibrated, seed)` is what the Benchmark Laboratory stores for each run: sample size, threshold, all confusion-matrix metrics with intervals, AUROC with a 500-sample bootstrap interval, ROC and PR curves, operating points at 1% and 0.1% FPR with estimability, calibration metrics (when applicable) and subgroup metrics (when groups are supplied).

19.13 Tests

`research/tests/test_reliability.py` checks the library against known values — for example the analytical PPV example, interval coverage properties, the minimum-negatives formula, AUROC on simple cases and Platt fitting — and `test_watermark_and_experiments.py` checks the watermark scheme and that every experiment runs and produces its documented structure. Both suites ran successfully while this document was being prepared.

CHAPTER 20

The Nine Reliability Experiments

`research/aegis_research/experiments/catalog.py` implements nine experiments that turn the principles of this book into numbers. Each returns a `kind` — **analytical** (a closed-form calculation under stated assumptions) or **simulation** (a seeded Monte Carlo run using the synthetic Markov language model and the reference watermark) — together with its full configuration, its seeds, the research question it answers and notes on interpretation. **None of these results measures a deployed detector on real content.**

20.1 Reproducing the results

```
make experiments                        # JSON + PNG + manifest.json → research/results/
python -m aegis_research.experiments exp9_fpr_calibration --out /tmp/x
```

The runner writes one JSON file per experiment, an optional PNG plot, and a `manifest.json` recording the generation time, the `aegis_research` version, the Python, NumPy and SciPy versions, the platform, and for each experiment its kind, file name, the SHA-256 of the result file and the SHA-256 of its configuration. The Research Bench in the web interface runs the same functions through `POST /api/v1/evaluation/experiments/{id}/runs` and stores each run, with a content hash, in the `experiment_runs` table.

> **Regenerated for this document.** All nine experiments were re-run while this book was being prepared (Python 3.13, NumPy 2.5, SciPy 1.18, with the seeds fixed in the catalogue). Every number in this chapter comes from that run, and every number matches the tables published in `docs/research/experiments.md`. The charts are the PNG files the runner produced; the runner does not draw charts for experiments 4 and 7, so those two are presented as tables only.

20.2 Experiment 1 — Base-rate effects (analytical)

Question. How does the share of flagged items that are truly AI-generated change with prevalence?

Configuration. TPR 0.90; FPR 0.05 (with curves for 0.01, 0.05 and 0.10); prevalence 5%; a population of 10,000; a prevalence grid from 0.1% to 50%.

Quantity	Value
PPV	**48.65%**
Expected true flags	450
Expected false flags	475
False flags per true flag	1.06

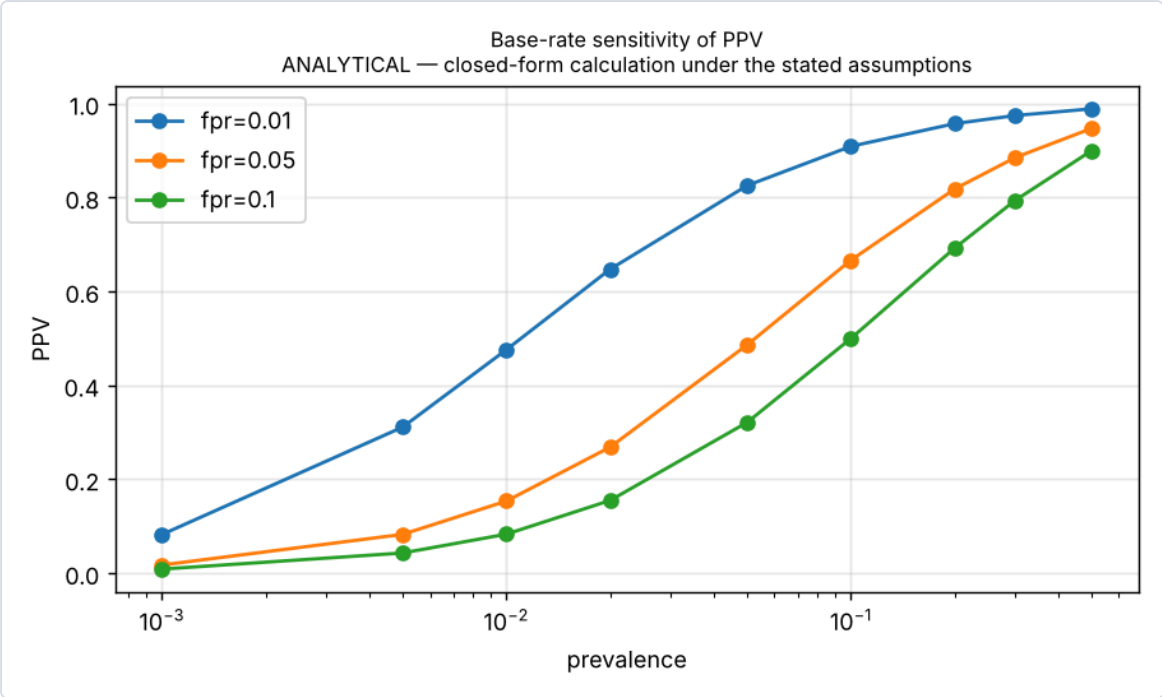


Figure 8. PPV as a function of prevalence for three false-positive rates (TPR fixed at 0.90). Even at a 1% FPR, PPV falls below 50% once prevalence drops under about 1%.

Interpretation. Under these assumptions more than half of all flags are false. This reproduces the analytical example from the research brief; it is a property of the assumed rates, not a measurement of AEGIS. The lesson drives the design of the whole system: a flag can only *start* a review.

20.3 Experiment 2 — Subgroup error concentration (analytical)

Question. If false-positive rates differ by group, where do false accusations land?

Configuration. Group A (“majority writers”) is 90% of the human-written population with FPR 2%; group B (“minority writers”) is 10% with FPR 10%. Rates are assumed and labelled as such.

Group	Population share	FPR	Expected false flags (per 10,000 humans)	Share of false flags
A	90%	2%	180	64.3%
B	10%	10%	100	**35.7%**

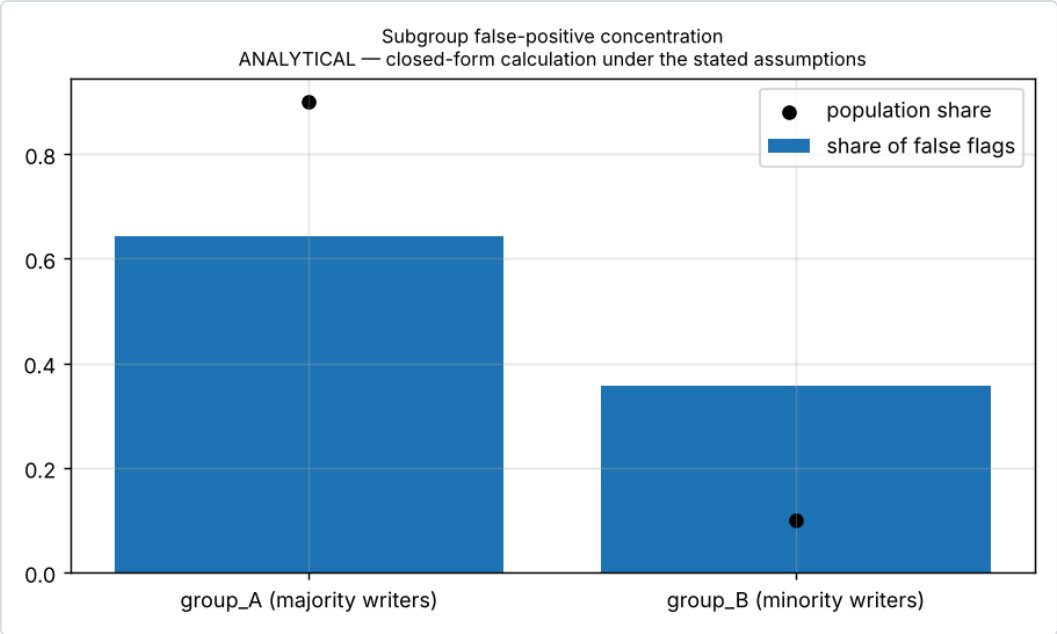


Figure 9. Population share (dots) versus share of false flags (bars) for the two groups.

Interpretation. A tenth of the population receives more than a third of the false accusations. Non-native writers are a frequently cited example of a group for whom detectors show elevated false-positive rates. This is why AEGIS's evaluation methodology requires subgroup FPR to be reported before any deployment, and why the Fairness laboratory exists.

20.4 Experiment 3 — Corroborating evidence (simulation)

Question. How much does requiring a second signal help, and what if the signals are correlated?

Configuration. Two detectors with (TPR 0.90, FPR 0.05) and (TPR 0.80, FPR 0.05), combined with an AND rule. Under conditional independence the joint FPR would be $0.05 \times 0.05 = 0.25\%$ and the joint TPR 72%. The simulation draws 200,000 samples per class from a bivariate normal latent model with within-class correlation ρ; each detector's threshold and class means are set from the stated marginal rates ($t = \Phi^{-1}(1 - \mathrm{FPR})$, $\mu = t - \Phi^{-1}(1 - \mathrm{TPR})$), and the joint rates of the AND rule are measured empirically (seed 7). PPV is computed at 5% prevalence.

ρ	Empirical joint FPR	× independence estimate	PPV empirical	PPV assumed
0.0	0.241%	0.96	0.940	0.938
0.3	0.697%	2.79	0.848	0.938
0.6	1.545%	6.18	0.721	0.938
0.9	3.228%	**12.9**	0.563	0.938

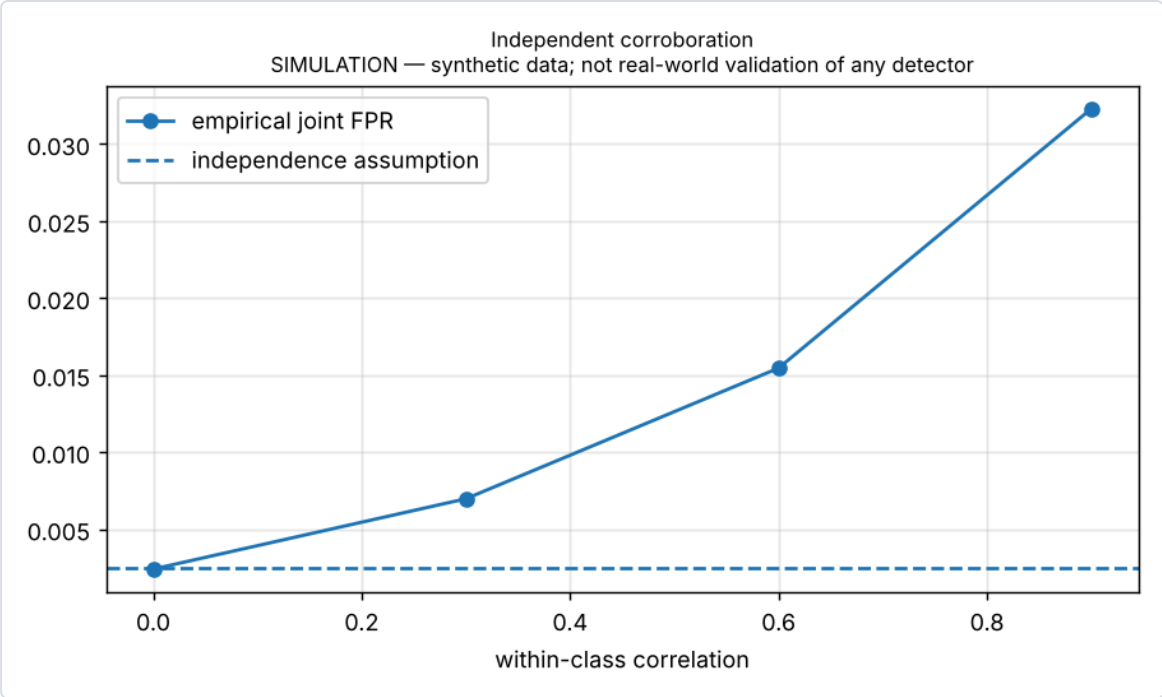


Figure 10. Empirical joint FPR of the AND rule as within-class correlation increases, against the independence assumption (dashed line).

Interpretation. The product rule is valid only if the detectors are independent. For correlated detectors — for example two that score text with the same language model — it overstates how much a second signal adds by up to an order of magnitude. This is exactly why the fusion policy refuses to multiply rates and records "known dependencies" between methods that share a model.

20.5 Experiment 4 — AUROC versus strict operating points (simulation)

Question. Do detectors with similar AUROC behave similarly at low false-positive operating points?

Configuration. 2,000 positives and 2,000 negatives (seed 11). Detector 1: positives $\mathcal{N}(1.8, 1)$, negatives $\mathcal{N}(0, 1)$. Detector 2: positives $\mathcal{N}(1.6, 0.6^2)$, negatives drawn from Student's *t* with 3 degrees of freedom — a heavy tail of high-scoring human texts.

Simulated detector	AUROC	TPR at 1% FPR	TPR at 0.1% FPR
Binormal, equal variance	0.898	28.6%	8.7% (**not estimable**: needs ≥ 2,995 negatives)
Heavy-tailed negatives (t_3)	0.887	**0.0%**	0.0% (not estimable)

Interpretation. Two detectors with nearly the same AUROC behave completely differently at screening operating points. The heavy tail of the human-written score distribution means that to keep false positives at 1%, the threshold must be set so high that nothing is caught. AUROC alone is therefore never reported by AEGIS. Note also that with 2,000 negatives the 0.1% operating point is not estimable; AEGIS says so instead of reporting 8.7% as if it were validated.

20.6 Experiment 5 — Watermark detection versus length and entropy (simulation)

Question. How many tokens are needed to detect the reference watermark, and how does that depend on the entropy of the text?

Configuration. The reference green-list scheme ($\gamma = 0.25$, $\delta = 2$) on the synthetic language model at four Zipf exponents (higher exponent = lower entropy), five lengths, 30 trials per cell, threshold $z \geq 4$ (seed 5). Rows show the Zipf exponent and the mean next-token entropy in nats.

Zipf (entropy)	10 tokens	25	50	100	200
0.6 (6.53)	0.43	0.97	1.00	1.00	1.00
1.0 (5.21)	0.47	0.83	1.00	1.00	1.00
1.5 (2.92)	0.07	0.70	0.97	1.00	1.00
2.5 (1.02)	0.03	0.13	0.27	0.63	1.00

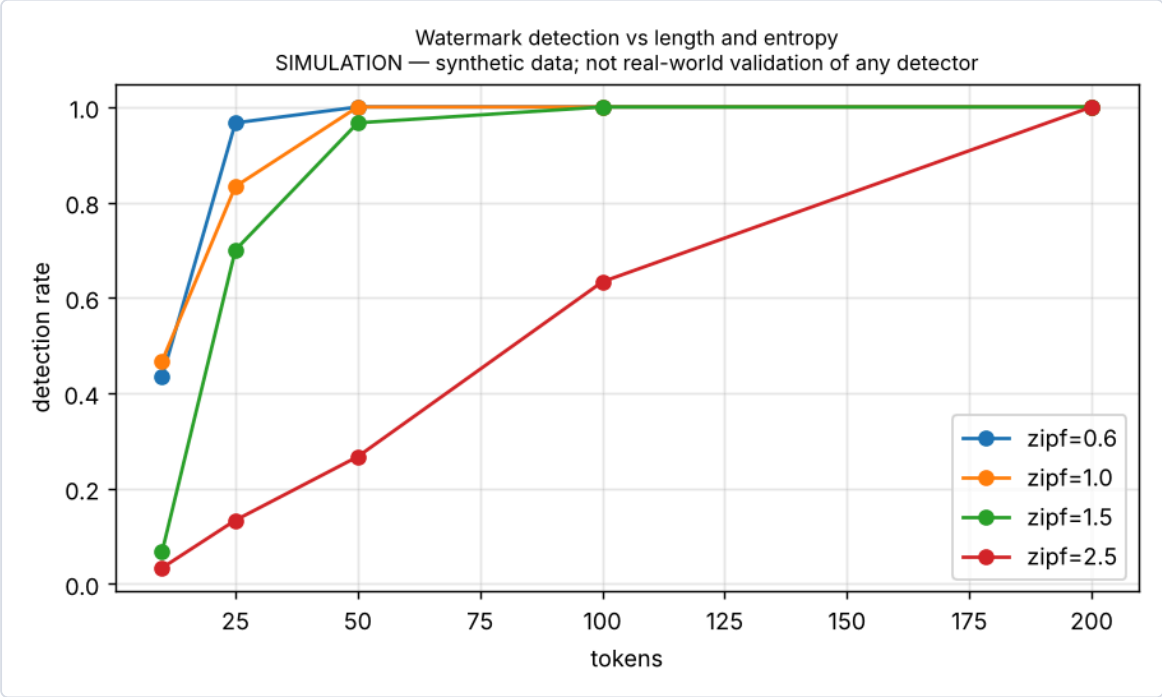


Figure 11. Detection rate versus text length for four Zipf exponents (higher exponent = lower entropy).

Interpretation. Across every cell the false-positive rate on unwatermarked text was 0/30. Low-entropy text — where the model has few plausible choices, as in formulaic writing — needs far more tokens before the watermark can be detected, because the bias δ can only shift probability towards green tokens when there are alternatives to choose from.

20.7 Experiment 6 — Watermark robustness under random token replacement (simulation)

Question. How quickly does detection degrade under controlled random token replacement?

Configuration. 200-token watermarked texts, Zipf 1.0, 30 trials per level, uniform random replacement of a fraction of tokens (seed 9).

Fraction replaced	0	0.1	0.2	0.3	0.4	0.5
Detection rate	1.00	1.00	1.00	1.00	0.97	**0.47**
Mean z	14.6	11.7	9.0	7.0	5.5	3.9

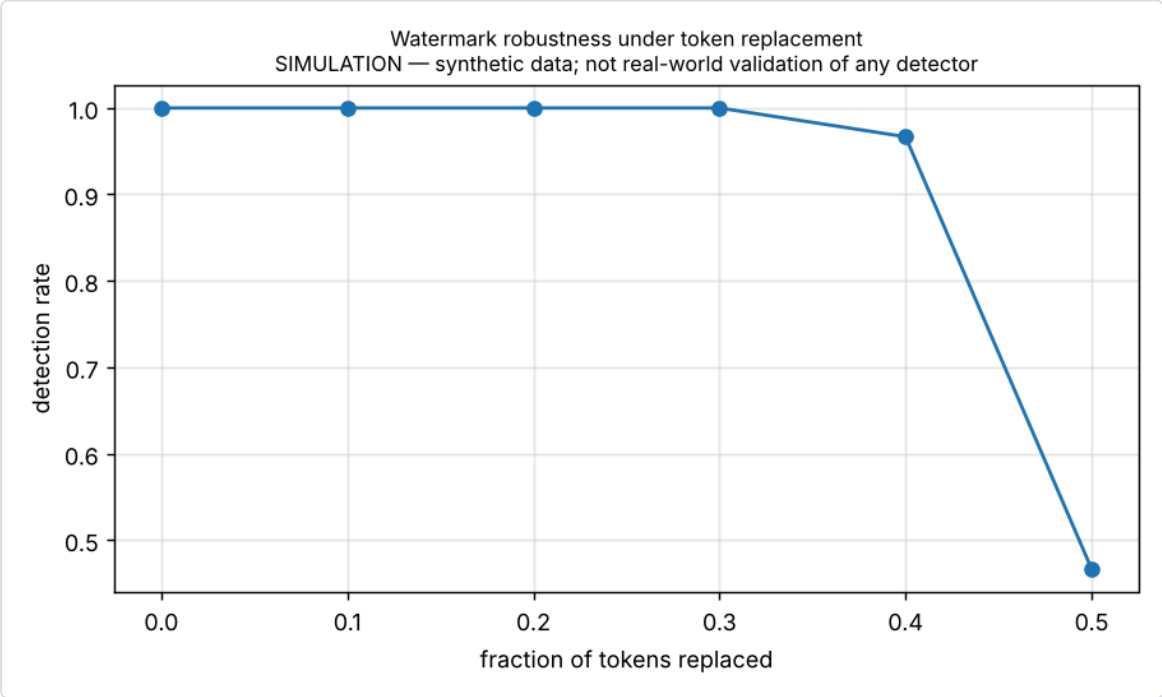


Figure 12. Detection rate as the fraction of randomly replaced tokens increases.

Interpretation. Each replaced token destroys its own green-list membership and the context of the next token, so z falls roughly linearly. This measures one specific implementation against uniform random replacement — a defensive robustness measurement — and is explicitly **not** a claim of robustness to paraphrasing.

20.8 Experiment 7 — Threshold selection (analytical)

Question. If a threshold is chosen from the normal approximation to meet an FPR tolerance, is the tolerance actually met?

Configuration. $\gamma = 0.25$; four target FPRs and text lengths.

Target FPR	Tokens	z (normal)	Exact min. green count	Exact FPR achieved	True FPR of normal threshold
10^{-2}	50	2.33	21	6.3×10^{-3}	1.4×10^{-2}
10^{-3}	50	3.09	24	3.7×10^{-4}	$2.6 \times \mathbf{10^{-3}}$
10^{-4}	200	3.72	75	6.2×10^{-5}	2.0×10^{-4}
10^{-6}	200	4.75	82	5.1×10^{-7}	2.2×10^{-6}

Interpretation. Thresholds taken from the normal approximation **exceed** the FPR tolerance they were chosen for, by up to 2.6×. Exact binomial thresholds always stay within it. The discreteness of the binomial distribution and its skew for small γ make the normal tail too thin.

20.9 Experiment 8 — Watermark strength versus distortion (simulation)

Question. How does increasing the bias δ trade detectability for distributional distortion?

Configuration. 100-token texts, Zipf 1.0, 30 trials per δ; distortion measured as the mean per-step KL divergence of the watermarked distribution from the base distribution over 300 random contexts (seed 13).

δ	0.5	1.0	2.0	4.0	8.0
Detection rate	0.03	0.77	1.00	1.00	1.00
Mean KL from base (nats/token)	0.026	0.113	0.455	1.139	1.418

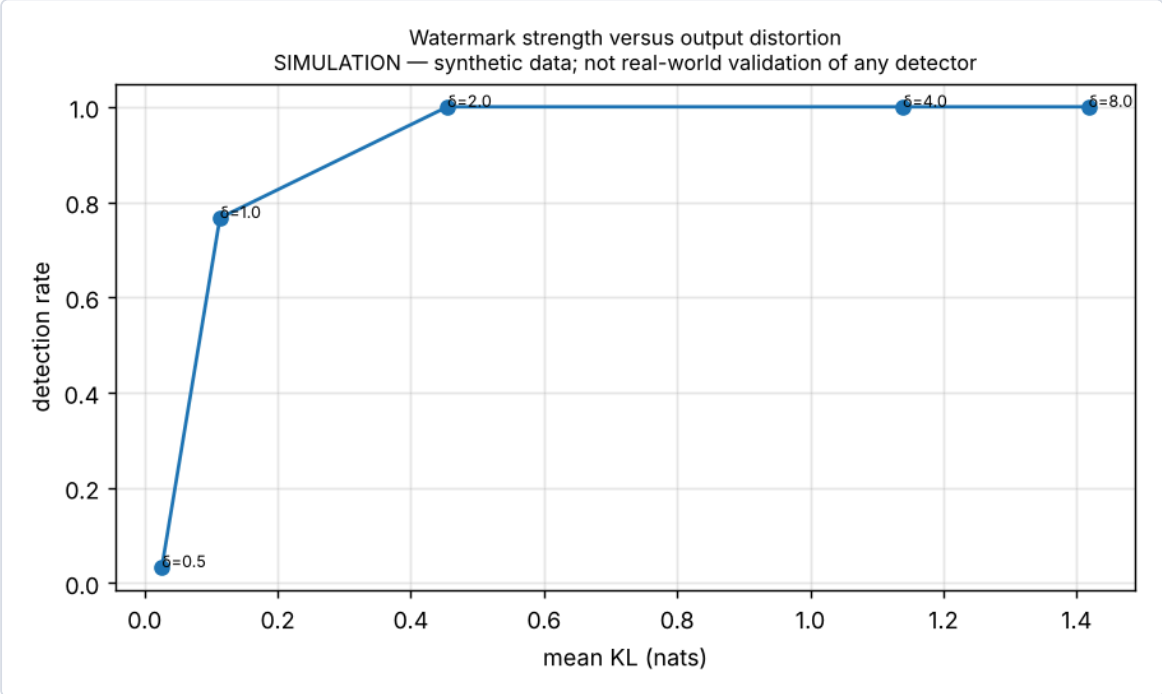


Figure 13. Detection rate versus mean KL divergence from the base distribution; each point is labelled with its bias δ.

Interpretation. Stronger watermarks are easier to detect but move the output distribution further from the model's own. KL divergence is only one mathematical measure of distortion; a low value does **not** show that semantic quality is preserved.

20.10 Experiment 9 — Exact versus approximate false-positive calibration (analytical)

Question. How accurate is the z-test's normal approximation for short texts?

Configuration. $\gamma = 0.25$; lengths from 10 to 1,000 tokens; thresholds $z \geq 2$ and $z \geq 4$. At $z \geq 4$ the normal approximation gives an FPR of 3.17×10^{-5} regardless of length.

Tokens	10	16	25	50	100	200	1000
Exact ÷ approximate FPR at $z \geq 4$	**13.1**	9.0	6.8	3.9	2.0	1.9	1.5

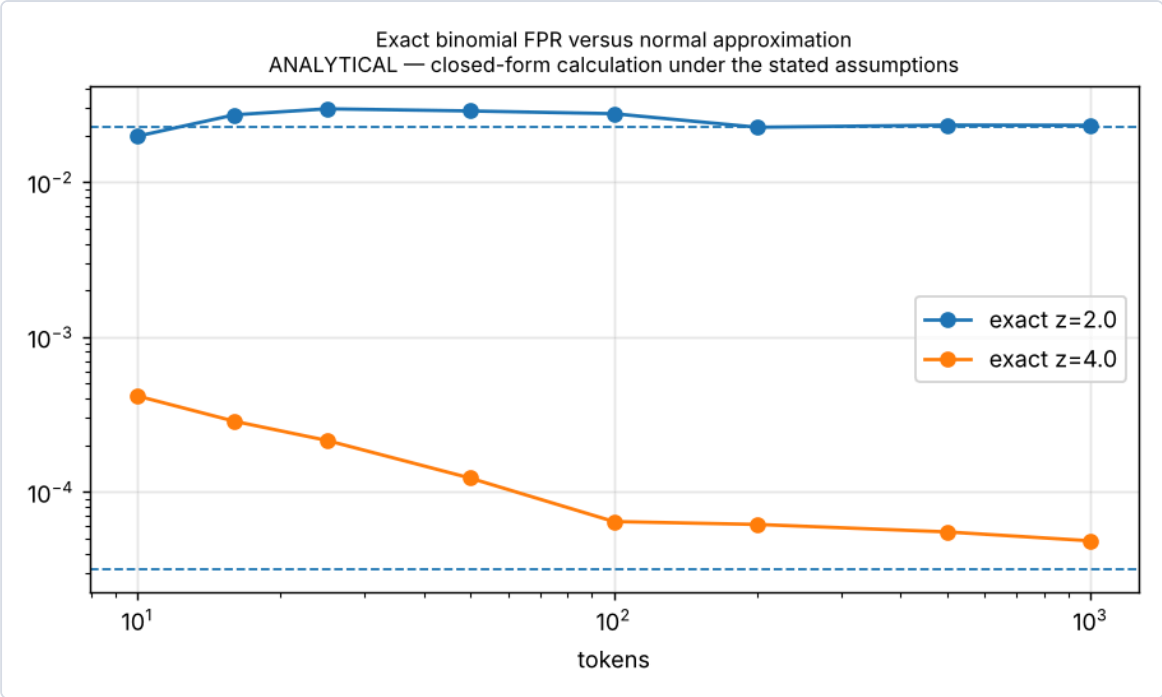


Figure 14. Exact binomial FPR by text length at z ≥ 2 and z ≥ 4 (solid lines), against the constant normal-approximation FPR for each threshold (dashed lines).

Interpretation. For short texts the z-test understates the false-positive rate by an order of magnitude. AEGIS therefore reports the exact binomial p-value alongside the z-score in every watermark result, and enforces minimum-length checks.

20.11 The built-in benchmark of the reference watermark

`make benchmark` (or `python -m app.cli benchmark`) additionally runs a complete Benchmark Laboratory run of the reference watermark verifier on a freshly generated synthetic dataset, stored with result kind **simulated**:

Item	Value
Dataset	`aegis-reference-watermark-synthetic`: 240 base samples (120 watermarked, 120 not); lengths 20, 40, 80 or 160 tokens; 60 validation and 180 test originals plus 360 transformed test variants
Key	A fresh random 32-byte key per generation, never stored; only its fingerprint is recorded
Threshold	Chosen on the validation split at a 1% FPR target
AUROC	0.9999 (95% bootstrap CI 0.9996–1.0)
TPR at selected threshold	98.9% (exact 95% CI 94.0–99.97%)
FPR at selected threshold	0.0% (exact 95% CI 0.0–4.0%)
TPR at 1% / 0.1% FPR	98.9%, **not estimable**: 90 negatives, at least 299 / 2,995 needed
20% / 40% token replacement	AUROC 0.995 / 0.957; TPR 86.7% / 45.6%

The interval reported for FPR is what this sample size actually supports. Observing zero false positives in 90 negatives does not show that the FPR is zero — only that it is probably below 4%.

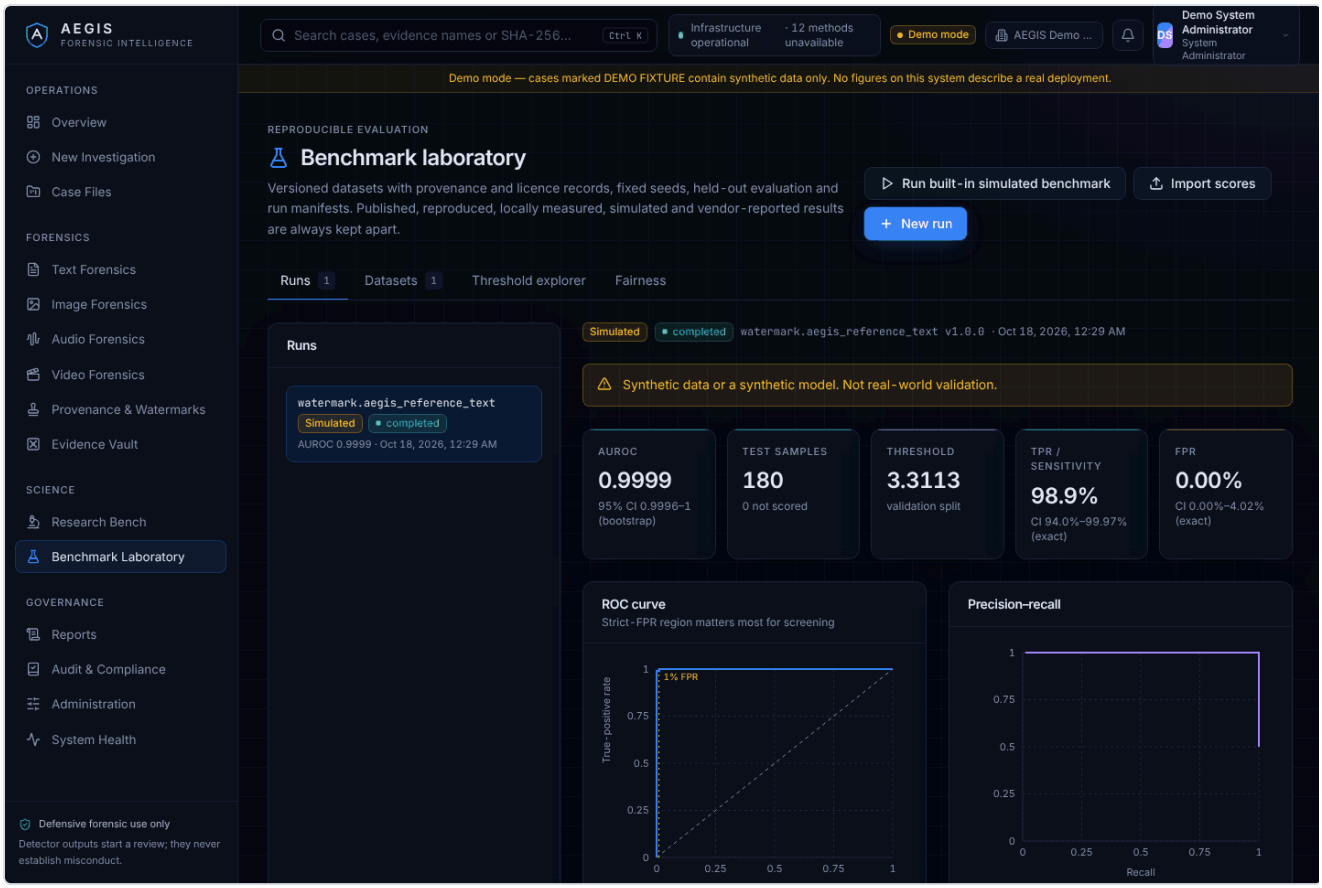


Figure 15. The Benchmark Laboratory: simulated results labelled as such, exact confidence intervals, and checks on whether a strict false-positive rate can be estimated.

CHAPTER 21

The Benchmark Laboratory and Research Bench

A detector can be validated in AEGIS only through locally measured evidence. The Benchmark Laboratory is where that evidence is produced: datasets are registered with provenance, methods are run on them, metrics are computed with honest uncertainty, and calibration profiles are fitted. The Research Bench provides calculators and the nine experiments for people who need to understand what a flag is worth. The code is in `apps/api/app/services/benchmarks.py` and `apps/api/app/api/v1/research.py`.

21.1 Evaluation methodology

The project's evaluation methodology (`docs/research/evaluation-methodology.md`) sets eight principles that the code enforces:

1. Report **sensitivity at strict false-positive rates** (1% and 0.1%), with sample sizes and exact Clopper–Pearson intervals. AUROC is reported but never on its own.
2. A strict FPR is **estimable** only with enough negatives: at least 299 for 1% and 2,995 for 0.1%. Operating points that fall short are marked "not validated".
3. **Choose thresholds on a validation split** and evaluate on a separate test split. A run without a validation split reports only threshold-free metrics.
4. **Exclude cross-split duplicates** before evaluation. Texts are normalised by case and whitespace and hashed; matches are excluded and counted.
5. Use **held-out evaluation**: restricting the test split to values of `generator`, `domain` or `language` absent from validation measures generalisation instead of memorisation.
6. **Keep result kinds separate**: `locally_measured`, `simulated`, `published`, `reproduced` and `vendor_reported` are stored in a constrained column and labelled in every view. A published number can be stored only with a citation.
7. **Calibration** (Platt) is fitted on the validation split. A profile counts as *validated* only for locally measured runs where the target operating point is estimable.
8. **Deployment PPV** is computed from TPR and FPR at the chosen threshold and a stated prevalence, with the source of each input shown.

21.2 Registering a dataset

Datasets are JSONL files, one record per line:

```
{"id": "s1", "text": "...", "label": 1, "split": "validation", "group": "L2-writers",
 "generator": "model-x", "domain": "news", "language": "hi", "transform": "original"}
```

Field	Meaning
`id`	Sample identifier, unique per transform
`text`	The sample text

Field	Meaning
`label`	1 = AI-generated, 0 = human-written
`split`	`train`, `validation` or `test`
`group`	Optional consented subgroup field, for fairness analysis
`generator`, `domain`, `language`	Optional fields usable for held-out evaluation
`transform`	`original`, or the name of a documented transformation of the same `id` (for example `paraphrase_v1`, `truncate_50`)

`POST /benchmarks/datasets` (requires `dataset.register`) parses and validates the file — invalid JSON, a missing `text` or `label`, an unknown split, duplicate ids or more than 50,000 samples are rejected with the line number — and **requires** three documentation fields: `provenance`, `license` and `consent_basis`. *"Dataset field '…' is required (provenance, licence and consent must be documented)."* The file's SHA-256 is recorded, the bytes are stored write-once, and a duplicate report counts within-dataset duplicates and cross-split duplicates. Runs verify the hash before use.

The dataset card template in `docs/research/dataset-cards.md` lists what a registered dataset should document: name and version, provenance (who collected it, how and when, with generator models and prompts for AI samples), licence and terms, consent or lawful basis, personal data and minimisation, composition, how splits were formed and what is held out, subgroup fields, transformations, known gaps and biases.

21.3 Running a benchmark

`POST /benchmarks/runs` (requires `benchmark.run`, returns `202`) creates a `BenchmarkRun` for a method on a dataset. Dataset benchmarks currently support text methods. The result kind is set automatically: `simulated` if the dataset is synthetic, `locally_measured` otherwise. The run is executed by `execute_benchmark()` on the `aegis-research` queue (or inline).

Execution proceeds as follows:

1. Load and verify the dataset; check the method is available (otherwise `409 method_unavailable`).
2. For each sample in the validation and evaluation splits:
   - skip it if its normalised text hash occurs in more than one split (**cross-split duplicate**), and count it;
   - apply the **hold-out** rule if configured: in the evaluation split keep only samples whose hold-out field has one of the held-out values; in the validation split drop them;
   - run the method through the normal `run()` wrapper with the sample's token count and language, exactly as in casework;
   - store a `BenchmarkSample` with the split, label, group, transform, score (only if the run completed) and status.
3. **Threshold**: use the configured threshold if given; otherwise choose it on the validation split's original samples at the target FPR (1% by default). The source of the threshold — `configured`, `validation_split` or `none` — is recorded.
4. **Metrics** on the evaluation split's original samples via `evaluate_scores()`: all confusion-matrix rates with intervals, AUROC with bootstrap interval, ROC and PR curves, operating points at 1% and 0.1% FPR with estimability, and subgroup metrics. Also recorded: how many samples could not be scored, how many duplicates were excluded, and the fraction of short texts (below 50 tokens).

5. **Robustness**: metrics are computed separately for each transform present in the evaluation split.
6. **Manifest**: dataset id, name, version, SHA-256, synthetic flag and licence; method id and version; full configuration and seed; hold-out definition; Python, platform, `aegis_research` and NumPy versions; result kind; start and finish times; number of samples scored.
7. Audit `benchmark.completed` with the method, result kind and AUROC.

Because the benchmark calls the same `AnalysisMethod.run()` that casework uses (ADR-02), a benchmark measures exactly the code that runs in production.

21.4 Importing scores

Sometimes a detector is run outside AEGIS — by a vendor or an independent evaluator. `POST /benchmarks/imported-scores` accepts a CSV with columns `label`, `score` and optionally `group` and `transform`. The result kind must be `locally_measured`, `vendor_reported`, `published` or `reproduced`, and **a citation or source is mandatory** for the last three. AEGIS computes metrics from the supplied scores and stores the CSV's SHA-256, but every view states: *"Imported scores: AEGIS computed metrics from the supplied labels and scores; it did not run the detector."*

21.5 The built-in reference-watermark benchmark

`POST /benchmarks/reference-watermark` (and `make benchmark`) builds a synthetic dataset with `build_reference_watermark_suite()` — 120 watermarked and 120 unwatermarked samples at lengths 20, 40, 80 and 160 tokens, with token-replacement transforms at 20% and 40% — under a fresh random 32-byte key, registers it as synthetic (licence CC0-1.0, no personal data), and benchmarks the reference verifier on it. Its results are listed in the Experiments chapter. Because the dataset is synthetic, the run is labelled **simulated** and can never be used to validate a method.

21.6 Calibration profiles

`POST /benchmarks/runs/{id}/calibration-profiles` (requires `admin.models`) fits a profile from a completed run with a target FPR and a description of the population:

```
cal = rel.PlattCalibrator.fit(y, sc)            # on validation-split originals only
op = rel.tpr_at_fpr(y, sc, target_fpr)          # threshold at the target FPR
validated = run.result_kind == "locally_measured" and op.estimable
```

- Only validation-split, original (untransformed), scored samples are used; both classes must be present ("Calibration requires a labelled validation split distinct from the test split.").
- The Platt parameters *a*, *b* and the threshold are stored with the target FPR, population, benchmark run and operating-point metrics.
- The profile is **validated only if** the run was locally measured **and** the operating point is estimable from the number of validation negatives.
- The audit event records whether it was validated and, if not, why: "simulated data" or the estimability note.

An administrator then attaches the run and profile to the method in the model registry (see the Administration chapter), which refuses validation without a completed, locally measured run and a validated profile of the same method. From then on, analysis jobs pass the profile to the method, which may

set `calibrated` and `flagged` — but only for evidence in the evaluated languages.

21.7 Research Bench endpoints

Endpoint	Purpose
`POST /evaluation/ppv`	Bayes PPV/NPV calculator with expected counts, a prevalence sensitivity table and the source of each input (`research.use`)
`POST /evaluation/threshold-explorer`	For a completed benchmark run, sweep thresholds and project TPR, FPR, PPV and expected false flags to a stated deployment prevalence and population
`GET /evaluation/fairness/{run_id}`	Per-subgroup FPR and FNR with exact intervals and small-group suppression, from a run's samples
`GET /evaluation/robustness/{run_id}`	Metrics per documented transform
`GET /evaluation/experiments`	The nine experiments with their questions and kinds
`POST /evaluation/experiments/{id}/runs`	Run an experiment with optional parameter overrides; store the result with a content hash
`GET /evaluation/experiment-runs[/{id}]`	Experiment history
`GET/POST /benchmarks/datasets`	List or register datasets
`GET/POST /benchmarks/runs` , `GET /benchmarks/runs/{id}` , `GET /benchmarks/runs/{id}/samples`	Benchmark runs and their samples
`GET /benchmarks/calibration-profiles`	All calibration profiles

The read-only researcher role can use the calculators and view benchmark results, but never sees case data.

21.8 Independent evaluation protocol

The repository includes a protocol for third parties, such as a university laboratory or a standards body, assessing an AEGIS deployment before a pilot:

1. **Freeze the system under test** — record the git commit, image digests, the model registry export and calibration profiles.
2. **Use evaluator-held data** the developers and operators have never seen, including held-out generators, domains, languages and documented transformations.
3. **Fix operating points in advance** from the operator's validation profiles; do not tune on test data.
4. **Run** in an isolated instance with datasets registered as locally measured, or import scores; archive the manifests.
5. **Measure** TPR at 1% and 0.1% FPR with exact intervals, AUROC, subgroup FPR/FNR, robustness per transform, short-text slices and calibration; state *n* for every number.
6. **Test the workflow** — end-to-end tests, security tests, and permission-denied, unsupported-media, corrupted-file, provider-outage and integrity-mismatch scenarios.
7. **Review governance** — compliance mapping, threat model, audit anchoring, retention, and the review and appeal procedure.

8. **Report**, including negative results, with the manifests needed to reproduce them; mark anything not covered as unverified.

21.9 Multilingual evaluation plan

Language identification works for English, Hindi and the major Indic scripts, but **no detector has been evaluated for any language** in this build. The plan proceeds in two phases:

- **Phase A, English baseline**: a licensed, consented corpus across at least four domains (academic, news, administrative, informal) with at least 3,000 human-written negatives per evaluation slice so that 0.1% FPR is estimable; several generator families with at least one held out; non-native writers included with consent and an explicit group field.
- **Phase B, Hindi and other Indian languages** (Bengali, Marathi, Tamil, Telugu, Gujarati, Kannada, Malayalam, Punjabi, Odia, Urdu): native-speaker annotators and institutional partners; code-mixed (for example Hinglish) and transliterated text as separate slices; translation sensitivity as documented transforms; revalidation of the 50-token minimum per language because token counts differ greatly by script. A language code is added to a model's `languages_evaluated` only after a locally measured run on that language with estimable operating points.

PART V

The Web Workstation

The Next.js investigative workstation, its architecture and every screen.

CHAPTER 22

The Next.js Workstation

The investigator-facing application is a Next.js 15 workstation in `apps/web`. It talks only to the versioned REST API and holds no secrets. This chapter describes its architecture — routing, authentication, data fetching, uploads, live progress, security headers and the design system — and then walks through every screen.

22.1 Technology

Concern	Choice
Framework	Next.js 15.5 with the App Router, React 18.3, TypeScript 5.7
Styling	Tailwind CSS 3.4 with CSS-variable colour tokens, a dark theme, Inter and JetBrains Mono variable fonts (self-hosted through Fontsource)
Data fetching	TanStack Query 5
Tables	TanStack Table 8
Forms	react-hook-form with zod schemas
Charts	Recharts 2.15 (ROC, PR, PPV curves, reliability diagrams, bars, sparklines)
Icons	lucide-react
Tests	Vitest (unit), Playwright (end to end)
Build	`output: "standalone"` for a small production image

22.2 Source layout

```
apps/web/src/
├── app/
│   ├── layout.tsx              root layout: dark theme, metadata (no indexing), Providers
│   ├── providers.tsx           TanStack Query client + AuthProvider
│   ├── globals.css             design tokens and base styles
│   ├── icon.svg                the AEGIS shield logo
│   ├── login/page.tsx          sign-in (with TOTP step)
│   └── (app)/                  authenticated route group, wrapped in AppShell
│       ├── page.tsx                    Overview dashboard
│       ├── investigations/new/page.tsx New investigation
│       ├── cases/page.tsx              Case files
│       ├── cases/[id]/page.tsx         Case workspace
│       ├── forensics/[modality]/page.tsx  Text / Image / Audio / Video forensics
│       ├── provenance/page.tsx         Provenance & watermarks
│       ├── evidence/page.tsx           Evidence vault
│       ├── research/page.tsx           Research bench
```

```
│       ├── benchmarks/page.tsx         Benchmark laboratory
│       ├── reports/page.tsx            Reports
│       ├── audit/page.tsx              Audit & compliance
│       ├── admin/page.tsx              Administration
│       └── health/page.tsx             System health
├── components/
│   ├── shell.tsx               sidebar, header, search, notifications, organisation switcher
│   ├── analysis.tsx            job view, fusion panel, run cards, artefact images
│   ├── charts.tsx              chart components
│   ├── badges.tsx              status, class, outcome and result-kind badges
│   ├── data-table.tsx          generic table
│   ├── methods.tsx             method capability lists
│   ├── system-map.tsx          live pipeline diagram
│   └── ui/index.tsx            design-system primitives
├── features/case/evidence.tsx  evidence cards, previews, upload and run dialogs
└── lib/
    ├── api.ts                  API client
    ├── auth.tsx                authentication context
    ├── upload.ts               uploads and local hashing
    └── format.ts               formatting and label dictionaries
```

22.3 One origin, no secrets in the browser

The browser never talks to the API directly. `next.config.mjs` rewrites every `/api/*` request to the internal API address (`AEGIS_API_INTERNAL_URL` , default `http://localhost:8000`):

```
async rewrites() {
  return [{ source: "/api/:path*", destination: `${api}/api/:path*` }];
},
```

Because the workstation and the API share one origin, the refresh cookie can be SameSite=Strict, CORS is not needed for the browser, and the Content Security Policy can restrict `connect-src` to `'self'` . The workstation sets its own security headers on every page: `nosniff` , `DENY` framing, `no-referrer` , a restrictive `Permissions-Policy` , and a CSP allowing only same-origin scripts, styles, images (plus `blob:` and `data:` for rendered artefacts), fonts and connections, with `frame-ancestors 'none'` , `base-uri 'self'` and `form-action 'self'` . The `X-Powered-By` header is disabled and search engines are told not to index the site.

22.4 The API client

`src/lib/api.ts` is a thin client with a few important behaviours:

- **The access token lives in a module variable** — memory only, never `localStorage` — as ADR-09 requires. Only the selected organisation id is kept in `sessionStorage` .
- **Every request** carries `Authorization: Bearer …` , `X-AEGIS-Organization` and `X-Requested-With: aegis-web` .

- **Automatic refresh.** When a request returns 401, the client calls `/auth/refresh` once and retries. Concurrent 401s share a single refresh promise, so ten parallel requests produce one refresh, not ten. If refreshing fails, the access token is cleared and session-expired listeners fire, which redirect to the sign-in page with a `next` parameter.
- **Typed errors.** Non-2xx responses become `ApiError` objects carrying the status, the API error code, message, details and correlation id, so error messages in the interface can show the correlation id an operator needs.
- **Authenticated binaries.** Images such as artefacts and thumbnails cannot be loaded with a plain `<img src>` because that would not send the bearer token; `blobUrl()` fetches them with the token and returns an object URL. `download()` does the same for files and honours the server's `Content-Disposition` filename.
- **Server-sent events over fetch.** The browser's `EventSource` cannot send an `Authorization` header, so `streamEvents()` reads the SSE stream through `fetch` and a `ReadableStream` reader, splitting on blank lines and parsing `data:` lines as JSON.

22.5 Authentication context

`src/lib/auth.tsx` provides `useAuth()` with the current user (`Me`), a loading flag, `can(permission)`, `signIn`, `signOut` and `switchOrganization`. On start-up it restores the last organisation from `sessionStorage`, attempts a silent refresh using the cookie and, if that succeeds, loads `/auth/me`. Signing out or switching organisation clears the TanStack Query cache so that no data from one organisation can appear in another.

`AppShell` redirects to `/login` when there is no session, and shows a branded loading bar while the session is being restored.

The TanStack Query client uses a 15-second stale time, does not refetch on window focus, and **does not retry client errors** (any status below 500): retrying a 403 or 404 would only hide a real problem.

22.6 Navigation and permissions

The sidebar groups screens into four sections. Each item declares the permission it needs, and items the user lacks are hidden:

Group	Screen	Permission
Operations	Overview	`case.read`
	New Investigation	`case.create`
	Case Files	`case.read`
Forensics	Text, Image, Audio, Video Forensics	`analysis.read`
	Provenance & Watermarks	`analysis.read`
	Evidence Vault	`evidence.read`
Science	Research Bench	`research.use`
	Benchmark Laboratory	`benchmark.read`
Governance	Reports	`report.read`
	Audit & Compliance	`case.read`
	Administration	`admin.users`

Group	Screen	Permission
	System Health	`system.health`

Hiding a menu item is a convenience, not a control: the server enforces every permission independently. A researcher, for example, sees only the Research Bench and Benchmark Laboratory, and the end-to-end test checks that "Case Files" does not appear for that role.

The sidebar footer carries a permanent reminder: **"Defensive forensic use only. Detector outputs start a review; they never establish misconduct."**

The header contains a global search box (Ctrl-K) that searches cases and evidence by name, reference or SHA-256 prefix; a live system-status pill (for example "Infrastructure operational · 12 methods unavailable"); the deployment mode badge (for example "Demo mode"); the organisation switcher; the notification bell; and the user menu. In demo mode, a banner across the top of every page states: *"Demo mode — cases marked DEMO FIXTURE contain synthetic data only. No figures on this system describe a real deployment."* The shell is responsive, with a slide-out menu on small screens, and includes a "Skip to content" link for keyboard users.

22.7 Uploads with local hashing

`src/lib/upload.ts` implements uploads:

- files up to 8 MB use a single multipart request;
- larger files use the chunked, resumable protocol (`init` → `PUT` chunks at the server's chunk size with the current offset → `complete`), reporting real byte progress;
- `sha256Hex()` computes the file's SHA-256 **in the browser** with the Web Crypto API before upload (for files up to 256 MB, and only in a secure context).

After upload, the interface compares the server's SHA-256 with the locally computed one and shows "matches local hash". This lets the person acquiring evidence confirm that the bytes the server recorded are exactly the bytes they selected. If the local hash is not available — plain HTTP on a non-localhost host, or a very large file — the interface says "local hash unavailable" rather than pretending; the server-side hash is recorded regardless.

The accepted-file list offered to the browser (`.txt, .pdf, .docx, .png, .jpg, .jpeg, .webp, .wav, .mp3, .m4a, .mp4`) is only a convenience; the server ignores extensions and sniffs content. `defaultProfile()` preselects the analysis profile for the modality: `text_triage` for text and documents, `image_investigation`, `synthetic_speech`, `deepfake_video`, otherwise `comprehensive`.

22.8 Live analysis view

`components/analysis.tsx` contains the components that display analysis results:

- `useJob(jobId)` loads the job and subscribes to its SSE stream, refetching the full job whenever a new snapshot arrives and stopping when it reaches a terminal state.
- `JobHeader` shows the status, profile, timestamps, whether external processing was permitted ("Local processing only") and a progress bar of `methods_done / methods_total`.
- `FusionPanel` displays the automated assessment: the primary outcome badge, the fixed statement, signals consistent with AI generation, conflicting signals, unavailable or blocked methods with reasons, failed methods ("not negative results"), uncertainty reasons and suggested next steps.

- `RunsByClass` groups run cards by evidence class, in a fixed order, so a statistical score is never shown next to a provenance verification as if they were comparable.
- `RunCard` shows a run's status, summary, score with its semantics, calibration status, flag (only if validated), warnings, limitations, configuration, duration, segments and artefacts.
- `ArtifactImage` renders each visualisation with its description and its *cannot establish* text directly beneath it.

22.9 Design system

`components/ui/index.tsx` provides the primitives used everywhere: `Badge` (with tones such as ok, warn, danger, accent, slate), `Button`, `Panel`, `PageHeader`, `Stat`, `KV` (key–value grid), `Callout`, `EmptyState`, `Skeleton`, `LoadingBlock`, `ErrorBlock` (which shows the API error code and correlation id), `Progress`, form `Field`, `Input`, `Textarea`, `Select`, `Tabs`, `Dialog`, `Hash` (a monospaced hash with copy-to-clipboard and optional shortening) and `SimpleTable`. `components/badges.tsx` maps domain values to badges: evidence class, run status, job status, case status, outcome, result kind, priority, implementation status and a "DEMO FIXTURE" badge.

`lib/format.ts` contains the label dictionaries — for example the human description of each evidence class and the note attached to each result kind ("Simulated benchmark. Not evidence of real-world performance.") — and number formatters designed **not to invent precision**: `fmtNum(0.99994, 3)` gives `0.9999` rather than rounding up to `1.000`, and `fmtPct(0.99999)` gives `99.999%` rather than `100.0%`. Rounding a 99.999% rate to 100% would imply a certainty the data do not support. These rules are covered by the Vitest unit tests.

Colours are defined as CSS variables (`--bg`, `--surface`, `--raised`, `--line`, `--ink`, `--muted`, `--accent`, `--cyan`, `--violet`, `--ok`, `--warn`, `--danger`) and exposed to Tailwind, giving a consistent dark "operations centre" look with a blue-to-cyan accent taken from the logo.

22.10 Shared types

`packages/shared-types/src/index.ts` (about 380 lines) mirrors the API's response schemas in TypeScript: `EvidenceClass`, `RunStatus` (including the transient `pending`, `running` and `cancelled`), `JobStatus`, `CaseStatus`, `Me`, `Page<T>`, `InvestigationSummary`, `Investigation`, `EvidenceVersion`, `Extraction`, `EvidenceItem`, `EvidenceListItem`, `Artifact`, `DetectorRun`, `Fusion`, `AnalysisJob`, `JobSummary`, `ProfileMethod`, `AnalysisProfile`, `Report`, `AuditEvent`, `MethodInfo`, `HealthComponent`, `RateEstimate`, `OperatingPoint`, `ResultKind`, `BenchmarkRun`, `Dataset` and `Notification`. The workstation imports these types, so a change to the API contract that is not reflected here shows up as a type error in CI.

CHAPTER 23

Screen by Screen

This chapter walks through each screen of the workstation, what it shows and which API calls it relies on.

23.1 Sign-in

The sign-in page asks for email and password. If the account has MFA enabled, the API's `mfa_required` response makes the page reveal a "One-time code" field ("Enter the 6-digit code from your authenticator app."). Errors are shown in a danger callout. The page also states two of AEGIS's principles: *"Absence of a watermark or provenance record proves nothing about authorship."* and *"Every finding carries its method version, input hash and limitations."* After sign-in, the user is returned to the page named in the `next` parameter.

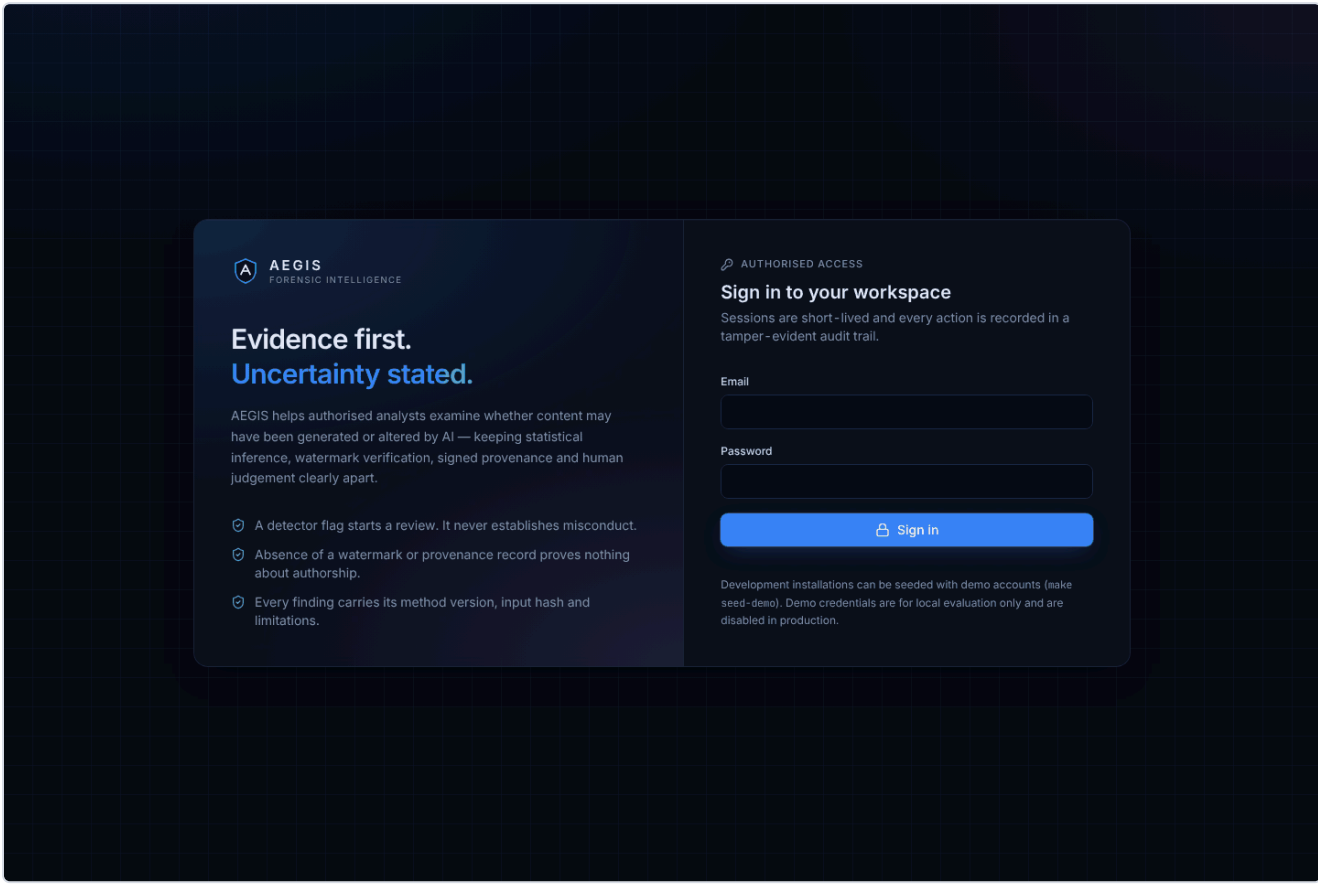


Figure 16. The sign-in screen.

23.2 Overview

The dashboard (`GET /overview` , `GET /health/components` , recent cases, latest benchmark run) shows statistic tiles — investigations, awaiting review, evidence items, completed analyses, inconclusive outcomes and queue depth — a **logical processing pipeline** diagram coloured by live component checks, recent cases, cases by status, automated outcomes (labelled "Descriptive categories from the fusion policy

— not determinations") and the latest completed benchmark run. If that run is simulated, a warning callout says so: *"Simulated benchmark. Not evidence of real-world performance."* Every number is a live count for the user's organisation.

23.3 New investigation

A form with sections for case details — title, reference, **purpose** ("Why this examination is needed and what question it should answer."), media category, priority, assigned independent reviewer ("Must be someone other than you."), access classification ("Restricted cases are visible only to you, the reviewer, auditors and administrators.") and retention class — under the note *"Recorded in the audit trail. Avoid unnecessary personal information."* Validation messages mirror the API's rules (title at least 3 characters, purpose at least 10). On success the user is taken to the new case.

23.4 Case files

A filterable, sortable, paginated table of the cases the user may see ("Investigations visible to you under your organisation and access classification."), with text search on title or reference and filters for status and priority. Restricted cases carry a "Restricted" badge; demo cases carry "DEMO FIXTURE".

23.5 The case workspace

The most important screen (`apps/web/src/app/(app)/cases/[id]/page.tsx`, about 550 lines). It contains:

- **A status stepper** showing where the case is in the workflow.
- **Case record**: reference, purpose, classification, retention, legal basis, reviewer, creator, and the external-processing status ("Permitted for this case (subject to deployment and organisation settings)" or "Not permitted — local processing only").
- **Latest automated outcomes** for each evidence item.
- **Safeguards**: reminders of the separation-of-duties rules that apply.
- **Evidence**: an evidence card per item with its thumbnail or text preview, SHA-256, size, MIME type, versions, extraction warnings, integrity status and actions — download, verify, redact (for text) and run analysis. The upload dialog shows the local and server hashes. The run dialog lists the profiles with each method's availability, so the user knows in advance what will run.
- **Analysis jobs**: the list of jobs and the live job view described above.
- **Review**: for the assigned reviewer while the case is in human review, a form to record a decision, a rationale (at least 20 characters) and agreement with the automated outcome; the recorded decision shows its decision hash.
- **Reports**: generate, finalise, export, verify and correct.
- **Annotations, appeals and deletion requests**.
- **Case history**: "From the hash-chained audit trail" — the case's audit events with their hashes.
- **Reopen** and **archive** dialogs. Reopening explains: *"Reopening is recorded with your reason. Finalised reports are preserved; changes produce a new report version."* and requires a reason of at least 10 characters.

Buttons for actions the user is not permitted to perform are hidden or disabled, and every action error is shown with its API message.

23.6 Text, Image, Audio and Video forensics

One dynamic route (`forensics/[modality]`) serves four views, each built on `GET /evidence?modality=…`. Each view opens with cards stating the limits of that modality — for example, for text, *"No detector can reliably establish authorship from text alone; a low score does not prove human authorship."*, *"Short texts (< 50 tokens), non-native writing, translation and heavy editing all degrade reliability."* and *"Sentence-level scores, where validated, do not establish who wrote each sentence."*; for images, *"Forensic visualisations are observations for an analyst — none is a classifier."*; for video, *"No facial recognition or identity judgement is performed."* — followed by a list of evidence of that modality across visible cases (filterable by name or SHA-256 prefix) and, for the selected item, its latest job and results. Items that have not been analysed are marked "Not yet analysed". A capability panel lists the methods for the modality and whether each is available.

23.7 Provenance & watermarks

This screen explains the three things provenance verification keeps apart, in three callouts:

- **Valid signature** — "The manifest is intact and was signed by the certificate holder."
- **Trusted signer** — "The certificate chains to a trust anchor configured by this deployment."
- **Truthful claim** — "Never established by verification alone. Claims require corroboration."

It then lists provenance records ("C2PA Content Credentials verified by the official c2pa-rs library"), the verification methods with their status, and a **watermark key management** panel that shows the reference key's fingerprint — never the key — and explains the scope of the reference verifier.

23.8 Evidence vault

Tiles for evidence items, stored originals, integrity alerts and process evidence; a searchable table of all evidence with modality filter and latest automated outcome. If any item has an integrity alert, a danger callout states: *"At least one stored item no longer matches its acquisition hash. It is blocked from analysis until investigated."*

23.9 Research bench

The **base-rate calculator** ("Bayes' rule: how many flags are correct in a given screening population?") takes sensitivity, false-positive rate, prevalence and the number of items screened, each with a selectable source (measured, assumed, published, vendor-reported, simulated), and shows the PPV, false flags per true flag, NPV and expected flags, a PPV-versus-prevalence curve and the assumptions statement. The **threshold explorer** projects a benchmark run's operating points to a deployment population. The **statistical reliability laboratory** lists the nine experiments; each can be run, and its results are displayed as tables and charts with its kind (analytical or simulation) clearly labelled. Fairness and robustness views operate on completed benchmark runs.

23.10 Benchmark laboratory

Datasets (with registration requiring provenance, licence and consent), runs and imported scores. For a selected run, a callout labels the result kind — green for locally measured, amber otherwise, with the citation for published or vendor-reported results — followed by tiles for AUROC, test samples, threshold, TPR and FPR; an ROC curve ("Strict-FPR region matters most for screening"); a precision–recall curve; a table of

sensitivity at strict false-positive operating points with estimability; the confusion matrix at the selected threshold (noting that "Precision/NPV reflect this sample's class balance, not a deployment population"); calibration (Brier score, expected calibration error and a reliability diagram); and robustness to documented transformations ("Defensive measurement only — not an evasion service"). Users with `admin.models` can create a calibration profile from a run.

23.11 Reports

A list of all reports with version, status and hashes, and three explanatory callouts: **Reproducible** ("The PDF is rendered deterministically from the canonical JSON payload; verification re-renders it and compares hashes."), **Immutable once final** ("Corrections create a new version and preserve the previous record.") and **Measured language** ("Reports use 'consistent with', 'inconclusive' and 'verification succeeded for the supported record' — never certainty claims."). Verification results are shown as, for example, "intact & reproducible".

23.12 Audit & compliance

Two tabs. **Audit trail** (for users with `audit.read`): a "Chain integrity" panel that verifies the chain on demand and shows either "Chain valid — *N* events recomputed" or "Chain broken at event #*n*: *reason*", together with the limitation of hash chaining; an "Anchor the chain head" panel showing the current head to copy into an external system; and an events table filterable by action prefix (for example `evidence.`). **Compliance mapping**: the controls from `compliance-mapping.json` with their statuses and evidence, under the disclaimer that the mapping is not a compliance claim.

23.13 Administration

Members (list, add with role and an initial password meeting the policy, change role, deactivate, revoke sessions); **Roles and permissions** ("Enforced on the server for every request"), showing the full matrix; **Model and detector registry** ("A detector can only be marked validated with a completed, locally measured benchmark run. All changes are audited."), where each method can be enabled or disabled and given a licence, validation status, evaluated languages, supporting benchmark run, calibration profile, threshold and note, always with a reason; **Settings & retention**: the organisation's external-processing switch, platform settings, retention periods and the list of cases due for retention review.

23.14 System health

The processing pipeline diagram, every component with its live status and detail, jobs by status, and the full list of analysis methods with implementation status, availability and the reason for any unavailability.

PART VI

Quality, Deployment and Operations

Tests, container images, deployment targets and operations.

CHAPTER 24

Testing Strategy

AEGIS makes strong claims — that it never fabricates a score, that tenants are isolated, that tampering is detected — and each of those claims is backed by an automated test. The README's badge reports **101 Python tests, 3 web unit tests and 3 browser end-to-end tests**. This chapter describes how the suites are organised, how the API harness works, and which test proves which property.

24.1 Test suites

Suite	Location	Needs	Focus
Research	`research/tests/`	Python only	Statistics, intervals, PPV, AUROC, operating points, Platt calibration, watermark scheme, experiments
Analysis methods	`ml/tests/`	Python, ffmpeg	Sniffing, extraction, language, segmentation, image/audio/video methods, C2PA states, provider contract, profiles, fusion
API	`apps/api/tests/`	PostgreSQL	Security, governance, evidence, workflow end to end, research API, hosted deployment
Web unit	`apps/web/tests/`	Node	Formatting helpers
Browser end to end	`apps/web/tests/e2e/`	A running, seeded stack	Full investigator → reviewer → report → audit workflow; permission-denied paths; a tour of every screen

One `pytest` run executes all three Python suites. While this document was being prepared, the research and analysis-method suites were run in the build container and **all 52 of their tests passed**; the API suite requires a PostgreSQL server, which was not available in that container, and was therefore not re-run here (it runs in CI against a PostgreSQL 16 service).

24.2 The API test harness

`apps/api/tests/conftest.py` sets up an isolated, realistic environment:

- environment variables are set **before** the application is imported: `AEGIS_ENVIRONMENT=test`, a dedicated database (`AEGIS_TEST_DATABASE_URL`, default `aegis_test`), a temporary evidence directory, `AEGIS_TASK_MODE=manual`, test-only secret and watermark keys;
- any provider, model or trust-anchor variables present in the developer's shell are deleted, so tests never accidentally contact a real provider or load real models;
- at session start, the `public` schema is **dropped and recreated** and `alembic upgrade head` is run from empty — every test run therefore exercises the full migration history — followed by `sync_roles()` and `sync_registry()`;

- with `task_mode=manual`, jobs are created but not dispatched; tests call the worker entry point (`execute_job`) explicitly, which makes asynchronous behaviour deterministic;
- the in-process rate-limit store is reset between tests;
- helpers create organisations, users with each role, signed-in clients and cases.

24.3 What each API test proves

Security (`test_security.py`)

Test	Property
`test_requires_authentication`	Protected endpoints return 401 without a token
`test_error_format_and_headers`	Errors use the uniform envelope; security headers and `X-Request-ID` are present
`test_role_permissions_enforced_server_side`	Each role is refused operations outside its permissions
`test_cross_tenant_isolation`	Another organisation's identifiers return 404
`test_restricted_classification`	Restricted cases are invisible to users without `case.read_restricted` unless they are creator or reviewer
`test_malicious_uploads_rejected`	Executables and unsupported content are refused
`test_zip_traversal_and_bomb_rejected`	Archive path traversal and compression bombs are refused
`test_oversized_upload`	Size limits return 413
`test_filename_is_sanitised`	Path components and control characters are stripped
`test_login_rate_limit_and_lockout`	Login rate limiting and lockout after 10 failures
`test_session_revocation`	A revoked session's access token stops working immediately
`test_refresh_requires_csrf_header`	Refresh fails without `X-Requested-With: aegis-web`
`test_no_secrets_in_responses`	Password hashes, MFA secrets and keys never appear in responses
`test_sql_injection_in_filters_is_inert`	SQL metacharacters in filters have no effect
`test_report_tampering_detected`	Editing a stored report payload makes verification fail
`test_audit_tampering_detected`	Editing an audit event makes chain verification fail at the right sequence number

Evidence and governance (`test_evidence_governance.py`)

Test	Property
`test_integrity_verification_and_mismatch_alert`	Altered stored bytes produce `integrity_alert`, a critical notification and an audit event
`test_storage_is_write_once`	A second write to an existing key fails
`test_s3_backend_with_moto`	The S3 backend works (against the `moto` mock)
`test_redaction_creates_new_version_preserving_original`	Redaction never modifies the original

Test	Property
`test_download_requires_permission_and_is_audited`	Downloads need `evidence.download` and write an audit event
`test_chunked_upload`	The resumable upload protocol, including offset checks
`test_deletion_two_person_rule_and_legal_hold`	Deletion needs a second person and is blocked by legal hold
`test_requester_cannot_approve_own_deletion`	The two-person rule holds
`test_state_machine_rules`	Invalid transitions are refused; reopening needs permission and a reason
`test_appeal_workflow`	Appeals cannot be resolved by the appellant
`test_process_evidence_role`	Process evidence is stored as its own role
`test_overview_counts_are_real`	Dashboard numbers equal database counts
`test_idempotent_job_creation_and_resume`	An `Idempotency-Key` returns the same job; re-executing a job does not repeat completed runs
`test_cancel_queued_job`	Cancelling a queued job cancels its runs
`test_failed_method_is_not_negative`	A crashing method is recorded as `failed`, never as a negative result
`test_provider_failure_and_policy`	External providers are blocked by policy by default and invalid responses fail

Workflow end to end (`test_workflow_e2e.py`)

- `test_full_text_workflow` — create a case, upload text, run `text_triage`, check the fusion outcome, request review, have a *different* user record the decision, generate a report, have a third user finalise it, verify the report, verify the audit chain; along the way, separation-of-duties refusals are checked.
- `test_inconclusive_and_insufficient_input` — short text yields `insufficient_input` and an honest outcome.
- `test_media_and_provenance_jobs` — image, audio and video jobs, including C2PA verification, run end to end.

Research API and hosted deployment

`test_research_api.py` checks that the PPV calculator matches the paper's example, that experiments are labelled analytical or simulation, that the reference watermark benchmark and threshold explorer work, that dataset registration requires documentation and detects cross-split contamination, that imported published scores require a citation, and that health and detector endpoints report truthfully.

`test_hosted_deployment.py` covers the single-container mode: the PostgreSQL storage backend round-trips data and is write-once, the evidence workflow works on it, hosted `postgres://` URLs are rewritten to psycopg, an unconfigured Redis is reported honestly as "not configured", and interrupted inline jobs are resumed at start-up.

24.4 Analysis-method tests

`ml/tests/test_sniff_and_text.py` checks content-based sniffing (a PNG named `.txt` is a PNG), rejections (executables, traversal, bombs), TXT/PDF/DOCX extraction, that prompt-injection phrases are recorded as an observation, segmentation and token counts, Hindi and English language identification, that

descriptive statistics never produce a score, that an unconfigured model is `unavailable` rather than negative, and the reference watermark end to end.

`ml/tests/test_media_provenance_fusion.py` checks the image methods, that ELA skips PNG, declared metadata, that corrupt images and audio fail explicitly, audio signal analysis, video methods, all C2PA states (signed, trusted, tampered, absent), provider-contract validation, that every profile references registered methods, and the fusion rules: failures and unavailability are never negative, uninterpreted scores never flag, and categories and contradictions are assigned correctly.

Fixtures (`ml/aegis_ml/fixtures.py`) generate every test file at run time — JPEGs with EXIF, PNGs with generator declarations, WAV tones, MP4 test patterns, PDFs, DOCX files, a zip-bomb-like archive, a local test certificate authority, C2PA-signed JPEGs and tampered copies — so no binary test data needs to be committed.

24.5 Research tests

`research/tests/test_reliability.py` (15 tests) checks the PPV paper example and edge cases, NPV and expected counts, that rate sources are recorded, Wilson and exact intervals, the minimum-negatives rule, confusion metrics, AUROC, TPR at FPR with estimability, ROC and threshold sweeps, calibration metrics, that Platt calibration improves log-loss, independence versus empirical joint rates, subgroup share and suppression, and the evaluation bundle. `test_watermark_and_experiments.py` (9 tests) checks key requirements and fingerprints, that the green fraction matches γ, that watermarked text is detected and plain text is not, that short input is insufficient, consistency of exact FPR and p-values, the token-replacement helper, the paper example in experiment 1, and that all experiments run and are labelled.

24.6 Web tests

The Vitest unit tests (`apps/web/tests/format.test.ts`) check the formatting helpers, in particular that numbers are formatted **without inventing precision**.

The Playwright suite (`apps/web/tests/e2e/workflow.spec.ts`) runs against a live, seeded stack:

1. **Investigator → analysis → reviewer decision → report → audit.** The investigator creates a case, uploads a text file (checking "local SHA-256" and "matches local hash" in a secure context, or "local hash unavailable" otherwise), runs text triage, sees the automated assessment ("Inconclusive", unavailable methods and the note "Only descriptive statistics and the reference watermark verifier ran."), requests review; the reviewer records a decision and sees its decision hash; the investigator generates report v1 and verifies it as "intact & reproducible"; the reviewer finalises it; the auditor verifies the chain ("Chain valid").
2. **Permission-denied paths and capability honesty.** The researcher sees the Research Bench but not Case Files; an investigator sees "Not permitted" where appropriate.
3. **A tour of every primary screen**, checking each page heading renders.

The Playwright configuration reads the target from `AEGIS_WEB_URL` (default `http://localhost:3000`). The Render guide records that the suite passed 3 of 3 against the single-container image running under free-plan resource limits.

24.7 Static checks in CI

Beyond tests, CI runs Ruff (lint and format check, including bandit security rules), ESLint with zero warnings allowed, the TypeScript compiler, a production Next.js build, `pip-audit` and `pnpm audit --prod`. The Docker images are built in a separate job. A dependency advisory fails the pipeline just like a failing test.

CHAPTER 25

Deployment

AEGIS can run in three ways: as a full multi-container stack with Docker Compose (on a workstation or a server), on a campus server behind an automatic HTTPS proxy, or as a single container on Render's free plan for demonstrations. This chapter describes the container images and each deployment target.

25.1 Container images

Three Dockerfiles live in `infrastructure/docker/`. All are multi-stage builds, run as an unprivileged user (`aegis`, UID 10001), and support an optional build-time CA certificate for TLS-inspecting corporate proxies, passed as a BuildKit secret so that it is **never stored in an image layer**:

```
DOCKER_BUILDKIT=1 docker build --secret id=extra_ca,src=/etc/ssl/proxy-ca.pem \
  -f infrastructure/docker/api.Dockerfile -t aegis-api .
```

API and worker image (`api.Dockerfile`)

- Base: `python:3.12-slim-bookworm`.
- **Static ffmpeg and ffprobe** copied from `mwader/static-ffmpeg:7.1` (ADR-11): no distribution package mirror is needed and the media toolchain version is pinned.
- uv 0.5.11 installs the locked dependencies into `/opt/venv`, first with only the lock and project metadata (a cacheable dependency layer), then with the source.
- Copies `research`, `ml`, `apps/api` and `docs/governance` (for the compliance mapping endpoint).
- Creates `/var/lib/aegis/evidence` owned by the `aegis` user and sets `AEGIS_STORAGE_ROOT` to it.
- A `HEALTHCHECK` calls `/api/v1/health/live` every 30 seconds.
- Default command: `uvicorn app.main:app --host 0.0.0.0 --port 8000 --proxy-headers --forwarded-allow-ips "*" --workers 2`.

The **worker** and **beat** use the same image with a Celery command, so the API and workers always share exactly the same code and method versions.

Web image (`web.Dockerfile`)

- Base: `node:22-bookworm-slim`; pnpm 10.28.0; frozen install of only the web package and its workspace dependencies.
- The Next.js rewrite target is fixed **at build time** through the `AEGIS_API_INTERNAL_URL` build argument (default `http://api:8000`).
- The runtime stage copies only Next.js's `standalone` output and static assets and runs `node apps/web/server.js` on port 3000.

Single-container image (`render.Dockerfile`)

Combines both: it builds the web workstation with the rewrite target `http://127.0.0.1:8000`, then builds the Python environment, copies the Node binary, the standalone web build, `infrastructure/render/serve.py` and `start.sh`, and listens on port 10000. Its start-up sequence is described in the Render section below.

25.2 Docker Compose

`docker-compose.yml` defines the full stack. All services publish ports on **127.0.0.1 only**; any shared use needs a TLS reverse proxy in front.

Service	Image	Purpose	Notes
`postgres`	`postgres:16-alpine`	Database	Health check with `pg_isready`; volume `pgdata`
`redis`	`redis:7-alpine`	Broker, rate limits, heartbeat	Append-only persistence; volume `redisdata`
`minio`	MinIO (profile `s3`)	S3-compatible evidence store	Optional
`clamav`	`clamav/clamav:1.4` (profile `scan`)	Malware scanning	Optional
`migrate`	API image	`alembic upgrade head && python -m app.cli sync`	Runs once after PostgreSQL is healthy
`api`	API image	REST API on 8000	Starts after `migrate` succeeds and Redis is healthy; volume `evidence`
`worker`	API image	Celery worker on queues `aegis` and `aegis-research`, concurrency 2	Health check with `celery inspect ping`; 60 s stop grace period so a running method can finish
`beat`	API image	Celery beat (stale-job recovery every 10 minutes)	
`web`	Web image	Workstation on 3000	

A YAML anchor shares the API environment between `migrate`, `api`, `worker` and `beat`. `AEGIS_SECRET_KEY` uses the `${VAR:?message}` form, so Compose **refuses to start** if it is not set. Images are tagged with `AEGIS_VERSION`, which is what the rollback procedure relies on.

```
cp .env.example .env                                  # set AEGIS_SECRET_KEY to a long random value
docker compose up --build -d
docker compose exec api python -m app.cli seed-demo   # development only
```

Optional profiles: `--profile s3` with `AEGIS_STORAGE_BACKEND=s3`, `AEGIS_S3_ENDPOINT_URL=http://minio:9000` and credentials; `--profile scan` with `AEGIS_CLAMAV_HOST=clamav` and `AEGIS_REQUIRE_MALWARE_SCAN=true`.

25.3 Production configuration

The deployment guide lists the settings that must change for any real use:

Setting	Production value
`AEGIS_SECRET_KEY`	64+ random characters (`python -c "import secrets;print(secrets.token_urlsafe(64))"`)
`POSTGRES_PASSWORD`	A strong password

Setting	Production value
`AEGIS_ENVIRONMENT`	`production` — the API refuses the development key, inline tasks and insecure cookies
`AEGIS_MODE`	`research` or `connected-production`
`AEGIS_COOKIE_SECURE`	`true`
`AEGIS_CORS_ORIGINS`	The public origin, e.g. `https://aegis.example.org`
`AEGIS_REFERENCE_WATERMARK_KEY`	A new secret, or empty to disable the verifier

Reverse proxy

An example nginx configuration is provided. The important points are a request-body limit at least as large as `AEGIS_MAX_UPLOAD_MB` (520 MB for the default 512 MB), `proxy_buffering off` for `/api/` so that server-sent events stream, and a long read timeout:

```
server {
  listen 443 ssl http2;
  server_name aegis.example.org;
  client_max_body_size 520m;
  location /api/ { proxy_pass http://127.0.0.1:8000; proxy_set_header Host $host;
                   proxy_set_header X-Forwarded-Proto https; proxy_buffering off;
                   proxy_read_timeout 3600s; }
  location /      { proxy_pass http://127.0.0.1:3000; proxy_set_header Host $host; }
}
```

Production checklist

- `AEGIS_ENVIRONMENT=production` (the API refuses to start with unsafe settings).
- TLS everywhere; `AEGIS_COOKIE_SECURE=true`; HSTS at the proxy.
- Secrets injected from a secret manager, not committed `.env` files.
- Disk encryption for PostgreSQL and evidence volumes, or an S3 bucket with SSE and Object Lock.
- ClamAV enabled and `AEGIS_REQUIRE_MALWARE_SCAN=true`.
- External processing off unless contracts and a data-processing assessment exist.
- Backups scheduled and a restore test executed.
- Audit-chain head anchored daily.
- Demo seeding never run (it is refused in production anyway).
- Legal, privacy and security review completed.

Rollback

Stop `api`, `worker`, `beat` and `web`; restore the database snapshot taken before the upgrade if a migration ran; start the previous image tag. Migrations are forward-only in production.

25.4 Campus or college server

`scripts/server-setup.sh` installs AEGIS on a Linux server with one command, for an IT team that wants everyone on campus to reach it at an address such as `https://aegis.yourcollege.edu`. The recommended machine is Ubuntu 22.04/24.04 LTS with 4 CPUs, 8 GB RAM and 100 GB disk, Docker Engine with

Compose v2, a DNS record, and inbound TCP 80 and 443.

```
git clone https://github.com/kryptonite012/AEGIS-AI-Content-Intelligence-Forensics-System.git aegis
cd aegis
sudo ./scripts/server-setup.sh aegis.yourcollege.edu
```

The script:

1. checks for Docker and Compose v2;
2. chooses the TLS mode — a public DNS name gets an automatic **Let's Encrypt** certificate; an IP address or an internal name (no dot, `.local`, `.lan`, `.internal`) gets a certificate from **Caddy's internal CA** (`tls internal`);
3. on first run, creates `.env` from the template with **random secrets**: a 64-character secret key, a 40-character watermark key, a random demo password and a 32-character PostgreSQL password; re-running keeps existing secrets;
4. sets `AEGIS_SITE`, the TLS directive, `AEGIS_ENVIRONMENT=production`, `AEGIS_MODE=research`, `AEGIS_COOKIE_SECURE=true` and `AEGIS_CORS_ORIGINS=https://<site>`, and makes `.env` readable by root only (`chmod 600`);
5. builds and starts the stack with the server overlay;
6. waits for `/health/ready`;
7. if no system administrator exists, prompts for the administrator's email, name, organisation and password and runs `create-admin`;
8. prints the address.

The server overlay (`infrastructure/deployment/docker-compose.server.yml`) adds **Caddy 2.8** on ports 80 and 443, removes the host port mappings of PostgreSQL, Redis, the API and the web service (`ports: !reset []`, so nothing but Caddy is reachable from outside), and keeps Caddy's certificates in volumes. The `Caddyfile` compresses responses, limits request bodies to 520 MB, adds HSTS, removes the `Server` header, proxies `/api/*` to the API with `flush_interval -1` (immediate flushing for server-sent events) and a one-hour read timeout, and proxies everything else to the web service.

Day-to-day commands:

```
C="docker compose -f docker-compose.yml -f infrastructure/deployment/docker-compose.server.yml"
$C ps                                       # status of every service
$C logs -f api worker                       # live logs
git pull && $C up -d --build                # update (back up first)
$C exec api python -m app.cli verify-audit  # check the audit chain
```

Demo accounts are not created on a server, because seeding is disabled in production mode. For an internal certificate, IT can distribute Caddy's root certificate to campus machines to remove the browser warning.

25.5 Render (free plan)

`render.yaml` is a Render Blueprint that deploys AEGIS as **one free web service plus one free PostgreSQL database** in the Singapore region, through the "Deploy to Render" button or New → Blueprint. It is a hosted demonstration with synthetic data only.

Setting	Value	Why
Runtime	Docker, `render.Dockerfile`	One container serves both web and API
Health check	`/api/v1/health/live`	
Build filter	Ignores Markdown, screenshots and browser tests	Saves monthly build minutes
`AEGIS_DATABASE_URL`	From the Render database	
`AEGIS_DEMO_PASSWORD`	Entered by the operator (`sync: false`)	
`AEGIS_SECRET_KEY` , `AEGIS_REFERENCE_WATERMARK_KEY`	`generateValue: true`	Random, never seen by anyone
`AEGIS_ENVIRONMENT`	`hosted-demo`	Production checks would refuse inline mode
`AEGIS_SEED_DEMO`	`true`	Load demo data at start-up
`AEGIS_TASK_MODE`	`inline`	No free background workers
`AEGIS_RESUME_INLINE_JOBS_ON_STARTUP`	`true`	Resume jobs interrupted by sleep
`AEGIS_STORAGE_BACKEND`	`database`	Free services lose their filesystem when they sleep
`AEGIS_COOKIE_SECURE`	`true`	Render serves HTTPS
`AEGIS_MAX_UPLOAD_MB`	`25`	Free-plan resources

The database has an empty IP allow-list, so it is reachable only from services in the same Render workspace. An alternative Blueprint, `infrastructure/render/render-external-database.yaml` , takes the database URL as an input instead, for use with Neon or any other PostgreSQL 14+ service — useful when a workspace's single free Render database is already in use or the demo must outlive Render's 30-day database expiry.

Start-up sequence

`infrastructure/render/start.sh` :

1. leaves `AEGIS_REDIS_URL` empty unless configured (in-process jobs and rate limits);
2. sets `AEGIS_CORS_ORIGINS` to Render's `RENDER_EXTERNAL_URL` if not set;
3. starts `serve.py` in the background and **waits until the API port answers** before starting the web server, so the public port opens only when the API can respond;
4. starts the Next.js server on `$PORT` with a 160 MB V8 heap limit;
5. exits when either process stops, so the platform restarts the container.

`infrastructure/render/serve.py` does everything in **one Python interpreter**, because on a 0.1-CPU instance every extra interpreter start costs tens of seconds and another copy of the scientific libraries in memory: it applies migrations, synchronises roles and the registry, and — if `AEGIS_SEED_DEMO=true` — starts a background thread that waits for the API to answer and then loads the demo data. It refuses to seed if the demo password is shorter than 12 characters or equals the publicly documented default, printing "Demo data not loaded: set AEGIS_DEMO_PASSWORD to a private value of at least 12 characters." Seeding waits until the API's start-up hook has resumed interrupted jobs, so no job is executed twice. Finally it runs uvicorn on 127.0.0.1:8000 with a 20-second graceful shutdown.

Measured behaviour under free-plan limits

The Render guide records measurements made by running the image locally with Docker limited to 0.1 CPU and 512 MB of memory, with the database in a separate container:

Check	Result
First start on an empty database until the public port answered	69 s
Loading the demo organisation and three DEMO FIXTURE cases, including analyses	49 s more, in the background
Restart after the container was deleted (files lost, as when Render puts the service to sleep)	54 s; all 9 evidence items verified OK; audit chain valid
A video job killed mid-run by deleting the container	Resumed at start-up (attempt 2) and completed; the finished step was not repeated
Peak memory (API, web, demo loading, browser tests)	308 MiB of 512 MiB
Browser test suite against the container	3 of 3 passed

Free-plan limits

Services sleep after 15 minutes without visitors (the first visit afterwards waits one to two minutes); 0.1 CPU and 512 MB memory; 750 free instance hours per month shared by all free services; one free PostgreSQL database per workspace, 1 GB, expiring after 30 days; no shell on free services. The guide is explicit that this mode is for demonstrations with synthetic data, never real evidence or personal data.

25.6 Choosing a deployment

Need	Use
Show AEGIS to someone with a public link	Render Blueprint
Develop or evaluate on one machine	Docker Compose, or local development with `make dev`
Shared use within an institution	College-server script (Compose + Caddy + production mode), after the governance review
Larger scale	Compose or an orchestrator with multiple API replicas, horizontally scaled workers (separate queues and GPU pools for model-backed methods), S3 with Object Lock, managed PostgreSQL with point-in-time recovery

CHAPTER 26

Operations

This chapter collects what an operator needs once AEGIS is running: enabling real detectors, backup and recovery, incident response, performance, and troubleshooting.

26.1 Configuring a real detector

Out of the box AEGIS runs every method that needs no model weights: provenance, the reference watermark, metadata and the forensic observations. Each learned detector reports `unavailable` until it is configured. The procedure in `docs/operations/configuring-detectors.md` has six steps.

1. Obtain weights legitimately. Check the licence, the publisher's documentation and the intended use. Record the SHA-256 of every weight file. Place the files on the worker hosts; AEGIS never downloads models at run time (threat T11, model supply chain).

2. Install the optional runtime.

```
uv sync --frozen --extra transformers      # torch + transformers for the text adapters
uv pip install onnxruntime                 # for the ONNX image/speech adapters
```

3. Point AEGIS at the model (worker environment):

Method	Variables
Supervised text classifier	`AEGIS_TEXT_CLASSIFIER_PATH` and `AEGIS_TEXT_CLASSIFIER_AI_LABEL` (AEGIS refuses to guess the label)
Token statistics, Fast-DetectGPT	`AEGIS_SCORING_LM_PATH` (a local Hugging Face causal LM directory)
Binoculars	`AEGIS_BINOCULARS_OBSERVER_PATH`, `AEGIS_BINOCULARS_PERFORMER_PATH` (must share a tokenizer)
Image / speech classifier	`AEGIS_IMAGE_MODEL_DIR`, `AEGIS_SPEECH_MODEL_DIR`, each containing `model.onnx` and `aegis_model.json`
Device	`AEGIS_INFERENCE_DEVICE=cuda` (falls back to CPU if CUDA is unavailable)

Restart the workers; System Health should list the method as operational.

4. Benchmark it locally in the Benchmark Laboratory on a registered, labelled dataset with validation and test splits, including held-out generators or domains, the intended languages, short texts and documented transformations.

5. Calibrate and validate. Create a calibration profile from the completed run (Platt on the validation split, threshold at the target FPR; validated only if locally measured and estimable). In Administration → Model registry, attach the run and the profile, enter the licence and evaluated languages, set the status to validated, and give a reason. Until then outputs are shown but never interpreted — they never flag evidence and are never presented as probabilities.

6. External providers follow the provider contract, and the deployment, organisation and case must all permit external processing.

The model card template (`docs/research/model-cards.md`) lists what to record for each configured model: identifier and source, weights checksum, licence, intended use ("triage signal for human review only"), supported media and evaluated languages, minimum input, training-data documentation, evaluation in AEGIS (benchmark run ids, result kind, dataset version, TPR at 1%/0.1% FPR with intervals), calibration, known limitations, and out-of-scope uses ("any decision about a person based on this score alone").

26.2 Backup and recovery

AEGIS state lives in three places, which must be backed up together so that hashes, audit events and objects stay consistent:

Store	Contents	Method
PostgreSQL	Cases, hashes, runs, report metadata, the audit chain (and evidence, with the database backend)	`pg_dump -Fc` nightly plus WAL archiving, or managed point-in-time recovery
Evidence store	Originals, derivatives, artefacts, report PDFs, datasets	Filesystem snapshot or `rsync` of `AEGIS_STORAGE_ROOT`; on S3, bucket versioning with Object Lock and cross-region replication
Configuration	`.env` or secret-manager entries, trust anchors, model directories	Secret-manager backup; record model-weight checksums

Redis holds only queue messages, rate-limit counters and the heartbeat. Losing it re-queues nothing by itself; run `python -m app.cli recover-jobs` afterwards.

Example with Compose:

```
docker compose exec -T postgres pg_dump -U aegis -Fc aegis > backup/aegis-$(date +%F).dump
docker run --rm -v aegis_evidence:/src -v "$PWD/backup":/dst alpine tar czf /dst/evidence-$(date
+%F).tgz -C /src .
docker compose exec api python -m app.cli verify-audit > backup/audit-head-$(date +%F).json
```

The last command also records the audit-chain head, which doubles as an external anchor.

Restore test (quarterly, with the result recorded):

1. Restore the dump into an empty database.
2. Restore the evidence archive to an empty volume.
3. Start an isolated stack against the restored copies.
4. Run `verify-audit`: it must report `valid: true` and its head must match the anchored value.
5. Run `verify-evidence`: every item must report `OK`; any `MISMATCH` means the backup is inconsistent.
6. Open a case and verify a finalised report: it must be intact and reproducible.

The documentation states plainly that the procedure and the verification commands exist and are tested, but that a restore test against real backups has not been run in the repository and must be run by the operator.

26.3 Incident response

The incident-response plan (`docs/operations/incident-response.md`) is a template to adapt to an institution's process:

Severity	Examples	Response target
Critical	Evidence integrity alert; audit-chain verification failure; suspected cross-tenant access; credential or watermark-key compromise	Immediate escalation; contain within 1 hour
High	Malware detected on upload; repeated authorisation failures from one account; anomalous provider responses	Same working day
Medium	Worker outage; storage nearly full; dependency advisory affecting a deployed component	2 working days

Playbooks:

- **Integrity alert** (`evidence.integrity_mismatch`): do not re-run analyses or export reports for the item; preserve the storage object, rows and logs; compare with backups to find when the change happened; check access logs and the audit trail around that time; notify the case owner and reviewer and record findings as annotations.
- **Audit chain broken** (`/audit/verify` returns `valid: false`): snapshot the database; compare the chain with anchored heads to locate the window; treat it as a privileged-access incident.
- **Credential compromise**: deactivate the account (which revokes its sessions); rotate `AEGIS_SECRET_KEY` (this invalidates every access token and every stored MFA secret, so users re-enrol MFA), provider tokens and the watermark key; review the account's audit trail.
- **Provider anomaly**: disable the method in the model registry (audited) and investigate; disabled methods are recorded as unavailable in later runs.

Every incident ends with a post-incident review: timeline, root cause, corrective actions and any changes needed to the compliance mapping.

26.4 Performance

All numbers in `docs/operations/performance.md` were **measured** with `scripts/loadtest.py` and describe only the stated conditions.

Condition	Value
Host	Cloud development container, 4 vCPU, 15 GiB RAM, everything on one host
Services	PostgreSQL 16, Redis 7, uvicorn with 2 workers, Celery concurrency 2, local storage
Software	Python 3.13, FastAPI 0.143, SQLAlchemy 2.0, Celery 5.6
Rate limit	Raised to 100,000/min for the test (the default 600/min per user would throttle a single-user test)
Command	`python scripts/loadtest.py --concurrency 8 --requests 400 --jobs 20`

Condition	Value
Date	2026-10-18

Phase	Requests	Errors	Throughput	p50	p95	p99
Read (`GET /investigations` , `GET /overview`), 8 concurrent	400	0	139 req/s	51 ms	98 ms	145 ms
PPV calculator, 8 concurrent	400	0	265 req/s	28 ms	48 ms	52 ms
Text evidence upload (≈3 KB, sequential)	20	0	18.6 req/s	45 ms	52 ms	273 ms

Twenty `text_triage` jobs queued together all completed in 2.17 s wall time (about 553 jobs/min), with completion latency p50 1.46 s and maximum 1.92 s. The documentation notes that these jobs are cheap *because* the four model-backed text methods are unavailable; jobs with transformer detectors will be dominated by model inference and need dedicated, possibly GPU-backed worker pools.

The load-test script has four phases — concurrent reads, concurrent PPV calculations, sequential text uploads (exercising hashing, sniffing, extraction and write-once storage), and a burst of jobs measured to completion through the worker pool — and reports latency percentiles, throughput and error counts as JSON together with the platform details.

Built-in resource limits

Limit	Value
Upload size	512 MB (text and documents 25 MB)
Chunk size	8 MB
Audio	Up to 3 h accepted; first 900 s analysed
Video	Up to 2 h accepted; at most 240 sampled frames
Images	Analysed downsampled to 1,024 px
Text	Extraction capped at 2 M characters; classifier scoring at 16 × 512 tokens
Jobs	Celery hard time limit 3,600 s

26.5 Troubleshooting

Symptom	Likely cause	Fix
API exits with “Unsafe production configuration”	Production with the development key, inline tasks or insecure cookies	Set a strong `AEGIS_SECRET_KEY` , `AEGIS_TASK_MODE=celery` , `AEGIS_COOKIE_SECURE=true`
`error parsing value for field "cors_origins"`	JSON list syntax in `.env`	Use a comma-separated list

Symptom	Likely cause	Fix
Jobs stay *queued*	No worker, or it does not consume queue `aegis`	Start the worker with `-Q aegis,aegis-research`; check the heartbeat in System Health
Jobs stuck *running* after a worker crash	Lost worker	Wait for beat (every 10 min) or run `recover-jobs`
Every audio/video method *unavailable*	ffmpeg/ffprobe missing on the worker	Install ffmpeg (images include static builds)
Text detectors *unavailable*	No model configured	Expected; see "Configuring a real detector"
External provider *blocked by policy*	External processing off at some level	Expected default; all three levels must allow it
Upload rejected `unsupported_media`	Content is not a supported format (extension ignored), or is an executable or macro document	Convert to a supported format
Upload rejected `malformed_media`	The file fails structural parsing	Obtain a clean original
429 `rate_limited`	A rate limit was reached	Wait, or tune `AEGIS_*_RATE_LIMIT_*`
UI says "local hash unavailable"	Not a secure context, or file over 256 MB	Serve over HTTPS; the server hash is recorded regardless
UI keeps redirecting to sign-in	Refresh cookie not set (HTTPS required with secure cookies, or a proxy strips cookies)	Serve over HTTPS, or `AEGIS_COOKIE_SECURE=false` for local HTTP development only
SSE progress never updates behind nginx	Proxy buffering	`proxy_buffering off` for `/api/` (the API also sends `X-Accel-Buffering: no`)
`alembic check` reports drift	Model changed without a migration	Generate a migration and review it
API tests cannot connect	Test database missing	`createdb -O aegis aegis_test`, or set `AEGIS_TEST_DATABASE_URL`

PART VII

Security and Governance

Threat model, security controls, governance, limitations and the roadmap.

CHAPTER 27

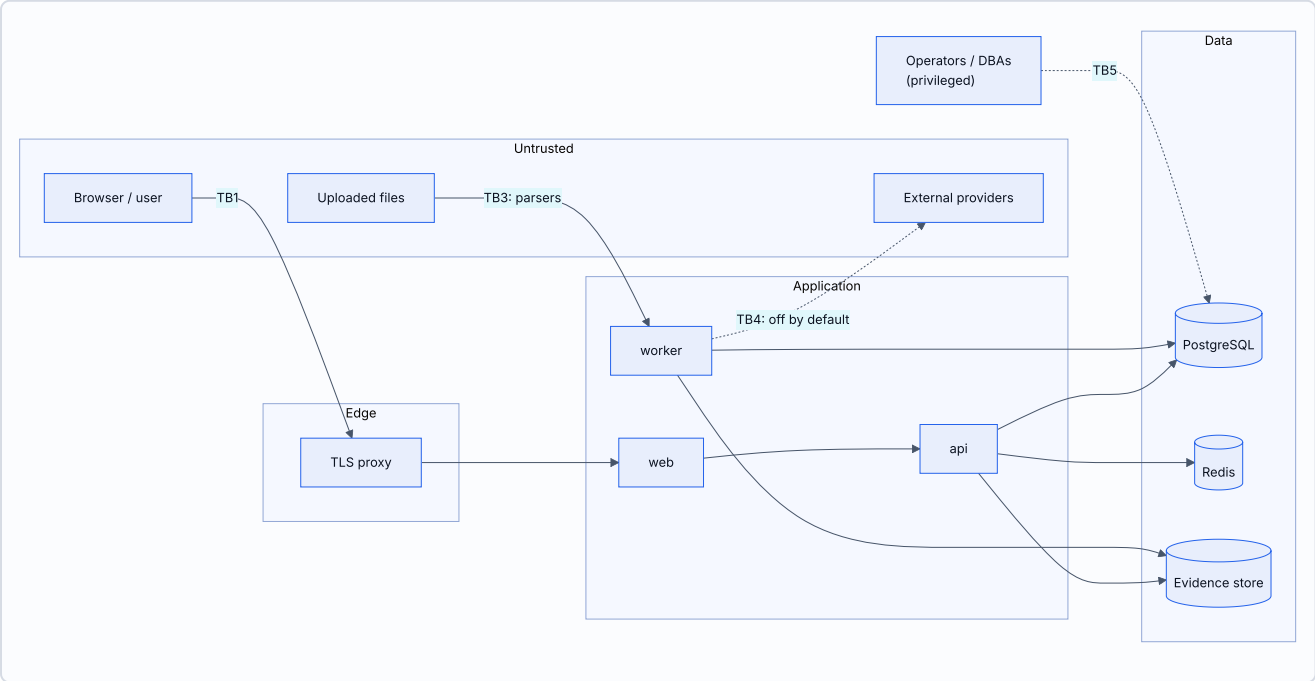
Threat Model and Security Controls

The threat model (`docs/security/threat-model.md`) covers an AEGIS deployment of web, API, workers, PostgreSQL, Redis and evidence storage, operated by one institution for one or more organisations. It uses a STRIDE-style enumeration per asset, and — characteristically — states for every threat both what AEGIS implements and what residual risk remains for the operator.

27.1 Assets and trust boundaries

Assets: evidence originals and derivatives; case records and reviewer conclusions; reports; the audit trail; credentials and sessions; watermark keys and provider tokens; model weights and calibration profiles; evaluation datasets.

Trust boundaries: browser ↔ reverse proxy ↔ web and API; API ↔ PostgreSQL, Redis and storage; worker ↔ uploaded content (untrusted parsers); worker ↔ external providers (optional); operators and administrators (privileged).



27.2 Threats and controls

#	Threat	Controls in AEGIS	Residual risk / operator action
T1	**Malicious uploads** (parser exploits, polyglots, executables)	Bytes-only type detection; executables and macro documents rejected; structural parse before acceptance; ffmpeg/ffprobe without a shell and with timeouts; Pillow decompression-bomb limit; optional ClamAV with mandatory-scan mode; content never executed	Parser vulnerabilities remain possible; keep dependencies patched; run workers with minimal privileges and no outbound network
T2	**Archive attacks** (zip bombs, path traversal)	DOCX accepted only with OOXML structure; entry count, uncompressed size, compression ratio, encryption and path checks; other ZIPs rejected	—
T3	**Denial of service through oversized media**	Global and text upload limits, chunk limits, media duration limits, analysis caps (sampled frames, first 900 s of audio, 16 text chunks), per-user upload and API rate limits, worker time limits	Large-scale DDoS protection belongs at the edge
T4	**Unauthorised evidence access**	Server-side RBAC on every endpoint; downloads need `evidence.download` and are audited; restricted classification; opaque storage keys; downloads as `application/octet-stream` attachments	—
T5	**Cross-tenant leakage**	Every query scoped by membership; foreign ids return 404; organisation header validated; tested	One shared database; separate deployments for strict physical isolation
T6	**Privileged-user abuse**	Separation of duties; every administrative change audited with a reason; org admins cannot record review decisions	A DBA can alter the database; hash chaining detects partial edits; anchor the head externally
T7	**Compromised provider credentials**	Tokens server-side only, never logged or returned; HTTPS required; external processing off by default and gated at three levels	Rotate tokens; scope provider accounts to inference only

#	Threat	Controls in AEGIS	Residual risk / operator action
T8	**Malicious or misleading provider responses**	Strict response validation; vendor probabilities stored as vendor-reported, never as AEGIS calibration; non-200 → failed	Plausible but wrong scores remain possible; benchmark providers locally
T9	**Tampering with reports or the audit trail**	Canonical JSON hash, PDF hash, deterministic re-render, immutable finals, corrections as new versions; hash chain with verification; tests prove detection	Whole-chain rewrite by a privileged actor; record `/audit/head` periodically in a separately controlled system
T10	**Prompt injection through documents**	No LLM agent processes documents; extracted text is data only; instruction-like phrases recorded as an extraction warning with no effect	Any future LLM-based method must not be given tools or policy authority
T11	**Model supply-chain risks**	Weights loaded only from operator-configured local paths with `local_files_only`; model identity and partial weight fingerprint recorded per run; registry records licence and validation	Verify weight checksums against the publisher and record them before enabling
T12	**Compromised worker**	Workers hold only database and storage credentials; artefacts content-addressed; job transitions audited	Separate network segment, no inbound access, least-privilege database roles
T13	**Session theft / CSRF**	15-minute in-memory access tokens; refresh rotation; httpOnly SameSite=Strict cookie scoped to the auth path; custom-header CSRF check; revocation on logout, password change and deactivation; CSP and frame denial	Enable secure cookies and HTTPS in production
T14	**Credential stuffing / brute force**	Argon2id; per-IP-and-email rate limit; lockout after 10 failures; optional TOTP	Integrate an institutional IdP with enforced MFA (roadmap)
T15	**Injection** (SQL, header, path)	SQLAlchemy parameterisation; filename sanitisation; storage-key validation; tests include SQL metacharacters	—

#	Threat	Controls in AEGIS	Residual risk / operator action
T16	**Misuse of findings** (treating a flag as proof)	Outcome categories describe evidence; standard limitations in every report; review decision required before finalisation; interface copy and banners	Organisational policy and training remain essential
T17	**Sensitive data in logs and errors**	Generic 500 messages with correlation ids; structured logs redact secret-like keys; sensitive EXIF fields redacted in views	Configure log retention and access at platform level

Out of scope by design: covert surveillance, account access, device monitoring, biometric identification, speaker or face recognition, and any inference of personal attributes. None of these exists in the code base.

27.3 Security controls matrix

`docs/security/security-controls.md` maps each control to its implementation and the test that verifies it:

Control	Implementation	Verified by
Authentication	Argon2id; JWT HS256 access tokens (15 min) bound to sessions; rotating refresh tokens (SHA-256 stored); optional TOTP	`test_session_revocation`, `test_refresh_requires_csrf_header`, `test_login_rate_limit_and_lockout`
Authorisation	`require(*permissions)` on every route; 32 permissions across 7 roles	`test_role_permissions_enforced_server_side`
Tenant isolation	`visible_cases()` scoping; 404 for foreign ids; membership-checked organisation header	`test_cross_tenant_isolation`, `test_restricted_classification`
Separation of duties	Creator ≠ reviewer; author ≠ finaliser; requester ≠ approver; appellant ≠ resolver; org admin cannot decide	`test_full_text_workflow`, `test_state_machine_rules`, `test_requester_cannot_approve_own_deletion`, `test_appeal_workflow`
Input validation	Pydantic schemas, size and length limits, content sniffing, structural parsing	`test_malicious_uploads_rejected`, `test_zip_traversal_and_bomb_rejected`, `test_oversized_upload`, `test_filename_is_sanitised`
Rate limiting	Redis fixed window with in-process fallback: login, uploads, per-user API	`test_login_rate_limit_and_lockout`
Security headers	CSP (`default-src 'none'` on the API), nosniff, DENY framing, no-referrer, HSTS over HTTPS, `no-store`	`test_error_format_and_headers`
CORS	Explicit origin allow-list	Configuration

Control	Implementation	Verified by
Secrets	Environment or secret store only; never in responses, logs or reports; production refuses unsafe settings	`test_no_secrets_in_responses`, `Settings.check_production_safety`
Evidence integrity	SHA-256 at acquisition; write-once storage; re-verification with critical alerts	`test_integrity_verification_and_mismatch_alert`, `test_storage_is_write_once`, `test_database_backend_round_trip_and_write_once`
Audit	Append-only hash chain, verification endpoint, head export	`test_audit_tampering_detected`
Report integrity	Canonical JSON hash, PDF hash, deterministic re-render	`test_report_tampering_detected`
External processing	Off by default; deployment AND organisation AND case must allow it	`test_provider_failure_and_policy`
Dependency hygiene	Locked dependencies; `make security-check` (pip-audit, pnpm audit, CycloneDX SBOM)	Clean at release time (`docs/governance/sbom/`)
Encryption in transit	Operator TLS proxy; secure cookies	Deployment guide
Encryption at rest	S3 SSE requested; MFA secrets Fernet-encrypted; disk encryption is the operator's responsibility	Partial
Backups	Procedure and restore test documented	Must be executed by the operator

27.4 Software bill of materials

`make security-check` exports the locked Python requirements, runs `pip-audit` against them (producing a CycloneDX JSON SBOM), runs `pnpm audit --prod`, and lists the licences of production Node dependencies. The release snapshot is committed in `docs/governance/sbom/` as `sbom-python.cdx.json` and `node-licenses.json`, and the CI pipeline runs `pip-audit` and `pnpm audit` on every change.

CHAPTER 28

Governance and Responsible Use

Technology alone cannot make an AI-content investigation fair. AEGIS therefore ships a governance package (`docs/governance/`) that collects what an institution needs to assess the system for a controlled pilot: procedures, templates, a machine-readable control mapping, a statement of limitations and a roadmap. This chapter summarises it.

28.1 Status and intended use

The package opens with an unambiguous statement: AEGIS is **not** approved, certified, classified or adopted by any government body and claims no such status. Its intended use is *investigative assistance and evidence triage under qualified human review*. For high-stakes decisions involving a person's rights, reputation, employment, education, liberty or access to services, the institution must require independent corroboration and qualified human review. **A detector score alone must never trigger punishment, rejection, prosecution or any other adverse decision.**

28.2 The pilot readiness package

Item	Where
System architecture and data-flow diagram	Architecture overview
Data model	Data-model document
Threat model and security controls	Security documents; `compliance-mapping.json`
Model and dataset registry	Administration → Model registry; model and dataset cards
Evaluation methodology	Research documents
False-positive / false-negative analysis and calibration	Experiments (no real-data calibration exists in this build)
Multilingual evaluation plan	Research documents
Bias and robustness results	Simulated only; the Fairness and Robustness laboratories are ready for real data
Evidence-handling procedure	Template
Audit and accountability design	Audit-chain document
Privacy impact assessment	Template
Data-retention policy	Template
Incident response, backup and recovery	Operations documents
Software bill of materials	`sbom/`
Deployment and rollback plan	Deployment guide
Limitations and residual risks	Limitations document
Independent evaluation protocol	Research documents
Human review and appeal procedure	Procedure

28.3 Human review and appeal procedure

Review.

1. The investigator requests review once analyses are complete. An independent reviewer must be assigned and cannot be the case creator.
2. The reviewer examines the evidence, every method result and its limitations, the automated assessment (including missing and failed methods and contradictions), and any process evidence.
3. The reviewer records an assessment, a rationale of at least 20 characters, and whether they agree with the automated outcome. Disagreement is recorded explicitly. The decision is hashed and chained into the audit trail.
4. A reviewer may instead request further analysis or record an inconclusive result.
5. A report can be finalised only after a review decision, and only by someone other than its author.

Language. Use evidence-describing language: “consistent with”, “inconclusive”, “verification succeeded for the supported provenance record”. Do not write “100% AI-generated”, “definitively human-written”, “criminally manipulated” or any other statement of certainty.

Decisions affecting people. Treat an AEGIS outcome as one input. Before any adverse decision, obtain independent corroboration (process evidence, interviews, original sources) and give the affected person an opportunity to respond.

Appeal. Anyone with the appeal permission can file an appeal or correction request, optionally on behalf of an affected party (using a role or reference rather than personal details). A reviewer other than the appellant moves it to under review and records upheld, corrected or rejected with a resolution. A correction creates a new report version that supersedes the previous one; the original remains preserved. Reopening a finalised case requires the reopen permission and a recorded reason.

28.4 Evidence-handling procedure (template)

1. **Authorisation** — open a case only with a documented purpose and, where required, a legal or organisational basis.
2. **Acquisition** — obtain the most original copy available (not screenshots or re-encodes) and upload it unchanged; AEGIS records the SHA-256, size, detected type, time and uploader and stores the original write-once.
3. **Verification** — compare the server SHA-256 with any hash provided by the source; run integrity verification before reporting and periodically in long cases.
4. **Derivatives** — work on AEGIS-generated derivatives; originals are never modified and every derivative records its parent and transformations.
5. **Access** — download originals only when needed (every download is audited); use restricted classification for sensitive cases.
6. **Process evidence** — attach drafts, revision histories and notes as `process_evidence`; reviewers evaluate them, not detectors.
7. **Review and reporting** — finalise only after an independent review decision; the report embeds evidence hashes, method versions and audit references.
8. **Retention and disposal** — follow the retention policy; deletion needs a request and a second person’s approval and is blocked under legal hold; hashes and audit records remain as tombstones.

The procedure must be adopted, adapted and validated by the institution; it is not a legally certified chain-of-custody process.

28.5 Data-retention policy (template)

Class	Default period	Typical use
`standard`	365 days	Routine triage cases
`extended`	7 years	Cases that may support formal proceedings
`legal_minimum`	90 days	Cases closed with no further action
`research`	3 years	Consented research evaluation

Rules enforced by the software: due cases are listed for review but nothing is deleted automatically; deletion requires a request with a reason and approval by a second authorised person; legal hold blocks requests and approvals; a purge removes stored bytes but keeps hashes, version tombstones and audit events; audit events are never purged by the application. The organisation must define who reviews due cases and how often, the legal-hold authority, and how destruction is verified in backups.

28.6 Privacy impact assessment (template)

The template, explicitly not legal advice and not establishing compliance with any law (it names India's Digital Personal Data Protection Act 2023 and the GDPR as examples), asks the institution to document: purpose and necessity; lawful basis and notice for each category of data subject (authors, depicted persons, speakers); a data inventory (names in text, faces in images, voices in audio, EXIF GPS); minimisation (AEGIS asks for no demographic data and performs no biometric identification; redaction is available); external processing and its agreements; retention; access control; accuracy and fairness (validated detectors, subgroup error analysis, communication of inconclusive results); rights and redress (the appeal workflow, corrections as new report versions); security (threat model and controls, residual risks with owners); and sign-off by the data protection officer, legal and security approvers.

28.7 Compliance control mapping

`docs/governance/compliance-mapping.json` (version 2026-10) records the technical status of each control with a disclaimer: *"This mapping records the technical status of controls in this repository. It is not a statement of compliance with any law, regulation or security standard. Legal and security review is required before operational deployment."* Each control has an id, an area, a requirement, a status — `implemented_and_tested`, `partially_implemented`, `documentation_required`, `external_assessment_required`, `not_implemented` or `not_applicable` — and the evidence (code paths and test names). The full list is reproduced in Appendix E. The Audit & Compliance screen displays it from the API, so the interface always shows the mapping shipped with the running code.

28.8 Limitations and residual risks

The limitations document is unusually direct. Its key points:

Scientific

- No validated AI-content detector ships with this build; every statistical-inference output is `unavailable` until an operator configures and validates a model.

- No calibrated probabilities: nothing is flagged and nothing is presented as a probability.
- All benchmark and experiment results in the repository are analytical or simulated.
- The reference watermark verifier detects only AEGIS-generated reference text; provider watermark verification is an unimplemented boundary.
- Fast-DetectGPT uses one model for sampling and scoring, and Binoculars has no validated threshold; neither reproduces published results.
- Mixed-authorship segment scores are withheld unless a model is validated at paragraph granularity, and none is.
- Forensic observations are not detectors and need expert interpretation.
- Languages: identification only; no detector has been evaluated for Hindi or any other language.
- OCR is not implemented; scanned PDFs produce an extraction warning.
- Lip-sync, face-swap and voice-cloning detectors are not implemented.

Security and operations

- The audit chain is tamper-evident, not tamper-proof, against privileged database access; external anchoring is manual.
- Encryption at rest for local storage and PostgreSQL depends on the host.
- The ClamAV integration has not been exercised against a live scanner.
- MFA is optional, with no recovery codes; there is no SAML/OIDC integration.
- Backups and restore are documented but not executed in the repository.
- Performance figures come from a single 4-vCPU development container.

Governance

- Compliance with any law or standard is not claimed.
- Chain-of-custody records are technical, not legally certified.
- The research documents named in the original brief were not supplied; the implementation follows the brief's descriptions of them.

28.9 Roadmap

1. **Validated text detection (English)** — procure a licensed model, build a consented corpus with at least 3,000 negatives per slice, calibrate and validate through the Benchmark Laboratory.
2. **Indian-language evaluation** — following the multilingual plan, Hindi first.
3. **Provider watermark verification** — SynthID-Text and media watermarks under authorised agreements, using the existing plugin boundary.
4. **Learned media detectors** — UnivFD-class image, AASIST-class speech and frame-level deepfake models as ONNX adapters with model cards.
5. **OCR** with per-token confidence, labelled separately from extracted text.
6. **Identity** — SAML/OIDC single sign-on, enforced MFA, recovery codes.
7. **Audit anchoring** — automatic RFC 3161 timestamping of the chain head.
8. **Retrieval-based corroboration** against authorised reference corpora.
9. **Evaluated fusion** — a learned combiner, only with held-out joint validation and documented dependence analysis.
10. **Scale-out** — a separate GPU worker queue, Object-Lock S3 by default, read replicas.

28.10 Responsible use

AEGIS is defensive investigative software. It has no surveillance, account-access or device-monitoring features. It performs no biometric identification and never infers a person's identity, nationality, ethnicity or intent. Any operational deployment needs a legal, privacy and security review first. The code reinforces these commitments at every layer — in the evidence classes that cannot be mixed, the statuses that cannot be faked, the fusion policy that will not average, the workflow that requires an independent human decision, the reports that carry their own limitations, and the audit chain that records who did what.

The project is released under the MIT licence.

APPENDIX A

Configuration Reference

AEGIS is configured entirely through environment variables. This appendix is generated from the `Settings` class in `apps/api/app/core/config.py` (API and worker settings) and from the environment variables read directly by the analysis methods in `aegis_ml`. Defaults shown are the code defaults; `.env.example` documents recommended values.

A.1 Application settings (`AEGIS_*`, read by `Settings`)

Variable	Default	Notes
`AEGIS_MODE`	`demo`	demo | research | connected-production (shown in the UI)
`AEGIS_ENVIRONMENT`	`development`	development | test | hosted-demo | production
`AEGIS_SECRET_KEY`	`dev-only-… (development placeholder; refused in production)`	production: 64+ random chars
`AEGIS_DATABASE_URL`	`postgresql+psycopg://aegis:aegis-dev-only@localhost:5432/aegis`	
`AEGIS_REDIS_URL`	`redis://localhost:6379/0`	empty: no Redis (inline tasks, in-process rate limits)
`AEGIS_TASK_MODE`	`celery`	celery | inline (inline = in-process thread, not for production)
`AEGIS_RESUME_INLINE_JOBS_ON_STARTUP`	`False`	inline mode with one API process: resume interrupted jobs
`AEGIS_STORAGE_BACKEND`	`local`	local | s3 | database (PostgreSQL, for hosts without a persistent disk)
`AEGIS_STORAGE_ROOT`	`var/evidence`	
`AEGIS_S3_BUCKET`	`aegis-evidence`	
`AEGIS_S3_ENDPOINT_URL`	*(empty)*	
`AEGIS_S3_REGION`	`us-east-1`	
`AEGIS_S3_OBJECT_LOCK`	`False`	
`AEGIS_CORS_ORIGINS`	`http://localhost:3000`	comma-separated
`AEGIS_ACCESS_TOKEN_MINUTES`	`15`	
`AEGIS_REFRESH_TOKEN_DAYS`	`7`	
`AEGIS_COOKIE_SECURE`	`False`	production: true (HTTPS only)
`AEGIS_MAX_UPLOAD_MB`	`512`	
`AEGIS_MAX_TEXT_UPLOAD_MB`	`25`	
`AEGIS_UPLOAD_CHUNK_MB`	`8`	
`AEGIS_REQUIRE_MALWARE_SCAN`	`False`	production: true, with ClamAV configured
`AEGIS_CLAMAV_HOST`	*(empty)*	
`AEGIS_CLAMAV_PORT`	`3310`	
`AEGIS_LOGIN_RATE_LIMIT_PER_MINUTE`	`10`	

Variable	Default	Notes
`AEGIS_UPLOAD_RATE_LIMIT_PER_HOUR`	`120`	
`AEGIS_API_RATE_LIMIT_PER_MINUTE`	`600`	
`AEGIS_ALLOW_EXTERNAL_PROCESSING`	`False`	

A.2 Analysis-method settings (read by `aegis_ml`)

Variable	Used by	Meaning
`AEGIS_REFERENCE_WATERMARK_KEY`	`watermark.aegis_reference_text`	Secret key (≥ 16 bytes) for the reference watermark; empty disables the verifier
`AEGIS_C2PA_TRUST_ANCHORS`	`provenance.c2pa`	Path to a PEM bundle of trusted signer roots; without it valid signatures are reported as signer untrusted
`AEGIS_TEXT_CLASSIFIER_PATH`	`text.supervised_classifier`	Local Hugging Face sequence-classification model directory
`AEGIS_TEXT_CLASSIFIER_AI_LABEL`	`text.supervised_classifier`	Name of the model label meaning "AI"; AEGIS refuses to guess
`AEGIS_SCORING_LM_PATH`	`text.token_statistics`, `text.fast_detectgpt_analytic`	Local causal language model directory
`AEGIS_BINOCULARS_OBSERVER_PATH`	`text.binoculars`	Observer causal LM directory
`AEGIS_BINOCULARS_PERFORMER_PATH`	`text.binoculars`	Performer causal LM directory (shared tokenizer)
`AEGIS_INFERENCE_DEVICE`	text model adapters	`cpu` (default) or `cuda` (falls back to CPU)
`AEGIS_IMAGE_MODEL_DIR`	`image.local_model`, `video.frame_model`	Directory with `model.onnx` and `aegis_model.json`
`AEGIS_SPEECH_MODEL_DIR`	`audio.synthetic_speech_model`	Directory with `model.onnx` and `aegis_model.json`
`AEGIS_PROVIDER_<ID>_URL`	`provider.<id>` (`image_detector`, `audio_detector`, `video_detector`)	HTTPS endpoint implementing the provider contract
`AEGIS_PROVIDER_<ID>_TOKEN`	`provider.<id>`	Bearer token, server-side only, never logged
`AEGIS_PROVIDER_<ID>_TIMEOUT`	`provider.<id>`	Request timeout in seconds (default 60)
`AEGIS_TRANSCRIPTION_ENGINE`	`audio.transcription`	Reserved for a future transcription engine (placeholder)

A.3 Operational and deployment variables

Variable	Where	Meaning
`AEGIS_DEMO_PASSWORD`	`app.cli seed-demo`, Render	Password for every demo account; the published default is refused on Render
`AEGIS_ADMIN_PASSWORD`	`app.cli create-admin`	Non-interactive administrator password (otherwise prompted)
`AEGIS_SEED_DEMO`	Render `serve.py`	`true` loads demo data at start-up
`AEGIS_TEST_DATABASE_URL`	API tests	PostgreSQL URL of the test database
`AEGIS_WEB_URL`	Playwright	Target of the browser tests (default `http://localhost:3000`)

Variable	Where	Meaning
`AEGIS_VERSION`	Docker Compose	Image tag, used for rollback
`POSTGRES_PASSWORD`	Docker Compose	Database password
`AEGIS_SITE`, `AEGIS_TLS_DIRECTIVE`	Server overlay / Caddy	Public name and TLS mode, set by `server-setup.sh`
`NEXT_PUBLIC_AEGIS_API_BASE`	Web	API base path seen by the browser (default `/api/v1`)
`AEGIS_API_INTERNAL_URL`	Web build	Where the Next.js rewrite sends `/api/*`
`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`	S3 backend	Object-store credentials (server-side only)

APPENDIX B

API Endpoint Reference

This appendix lists every operation of the AEGIS REST API v1. It is generated from the route definitions in `apps/api/app/api/v1/` (method, path and the permission enforced by the route's dependencies) and from the OpenAPI snapshot `docs/api/openapi.json` (92 paths). "Permission" lists the permissions passed to `require()`; *any authenticated user* means the route only needs a valid session; *public* routes need no authentication. All routes share the error envelope, correlation id and rate limits described in the API Foundations chapter.

Authentication and sessions

Method	Path	Permission	Summary
POST	`/auth/login`	public	Login — body: `LoginIn`
POST	`/auth/refresh`	refresh cookie + `X-Requested-With` header	Refresh
POST	`/auth/logout`	any authenticated user	Logout
GET	`/auth/me`	any authenticated user	Me
POST	`/auth/change-password`	any authenticated user	Change Password — body: `PasswordChange`
POST	`/auth/mfa/setup`	any authenticated user	Mfa Setup
POST	`/auth/mfa/enable`	any authenticated user	Mfa Enable — body: `CodeIn`
GET	`/auth/sessions`	any authenticated user	Sessions
DELETE	`/auth/sessions/{session_id}`	any authenticated user	Revoke Session

Investigations, review, appeals and retention

Method	Path	Permission	Summary
GET	`/investigations`	case.read	List Investigations — query: `status, priority, q, assigned_to_me, sort, order, page, page_size`
POST	`/investigations`	case.create	Create Investigation — body: `InvestigationIn`
GET	`/investigations/{case_id}`	case.read	Get Investigation
PATCH	`/investigations/{case_id}`	case.update	Update Investigation — body: `InvestigationPatch`
POST	`/investigations/{case_id}/assign`	case.assign	Assign Reviewer — body: `AssignIn`
POST	`/investigations/{case_id}/transition`	case.read	Transition — body: `TransitionIn`
POST	`/investigations/{case_id}/legal-hold`	admin.settings	Legal Hold — body: `LegalHoldIn`

Method	Path	Permission	Summary
GET	`/investigations/{case_id}/annotations`	case.read	List Annotations
POST	`/investigations/{case_id}/annotations`	review.annotate	Add Annotation — body: `AnnotationIn`
GET	`/investigations/{case_id}/reviews`	case.read	List Reviews
POST	`/investigations/{case_id}/reviews`	review.decide	Record Review — body: `ReviewIn`
GET	`/investigations/{case_id}/appeals`	case.read	List Appeals
POST	`/investigations/{case_id}/appeals`	appeal.submit	Submit Appeal — body: `AppealIn`
POST	`/investigations/{case_id}/appeals/{appeal_id}/resolve`	appeal.resolve	Resolve Appeal — body: `AppealResolve`
POST	`/investigations/{case_id}/deletion-requests`	retention.request_deletion	Request Deletion — body: `DeletionIn`
POST	`/investigations/{case_id}/deletion-requests/{request_id}/decide`	retention.approve_deletion	Decide Deletion — body: `DeletionDecision`
GET	`/investigations/{case_id}/deletion-requests`	case.read	List Deletion Requests
GET	`/investigations/{case_id}/history`	case.read	Case History — query: `limit`
GET	`/investigations/{case_id}/reviewers`	case.read	Eligible Reviewers

Evidence vault and uploads

Method	Path	Permission	Summary
POST	`/investigations/{case_id}/evidence`	evidence.upload	Upload Evidence — form-data form
POST	`/investigations/{case_id}/uploads`	evidence.upload	Init Chunked Upload — body: `UploadInit`
PUT	`/uploads/{upload_id}`	evidence.upload	Put Chunk — query: `offset`
POST	`/uploads/{upload_id}/complete`	evidence.upload	Complete Chunked Upload
GET	`/investigations/{case_id}/evidence`	evidence.read	List Case Evidence
GET	`/evidence`	evidence.read	Evidence across visible cases, with the latest analysis job for each item (used by modality views). — query: `modality, q, limit`
GET	`/evidence/{evidence_id}`	evidence.read	Get Evidence
GET	`/evidence/{evidence_id}/versions/{version_id}/download`	evidence.download	Download Version

Method	Path	Permission	Summary
GET	`/evidence/{evidence_id}/thumbnail`	evidence.read	Thumbnail
GET	`/evidence/{evidence_id}/text`	evidence.read	Extracted Text — query: `version`
POST	`/evidence/{evidence_id}/verify`	evidence.verify	Verify
POST	`/evidence/{evidence_id}/redactions`	evidence.redact	Redact — body: `RedactIn`
GET	`/artifacts/{artifact_id}`	analysis.read	Get Artifact

Analysis jobs

Method	Path	Permission	Summary
GET	`/analysis/profiles`	analysis.read	Profiles
POST	`/analysis/investigations/{case_id}/jobs`	analysis.run	Create Job — body: `JobIn`
GET	`/analysis/investigations/{case_id}/jobs`	analysis.read	List Jobs
GET	`/analysis/jobs/{job_id}`	analysis.read	Get Job
POST	`/analysis/jobs/{job_id}/cancel`	analysis.run	Cancel
GET	`/analysis/runs/{run_id}/segments`	analysis.read	Run Segments
GET	`/analysis/jobs/{job_id}/events`	analysis.read	Server-sent events reflecting persisted job state. Each event is emitted only when the stored

Reports

Method	Path	Permission	Summary
GET	`/investigations/{case_id}/reports`	report.read	List Reports
GET	`/reports`	report.read	All Reports
POST	`/investigations/{case_id}/reports`	report.generate	Generate
POST	`/reports/{report_id}/corrections`	report.generate	Correction — body: `CorrectionIn`
POST	`/reports/{report_id}/finalize`	report.finalize	Finalize
GET	`/reports/{report_id}/json`	report.read	Report Json
GET	`/reports/{report_id}/pdf`	report.read	Report Pdf
POST	`/reports/{report_id}/verify`	report.read	Verify

Research bench and benchmark laboratory

Method	Path	Permission	Summary
POST	`/evaluation/ppv`	research.use	Ppv Calculator — body: `PPVIn`
POST	`/evaluation/threshold-explorer`	research.use	Threshold Explorer — body: `ThresholdIn`
GET	`/evaluation/fairness/{run_id}`	research.use	Fairness — query: `threshold, min_group_size`
GET	`/evaluation/robustness/{run_id}`	research.use	Robustness
GET	`/evaluation/experiments`	research.use	List Experiments
POST	`/evaluation/experiments/{experiment_id}/runs`	research.use	Run Experiment — body: `ExperimentIn`
GET	`/evaluation/experiment-runs`	research.use	Experiment Runs — query: `experiment_id`
GET	`/evaluation/experiment-runs/{run_id}`	research.use	Experiment Run
GET	`/benchmarks/datasets`	benchmark.read	Datasets
POST	`/benchmarks/datasets`	dataset.register	Register Dataset — form-data form
GET	`/benchmarks/runs`	benchmark.read	Runs — query: `method_id`
GET	`/benchmarks/runs/{run_id}`	benchmark.read	Get Run
POST	`/benchmarks/runs`	benchmark.run	Create Run — body: `RunIn`
POST	`/benchmarks/reference-watermark`	benchmark.run	Built-in, clearly labelled SIMULATED benchmark of the AEGIS reference watermark verifier.
POST	`/benchmarks/imported-scores`	benchmark.run	Imported Scores — form-data form
POST	`/benchmarks/runs/{run_id}/calibration-profiles`	admin.models	Calibration Profile — body: `CalibrationIn`
GET	`/benchmarks/calibration-profiles`	benchmark.read	Calibration Profiles
GET	`/benchmarks/runs/{run_id}/samples`	benchmark.read	Run Samples — query: `limit`

Platform: overview, audit, health, registry, administration, governance

Method	Path	Permission	Summary
GET	`/overview`	case.read	Dashboard figures. Every number is a live database count for the caller's organisation.
GET	`/search`	case.read	Search — query: `q`
GET	`/notifications`	any authenticated user	Notifications — query: `unread_only`
POST	`/notifications/read`	any authenticated user	Mark Read

Method	Path	Permission	Summary
GET	`/audit/events`	audit.read	Audit Events — query: `action, investigation_id, before_seq, limit`
POST	`/audit/verify`	audit.read	Verify Audit
GET	`/audit/head`	audit.read	Audit Head
GET	`/health/live`	public	Live
GET	`/health/ready`	public	Ready
GET	`/health/components`	system.health	Components
GET	`/detectors`	analysis.read	Detectors
GET	`/provenance/records`	analysis.read	Provenance Records
GET	`/watermarks/verifiers`	analysis.read	Watermark Verifiers
GET	`/reviewers`	case.read	Active organisation members who hold the review permission (excluding the caller).
GET	`/users`	admin.users	List Users
POST	`/users`	admin.users	Create User — body: `UserCreate`
PATCH	`/users/{user_id}`	admin.users	Update Member — body: `MemberPatch`
POST	`/users/{user_id}/revoke-sessions`	admin.users	Revoke Sessions
GET	`/roles`	any authenticated user	Roles
GET	`/organizations`	any authenticated user	Organizations
POST	`/organizations`	system administrator	Create Org — body: `OrgIn`
PUT	`/organizations/current/settings`	admin.settings	Org Settings — body: `OrgSettings`
GET	`/models`	analysis.read	Models
PATCH	`/models/{registry_id}`	admin.models	Update Model — body: `RegistryPatch`
GET	`/admin/settings`	admin.settings	Get Settings View
PUT	`/admin/settings/{key}`	system administrator	Put Setting — body: `SettingIn`
GET	`/admin/retention`	case.read	Retention
PUT	`/admin/retention/{classification}`	admin.settings	Put Retention — body: `RetentionIn`
POST	`/admin/jobs/recover`	system administrator	Recover
GET	`/governance/compliance`	any authenticated user	Compliance
GET	`/governance/registry-summary`	any authenticated user	Registry Summary

Request and response schemas

The main Pydantic schemas published in the OpenAPI document, with their fields. Required fields are marked with an asterisk.

AnnotationIn — `body` * (string); `kind` (one of note, finding, question, contradiction); `evidence_item_id` (string or null); `detector_run_id` (string or null)

AnnotationOut — `id` * (string); `kind` * (string); `body` * (string); `evidence_item_id` * (string or null); `detector_run_id` * (string or null); `author` * (UserRef); `created_at` * (string)

AppealIn — `grounds` * (string); `report_id` (string or null); `on_behalf_of` (string or null)

AppealOut — `id` * (string); `report_id` * (string or null); `on_behalf_of` * (string or null); `grounds` * (string); `status` * (string); `resolution` * (string or null); `created_at` * (string); `resolved_at` * (string or null)

AppealResolve — `status` * (one of under_review, upheld, corrected, rejected); `resolution` * (string)

ArtifactOut — `id` * (string); `name` * (string); `media_type` * (string); `sha256` * (string); `description` * (string); `cannot_establish` * (string)

AssignIn — `reviewer_id` * (string or null)

CalibrationIn — `target_fpr` * (number); `population` * (string)

CodeIn — `code` * (string)

CorrectionIn — `reason` * (string)

DatasetOut — `id` * (string); `name` * (string); `version` * (string); `modality` * (string); `provenance` * (string); `license` * (string); `consent_basis` * (string); `documentation` * (string); `is_synthetic` * (boolean); `sha256` * (string); `n_samples` * (integer); `splits` * (object); `fields` * (array); `duplicate_report` * (object); `created_at` * (string)

DeletionDecision — `approve` * (boolean); `reason` * (string)

DeletionIn — `reason` * (string)

EvidenceOut — `id` * (string); `investigation_id` * (string); `display_name` * (string); `role` * (string); `modality` * (string); `mime_type` * (string); `status` * (string); `description` * (string or null); `acquired_at` * (string); `uploaded_by` * (UserRef); `is_demo_fixture` * (boolean); `extraction` * (object or null); `versions` * (array)

ExperimentIn — `config` (object)

InvestigationIn — `title` * (string); `case_reference` * (string); `purpose` * (string); `media_category` * (one of text, document, image, audio, video, mixed); `description` (string); `priority` (one of low, normal, high, urgent); `assigned_reviewer_id` (string or null); `legal_basis` (string or null); `retention_class` (one of standard, extended, legal_minimum, research); `access_classification` (one of standard, restricted); `allow_external_processing` (boolean)

InvestigationOut — `id` * (string); `case_reference` * (string); `title` * (string); `purpose` * (string); `media_category` * (string); `description` * (string); `priority` * (string); `status` * (string); `legal_basis` * (string or null); `retention_class` * (string); `access_classification` * (string); `legal_hold` * (boolean); `allow_external_processing` * (boolean); `disposition` * (string or null); `pending_questions` * (array); `assigned_reviewer` * (UserRef or null); `created_by` * (UserRef); `is_demo_fixture` * (boolean); `created_at` * (string); `updated_at` * (string); `archived_at` * (string or null)

InvestigationPatch — `title` (string or null); `description` (string or null); `priority` (string or null); `legal_basis` (string or null); `access_classification` (string or null); `retention_class` (string or null); `allow_external_processing` (boolean or null); `pending_questions` (array or null)

InvestigationSummary — `id` * (string); `case_reference` * (string); `title` * (string); `media_category` * (string); `priority` * (string); `status` * (string); `access_classification` * (string); `is_demo_fixture` * (boolean); `assigned_reviewer` * (UserRef or null); `created_at` * (string); `updated_at` * (string)

JobIn — `evidence_item_id` * (string); `profile_id` * (string); `options` (object)

JobOut — `id` * (string); `investigation_id` * (string); `evidence_item_id` * (string); `evidence_version_id` * (string); `profile_id` * (string); `status` * (string); `methods_total` * (integer); `methods_done` * (integer); `options` * (object); `external_processing_permitted` * (boolean); `attempts` * (integer); `cancel_requested` * (boolean); `fusion` * (object or null); `error` * (string or null); `created_at` * (string); `started_at` * (string or null); `finished_at` * (string or null); `runs` * (array)

JobSummary — `id` * (string); `evidence_item_id` * (string); `profile_id` * (string); `status` * (string); `methods_total` * (integer); `methods_done` * (integer); `created_at` * (string); `finished_at` * (string or null)

LegalHoldIn — `legal_hold` * (boolean); `reason` * (string)

LoginIn — `email` * (string); `password` * (string); `totp` (string or null)

MemberPatch — `role` (string or null); `is_active` (boolean or null)

OrgIn — `name` * (string); `slug` * (string)

OrgSettings — `allow_external_processing` * (boolean); `reason` * (string)

PPVIn — `prevalence` * (number); `tpr` * (number); `fpr` * (number); `sample_size` (integer or null); `tpr_source` (one of measured, assumed, published, vendor_reported, simulated); `fpr_source` (one of measured, assumed, published, vendor_reported, simulated); `prevalence_source` (one of measured, assumed, published, vendor_reported, simulated); `benchmark_run_id` (string or null)

Page_InvestigationSummary_ — `items` * (array); `total` * (integer); `page` * (integer); `page_size` * (integer)

PasswordChange — `current_password` * (string); `new_password` * (string)

RedactIn — `spans` * (array); `reason` * (string)

RegistryPatch — `enabled` (boolean or null); `license` (string or null); `model_identifier` (string or null); `languages_evaluated` (array or null); `validation_status` (string or null); `validation_benchmark_run_id` (string or null); `calibration_profile_id` (string or null); `operating_threshold` (number or null); `validation_note` (string or null); `reason` * (string)

ReportOut — `id` * (string); `investigation_id` * (string); `version` * (integer); `status` * (string); `supersedes_id` * (string or null); `correction_reason` * (string or null); `schema_version` * (string); `json_sha256` * (string); `pdf_sha256` * (string or null); `generated_at` * (string); `finalized_at` * (string or null)

RetentionIn — `retain_days` * (integer); `description` (string); `reason` * (string)

ReviewIn — `decision` * (one of no_reliable_conclusion, inconclusive, some_signals_consistent_with_ai, multiple_compatible_signals_consistent_with_ai, evidence_of_recognized_watermark, provenance_record_verified, provenance_verification_failed, contradictory_evidence, further_analysis_requested); `rationale` * (string); `agrees_with_automated` (boolean or null); `supersedes_id` (string or null)

ReviewOut — `id` * (string); `decision` * (string); `rationale` * (string); `agrees_with_automated` * (boolean or null); `automated_outcome` * (string or null); `supersedes_id` * (string or null); `content_sha256` * (string); `audit_event_hash` * (string or null); `reviewer` * (UserRef); `created_at` * (string)

RunIn — `method_id` * (string); `dataset_id` * (string); `split` (one of test, validation); `threshold` (number or null); `target_fpr` (number); `seed` (integer); `holdout_field` (string or null); `holdout_values` (array)

SegmentOut — `start` * (number); `end` * (number); `unit` * (string); `label` * (string); `score` * (number or null); `detail` * (object)

SettingIn — `value` * (any); `reason` * (string)

ThresholdIn — `benchmark_run_id` * (string); `deployment_prevalence` * (number); `population_size` (integer); `split` (string)

TokenOut — `access_token` * (string); `token_type` (string); `expires_in` * (integer)

TransitionIn — `target` * (one of human_review, analysis_queued, inconclusive, archived); `reason` (string or null)

UploadInit — `filename` * (string); `total_size` * (integer); `role` (one of primary, process_evidence)

UserCreate — `email` * (string); `display_name` * (string); `password` * (string); `role` * (one of org_admin, investigator, forensic_analyst, independent_reviewer, auditor, researcher)

UserRef — `id` * (string); `display_name` * (string); `email` * (string)

VersionOut — `id` * (string); `version_no` * (integer); `kind` * (string); `parent_version_id` * (string or null); `sha256` * (string); `size_bytes` * (integer); `mime_type` * (string); `original_filename` * (string or null); `transformations` * (array); `metadata_status` * (object); `scan_status` * (string); `reason` * (string or null); `created_at` * (string); `purged_at` * (string or null)

app__api__v1__analysis__RunOut — `id` * (string); `position` * (integer); `method_id` * (string); `method_version` * (string); `method_name` * (string); `evidence_class` * (string); `status` * (string); `input_sha256` * (string); `config` * (object); `threshold` * (number or null); `summary` * (string or null); `output` * (object); `output_schema` * (string); `score` * (number or null); `score_semantics` * (string or null); `calibrated` * (boolean); `flagged` * (boolean or null); `uncertainty` * (object or null); `warnings` * (array); `limitations` * (array); `error` * (string or null); `started_at` * (string or null); `finished_at` * (string or null); `duration_ms` * (integer or null); `artifacts` * (array)

app__api__v1__research__RunOut — `id` * (string); `dataset_id` * (string or null); `method_id` * (string); `method_version` * (string); `result_kind` * (string); `status` * (string); `seed` * (integer); `config` * (object); `manifest` * (object); `metrics` * (object); `source_citation` * (string or null); `error` * (string or null); `created_at` * (string); `finished_at` * (string or null)

APPENDIX C

Database Schema Reference

Generated from the SQLAlchemy metadata in `apps/api/app/models/__init__.py`. Types are shown as SQLAlchemy types; `JSON` columns are stored as `JSONB` on PostgreSQL. PK = primary key, FK = foreign key, ✓ = nullable.

`analysis_artifacts`

Column	Type	Null	Key / reference	Default
`id`	Uuid		PK	uuid4()
`detector_run_id`	Uuid		FK → `detector_runs.id` (on delete cascade), indexed	
`name`	String(120)			
`media_type`	String(80)			
`storage_key`	String(400)			
`sha256`	String(64)			
`description`	Text			
`cannot_establish`	Text			

`analysis_jobs`

Column	Type	Null	Key / reference	Default
`id`	Uuid		PK	uuid4()
`organization_id`	Uuid		FK → `organizations.id`, indexed	
`investigation_id`	Uuid		FK → `investigations.id`, indexed	
`evidence_item_id`	Uuid		FK → `evidence_items.id`, indexed	
`evidence_version_id`	Uuid		FK → `evidence_versions.id`	
`profile_id`	String(60)			
`status`	String(30)			'queued'
`methods_total`	Integer			0
`methods_done`	Integer			0
`options`	JSON			{}
`external_processing_permitted`	Boolean			False
`idempotency_key`	String(120)	✓	unique	
`task_id`	String(80)	✓		
`attempts`	Integer			0
`cancel_requested`	Boolean			False
`fusion`	JSON	✓		

Column	Type	Null	Key / reference	Default
error	Text	✓		
requested_by_id	Uuid		FK → users.id	
created_at	DateTime			utcnow()
started_at	DateTime	✓		
finished_at	DateTime	✓		

analysis_signals

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
job_id	Uuid		FK → analysis_jobs.id (on delete cascade), indexed	
detector_run_id	Uuid		FK → detector_runs.id (on delete cascade), unique	
evidence_class	String(40)			
verification_status	String(60)	✓		
calibration_status	String(30)			
applicable_population	Text	✓		
input_characteristics	JSON			{}
uncertainty	JSON	✓		
known_dependencies	JSON			[]
supporting_reference	String(200)	✓		
created_at	DateTime			utcnow()

annotations

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
investigation_id	Uuid		FK → investigations.id (on delete cascade), indexed	
evidence_item_id	Uuid	✓	FK → evidence_items.id	
detector_run_id	Uuid	✓	FK → detector_runs.id	
author_id	Uuid		FK → users.id	
kind	String(30)			'note'
body	Text			
created_at	DateTime			utcnow()

appeals

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
investigation_id	Uuid		FK → investigations.id (on delete cascade), indexed	
report_id	Uuid	✓	FK → reports.id	
submitted_by_id	Uuid		FK → users.id	
on_behalf_of	String(200)	✓		
grounds	Text			
status	String(30)			'open'
resolution	Text	✓		
resolved_by_id	Uuid	✓	FK → users.id	
created_at	DateTime			utcnow()
resolved_at	DateTime	✓		

audit_events

Column	Type	Null	Key / reference	Default
seq	BigInteger		PK	identity (monotonic)
id	Uuid		unique	uuid4()
occurred_at	DateTime			utcnow()
organization_id	Uuid	✓	FK → organizations.id	
actor_user_id	Uuid	✓	FK → users.id	
actor_label	String(200)			
action	String(80)		indexed	
entity_type	String(40)			
entity_id	String(64)	✓		
investigation_id	Uuid	✓	indexed	
reason	Text	✓		
details	JSON			{}
correlation_id	String(64)	✓		
ip_address	String(64)	✓		
prev_hash	String(64)			
event_hash	String(64)		unique	

Constraints: Index ix_audit_events_org_time on (organization_id, occurred_at); Index ix_audit_events_entity on (entity_type, entity_id).

benchmark_runs

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
suite_id	Uuid	✓	FK → benchmark_suites.id	
dataset_id	Uuid	✓	FK → datasets.id	
method_id	String(80)			
method_version	String(40)			
result_kind	String(30)			
status	String(20)			'queued'
seed	Integer			0
config	JSON			{}
manifest	JSON			{}
metrics	JSON			{}
source_citation	Text	✓		
error	Text	✓		
created_by_id	Uuid	✓	FK → users.id	
created_at	DateTime			utcnow()
finished_at	DateTime	✓		

Constraints: Check: `result_kind in ('published','reproduced','locally_measured','simulated','vendor_reported')`.

benchmark_samples

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
run_id	Uuid		FK → benchmark_runs.id (on delete cascade), indexed	
sample_key	String(120)			
split	String(20)			'test'
label	Integer			
group	String(80)	✓		
transform	String(60)			'original'
score	Float	✓		
status	String(30)			

Constraints: Unique (run_id, sample_key, transform).

benchmark_suites

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
name	String(200)		unique	
description	Text			
dataset_id	Uuid	✓	FK → datasets.id	
task	String(60)			
config	JSON			{}
created_at	DateTime			utcnow()

calibration_profiles

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
method_id	String(80)		indexed	
method_version	String(60)			
calibration_method	String(30)			
parameters	JSON			{}
threshold	Float	✓		
target_fpr	Float	✓		
population	Text			
languages	JSON			[]
benchmark_run_id	Uuid	✓	FK → benchmark_runs.id	
metrics	JSON			{}
validated	Boolean			False
created_by_id	Uuid	✓	FK → users.id	
created_at	DateTime			utcnow()

datasets

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
name	String(200)			
version	String(60)			
modality	String(20)			'text'
provenance	Text			
license	String(200)			
consent_basis	Text			
documentation	Text			''
is_synthetic	Boolean			False

Column	Type	Null	Key / reference	Default
sha256	String(64)			
storage_key	String(400)			
n_samples	Integer			
splits	JSON			{}
fields	JSON			[]
duplicate_report	JSON			{}
created_by_id	Uuid	✓	FK → users.id	
created_at	DateTime			utcnow()

Constraints: Unique (name, version).

deletion_requests

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
investigation_id	Uuid		FK → investigations.id , indexed	
requested_by_id	Uuid		FK → users.id	
reason	Text			
status	String(20)			'pending'
decided_by_id	Uuid	✓	FK → users.id	
decided_at	DateTime	✓		
executed_at	DateTime	✓		
created_at	DateTime			utcnow()

detector_runs

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
job_id	Uuid		FK → analysis_jobs.id (on delete cascade), indexed	
position	Integer			0
method_id	String(80)			
method_version	String(40)			
method_name	String(200)			
evidence_class	String(40)			
status	String(30)			'pending'
input_sha256	String(64)			
config	JSON			{}
threshold	Float	✓		

Column	Type	Null	Key / reference	Default
summary	Text	✓		
output	JSON			{}
output_schema	String(60)			'aegis.method_result/v1'
score	Float	✓		
score_semantics	Text	✓		
calibrated	Boolean			False
flagged	Boolean	✓		
uncertainty	JSON	✓		
warnings	JSON			[]
limitations	JSON			[]
error	Text	✓		
started_at	DateTime	✓		
finished_at	DateTime	✓		
duration_ms	Integer	✓		

Constraints: Unique (job_id, method_id).

detector_versions

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
method_id	String(80)		indexed	
version	String(60)			
name	String(200)			
provider_id	String(60)	✓	FK → model_providers.id	
model_identifier	String(300)	✓		
license	String(200)	✓		
supported_media	JSON			[]
languages_evaluated	JSON			[]
minimum_input	JSON			{}
known_limitations	JSON			[]
data_documentation	Text	✓		
evidence_class	String(40)			
implementation_status	String(40)			
validation_status	String(30)			'unvalidated'
validation_benchmark_run_id	Uuid	✓	FK → benchmark_runs.id	
validation_note	Text	✓		

Column	Type	Null	Key / reference	Default
calibration_profile_id	Uuid	✓	FK → calibration_profiles.id	
operating_threshold	Float	✓		
enabled	Boolean			True
updated_at	DateTime			utcnow() · on update utcnow()
created_at	DateTime			utcnow()

Constraints: Unique (method_id, version).

evidence_items

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
investigation_id	Uuid		FK → investigations.id, indexed	
organization_id	Uuid		FK → organizations.id, indexed	
display_name	String(255)			
role	String(30)			'primary'
modality	String(20)			
mime_type	String(120)			
status	String(30)			'active'
description	Text	✓		
acquired_at	DateTime			utcnow()
uploaded_by_id	Uuid		FK → users.id	
is_demo_fixture	Boolean			False
extraction	JSON	✓		

evidence_versions

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
evidence_item_id	Uuid		FK → evidence_items.id, indexed	
version_no	Integer			
kind	String(30)			
parent_version_id	Uuid	✓	FK → evidence_versions.id	
sha256	String(64)		indexed	
size_bytes	BigInteger			
mime_type	String(120)			
storage_key	String(400)			
original_filename	String(255)	✓		

Column	Type	Null	Key / reference	Default
transformations	JSON			[]
metadata_status	JSON			{}
scan_status	String(60)			'not_scanned'
reason	Text	✓		
created_by_id	Uuid		FK → users.id	
created_at	DateTime			utcnow()
purged_at	DateTime	✓		

Constraints: Unique (evidence_item_id, version_no).

experiment_runs

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
experiment_id	String(60)		indexed	
kind	String(20)			
config	JSON			{}
results	JSON			
manifest	JSON			{}
created_by_id	Uuid	✓	FK → users.id	
created_at	DateTime			utcnow()

file_hashes

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
evidence_version_id	Uuid		FK → evidence_versions.id , indexed	
algorithm	String(20)			'sha256'
value	String(128)		indexed	
purpose	String(30)			
matches_recorded	Boolean	✓		
computed_by_id	Uuid	✓	FK → users.id	
computed_at	DateTime			utcnow()

investigations

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
organization_id	Uuid		FK → organizations.id , indexed	

Column	Type	Null	Key / reference	Default
workspace_id	Uuid	✓	FK → workspaces.id	
case_reference	String(80)			
title	String(300)			
purpose	Text			
media_category	String(40)			
description	Text			''
priority	String(20)			'normal'
status	String(40)			'draft'
legal_basis	Text	✓		
retention_class	String(40)			'standard'
access_classification	String(40)			'standard'
legal_hold	Boolean			False
allow_external_processing	Boolean			False
disposition	String(80)	✓		
pending_questions	JSON			[]
assigned_reviewer_id	Uuid	✓	FK → users.id	
created_by_id	Uuid		FK → users.id	
is_demo_fixture	Boolean			False
created_at	DateTime			utcnow()
updated_at	DateTime			utcnow() · on update utcnow()
archived_at	DateTime	✓		

Constraints: Unique (organization_id, case_reference); Index ix_investigations_org_status on (organization_id, status).

memberships

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
user_id	Uuid		FK → users.id (on delete cascade), indexed	
organization_id	Uuid		FK → organizations.id (on delete cascade), indexed	
role_id	String(40)		FK → roles.id	
created_at	DateTime			utcnow()

Constraints: Unique (user_id, organization_id).

model_providers

Column	Type	Null	Key / reference	Default
id	String(60)		PK	
name	String(200)			
kind	String(20)			
external_processing	Boolean			False
terms_reference	Text	✓		
notes	Text	✓		

notifications

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
user_id	Uuid		FK → users.id (on delete cascade)	
organization_id	Uuid	✓	FK → organizations.id	
kind	String(40)			
severity	String(20)			'info'
title	String(200)			
body	Text			''
link	String(300)	✓		
created_at	DateTime			utcnow()
read_at	DateTime	✓		

Constraints: Index ix_notifications_user_unread on (user_id, read_at).

organizations

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
name	String(200)			
slug	String(80)		unique	
allow_external_processing	Boolean			False
created_at	DateTime			utcnow()

permissions

Column	Type	Null	Key / reference	Default
id	String(60)		PK	
description	Text			''

provenance_records

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
evidence_item_id	Uuid		FK → evidence_items.id, indexed	
detector_run_id	Uuid		FK → detector_runs.id (on delete cascade), unique	
standard	String(40)			'c2pa'
state	String(60)			
summary	JSON			{}
trust_anchor	String(200)	✓		
verified_at	DateTime			utcnow()

reports

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
investigation_id	Uuid		FK → investigations.id, indexed	
version	Integer			
status	String(20)			'draft'
supersedes_id	Uuid	✓	FK → reports.id	
correction_reason	Text	✓		
schema_version	String(30)			'aegis.report/v1'
payload	JSON			
json_sha256	String(64)			
pdf_storage_key	String(400)	✓		
pdf_sha256	String(64)	✓		
generated_by_id	Uuid		FK → users.id	
generated_at	DateTime			utcnow()
finalized_by_id	Uuid	✓	FK → users.id	
finalized_at	DateTime	✓		

Constraints: Unique (investigation_id, version).

retention_policies

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
organization_id	Uuid		FK → organizations.id, indexed	
classification	String(40)			

Column	Type	Null	Key / reference	Default
retain_days	Integer			
description	Text			''
updated_at	DateTime			utcnow() · on update utcnow()

Constraints: Unique (organization_id, classification).

review_decisions

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
investigation_id	Uuid		FK → investigations.id (on delete cascade), indexed	
reviewer_id	Uuid		FK → users.id	
decision	String(60)			
rationale	Text			
agrees_with_automated	Boolean	✓		
automated_outcome	String(80)	✓		
supersedes_id	Uuid	✓	FK → review_decisions.id	
content_sha256	String(64)			
audit_event_hash	String(64)	✓		
created_at	DateTime			utcnow()

role_permissions

Column	Type	Null	Key / reference	Default
role_id	String(40)		PK, FK → roles.id (on delete cascade)	
permission_id	String(60)		PK, FK → permissions.id (on delete cascade)	

roles

Column	Type	Null	Key / reference	Default
id	String(40)		PK	
display_name	String(80)			
description	Text			''

segment_findings

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
detector_run_id	Uuid		FK → detector_runs.id (on delete cascade), indexed	
start	Float			
end	Float			
unit	String(20)			
label	String(80)			
score	Float	✓		
detail	JSON			{}

stored_object_chunks

Column	Type	Null	Key / reference	Default
key	String(512)		PK, FK → stored_objects.key (on delete cascade)	
seq	Integer		PK	
data	LargeBinary			

stored_objects

Column	Type	Null	Key / reference	Default
key	String(512)		PK	
content_type	String(200)			
size_bytes	BigInteger			
created_at	DateTime			utcnow()

system_health_events

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
component	String(80)		indexed	
status	String(20)			
detail	Text			''
created_at	DateTime			utcnow()

system_settings

Column	Type	Null	Key / reference	Default
key	String(80)		PK	
value	JSON			
updated_by_id	Uuid	✓	FK → users.id	
updated_at	DateTime			utcnow() · on update utcnow()

upload_sessions

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
investigation_id	Uuid		FK → investigations.id , indexed	
user_id	Uuid		FK → users.id	
filename	String(255)			
role	String(30)			'primary'
total_size	BigInteger			
received	BigInteger			0
status	String(20)			'open'
created_at	DateTime			utcnow()

user_sessions

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
user_id	Uuid		FK → users.id (on delete cascade), indexed	
refresh_token_hash	String(64)		unique	
created_at	DateTime			utcnow()
expires_at	DateTime			
revoked_at	DateTime	✓		
user_agent	String(300)	✓		
ip_address	String(64)	✓		

users

Column	Type	Null	Key / reference	Default
id	Uuid		PK	uuid4()
email	String(320)		unique	
display_name	String(200)			
password_hash	String(300)			

Column	Type	Null	Key / reference	Default
`is_active`	Boolean			True
`is_system_admin`	Boolean			False
`mfa_secret_encrypted`	Text	✓		
`mfa_enabled`	Boolean			False
`failed_login_count`	Integer			0
`locked_until`	DateTime	✓		
`last_login_at`	DateTime	✓		
`is_demo_account`	Boolean			False
`created_at`	DateTime			utcnow()

`workspaces`

Column	Type	Null	Key / reference	Default
`id`	Uuid		PK	uuid4()
`organization_id`	Uuid		FK → `organizations.id` (on delete cascade), indexed	
`name`	String(200)			
`created_at`	DateTime			utcnow()

Constraints: Unique (organization_id, name).

APPENDIX D

Role and Permission Matrix

Generated from `apps/api/app/security/permissions.py`, the single source of truth for authorisation (copied into the database by `python -m app.cli sync`).

D.1 Roles

Role id	Name	Description	Permissions
`system_admin`	System Administrator	Full access, including system settings.	32
`org_admin`	Organization Administrator	Administers an organisation's users, settings and models.	30
`investigator`	Investigator	Opens cases, acquires evidence, runs analyses, drafts reports.	19
`forensic_analyst`	Forensic Analyst	Performs analyses and research evaluations.	17
`independent_reviewer`	Independent Reviewer	Reviews evidence and records the human assessment.	15
`auditor`	Auditor	Read-only access to cases, reports and the audit trail.	9
`researcher`	Read-Only Researcher	Research bench and benchmark results only; no case data.	2

D.2 Matrix

Permission	Description	SysAdm	OrgAdm	Inv	Analyst	Reviewer	Auditor	Research
`case.create`	Create investigations	●	●	●				
`case.read`	View investigations within the organisation (subject to access classification)	●	●	●	●	●	●	
`case.update`	Edit investigation details and advance workflow	●	●	●	●			
`case.assign`	Assign reviewers	●	●	●				
`case.archive`	Archive finalised investigations	●	●					

Permission	Description	SysAdm	OrgAdm	Inv	Analyst	Reviewer	Auditor	Research
`case.reopen`	Reopen finalised or archived investigations with a recorded reason	●	●			●		
`case.read_restricted`	View investigations with restricted access classification	●	●			●	●	
`evidence.upload`	Upload evidence	●	●	●	●			
`evidence.read`	View evidence metadata and derived views	●	●	●	●	●	●	
`evidence.download`	Download original evidence	●	●	●	●	●		
`evidence.verify`	Run integrity verification	●	●	●	●	●	●	
`evidence.redact`	Create redacted derivative versions	●	●	●	●			
`analysis.run`	Queue analyses	●	●	●	●			
`analysis.read`	View analysis results	●	●	●	●	●	●	
`review.annotate`	Add annotations and questions	●	●	●	●	●		
`review.decide`	Record a human review decision	●				●		
`report.generate`	Generate draft reports	●	●	●	●			
`report.finalize`	Finalise reports	●				●		
`report.read`	View and export reports	●	●	●	●	●	●	
`appeal.submit`	Submit an appeal or correction request	●	●	●				
`appeal.resolve`	Resolve appeals	●	●			●		

Permission	Description	SysAdm	OrgAdm	Inv	Analyst	Reviewer	Auditor	Research
`retention.request_deletion`	Request deletion of an investigation's evidence	●	●	●				
`retention.approve_deletion`	Approve deletion requests (second person)	●	●			●		
`audit.read`	Read audit trail and verify the hash chain	●	●				●	
`research.use`	Use the research bench and calculators	●	●	●	●	●		●
`benchmark.read`	View benchmark runs	●	●	●	●	●	●	●
`benchmark.run`	Run benchmarks	●	●		●			
`dataset.register`	Register evaluation datasets	●	●		●			
`admin.users`	Manage users and memberships	●	●					
`admin.models`	Manage the model registry, calibration profiles and thresholds	●	●					
`admin.settings`	Manage organisation settings, retention policies and feature flags	●	●					
`system.health`	View system health and queues	●	●	●	●		●	

APPENDIX E

Compliance Control Mapping

Reproduced from `docs/governance/compliance-mapping.json` (version 2026-10).

> **Disclaimer.** This mapping records the technical status of controls in this repository. It is not a statement of compliance with any law, regulation or security standard. Legal and security review is required before operational deployment.

Status summary: `documentation_required` 3; `external_assessment_required` 6; `implemented_and_tested` 22; `not_applicable` 1; `not_implemented` 2; `partially_implemented` 6 — 40 controls in total.

Id	Area	Requirement	Status	Evidence
SEC-01	Access control	Role-based access control enforced server-side on every endpoint	`implemented_and_tested`	security/deps.py require(); apps/api/tests/test_security.py::test_role_permissions_enforced_server_side
SEC-02	Access control	Organisation (tenant) isolation; identifiers from other tenants return 404	`implemented_and_tested`	test_cross_tenant_isolation
SEC-03	Access control	Restricted access classification for sensitive cases	`implemented_and_tested`	test_restricted_classification
SEC-04	Authentication	Argon2id password hashing, lockout after repeated failures, login rate limiting	`implemented_and_tested`	security/passwords.py; test_login_rate_limit_and_lockout
SEC-05	Authentication	Short-lived access tokens bound to revocable server-side sessions; refresh rotation	`implemented_and_tested`	security/tokens.py; test_session_revocation; test_refresh_requires_csrf_header
SEC-06	Authentication	Optional TOTP multi-factor authentication	`partially_implemented`	Enrolment and login enforcement implemented; no recovery codes or admin-enforced MFA policy; not covered by automated tests

Id	Area	Requirement	Status	Evidence
SEC-07	Authentication	Integration with institutional identity providers (SAML/OIDC)	`not_implemented`	Roadmap item
SEC-08	Input handling	Content-based file validation; executables, macros, unsafe archives and malformed media rejected	`implemented_and_tested`	aegis_ml/evidence/sniff.py; test_malicious_uploads_rejected; test_zip_traversal_and_bomb_rejected
SEC-09	Input handling	Malware scanning of uploads	`partially_implemented`	ClamAV clamd adapter implemented; not exercised in this build environment; AEGIS_REQUIRE_MALWARE_SCAN enforces presence
SEC-10	Input handling	Uploaded content treated as untrusted data; instruction-like text recorded as an observation only	`implemented_and_tested`	text/extract.py; test_prompt_injection_text_is_flagged_as_observation
SEC-11	Transport & headers	Security headers, CSP, CORS restrictions, correlation IDs	`implemented_and_tested`	core/middleware.py; test_error_format_and_headers
SEC-12	Transport & headers	TLS termination and HSTS in production	`documentation_required`	Deployment guide documents a TLS-terminating reverse proxy; not configured in development
SEC-13	Secrets	No secrets in responses, logs or reports; watermark keys represented by fingerprints	`implemented_and_tested`	test_no_secrets_in_responses; reference watermark key fingerprint
SEC-14	Secrets	Managed secret store (e.g. Vault, cloud KMS)	`external_assessment_required`	Environment-variable injection only; integration depends on host platform

Id	Area	Requirement	Status	Evidence
SEC-15	Encryption at rest	Encryption of stored evidence and database at rest	`partially_implemented`	S3 backend requests SSE (AES256); local filesystem and PostgreSQL rely on host disk encryption; MFA secrets encrypted at field level
SEC-16	Supply chain	Locked dependencies and vulnerability scanning	`partially_implemented`	uv.lock and pnpm-lock.yaml committed; make security-check runs pip-audit and pnpm audit; results depend on scan date
EVD-01	Evidence handling	SHA-256 at acquisition, write-once content-addressed storage, versioned derivatives	`implemented_and_tested`	services/evidence.py; storage/local.py; test_storage_is_write_once; test_s3_backend_with_moto
EVD-02	Evidence handling	Integrity re-verification with high-priority alert on mismatch	`implemented_and_tested`	test_integrity_verification_and_mismatch_alert
EVD-03	Evidence handling	Legally certified chain of custody	`external_assessment_required`	Technical records exist; legal certification requires a validated operational process and legal review
AUD-01	Audit	Append-only, hash-chained audit trail with verification	`implemented_and_tested`	audit/service.py; test_audit_tampering_detected
AUD-02	Audit	External anchoring of the audit chain head	`partially_implemented`	/api/v1/audit/head exposes the head for anchoring; no automatic anchoring service
GOV-01	Human oversight	Automated outcomes never trigger adverse action; human review decision required to finalise reports	`implemented_and_tested`	services/reports.py finalize(); test_full_text_workflow

Id	Area	Requirement	Status	Evidence
GOV-02	Human oversight	Separation of duties (case creator cannot review; author cannot finalise; requester cannot approve own deletion)	`implemented_and_tested`	test_state_machine_rules; test_requester_cannot_approve_own_deletion
GOV-03	Human oversight	Appeal and correction workflow	`implemented_and_tested`	test_appeal_workflow; report corrections create new versions
GOV-04	Privacy	Data minimisation: no demographic collection, no biometric identification, no speaker/face identification	`implemented_and_tested`	No such fields or methods exist; subgroup data only from evaluation datasets
GOV-05	Privacy	Retention policies, legal hold, two-person deletion with tombstones	`implemented_and_tested`	test_deletion_two_person_rule_and_legal_hold
GOV-06	Privacy	Redaction producing derivative versions while preserving the original	`implemented_and_tested`	test_redaction_creates_new_version_preserving_original
GOV-07	Privacy	External processing disabled by default; three-level authorisation (deployment, organisation, case)	`implemented_and_tested`	test_provider_failure_and_policy
GOV-08	Privacy	Privacy impact assessment	`documentation_required`	Template in docs/governance/privacy-impact-assessment-template.md
GOV-09	Law	India Digital Personal Data Protection Act 2023 compliance	`external_assessment_required`	No compliance is claimed; requires legal review of the operational deployment

Id	Area	Requirement	Status	Evidence
GOV-10	Law	EU AI Act obligations (if applicable to the deployer)	`external_assessment_required`	No compliance is claimed
GOV-11	Law	Admissibility of reports as evidence	`external_assessment_required`	No evidentiary status is claimed
SCI-01	Scientific validity	Simulated, measured, published and vendor-reported results kept separate	`implemented_and_tested`	benchmark_runs.result_kind check constraint; test_reference_watermark_benchmark_and_threshold_explorer
SCI-02	Scientific validity	Detectors cannot be marked validated without locally measured benchmark evidence	`implemented_and_tested`	platform.update_model; same test
SCI-03	Scientific validity	Strict-FPR operating points flagged as not estimable when sample size is insufficient	`implemented_and_tested`	reliability.tpr_at_fpr; test_tpr_at_fpr_and_estimability
SCI-04	Scientific validity	Validated AI-text detector for English and Indian languages	`not_implemented`	No model weights are bundled; adapters exist and require locally supplied weights plus benchmark validation
SCI-05	Scientific validity	Independent third-party evaluation	`external_assessment_required`	Protocol in docs/research/independent-evaluation-protocol.md
OPS-01	Operations	Backups and tested restore	`documentation_required`	Procedure in docs/operations/backup-and-recovery.md; restore test must be executed by the operator
OPS-02	Operations	Load testing with documented results	`partially_implemented`	scripts/loadtest.py provided; results in docs/operations/performance.md are from the development container only

Id	Area	Requirement	Status	Evidence
OPS-03	Operations	Government security certification	`not_applicable`	AEGIS holds no certification; none is claimed

APPENDIX F

Source File Inventory

Every one of the 213 files tracked in the repository, with its size in lines (or bytes for binary files) and a one-line description taken from its module docstring, heading or leading comment where one exists. In total the text files contain 47,085 lines.

(root)

File	Size	Description
.dockerignore	15	Docker build-context exclusions
.env.example	59	Configuration template
.gitignore	43	Git ignore rules
LICENSE	21	MIT licence
Makefile	91	Task runner
README.md	190	Project overview, run options, sign-in table, capability status
docker-compose.yml	131	Full Docker Compose stack
package.json	20	pnpm workspace root
pnpm-lock.yaml	5,182	pnpm lock file
pnpm-workspace.yaml	3	pnpm workspace members
pyproject.toml	43	uv workspace root; Ruff and pytest configuration
render.yaml	54	Render Blueprint
uv.lock	4,513	uv lock file

.github

File	Size	Description
workflows/ci.yml	63	

apps/api

File	Size	Description
alembic.ini	37	
app/__init__.py	3	AEGIS API application package.
app/api/__init__.py	0	
app/api/v1/__init__.py	0	
app/api/v1/_util.py	12	
app/api/v1/analysis.py	247	
app/api/v1/auth.py	259	

File	Size	Description
app/api/v1/evidence.py	378	
app/api/v1/investigations.py	786	
app/api/v1/platform.py	876	Overview, search, notifications, audit, health, detectors, model registry and administration.
app/api/v1/reports.py	188	
app/api/v1/research.py	464	Statistical Reliability Laboratory, Research Bench and Benchmark Laboratory endpoints.
app/audit/__init__.py	0	
app/audit/service.py	107	Append-only, hash-chained audit trail.
app/cli.py	358	Operational commands.
app/core/__init__.py	0	
app/core/config.py	86	Runtime configuration from environment variables (see .env.example).
app/core/errors.py	77	Consistent error responses: {“error”: {“code”, “message”, “correlation_id”, “details”}}.
app/core/logging.py	39	Structured JSON logging with correlation IDs.
app/core/middleware.py	56	Correlation IDs, security headers, request logging.
app/core/ratelimit.py	78	Fixed-window rate limiting backed by Redis, with an in-process fallback for development/tests.
app/db.py	68	Database engine and session management.
app/main.py	93	AEGIS API entrypoint.
app/models/__init__.py	658	SQLAlchemy models. UUID primary keys everywhere except the audit sequence number.
app/schemas/__init__.py	0	
app/schemas/common.py	40	
app/security/__init__.py	0	
app/security/deps.py	96	Authentication and authorisation dependencies.
app/security/passwords.py	30	
app/security/permissions.py	140	Roles and permissions. Enforced server-side on every endpoint via `require`.

File	Size	Description
app/security/tokens.py	73	Short-lived JWT access tokens bound to revocable server-side sessions; opaque refresh tokens.
app/services/__init__.py	0	
app/services/analysis.py	539	Analysis job orchestration. Jobs are persisted; workers execute them idempotently.
app/services/benchmarks.py	557	Benchmark Laboratory: dataset registration, reproducible runs, calibration profiles.
app/services/bootstrap.py	61	Idempotent bootstrap: roles/permissions and model-registry records for every registered method.
app/services/evidence.py	518	Evidence acquisition, versioning, integrity verification and redaction.
app/services/health.py	118	Truthful component health: every status comes from an actual check at request time.
app/services/investigations.py	125	Investigation access control and workflow state machine.
app/services/notifications.py	24	
app/services/reports.py	662	Reproducible forensic reports (JSON schema `aegis.report/v1` + PDF).
app/services/scanning.py	35	Malware scanning via ClamAV's clamd INSTREAM protocol, when configured.
app/storage/__init__.py	0	
app/storage/base.py	88	Evidence object storage. Objects are content-addressed and write-once: an existing key is never overwritten. Deletion is only possible through the retention pur
app/storage/database.py	91	
app/storage/local.py	83	
app/storage/s3.py	62	
app/workers/__init__.py	0	
app/workers/celery_app.py	53	Celery application: Redis broker, late acknowledgement, persisted job state in PostgreSQL.
app/workers/tasks.py	34	
migrations/env.py	38	
migrations/script.py.mako	26	
migrations/versions/20261017_0001_initial_schema.py	1,017	initial schema

File	Size	Description
`migrations/versions/20261018_0002_database_object_store.py`	47	database object store
`pyproject.toml`	34	
`tests/conftest.py`	187	API test harness: a dedicated PostgreSQL database migrated with Alembic from clean, a temporary evidence store, and manual job execution (tests call the worker
`tests/test_evidence_governance.py`	353	Evidence integrity, versioning, redaction, retention, appeals, workflow rules.
`tests/test_hosted_deployment.py`	118	Settings used by hosted deployments without a persistent disk or Redis (see docs/operations/render.md).
`tests/test_research_api.py`	137	
`tests/test_security.py`	230	Access control, tenant isolation, malicious input, rate limiting and secret handling.
`tests/test_workflow_e2e.py`	157	End-to-end workflow: sign in, create case, upload text, run a real method, review, annotate, report, verify report references, audit trail.

`apps/web`

File	Size	Description
`.eslintrc.json`	1	
`next.config.mjs`	38	@type {import('next').NextConfig} */
`package.json`	43	
`playwright.config.ts`	16	Runs against an already-running stack (make dev or docker compose up) seeded with `make seed-demo`.
`postcss.config.mjs`	1	
`src/app/(app)/admin/page.tsx`	199	Administration
`src/app/(app)/audit/page.tsx`	100	Audit and compliance
`src/app/(app)/benchmarks/page.tsx`	353	Benchmark laboratory
`src/app/(app)/cases/[id]/page.tsx`	553	Case workspace
`src/app/(app)/cases/page.tsx`	101	Case files list
`src/app/(app)/evidence/page.tsx`	63	Evidence vault
`src/app/(app)/forensics/[modality]/page.tsx`	120	Text, image, audio and video forensics views
`src/app/(app)/health/page.tsx`	44	System health
`src/app/(app)/investigations/new/page.tsx`	303	New investigation form

File	Size	Description
src/app/(app)/layout.tsx	5	Authenticated layout (AppShell)
src/app/(app)/page.tsx	125	Overview dashboard
src/app/(app)/provenance/page.tsx	65	Provenance and watermarks
src/app/(app)/reports/page.tsx	48	Reports
src/app/(app)/research/page.tsx	251	Research bench
src/app/globals.css	88	Design tokens and base styles
src/app/icon.svg	1	AEGIS logo
src/app/layout.tsx	23	Root layout
src/app/login/page.tsx	123	Sign-in page
src/app/providers.tsx	27	Query client and auth providers
src/components/analysis.tsx	629	Job view, fusion panel, run cards, artefacts
src/components/badges.tsx	71	Domain badges
src/components/charts.tsx	233	Recharts chart components
src/components/data-table.tsx	107	Generic data table
src/components/methods.tsx	44	Method capability lists
src/components/shell.tsx	425	Application shell: navigation, header, search, notifications
src/components/system-map.tsx	66	Live processing-pipeline diagram
src/components/ui/index.tsx	498	Design-system primitives
src/features/case/evidence.tsx	290	Evidence cards, previews, upload and run dialogs
src/lib/api.ts	188	Thin API client. The access token lives in memory only; the refresh token is an httpOnly,
src/lib/auth.tsx	85	Authentication context
src/lib/format.ts	130	Formatters and label dictionaries
src/lib/upload.ts	49	Uploads and browser-side hashing
tailwind.config.ts	38	
tests/e2e/workflow.spec.ts	124	Playwright end-to-end tests
tests/format.test.ts	24	Vitest unit tests
tsconfig.json	22	
vitest.config.ts	3	

apps/worker

File	Size	Description
README.md	14	AEGIS worker

docs/api

File	Size	Description
README.md	54	API guide
openapi.json	11,068	OpenAPI specification snapshot
provider-contract.md	45	External detection provider contract

docs/architecture

File	Size	Description
data-model.md	53	Data model
decisions.md	27	Architecture decisions
overview.md	109	Architecture overview

docs/assets

File	Size	Description
screenshot-benchmarks.png	96 KB	Screenshot
screenshot-case-analysis.png	125 KB	Screenshot
screenshot-health.png	90 KB	Screenshot
screenshot-image.png	103 KB	Screenshot
screenshot-login.png	129 KB	Screenshot
screenshot-overview.png	98 KB	Screenshot
screenshot-provenance.png	101 KB	Screenshot
screenshot-research.png	94 KB	Screenshot
screenshot-text.png	104 KB	Screenshot

docs/governance

File	Size	Description
README.md	29	Institutional pilot readiness package
compliance-mapping.json	294	Machine-readable control mapping
data-retention-policy-template.md	20	Data-retention policy (template)
evidence-handling-procedure.md	12	Evidence-handling procedure (template)
human-review-and-appeal-procedure.md	20	Human review and appeal procedure
limitations-and-residual-risks.md	27	Limitations and residual risks
privacy-impact-assessment-template.md	15	Privacy impact assessment (template)
roadmap.md	12	Roadmap

File	Size	Description
sbom/node-licenses.json	1,024	Node production licences
sbom/sbom-python.cdx.json	1	CycloneDX SBOM (Python)

docs/operations

File	Size	Description
backup-and-recovery.md	30	Backup and recovery
college-server.md	63	Running AEGIS on a college server
configuring-detectors.md	48	Configuring a real detector
deployment.md	71	Deployment
incident-response.md	32	Incident response plan (template)
performance.md	39	Performance
render.md	93	Deploying on Render (free plan)
troubleshooting.md	20	Troubleshooting

docs/research

File	Size	Description
dataset-cards.md	34	Dataset cards
evaluation-methodology.md	37	Evaluation methodology
experiments.md	124	Reliability experiments and results
independent-evaluation-protocol.md	12	Independent evaluation protocol
methods-and-literature.md	42	Methods and literature map
model-cards.md	43	Model cards
multilingual-evaluation-plan.md	18	Multilingual evaluation plan (English first, then Indian languages)

docs/security

File	Size	Description
audit-chain.md	33	Audit chain
security-controls.md	23	Security controls matrix
threat-model.md	37	Threat model

infrastructure/deployment

File	Size	Description
Caddyfile	26	HTTPS front door for AEGIS. AEGIS_SITE is set by scripts/server-setup.sh:

File	Size	Description
`docker-compose.server.yml`	28	Server overlay: adds the Caddy HTTPS proxy and production restart policies.

`infrastructure/docker`

File	Size	Description
`api.Dockerfile`	49	AEGIS API and worker image (same image, different command).
`render.Dockerfile`	67	AEGIS in one container, for platforms that run a single web process per service (for example the
`web.Dockerfile`	30	AEGIS web workstation. Build from the repository root:

`infrastructure/render`

File	Size	Description
`render-external-database.yaml`	42	Render Blueprint: AEGIS as one free web service using a PostgreSQL database hosted elsewhere
`serve.py`	58	Applies migrations, synchronises roles and the model registry, optionally loads the demo data, and serves the API on 127.0.0.1:8000.
`start.sh`	36	Entry point of infrastructure/docker/render.Dockerfile. Starts the API on 127.0.0.1:8000 (serve.py:

`ml`

File	Size	Description
`aegis_ml/__init__.py`	9	AEGIS forensic analysis methods.
`aegis_ml/audio/__init__.py`	0	
`aegis_ml/audio/analysis.py`	223	Audio forensics: container inspection (ffprobe), PCM decoding (ffmpeg), waveform/spectrogram and signal-quality observations. No speaker identification or attri
`aegis_ml/core.py`	180	Shared types for analysis methods.
`aegis_ml/evidence/__init__.py`	0	
`aegis_ml/evidence/sniff.py`	131	Content-based file type identification and archive safety checks.
`aegis_ml/fixtures.py`	272	Synthetic demo/test fixtures generated at runtime (never committed as evidence).

File	Size	Description
`aegis_ml/fusion.py`	251	Evidence Fusion Service (decision policy `aegis-fusion-v1`).
`aegis_ml/image/__init__.py`	0	
`aegis_ml/image/analysis.py`	389	Image forensics: decoding, metadata, frequency spectrum, noise residual, error level analysis.
`aegis_ml/provenance/__init__.py`	0	
`aegis_ml/provenance/c2pa_verifier.py`	321	Content Credentials (C2PA) verification using the official `c2pa-python` bindings (c2pa-rs).
`aegis_ml/providers.py`	224	Integration boundaries for detectors that AEGIS does not bundle.
`aegis_ml/registry.py`	273	Method registry and analysis profiles.
`aegis_ml/text/__init__.py`	0	
`aegis_ml/text/extract.py`	160	Text extraction from TXT, PDF and DOCX with preserved original and explicit warnings.
`aegis_ml/text/features.py`	99	Descriptive text statistics.
`aegis_ml/text/language.py`	62	Language identification.
`aegis_ml/text/model_detectors.py`	463	Model-based text detectors (supervised classifier, token statistics, Fast-DetectGPT, Binoculars).
`aegis_ml/text/reference_watermark.py`	164	Verifier (and demo generator) for the AEGIS reference text watermark.
`aegis_ml/text/segment.py`	47	Sentence and paragraph segmentation with character offsets.
`aegis_ml/text/tokenize.py`	26	Documented, dependency-free tokenizer used for length estimates and descriptive statistics.
`aegis_ml/video/__init__.py`	0	
`aegis_ml/video/analysis.py`	256	Video forensics: container/stream inspection, keyframes, sampled-frame temporal consistency and container-level audio/video alignment. No facial recognition or
`aegis_ml/visual.py`	50	Small rendering helpers for forensic visualisations (no matplotlib dependency in the API).
`pyproject.toml`	28	
`tests/conftest.py`	29	
`tests/test_media_provenance_fusion.py`	174	
`tests/test_sniff_and_text.py`	134	

packages

File	Size	Description
shared-types/package.json	9	
shared-types/src/index.ts	381	API contract types
shared-types/tsconfig.json	1	

research

File	Size	Description
aegis_research/__init__.py	8	AEGIS research package: statistical reliability engine, reference watermark, experiments.
aegis_research/experiments/__init__.py	3	
aegis_research/experiments/__main__.py	89	Run reliability experiments and write machine-readable outputs plus a manifest.
aegis_research/experiments/catalog.py	392	The nine reliability experiment categories.
aegis_research/experiments/plots.py	74	Static PNG plots for experiment outputs (optional; requires matplotlib).
aegis_research/reliability.py	697	Statistical reliability engine.
aegis_research/watermark.py	226	Reference green-list watermark (Kirchenbauer et al., 2023, “A Watermark for Large Language Models”) over integer token ids, plus a synthetic Markov language mod
pyproject.toml	19	
tests/test_reliability.py	144	
tests/test_watermark_and_experiments.py	79	

scripts

File	Size	Description
dev.sh	14	Runs the API, a Celery worker and the web app for local development.
loadtest.py	175	Repeatable load test for the AEGIS API.
server-setup.sh	62	One-command AEGIS installation on a Linux server (Ubuntu/Debian/RHEL with Docker).

APPENDIX G

Glossary

Term	Meaning in AEGIS
Access classification	`standard` or `restricted`; restricted cases are visible only to the creator, the assigned reviewer and holders of `case.read_restricted`
Adapter	A real implementation of a method that needs operator-supplied weights or credentials before it can run
ADR	Architecture decision record (`docs/architecture/decisions.md`)
Analysis job	One request to run one profile on one evidence version; contains one detector run per method
Analysis profile	A named set of methods for an investigation type, e.g. `text_triage`
Artefact	A generated visualisation (spectrum, spectrogram, contact sheet), stored write-once with its hash and a *cannot establish* statement
Audit chain	The append-only, hash-chained `audit_events` table
AUROC	Area under the ROC curve; computed exactly with the Mann–Whitney statistic; never reported alone
Base rate / prevalence	The proportion of AI-generated items in the population actually screened
Binoculars	A zero-shot detector comparing perplexity with cross-perplexity between two language models (Hans et al., 2024)
Blocked by policy	Run status of a method that would send evidence outside the deployment when external processing is not permitted
C2PA	Coalition for Content Provenance and Authenticity; its Content Credentials are signed manifests embedded in media files
Calibration profile	A Platt calibration and threshold fitted on a validation split; *validated* only for locally measured, estimable runs
Chain head	The latest `(seq, event_hash)` of the audit chain, to be anchored externally
Clopper–Pearson interval	The exact binomial confidence interval, preferred for rare events
Content addressing	Naming stored objects by the SHA-256 of their contents
Contradictory evidence	Fusion outcome when signals point in different directions, e.g. a signed capture claim plus AI-consistent detectors
Correlation id	The `X-Request-ID` value attached to every response, log line and audit event of a request
Declared metadata	Unsigned claims in a file (EXIF Software, PNG text chunks, XMP DigitalSourceType); an evidence class of its own
Detector run	One method's execution within a job; unique per job and method
Digital source type	IPTC vocabulary describing how media was made (e.g. `digitalCapture`, `trainedAlgorithmicMedia`)
ELA	Error level analysis: differences after JPEG re-compression; qualitative and widely misinterpreted

Term	Meaning in AEGIS
Estimable	An operating point whose FPR can be bounded from the available negatives: ≥ 299 for 1%, ≥ 2,995 for 0.1%
Evidence class	One of seven kinds of evidence that are never mixed or averaged
Evidence version	An immutable set of bytes belonging to an evidence item: original, normalised text, thumbnail, redacted or correction
External processing	Sending evidence to a provider outside the deployment; requires deployment, organisation and case permission
Fast-DetectGPT	A zero-shot detector based on conditional probability curvature (Bao et al., 2024)
Flagged	Set only when a validated operating threshold exists; otherwise a score is shown but not interpreted
FPR / TPR	False-positive rate and true-positive rate (sensitivity)
Fusion policy	`aegis-fusion-v1`, which combines categorical facts by evidence class and never averages scores
Green list	In the Kirchenbauer watermark, the keyed subset of the vocabulary favoured during generation
Hold-out evaluation	Testing on generators, domains or languages absent from the validation split
Idempotency key	Header that makes repeated job-creation requests return the original job
Inline task mode	Running jobs in a background thread of the API process; for single-container hosts, refused in production
Insufficient input	Run status when the input is below a method's minimum (e.g. fewer than 50 tokens)
Legal hold	A case flag that blocks deletion requests and approvals
MAD	Median absolute deviation, a robust measure of spread used in residual and video analysis
NPV / PPV	Negative and positive predictive value: the share of negative or positive calls that are correct at a stated prevalence
Platt scaling	Logistic calibration of raw scores, $p = 1/(1 + e^{-(as + b)})$
Process evidence	Drafts, revision histories and notes uploaded with the `process_evidence` role; evaluated by reviewers, never scored
Reference watermark	AEGIS's own implementation of the green-list scheme, able to verify only AEGIS-generated reference text
Result kind	`locally_measured`, `simulated`, `published`, `reproduced` or `vendor_reported`; enforced by a CHECK constraint
Separation of duties	Rules preventing one person from both performing and approving an action (creator ≠ reviewer, author ≠ finaliser, requester ≠ approver, appellant ≠ resolver)
SSE	Server-sent events; used to stream persisted job state to the browser
Tenant	An organisation; all data is scoped to organisation membership
Tombstone	The database record of a purged evidence version, kept with its hash after the bytes are deleted
TOTP	Time-based one-time password (RFC 6238), used for optional MFA

Term	Meaning in AEGIS
Two-person rule	Deletion requires approval by someone other than the requester
Unavailable	Run status of a method that cannot run in this deployment; never a negative result
Write-once storage	Storage in which an existing key can never be overwritten; deletion only through the audited retention purge